



Degree Project in Computer Simulations for Science and Engineering (Applied Mathematics)

Second cycle, 30 credits

# **Molecular-dynamics-assisted hierarchical machine learning for modified siRNA activity prediction**

**JAVIER GALLARDO SAENZ**



# **Molecular-dynamics-assisted hierarchical machine learning for modified siRNA activity prediction**

JAVIER GALLARDO SAENZ

Master's Programme, Computer Simulations for Science and Engineering, 120  
credits

Date: September 22, 2026

Supervisors: Emilie Gerard, Marco Klähn, Olof Runborg

Examiner: Olof Runborg

Host company: AstraZeneca AB - MolecularAI department

Swedish title: Molekyldynamikassisterad hierarkisk maskininlärning för  
aktivitetsprediktion av modifierade siRNA



## Abstract

Small interfering RNA (siRNA) molecules are short double-stranded oligonucleotides capable of silencing disease-associated genes through the RNA interference pathway. Their therapeutic promise is conditioned on the introduction of chemical modifications that simultaneously ensure stability, cellular delivery, target engagement and minimal off-target activity. The resulting design space is combinatorially vast and, at present, essentially inaccessible to brute-force experimental or fully atomistic exploration: a single 21-nucleotide modified duplex requires several hours of GPU time to simulate in explicit water, while the number of chemically distinct duplexes of that length easily exceeds  $10^{40}$  even under conservative assumptions on the available modifications.

This thesis investigates whether the messenger RNA knockdown activity of a modified siRNA duplex — a global, experimentally measured property — can be predicted from the structural and dynamical properties of its constituent double-stranded trimers, and whether a model built on this premise can generalise to unseen sequences and chemical modifications. To address this question, we curate a dataset of more than 88 000 experimentally characterised modified siRNAs, develop an automated fragmentation procedure that separates each modified nucleotide into backbone, sugar and base fragments, and cross-reference the extracted fragments with the *modXNA* parametrisation catalogue in order to identify the unique nucleotides and overlapping trimers that can be simulated. The compatible trimers are embedded in standardised pentamer constructs and simulated by molecular dynamics in an incremental, high-throughput pipeline that also extracts a set of structural and dynamical descriptors per trimer. In parallel, we conduct a mathematical analysis of graph-based machine learning models on siRNA molecular graphs, formalising the problem as a hierarchical learning task over a graph of graphs and proving that standard message-passing neural networks suffer from exponential over-squashing on siRNA duplexes even when hydrogen bonds and base-stacking interactions are explicitly encoded as edges. Motivated by this analysis, we introduce a modular hierarchical architecture that integrates atom-, nucleotide-, trimer- and molecule-level representations, together with the notion of *airtight* hierarchical models, in which within-level propagation is restricted to elements that share a common aggregation context.

Evaluated on a demanding, generalisation-oriented split, the proposed hierarchical models clearly surpass the best previously available baseline for modified siRNA activity prediction, and ablations show that the

molecular dynamics-derived trimer features contribute measurably to this improvement. The resulting workflow provides an end-to-end, reproducible pipeline covering dataset curation, trimer identification, molecular dynamics simulation, feature extraction and hierarchical model training, and constitutes a step towards a physics-assisted, computationally tractable exploration of the modified siRNA chemical space.

## **Keywords**

Small interfering RNA, Chemical modifications, Molecular dynamics, Hierarchical Machine Learning, Physics-assisted Machine Learning, Graph Neural Networks, Transformers, Convolutional Neural Network, Over-squashing, Activity prediction.

## Sammanfattning

Små interfererande RNA-molekyler (siRNA) är korta dubbelsträngade oligonukleotider som kan tysta sjukdomsassocierade gener genom RNA-interferensvägen. Deras terapeutiska potential förutsätter att kemiska modifieringar införs, vilka samtidigt måste säkerställa stabilitet, cellulär leverans, bindning till målsekvensen och minimal oavsiktlig påverkan på andra gener. Det resulterande designutrymmet är kombinatoriskt enormt och för närvarande i praktiken otillgängligt för uttömmande experimentell eller helt atomistisk utforskning: en enda modifierad duplex bestående av 21 nukleotider kräver flera timmars beräkningstid på en grafikprocessor för att simuleras i explicit vatten, medan antalet kemiskt skilda duplex av samma längd med lätthet överstiger  $10^{40}$  även under konservativa antaganden om vilka modifieringar som finns tillgängliga.

Denna avhandling undersöker om förmågan hos en modifierad siRNA-duplex att tysta budbärar-RNA – en global, experimentellt mätbar egenskap – kan förutsägas utifrån de strukturella och dynamiska egenskaperna hos duplexens ingående dubbelsträngade trimerer, och om en modell som bygger på denna premis kan generalisera till sekvenser och kemiska modifieringar som inte förekommit under träningen. För att besvara denna fråga sammanställer vi ett dataset med fler än 88 000 experimentellt karakteriserade modifierade siRNA, utvecklar en automatiserad fragmenteringsprocedur som delar upp varje modifierad nukleotid i ryggrads-, socker- och basfragment och korsrefererar de extraherade fragmenten mot parametriseringskatalogen *modXNA* för att identifiera de unika nukleotider och överlappande trimerer som kan simuleras. De kompatibla trimererna bäddas in i standardiserade pentamerkonstrukt och simuleras med molekylodynamik i ett inkrementellt arbetsflöde med hög genomströmning som även extraherar en uppsättning strukturella och dynamiska deskriptorer för varje trimer. Parallellt genomför vi en matematisk analys av grafbaserade maskininlärningsmodeller tillämpade på molekylära grafer för siRNA, formulerar problemet som en hierarkisk inlärningsuppgift över en graf av grafer och visar att konventionella neurala nätverk baserade på meddelandeöverföring uppvisar exponentiell överkomprimering (*over-squashing*) på siRNA-duplex, även när vätebindningar och basstaplingsinteraktioner uttryckligen representeras som kanter. Med utgångspunkt i denna analys introducerar vi en modulär hierarkisk arkitektur som integrerar representationer på atom-, nukleotid-, trimer- och molekylnivå, tillsammans med begreppet *lufttäta* hierarkiska modeller, där informationsspridningen inom en nivå begränsas till element som delar ett

gemensamt aggregeringssammanhang.

Vid utvärdering på en krävande datauppdelning, utformad för att pröva modellernas generaliseringsförmåga, överträffar de föreslagna hierarkiska modellerna tydligt den bästa tidigare tillgängliga referensmodellen för aktivitetsprediktion av modifierade siRNA, och ablationsstudier visar att de trimeregenskaper som härletts ur molekylodynamik bidrar mätbart till denna förbättring. Det resulterande arbetsflödet tillhandahåller en reproducerbar helhetslösning som omfattar kurering av data, identifiering av trimerer, molekylodynamiksimuleringar, extraktion av egenskaper och träning av hierarkiska modeller, och utgör ett steg mot en fysikassisterad och beräkningsmässigt hanterbar utforskning av det kemiska rummet för modifierade siRNA.

## Nyckelord

Små interfererande RNA, kemiska modifieringar, molekylodynamik, hierarkisk maskininlärning, fysikassisterad maskininlärning, grafneurala nätverk, transformers, konvolutionellt neuralt nätverk, överkomprimering (over-squashing), aktivitetsprediktion.

## Acknowledgments

I feel incredibly lucky that I get to try writing an acknowledgement section while not knowing where to even begin.

I will try (and probably fail) to be brief. Thanks to the COSSE programme for giving me the opportunity to spend two amazing years learning applied mathematics in some great cities with some even greater company.

Danke schön, Reinhard, for welcoming me in TU Berlin during the COSSE Workshop. Tack så mycket, Elias, for welcoming me in KTH and for all your help. Dank je wel, Kees, for welcoming me in TU Delft and for your constant support.

Regarding this thesis academic content, I would like to thank Alexander Heinlein for pointing me towards hierarchical models; my opponent Anastasios Karakoulakis for his constructive feedback; my academic supervisor Olof Runborg for his attention to detail and his patience and suggestions regarding the proof of the permutation invariance lemma, and my industry supervisors Marco Klähn and Emilie Gerard for putting together the main dataset for this thesis and devising the idea of decomposing **Small interfering RNA (siRNA)** chains into overlapping trimers.

Now for everyone beyond this thesis academic content! Thank you, Marco, for welcoming me in your team within AstraZeneca's MolecularAI department. Thank you, Emilie, for your patience, knowledge, and daily support. Thank you to everybody I met during my time at AstraZeneca - you guys made my time in Gothenburg a memory I will always cherish.

Thank you, Alessandro and Guillaume, for sharing these two years with me, from Delft to Stockholm. I will really miss our daily coexistence, and I am grateful to call you friends. Thank you, Jo, for also being a great friend, even though you abandoned us when we set course to Stockholm. Was it our absurd daily chats that made you do it? Thank you to even more good friends: Ezra, Iraklis, and all the wonderful people I have met through the COSSE programme.

A big thank you to my beloved San Diego crew, for in some way this journey started there, with you. I miss you dearly, and our (incomplete) reunions never fail to make my heart full.

Moltes gràcies a tots els amics que he fet a Barcelona i que ara estan repartits arreu del món: us estimo i us vull tornar a veure.

Muchas gracias a mis queridos amigos calagurritanos, porque siempre que no estoy allí sueño con volver y, cuando lo hago, es como si el tiempo no hubiera pasado.

Gracias infinitas a mi familia, a mis padres, por su apoyo constante, por aguantarme cada día y por su amor incondicional, que me ha convertido en la persona que soy hoy.

Last but certainly not least: thank you, Evie, for being the best partner one could wish for. This thesis made it through the finish line thanks to your unwavering love and support. I hope I can give you back everything you give to me, brighten your days like you brighten mine.

## AI use disclosure

I acknowledge the use of generative AI for the following purposes during the development of this thesis:

- To generate code which does not concern molecular dynamics simulations or feature extraction from molecular dynamics trajectories (current generative AI deeply struggles with these tasks).

Every time an AI system was prompted to generate some code, it was provided a detailed set of steps&instructions that acted as the script foundations/skeleton. This ensures that all code stems from previous human intentional planning and facilitates code reviewing.

- To review the arguments and mathematical proofs included in this work. For mathematical content, Opus 5 on high effort was used in the following way: only after the author had written down a proof for a particular result was the proof's `LaTeX` code (together with the necessary background) given to Opus, which identified alleged logical gaps, small errors, notation inconsistencies and steps with a missing rigorous justification. Some of these observations turned out to (relatively frequently) be false, so all outputs were thoroughly checked by the author.

If a logical gap was confirmed, the author went back to independently work on the proof and then restarted this process only after the reworked proof was complete.

If a small error was confirmed, it was easily fixed by the author.

In the case of identified alleged notation inconsistencies, Opus 5 was allowed to propose reworked and/or complementary definitions. These were thoroughly checked by the author and sometimes incorporated (with the pertinent changes to guarantee their appropriateness) to the thesis.

In the case of identified alleged steps with missing rigorous justification, Opus 5 was allowed to propose background lemmas to ground the derivation. Some of the proposed additional justifications were entirely obvious and therefore dismissed, and the remaining cases were thoroughly checked by the author. The proposed supplementary lemmas' claims were (almost) always found to be correct, but in almost all non-trivial cases they were missing vital conditions in their statement. Opus 5 (seemingly) randomly decided to additionally include proofs for some of the suggested lemmas, but these were disregarded by the author to independently arrive at a proof which was subsequently checked with Opus 5 following this same procedure.

- To streamline some of the written explanations in the present document.

Stockholm, September 2026

Javier Gallardo Saenz



# Contents

|           |                                                                                                   |          |
|-----------|---------------------------------------------------------------------------------------------------|----------|
| <b>1</b>  | <b>Introduction</b>                                                                               | <b>1</b> |
| 1.1       | Background . . . . .                                                                              | 2        |
| 1.2       | Problem Statement . . . . .                                                                       | 4        |
| 1.2.1     | Scientific and engineering issues . . . . .                                                       | 4        |
| 1.3       | Purpose . . . . .                                                                                 | 6        |
| 1.4       | Goals . . . . .                                                                                   | 6        |
| 1.5       | Research Methodology . . . . .                                                                    | 6        |
| 1.6       | Delimitations . . . . .                                                                           | 7        |
| 1.7       | Structure of the thesis . . . . .                                                                 | 7        |
| <b>2</b>  | <b>Background</b>                                                                                 | <b>9</b> |
| 2.1       | An overview of small interfering Ribonucleic Acid (siRNA)<br>for applied mathematicians . . . . . | 9        |
| 2.1.1     | Nucleic Acids . . . . .                                                                           | 9        |
| 2.1.2     | The role of small interfering Ribonucleic Acids . . . . .                                         | 14       |
| 2.2       | Statistical Physics and Molecular dynamics . . . . .                                              | 16       |
| 2.2.1     | Statistical Ensembles . . . . .                                                                   | 16       |
| 2.2.2     | Molecular Dynamics Simulations . . . . .                                                          | 17       |
| 2.2.3     | Statistical Considerations for Molecular Dynamics<br>Observables . . . . .                        | 18       |
| 2.2.4     | Existing methods . . . . .                                                                        | 20       |
| 2.2.4.1   | GROMACS . . . . .                                                                                 | 21       |
| 2.2.4.2   | AMBER force fields . . . . .                                                                      | 21       |
| 2.2.4.3   | ModXNA . . . . .                                                                                  | 21       |
| 2.3       | Graph Theory . . . . .                                                                            | 22       |
| 2.3.0.0.1 | BFS . . . . .                                                                                     | 29       |
| 2.3.0.0.2 | DFS . . . . .                                                                                     | 30       |
| 2.3.1     | Molecular Graphs and RDKit . . . . .                                                              | 30       |
| 2.4       | Machine Learning . . . . .                                                                        | 35       |

|           |                                                                                                 |           |
|-----------|-------------------------------------------------------------------------------------------------|-----------|
| 2.4.1     | Principal Component Analysis . . . . .                                                          | 35        |
| 2.4.2     | Clustering . . . . .                                                                            | 36        |
| 2.4.2.1   | K-Medoids . . . . .                                                                             | 37        |
| 2.4.3     | Random Forest . . . . .                                                                         | 37        |
| 2.4.4     | XGBoost . . . . .                                                                               | 38        |
| 2.4.5     | Neural Networks . . . . .                                                                       | 39        |
| 2.4.5.1   | Optimizers . . . . .                                                                            | 40        |
| 2.4.6     | Convolutional Neural Networks . . . . .                                                         | 40        |
| 2.4.7     | Message Passing Graph Neural Networks . . . . .                                                 | 41        |
| 2.4.7.0.1 | Over-squashing. . . . .                                                                         | 43        |
| 2.4.7.0.2 | Over-smoothing . . . . .                                                                        | 43        |
| 2.4.7.1   | Chemprop . . . . .                                                                              | 43        |
| 2.4.8     | Transformers . . . . .                                                                          | 44        |
| 2.4.8.1   | Scaled Dot-Product Attention . . . . .                                                          | 46        |
| 2.4.8.2   | Multi-Head Attention . . . . .                                                                  | 47        |
| 2.4.8.3   | Positional Embedding . . . . .                                                                  | 47        |
| 2.4.9     | Graph Attention Transformers . . . . .                                                          | 47        |
| 2.4.10    | Cm-siRPred . . . . .                                                                            | 48        |
| 2.4.10.1  | Hierarchical and Multi-Level ML Models . . . . .                                                | 50        |
| <b>3</b>  | <b>Theoretical Analysis of ML models on modified siRNA graphs</b>                               | <b>53</b> |
| 3.1       | Mathematical Formulation of the Research Question . . . . .                                     | 53        |
| 3.2       | The graph of graphs formulation . . . . .                                                       | 55        |
| 3.3       | Analysis of graph-based ML methods on siRNA molecular graphs . . . . .                          | 59        |
| 3.3.1     | The limitations of standard Message Passing Neural Networks on siRNA molecular graphs . . . . . | 59        |
| 3.3.1.1   | The importance of hydrogen bonds and stacking interactions . . . . .                            | 59        |
| 3.3.1.2   | Further results on over-squashing in a standard Message Passing Neural Network . . . . .        | 76        |
| 3.3.2     | Theoretical Analysis of Chemprop . . . . .                                                      | 79        |
| 3.3.3     | A hybrid hierarchical machine learning framework . . . . .                                      | 80        |
| <b>4</b>  | <b>Methods</b>                                                                                  | <b>87</b> |
| 4.1       | Research Process . . . . .                                                                      | 87        |
| 4.2       | Data Analysis and Pre-Processing . . . . .                                                      | 88        |
| 4.2.1     | Dataset Analysis . . . . .                                                                      | 88        |

|           |                                                                    |     |
|-----------|--------------------------------------------------------------------|-----|
| 4.2.2     | Extracting unique trimers, nucleotides and modifications . . . . . | 89  |
| 4.3       | Molecular dynamics preparation and simulation . . . . .            | 93  |
| 4.3.1     | Boundary effects and trimer caps . . . . .                         | 93  |
| 4.3.2     | Obtaining initial pentamer structures . . . . .                    | 94  |
| 4.3.3     | Building a modified nucleotide library with modXNA . . . . .       | 95  |
| 4.3.4     | Format normalization . . . . .                                     | 96  |
| 4.3.5     | Molecular dynamics input files generation . . . . .                | 97  |
| 4.3.6     | Pentamer simulation details . . . . .                              | 98  |
| 4.3.7     | Load optimization . . . . .                                        | 99  |
| 4.4       | Feature Extraction . . . . .                                       | 104 |
| 4.4.1     | Trajectory Cleaning and New Index File Creation . . . . .          | 105 |
| 4.4.1.1   | Trajectory Cleaning . . . . .                                      | 105 |
| 4.4.1.2   | New Index File Creation . . . . .                                  | 105 |
| 4.4.2     | Observable time-series and equilibrium detection . . . . .         | 106 |
| 4.4.3     | Processing equilibrium time-series as histograms . . . . .         | 107 |
| 4.4.4     | Extracted features . . . . .                                       | 108 |
| 4.4.4.1   | RMSD and RMSF . . . . .                                            | 108 |
| 4.4.4.2   | Base-pair, base-pair step and helical parameters . . . . .         | 110 |
| 4.4.4.3   | Pucker angle . . . . .                                             | 111 |
| 4.4.4.4   | Hydrogen bonds . . . . .                                           | 112 |
| 4.4.4.5   | Interaction Energies . . . . .                                     | 120 |
| 4.4.5     | Observable distributions under duplex dissociation . . . . .       | 122 |
| 4.4.5.1   | Run-weighted aggregation in the double-stranded region . . . . .   | 124 |
| 4.4.5.1.1 | Circular mean and circular standard deviation. . . . .             | 125 |
| 4.4.5.2   | Weighted aggregation of normalized histograms. . . . .             | 125 |
| 4.4.5.2.1 | Toy example . . . . .                                              | 127 |
| 4.4.6     | The modified siRNA trimers' dynamical properties dataset . . . . . | 127 |
| 4.5       | Machine Learning models . . . . .                                  | 128 |
| 4.5.1     | Dataset Splits . . . . .                                           | 128 |
| 4.5.1.1   | The "hard" data split . . . . .                                    | 128 |
| 4.5.1.2   | The "easy" data split . . . . .                                    | 129 |
| 4.5.1.3   | Trimer and sequence clustering . . . . .                           | 130 |
| 4.5.1.3.1 | Trimer Clustering . . . . .                                        | 130 |

|           |                                                                  |     |
|-----------|------------------------------------------------------------------|-----|
| 4.5.1.3.2 | Sequence Clustering . . . . .                                    | 130 |
| 4.5.2     | Feature construction and preprocessing . . . . .                 | 131 |
| 4.5.2.1   | Chemical features . . . . .                                      | 132 |
| 4.5.2.2   | Experimental covariates . . . . .                                | 137 |
| 4.5.2.3   | MD-derived trimer features . . . . .                             | 137 |
| 4.5.2.4   | Tensorization and model-specific input for-<br>mats . . . . .    | 139 |
| 4.5.3     | A modular design for hierarchical models . . . . .               | 140 |
| 4.5.3.1   | Nucleotide encoder . . . . .                                     | 141 |
| 4.5.3.2   | Nucleotide to sequence encoder . . . . .                         | 142 |
| 4.5.3.2.1 | CNN nucleotide-to-sequence en-<br>coder . . . . .                | 142 |
| 4.5.3.2.2 | Transformer nucleotide-to-sequence<br>encoder . . . . .          | 143 |
| 4.5.3.3   | Nucleotide-to-trimer encoder . . . . .                           | 144 |
| 4.5.3.3.1 | MLP nucleotide-to-trimer encoder                                 | 145 |
| 4.5.3.3.2 | CNN nucleotide-to-trimer encoder                                 | 145 |
| 4.5.3.4   | Trimer encoder . . . . .                                         | 146 |
| 4.5.3.4.1 | Pass-through chemical trimer en-<br>coder . . . . .              | 146 |
| 4.5.3.4.2 | Learned-only trimer encoder . . .                                | 147 |
| 4.5.3.4.3 | Hybrid chemical/learned trimer<br>encoder . . . . .              | 147 |
| 4.5.3.4.4 | Including trimer Molecular Dy-<br>namics (MD) features . . . . . | 147 |
| 4.5.3.4.5 | Additive chemical + MD embed-<br>ding . . . . .                  | 148 |
| 4.5.3.4.6 | Masked concatenation + MLP . .                                   | 148 |
| 4.5.3.5   | Trimer-to-sequence encoder . . . . .                             | 149 |
| 4.5.3.5.1 | Transformer trimer-to-sequence en-<br>coder. . . . .             | 149 |
| 4.5.3.5.2 | CNN trimer-to-sequence encoder .                                 | 150 |
| 4.5.3.6   | Prediction head . . . . .                                        | 151 |
| 4.5.4     | Baselines and evaluation methodology . . . . .                   | 152 |
| 4.5.4.1   | Ablation and Feature importance analyses .                       | 153 |
| 4.5.4.2   | XGBoost baselines . . . . .                                      | 153 |
| 4.5.4.3   | Cm-siRPred . . . . .                                             | 154 |
| 4.5.4.4   | Hierarchical Models . . . . .                                    | 155 |

|           |                                                                                     |            |
|-----------|-------------------------------------------------------------------------------------|------------|
| 4.5.4.4.1 | Transformer-based Hierarchical Models . . . . .                                     | 156        |
| 4.5.4.4.2 | CNN-based Hierarchical Models . . . . .                                             | 157        |
| <b>5</b>  | <b>Results and discussion</b>                                                       | <b>161</b> |
| 5.1       | Data analysis and pre-processing . . . . .                                          | 161        |
| 5.1.1     | Dataset analysis . . . . .                                                          | 161        |
| 5.1.2     | Trimers, nucleotides and modifications . . . . .                                    | 163        |
| 5.2       | Molecular dynamics preparation and simulation . . . . .                             | 169        |
| 5.2.1     | Structure generation and system preparation . . . . .                               | 169        |
| 5.2.2     | Determination of the number of ions . . . . .                                       | 170        |
| 5.2.3     | Selection of the production run length . . . . .                                    | 170        |
| 5.2.4     | Load optimization and cluster usage . . . . .                                       | 172        |
| 5.3       | Feature Extraction . . . . .                                                        | 172        |
| 5.3.1     | Feature time-series and equilibrium detection . . . . .                             | 172        |
| 5.3.2     | Choice of histogram bins . . . . .                                                  | 173        |
| 5.3.3     | Extracted trimer features . . . . .                                                 | 174        |
| 5.3.4     | Feature distributions and discretization . . . . .                                  | 174        |
| 5.3.4.1   | Base-pair parameters . . . . .                                                      | 174        |
| 5.3.4.2   | Pucker angle . . . . .                                                              | 177        |
| 5.3.4.3   | RMSD . . . . .                                                                      | 178        |
| 5.3.4.4   | Bonded, open and dissociated probabilities . . . . .                                | 179        |
| 5.3.4.5   | Interaction energies . . . . .                                                      | 180        |
| 5.3.4.6   | Hydrogen bonds . . . . .                                                            | 183        |
| 5.3.5     | Errors and excluded systems . . . . .                                               | 186        |
| 5.4       | ML Results . . . . .                                                                | 187        |
| 5.4.1     | Dataset split and pre-processing . . . . .                                          | 187        |
| 5.4.1.1   | Hard data split . . . . .                                                           | 187        |
| 5.4.1.2   | Easy data split . . . . .                                                           | 192        |
| 5.4.2     | Cm-siRPred Baseline Evaluation on the Hard Data Split                               | 195        |
| 5.4.3     | XGBoost Baseline Evaluation on the Hard Data Split . . . . .                        | 195        |
| 5.4.4     | Transformer-based Hierarchical Models Evaluation on the Hard Data Split . . . . .   | 200        |
| 5.4.5     | CNN-based Hierarchical Models Evaluation on the Hard Data Split . . . . .           | 208        |
| 5.4.6     | Best Model Hyper-parameter Optimization & Ablation on the Hard Data Split . . . . . | 213        |
| 5.4.7     | Best Model & Baseline Evaluation on the Easy Data Split . . . . .                   | 219        |

|          |                                                                |            |
|----------|----------------------------------------------------------------|------------|
| <b>6</b> | <b>Conclusions and Future work</b>                             | <b>221</b> |
| 6.1      | Conclusions . . . . .                                          | 221        |
| 6.2      | Limitations . . . . .                                          | 223        |
| 6.3      | Future work . . . . .                                          | 223        |
| 6.4      | Reproducibility . . . . .                                      | 225        |
| 6.5      | Impact Statement . . . . .                                     | 225        |
|          | <b>References</b>                                              | <b>227</b> |
| <b>A</b> | <b>Supporting materials</b>                                    | <b>237</b> |
| A.1      | Short discussion on the ergodicity of MD simulations . . . . . | 238        |
| A.2      | Errors . . . . .                                               | 238        |
| A.2.1    | Initial load re-optimization bug . . . . .                     | 238        |
| A.3      | Complementary Figures . . . . .                                | 239        |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                   |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Diagrammatic representation of the implemented pipeline for the case of a single siRNA molecule. The maps $t_i(\cdot)$ and $n_i(\cdot)$ are defined in Equations (4.55) and (4.48), respectively. The map $MD(\cdot)$ simply refers to the concatenated MD features of the input trimer. Finally, $a(\cdot)$ denotes the predicted activity for the input siRNA molecule. . . . . | 2  |
| 2.1 | 2D representation of a Ribonucleic Acid (RNA) A nucleotide. The phosphate group is highlighted in blue, the sugar in green and the base (adenine) in red. The carbons in the sugar group are labelled with their standard notation. . . . .                                                                                                                                       | 10 |
| 2.2 | Differences between RNA (left column) and DNA (right column) nucleotides. Taken from [7, Figure 8] . . . . .                                                                                                                                                                                                                                                                      | 11 |
| 2.3 | Overview of the double helix structure (top-left), hydrogen bonds in base pairing (top right) and base staking (bottom) on Deoxyribonucleic Acid (DNA). Taken from [7, Figure 2] . . . . .                                                                                                                                                                                        | 12 |
| 2.4 | Helical parameters. Taken from [11, Figure 1] . . . . .                                                                                                                                                                                                                                                                                                                           | 13 |
| 2.5 | An overview of the central dogma of biology, representing information flow in eukaryote cells. Taken from [7, Figure 9] . . . . .                                                                                                                                                                                                                                                 | 15 |
| 2.6 | Original transformer architecture - Taken from [48, Figure 1]. The encoder architecture corresponds to the left stack and the decoder architecture corresponds to the right stack. . . . .                                                                                                                                                                                        | 45 |
| 2.7 | Original Edge Set Transformer architecture - Taken from [50, Figure 4] . . . . .                                                                                                                                                                                                                                                                                                  | 49 |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Example graph of graphs structure over an underlying, standard graph (left) and its corresponding coarse-grained graph (right).<br>The nodes in the coarse grained interfaces and the edges between them are highlighted in red. Observe that $\partial_{G_1}(G_2) = \{2, 4\}$ , $\partial_{G_1}(G_3) = \{4\}$ , $\partial_{G_2}(G_1) = \{3\}$ , $\partial_{G_2}(G_3) = \{5\}$ , $\partial_{G_3}(G_1) = \partial_{G_3}(G_2) = \{6\}$ .<br>The blue arrows on the underlying graph (left) depict a path $p = (1, 2, 3, 4, 6, 7, 8)$ with length $l(p) = 6$ . The blue arrows on the coarse-grained graph (right) depict the coarse-grained walk $p^{CG} = (G_1, G_2, G_1, G_3)$ associated to it, with coarse-grained length $c(p) = 3$ . . . . . | 56 |
| 3.2 | Ball of radius 4 around the generic $\partial^+$ (in red) and $\partial^-$ (in blue) interfaces of a RNA G nucleotide in a strand with direction 5' to 3'. Any nucleotide with the same backbone and sugar but an arbitrary base – as long as it has a Nitrogen which binds it to the sugar and has two Carbon neighbours in the base – would yield the same balls. . . . .                                                                                                                                                                                                                                                                                                                                                                      | 68 |
| 3.3 | G-C base pair with generic $\partial^+$ (in red) and $\partial^-$ (in blue) interfaces. Hydrogen bonds between the two nucleotides are included as edges (dashed lines). Hydrogen atoms are implicit and not included as graph nodes.<br>The golden arrows showcase the two paths for which $g_{\min} = 5$ is obtained.<br>The cyan arrows showcase the three paths for which $g_{\max} = 21$ is obtained. These three paths also reveal the three possible geodesic crossings from one nucleotide in the base pair to the other; which dictate the maximum number of geodesic paths between any two nodes in $\partial^+ \cup \partial^-$ , i.e. $M_g = 3$ .<br>See Lemma 42 for the definition of $g_{\min}$ , $g_{\max}$ and $M_g$ . . . . .  | 75 |
| 3.4 | Molecular highway on two consecutive G-C base pairs. Hydrogen bonds between the two nucleotides in each base pair are included as edges (dashed lines). Hydrogen atoms are implicit and not included as graph nodes. Stacking interactions are also included as edges (dotted lines). Interfaces with the previous nucleotide pair appear in blue, while interfaces with the next nucleotide pair appear in red. . . . .                                                                                                                                                                                                                                                                                                                         | 77 |
| 3.5 | Overview of the proposed architecture . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 86 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |     |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 4.1  | Illustration of the residue pair sets $\mathcal{P}_{bp}$ (4.11) (left, in red), $\mathcal{P}_{dl}$ (4.13) (middle, in green) and $\mathcal{P}_{cl}$ (4.12) (right, in blue) on a generic pentamer with numbered residues . . . . .                                                                                                                                                                                                                                  | 114 |
| 4.2  | Illustration of a triplet ( $D, H, A$ ) in its natural plane and the variables $d_{DH}, d_{DA}, d_{AH}, \theta_{DHA}, \theta_{ADH}, \theta_{DAH}$ . . . . .                                                                                                                                                                                                                                                                                                         | 115 |
| 5.1  | Distribution of activity values, concentrations, incubation times, target cell types, transfection methods and activity detection methods across the dataset. . . . .                                                                                                                                                                                                                                                                                               | 164 |
| 5.2  | Nucleotide structures covering all chemical modifications present in the dataset with base fragments highlighted in red, sugar fragments highlighted in green and backbone fragments highlighted in blue. Each subfigure caption lists the backbone, sugar, and base modXNA IDs. A Simplified Molecular Input Line Entry System (SMILES) string in the place of the sugar fragment modXNA ID is used for those sugar modifications not available in modXNA. . . . . | 167 |
| 5.3  | 2D representation of trimer #75 in the <code>sirna_unique_trimers_modXNA_noterm</code> dataset guide strand obtained by linking its conforming nucleotides . . . . .                                                                                                                                                                                                                                                                                                | 168 |
| 5.4  | Whole-pentamer Root Mean Square Deviation (RMSD) plot for the test run of trimer 3 in <code>sirna_unique_trimers_modXNA_noterm</code> showcasing two snapshots of the pentamer's configuration: one right after the established equilibrium point (around 5 ns, left); and another one after the RMSD transitions to a different mode which corresponds to the pentamer opening (around 22 ns, right). . . . .                                                      | 171 |
| 5.5  | Empirical pooled distribution of the shear BP parameter . . . .                                                                                                                                                                                                                                                                                                                                                                                                     | 175 |
| 5.6  | Empirical pooled distribution of the stretch BP parameter . . . .                                                                                                                                                                                                                                                                                                                                                                                                   | 175 |
| 5.7  | Empirical pooled distribution of the stagger BP parameter . . . .                                                                                                                                                                                                                                                                                                                                                                                                   | 175 |
| 5.8  | Empirical pooled distribution of the buckle BP parameter . . . .                                                                                                                                                                                                                                                                                                                                                                                                    | 176 |
| 5.9  | Empirical pooled distribution of the propeller BP parameter . . . .                                                                                                                                                                                                                                                                                                                                                                                                 | 176 |
| 5.10 | Empirical pooled distribution of the opening BP parameter . . . .                                                                                                                                                                                                                                                                                                                                                                                                   | 176 |
| 5.11 | Empirical pooled distribution of the major BP parameter . . . .                                                                                                                                                                                                                                                                                                                                                                                                     | 176 |
| 5.12 | Empirical pooled distribution of the minor BP parameter . . . .                                                                                                                                                                                                                                                                                                                                                                                                     | 177 |
| 5.13 | Empirical pooled distribution of the sugar pucker angle. . . . .                                                                                                                                                                                                                                                                                                                                                                                                    | 178 |
| 5.14 | Empirical pooled distribution of the whole-pentamer RMSD . . . .                                                                                                                                                                                                                                                                                                                                                                                                    | 179 |
| 5.15 | Empirical pooled distribution of the per-residue RMSD . . . . .                                                                                                                                                                                                                                                                                                                                                                                                     | 179 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |     |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.16 | Empirical pooled distribution of the interaction energy between the pentamer and the ions in the system . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 181 |
| 5.17 | Empirical pooled distribution of the interaction energy between same-strand consecutive residue pairs, i.e., residue pairs belonging to $\mathcal{P}_{c1}$ . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 181 |
| 5.18 | Empirical pooled distribution of the interaction energy between same-strand residue pairs that lie two positions away from each other, i.e., residue pairs belonging to $\mathcal{P}_{c2}$ . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 181 |
| 5.19 | Empirical pooled distribution of the interaction energy between inter-strand diagonally consecutive residue pairs, i.e., residue pairs belonging to $\mathcal{P}_{d1}$ . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | 182 |
| 5.20 | Empirical pooled distribution of the interaction energy between inter-strand once diagonally removed residue pairs, i.e., residue pairs belonging to $\mathcal{P}_{d2}$ . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 182 |
| 5.21 | Empirical pooled distribution of the interaction energy between base pairs, i.e., residue pairs belonging to $\mathcal{P}_{bp}$ . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 182 |
| 5.22 | Empirical pooled distributions of hydrogen-bond counts for the four residue-pair classes considered in the dataset . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 184 |
| 5.23 | Empirical pooled distribution of the donor–acceptor distance in hydrogen bonds between base-pairs . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 185 |
| 5.24 | Empirical pooled distribution of the donor–hydrogen–acceptor angle in hydrogen bonds between base-pairs . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 186 |
| 5.25 | Mean and one standard deviation of the hydrogen bond lifetime curves across all hydrogen bonds between residue pairs in $\mathcal{P}_{bp}$ (blue), $\mathcal{P}_{c1}$ (orange) and $\mathcal{P}_{d1}$ (green) . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | 186 |
| 5.26 | Exploration of the similarity between the medoids of the selected trimer clusters (labelled 840, 668, 120, 750 and 606) to be held out from the Hard split training set. Top-left plot illustrates the distribution of pairwise similarities between all medoid pairs and selected-clusters-only medoid pairs. Top-right plot illustrates the distribution of pairwise similarities between medoid pairs where one is the medoid of a selected cluster and the other is the medoid of an unselected cluster. Bottom-left plot illustrates the distribution of the coverage, defined as the pairwise similarity between the medoid of an unselected cluster and its closest (i.e., most similar) medoid of a selected cluster. Bottom-right plot illustrates the selected clusters medoids pair-wise similarity matrix . . . . . | 189 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                             |     |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.27 | Explained variance curves for the PCA-based preprocessing steps used in the chemical feature construction pipeline for the Hard data split. Top left: nucleotide fingerprints and nucleotide fingerprint differences. Top right: trimer fingerprint-based representations and trimer fingerprint-difference representations. Bottom: nucleotide RDKit descriptors. . . . .                                                                                  | 191 |
| 5.28 | Explained variance curves for the PCA-based preprocessing steps used in the chemical feature construction pipeline for the Hard data split. Top left: nucleotide fingerprints and nucleotide fingerprint differences. Top right: trimer fingerprint-based representations and trimer fingerprint-difference representations. Bottom: nucleotide RDKit descriptors. . . . .                                                                                  | 194 |
| 5.29 | Feature-importance analysis for the nucleotide-level XGBoost baseline N2. Left: normalized importance aggregated by feature group. Right: positional importance of the guide-strand fingerprint-difference features. . . . .                                                                                                                                                                                                                                | 197 |
| 5.30 | Feature-importance analysis for the trimer-level XGBoost baseline T2. Left: normalized importance aggregated by feature group. Right: positional importance of the trimer fingerprint features. . . . .                                                                                                                                                                                                                                                     | 198 |
| 5.31 | Comparison of normalized feature-group importances for the trimer-level XGBoost baselines. Left: chemical-only model (T3). Right: model augmented with cross-run means of the extracted MD observables (T4). . . . .                                                                                                                                                                                                                                        | 200 |
| 5.32 | Validation-set MAE as a function of the convolutional kernel width $k^{(w)}$ for the best-performing nucleotide-to-sequence and trimer-to-sequence Convolutional Neural Network (CNN) backbones. . . . .                                                                                                                                                                                                                                                    | 210 |
| 5.33 | Diagram of the best performing model’s generalized architecture, with unspecified hyper-parameters and MD features. The map $t_i(\cdot)$ is defined rigorously in (4.55). The map MD( $\cdot$ ) simply refers to the concatenated MD features of the input trimer, while the map mask( $\cdot$ ) indicates whether MD features for the input trimer are available. Finally, $a(\cdot)$ denotes the predicted activity for the input siRNA molecule. . . . . | 214 |
| A.1  | Distribution of targeted genes across the dataset . . . . .                                                                                                                                                                                                                                                                                                                                                                                                 | 240 |



# List of Tables

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |     |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 4.1  | Simulation protocol used for capped trimer (pentamer) duplexes.                                                                                                                                                                                                                                                                                                                                                                                                               | 100 |
| 4.2  | Main molecular dynamics parameters used across the pentamer simulation workflow. . . . .                                                                                                                                                                                                                                                                                                                                                                                      | 101 |
| 4.3  | Attributes of the <code>PartitionConfig</code> objects used in load optimization. . . . .                                                                                                                                                                                                                                                                                                                                                                                     | 102 |
| 4.4  | Summary of RMSD and Root Mean Square Fluctuation (RMSF) extraction. . . . .                                                                                                                                                                                                                                                                                                                                                                                                   | 110 |
| 4.5  | Base-pair parameters extracted with <code>nastruct</code> . . . . .                                                                                                                                                                                                                                                                                                                                                                                                           | 111 |
| 4.6  | Base-pair step parameters extracted with <code>nastruct</code> . . . . .                                                                                                                                                                                                                                                                                                                                                                                                      | 111 |
| 4.7  | Helical parameters extracted with <code>nastruct</code> . . . . .                                                                                                                                                                                                                                                                                                                                                                                                             | 112 |
| 4.8  | Classification of sugar pucker phase angles into standard conformational regions. . . . .                                                                                                                                                                                                                                                                                                                                                                                     | 112 |
| 4.9  | Tensors produced by the preprocessing pipeline and passed to the modular hierarchical model. Note that the default trimer input tokens $[B, T, d_{\text{trimer},i}]$ as described in (4.55) are built by concatenating the trimer-level fingerprint-based representations and the nucleotide-level features for trimer tokens tensors across their last dimension, i.e., $d_{\text{trimer},i} = d_{\text{trimer},\text{fp}} + 6 \cdot d_{\text{nuc},\text{trimer}}$ . . . . . | 139 |
| 4.10 | Hyperparameter values kept fixed across the evaluation of the machine learning models . . . . .                                                                                                                                                                                                                                                                                                                                                                               | 153 |
| 4.11 | Tested nucleotide-level XGBoost hyper-parameter configurations. . . . .                                                                                                                                                                                                                                                                                                                                                                                                       | 154 |
| 4.12 | Tested trimer-level XGBoost hyper-parameter configurations. . . . .                                                                                                                                                                                                                                                                                                                                                                                                           | 155 |
| 5.1  | Top 10 repeated siRNA sequence pairs in the dataset, ranked by occurrence count. The anti-sense strand ( <code>as_seq</code> ) is the guide strand written in 5'–3' orientation, and the sense strand ( <code>s_seq</code> ) is the passenger strand written in 3'–5' orientation. . . . .                                                                                                                                                                                    | 162 |

|      |                                                                                                                                                                                                                                        |     |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.2  | Representation of <code>sugars_siRNA</code> database, including sugar SMILES and modXNA IDs but excluding IUPAC International Chemical Identifier Key (InChIKey)                                                                       | 165 |
| 5.3  | Comparison of total components in the dataset versus those who additionally fulfill the criteria of being parametrized by modXNA, and those who satisfy both the previous criteria and the condition of spanning no termini            | 165 |
| 5.4  | Summary of success and failure counts during molecular dynamics system construction and preparation for the 5925 unique non-cap trimers parameterized in modXNA. Percentages are reported relative to the initial set of 5925 trimers. | 169 |
| 5.5  | Distribution summaries and selected histogram ranges for base-pair parameters.                                                                                                                                                         | 175 |
| 5.6  | Distribution summaries and selected histogram ranges for interaction-energy features.                                                                                                                                                  | 180 |
| 5.7  | Summary statistics and discretization choices for hydrogen-bond count between different types of residue pairs                                                                                                                         | 184 |
| 5.8  | Summary statistics and discretization choices for geometric hydrogen-bond observables in hydrogen bonds between base pairs                                                                                                             | 185 |
| 5.9  | Summary of the mean hydrogen-bond lifetime curves across all detected hydrogen bonds within each residue-pair class                                                                                                                    | 185 |
| 5.10 | Predefined categories of siRNA molecules forced into the test split. Percentages are reported relative to the full dataset of 88,833 molecules.                                                                                        | 188 |
| 5.11 | Summary of the PCA dimensionality reduction results on the Hard training split.                                                                                                                                                        | 189 |
| 5.12 | Summary of the PCA dimensionality reduction results on the Hard training split.                                                                                                                                                        | 192 |
| 5.13 | Validation-set MAE for the tested <code>Cm-siRPre</code> configurations.                                                                                                                                                               | 195 |
| 5.14 | Validation-set MAE for the nucleotide-level XGBoost baselines.                                                                                                                                                                         | 196 |
| 5.15 | Validation-set MAE for the trimer-level chemical-only XGBoost baselines.                                                                                                                                                               | 197 |
| 5.16 | Validation-set MAE for the trimer-level XGBoost models used in the MD-feature importance study.                                                                                                                                        | 199 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |     |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.17 | Validation-set MAE for the initial exploration of transformer backbones. Nucleotide-to-sequence models with $d_{nuc} = 64$ use a one-layer nucleotide encoder with hidden dimension 64, while nucleotide-to-sequence models with $d_{nuc} = 32$ simply implement a projection layer from the input $d_{nuc,i} = 37$ . Trimer-to-sequence models use the default trimer token representation with $d_{trimer} = 128$ . . . . .                                                                                                                                                                    | 201 |
| 5.18 | Validation-set Mean Absolute Error (MAE) for the further exploration of nucleotide-to-sequence and nucleotide base-pair-to-sequence transformer architectures. Nucleotide-to-sequence models all have $d_{nuc} = 128$ and use a one-layer nucleotide encoder with hidden dimension 64. Nucleotide base-pair-to-sequence models with $d_{nuc,bp} = 64$ implement a projection layer from the input $d_{nuc,i} = 37$ to $d_{nuc}$ ; while nucleotide base-pair-to-sequence models with $d_{nuc,bp} = 128$ use a one-layer nucleotide encoder with hidden dimension 64 and $d_{nuc} = 64$ . . . . . | 202 |
| 5.19 | Validation-set MAE for different pooling strategies applied to the best-performing transformer backbone from Table 5.17. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 203 |
| 5.20 | Validation-set MAE for transformer-based trimer models using learned and hybrid trimer encoders. All models inherit the best-performing trimer-to-sequence transformer backbone and pooling strategy selected in Stage 1; corresponding to 8 heads, 2 layers, feed-forward dimension 256 and Max pooling. The nucleotide encoder is a one-layer Multi Layer Perceptron (MLP) with hidden dimension 64 and output dimension $d_{nuc} = 32$ . . . . .                                                                                                                                              | 204 |
| 5.21 | Validation-set MAE for MD-aware transformer-based trimer encoders using cross-run means and standard deviations of all extracted MD observables. All models inherit the best-performing transformer backbone selected in the previous stages; corresponding to 8 heads, 2 layers, feed-forward dimension 256 and Max pooling. The initial MD dropout is fixed to 0.1. . . . .                                                                                                                                                                                                                    | 205 |

|      |                                                                                                                                                                                                                                                                                                                                                                                                               |     |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.22 | Validation-set MAE for different MD dropout values applied to the best-performing MD-aware trimer encoder from Table 5.21; consisting on the concatenation MD encoder with a one-layer MLP of hidden dimension 256 on the concatenated chemical & MD inputs and a trimer-to-sequence transformer with 8 heads, 2 layers, feed-forward dimension 256 and Max pooling. . . . .                                  | 206 |
| 5.23 | Validation-set MAE for different trimer-level MD feature representations using the best-performing MD-aware trimer encoder and MD dropout value. . . . .                                                                                                                                                                                                                                                      | 207 |
| 5.24 | Summary of the best-performing transformer-based hierarchical models. Both models employ the observed optimal trimer-to-sequence transformer with 8 heads, 2 layers, feed-forward dimension 256 and Max pooling . . . . .                                                                                                                                                                                     | 208 |
| 5.25 | Validation-set MAE for the initial exploration of CNN backbones. Nucleotide-to-sequence models use $d_{\text{nuc}} = 32$ and a one-layer MLP nucleotide encoder with hidden dimension 64. Trimer-to-sequence models use $d_{\text{trimer}} = 128$ . All models use mean pooling . . . . .                                                                                                                     | 208 |
| 5.26 | Validation-set MAE for the kernel-width sweep of the best-performing nucleotide-to-sequence CNN backbone from Table 5.25. . . . .                                                                                                                                                                                                                                                                             | 209 |
| 5.27 | Validation-set MAE for the kernel-width sweep of the best-performing trimer-to-sequence CNN backbone from Table 5.25.                                                                                                                                                                                                                                                                                         | 210 |
| 5.28 | Validation-set MAE for different pooling strategies applied to the best-performing CNN backbone identified in Stage 1; corresponding to a trimer-to-sequence CNN model with 3 convolutional layers, channel dimension 128, kernel width $k^{(w)} = 6$ and sequence padding. . . . .                                                                                                                           | 211 |
| 5.29 | Validation-set MAE for CNN-based hierarchical models using learned and hybrid trimer encoders, together with alternative nucleotide-to-trimer encoders. All models inherit the best-performing trimer-to-sequence CNN backbone and pooling strategy selected in Stage 1; consisting of 3 convolutional layers, channel dimension 128, kernel width $k^{(w)} = 6$ , sequence padding and mean pooling. . . . . | 211 |
| 5.30 | Validation-set MAE for a MD-aware trimer-to-sequence CNN, varying the MD dropout rate and setting the convolutional kernel to $k^{(w)} = 6$ . . . . .                                                                                                                                                                                                                                                         | 212 |

|      |                                                                                                                                                                                                                                                                                                                                 |     |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 5.31 | Validation-set MAE for different MD feature representations applied to the best-performing CNN-based trimer architecture. The MD-aware trimer encoder is inherited from the transformer-based exploration and the initial MD dropout is fixed to 0.3. . . . .                                                                   | 212 |
| 5.32 | Summary of the best-performing transformer-based hierarchical models. Both models employ the observed optimal trimer-to-sequence CNN with $d_{\text{trimer}} = d_{\text{seq}} = 128$ , 3 convolutional layers with channel size 128, kernel width $k^{(w)} = 6$ , mean pooling and sequence padding. . . . .                    | 213 |
| 5.33 | Architecture summary of the best-performing model. . . . .                                                                                                                                                                                                                                                                      | 215 |
| 5.34 | Optuna search space for the hyper-parameter optimization of the best-performing MD-aware trimer-to-sequence CNN model. Fixed architectural choices inherited from Table 5.33 were $k^{(w)} = 6$ , sequence padding enabled and mean pooling. . . . .                                                                            | 216 |
| 5.35 | Cross-seed ablation of the MD-derived trimer features . . . . .                                                                                                                                                                                                                                                                 | 216 |
| 5.36 | One-factor-at-a-time ablation study of the best-performing model. Each row reports a single variation with respect to the reference configuration in Table 5.33, while all other hyper-parameters are kept fixed. . . . .                                                                                                       | 217 |
| 5.37 | Feature-group ablation study for the MD descriptors in the best-performing model. Each row reports the result obtained after removing one MD feature group from the full MD input while keeping all other feature groups unchanged. The feature group with the largest negative effect is indicated with an asterisk *. . . . . | 218 |
| 5.38 | Final comparison of the best-performing baseline models and the best overall model on the validation and test sets of the Hard data split. . . . .                                                                                                                                                                              | 218 |
| 5.39 | Final comparison of the best-performing baseline models and the best overall model on the validation and test sets of the Easy data split. . . . .                                                                                                                                                                              | 219 |



## List of acronyms and abbreviations

|          |                                             |
|----------|---------------------------------------------|
| BFS      | Breadth First Search                        |
| CNN      | Convolutional Neural Network                |
| D-MPNN   | Directed Message Passing Neural Network     |
| DFS      | Depth First Search                          |
| DNA      | Deoxyribonucleic Acid                       |
| FFN      | Feed Forward Network                        |
| GNN      | Graph Neural Network                        |
| InChI    | IUPAC International Chemical Identifier     |
| InChIKey | IUPAC International Chemical Identifier Key |
| JS       | Jensen–Shannon                              |
| MAE      | Mean Absolute Error                         |
| MD       | Molecular Dynamics                          |
| MILP     | Mixed Linear Integer Programming            |
| ML       | Machine Learning                            |
| MLP      | Multi Layer Perceptron                      |
| MPNN     | Message Passing Neural Network              |
| mRNA     | Messenger RNA                               |
| PBC      | Periodic Boundary Condition                 |
| PCA      | Principal Component Analysis                |
| PDE      | Partial Differential Equation               |
| RISC     | RNA-induced silencing complex               |
| RMSD     | Root Mean Square Deviation                  |
| RMSF     | Root Mean Square Fluctuation                |
| RNA      | Ribonucleic Acid                            |
| RNAi     | RNA interference                            |
| SDG      | Sustainable Development Goal                |

|        |                                              |
|--------|----------------------------------------------|
| siRNA  | Small interfering RNA                        |
| SMILES | Simplified Molecular Input Line Entry System |
| UN     | United Nations                               |

# Chapter 1

## Introduction

**Small interfering RNA (siRNA)** molecules are double-stranded oligonucleotides which are able to silence gene expression through a process denominated as the **RNA interference (RNAi)** pathway. This process ultimately achieves gene silencing by the knockdown (i.e., destruction) of the **Messenger RNA (mRNA)** molecules that would otherwise be read by ribosomes - leading to the synthesis of the proteins through which genes express themselves. Such behaviour is of strong therapeutical promise: **siRNA** enables precise, programmable knockdown of disease-associated genes across diverse indications, including metabolic, hepatic, and rare genetic disorders.

However, effective **siRNA** design faces several challenges. Therapeutic **siRNA** must deliver strong knockdown while minimizing off-target effects (it must avoid silencing non-problematic genes) and immunological response. Achieving this requires overcoming delivery barriers (e.g. cellular uptake) and ensuring the **siRNA** molecule is stable enough to partake in the **RNAi** pathway without compromising its central role in this process.

These constraints create a multi-objective optimization problem in which local structural and physicochemical features strongly influence global efficacy, specificity, and safety. The primary strategy to balance these objectives is the use of chemically modified **siRNA** [1], [2], [3]. This is a complex problem for which experimental trial and error is the current method to measure how chemical modifications affect the desired gene knockdown.

The aims of this project are to test whether **mRNA** knockdown efficacy (a global property) of modified **siRNA** duplexes can be predicted from

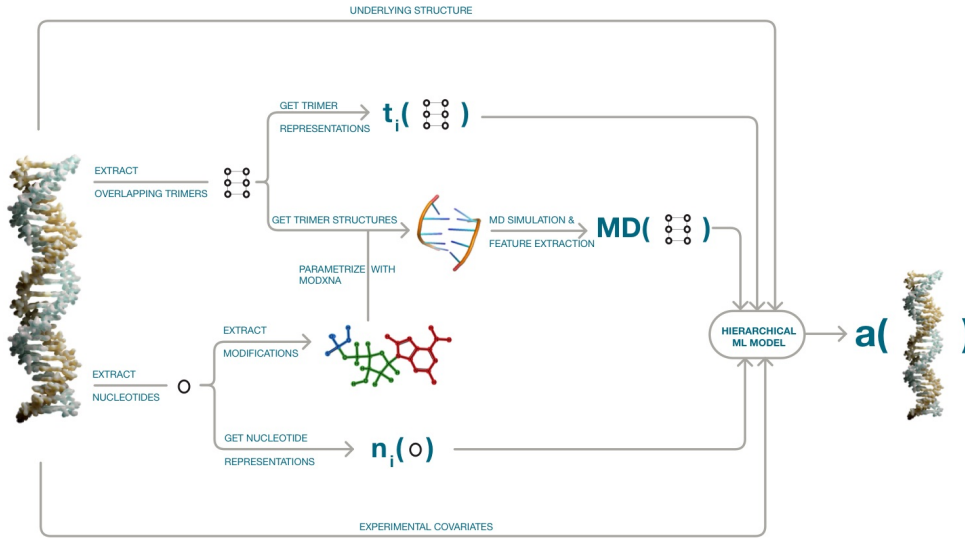


Figure 1.1: Diagrammatic representation of the implemented pipeline for the case of a single **siRNA** molecule. The maps  $t_i(\cdot)$  and  $n_i(\cdot)$  are defined in Equations (4.55) and (4.48), respectively. The map  $MD(\cdot)$  simply refers to the concatenated **MD** features of the input trimer. Finally,  $a(\cdot)$  denotes the predicted activity for the input **siRNA** molecule.

the structural and dynamical properties of the **siRNA**'s constituent double-stranded trimers. The **siRNA** duplexes are approximately 21-nucleotides long and contain modifications on either strand. We propose a modular workflow combining curated modified-**siRNA** data, **Molecular Dynamics (MD)** simulations of representative pentamer constructs embedding target trimers, automated feature extraction, and a hierarchical **Machine Learning (ML)** framework. This model architecture establishes the integration of multi-level information from individual nucleotides, overlapping trimers and the molecule as a whole to predict **siRNA** activity (knockdown). Figure 1.1 illustrates the implemented pipeline for the case of a single **siRNA** molecule.

## 1.1 Background

In order to move from pure experiment-based exploration to a computationally-informed approach for modified **siRNA** design, one would first need to be able

to run MD simulations of modified siRNA. However, pre-existing force fields only considered standard oligonucleotides. The modXNAproject [4] provides a library of parametrised modifications\* and a systematic way of combining them, offering the possibility to up to simulate 132,685 modified nucleic acids.

While modXNA has greatly facilitated the MD simulation of modified siRNA molecules, it is also important to recognize that parallel work without it has achieved significant progress in the understanding of the individual behaviour of modified siRNA [2].

Unfortunately, this approach is too resource-intensive and thus incompatible with the efficient exploration of the chemical space of modified siRNA molecules. An alternative approach is to tackle modified siRNA activity prediction through data-based methods. This approach relies on publicly available experimental activities of modified siRNA molecules to train models that can extract patterns to make activity predictions [3]. To the best of our knowledge, Cm-siRPred [5] remains the most advanced and ambitious approach to modified siRNA activity prediction. Initially trained on a limited dataset consisting of only 4278 siRNA molecules, the model has also been applied to the new CMsiRNAdb [6], which consists of 43153 modified siRNA molecules with experimentally measured activities.

While the performance of Cm-siRPred on the new CMsiRNAdb dataset has not been disclosed, the original Cm-siRPred paper reported a Mean Absolute Error (MAE) of 0.1152 on the test split of the dataset it was trained on, and a MAE of 0.13 on an independent dataset. However, it was trained using a random split (ensuring only a uniform distribution of different activity values in the train and test splits), which is not enough to evaluate the model's generalization capabilities to unseen sequences and modifications. Furthermore, its input data neglects important features for oligonucleotide duplex characterization such as hydrogen bonds or sugar ring puckering.

Our literature research did not identify any previous approaches combining data-driven methods with physical features obtained from MD simulation for siRNA activity prediction.

---

\*In the context of MD simulations, a modification having been parametrised means that the force field parameters concerning said modification have been assigned a value. For example, we can imagine a simple force field  $U(q; \lambda) = \lambda q^2$  where the value of  $\lambda$  must be adjusted depending on the type of particle being simulated. Parametrising this force field for a given particle type  $a$  means determining the appropriate value  $\lambda_a$  for the parameter  $\lambda$ .

## 1.2 Problem Statement

Our research question is the following: Can **mRNA** knockdown – a global **siRNA** property – be predicted from the structural and dynamical properties of the **siRNA**'s constituent double-stranded trimers? Furthermore, can a **ML** model that positively answers our inquiry generalize to unseen sequences and chemical modifications?

To answer these questions, we introduce a dataset consisting of more than 88000 **siRNA** molecules whose activities have been experimentally measured. The structure of our problem prompts a series of accompanying questions:

- What modifications are present in our dataset? How many unique trimers are needed to cover all the **siRNA** molecules in our dataset?
- Can we run MD simulations to extract the physical properties of those trimers and, if so, are these results informative of the trimer's behaviour inside a larger chain?
- Can these physical (dynamical) features be effectively integrated with the readily available chemical (structural) features for every molecule, with the final objective of improving **ML** models' **siRNA** activity prediction?
- What **ML** model architecture is appropriate to integrate information at different scales within a modified **siRNA** molecule, and to exploit it for activity prediction?

A successful model would be able to take an input modified **siRNA** molecule, identify its overlapping, constituent trimers, retrieve their structural and dynamical data, integrate those and attend to the other trimers in the chain in order to make an accurate prediction of the input **siRNA**'s activity.

### 1.2.1 Scientific and engineering issues

Some of the main challenges associated to predicting **siRNA** activity are related to the high computational cost of full **siRNA** simulations and the exploding combinatorial space resulting from the introduction of chemical modifications into **siRNA**. These two conditions force us to search for an efficient, accurate algorithm that can enable high-throughput exploration of the **siRNA** chemical space.

In order to grasp the sheer size of this combinatorial space, let us get slightly ahead of ourselves for just one moment. Every nucleotide can be divided in three fragments: backbone, sugar and base. Canonical **Ribonucleic Acid (RNA)** bases are denoted A, U, C, G; with A-U and C-G being the canonical base pairs found in double-stranded **RNA** molecules. Please see Section 2.1 for a more in-depth explanation of the biology of **siRNA**.

Let now  $m_B$  be the number of available modifications to the backbone group of a nucleotide,  $m_S$  the number of available modifications to the sugar and  $m_A, m_C, m_G, m_U$  the number of available modifications to the canonical **RNA** bases of a **RNA** nucleotide. Then, the number of double stranded **siRNA** chains of length  $l$  that we can build is simply

$$[2(m_A m_U + m_C m_G) m_B^2 m_S^2]^l$$

Suppose that we do not modify the nucleotides' bases, this is,  $m_A = m_U = m_C = m_G = 1$ , and suppose also that we have only 3 possible backbone and sugar fragments to choose from,  $m_B = m_S = 3$ . Then, choosing  $l = 21$  we see that there are approximately  $5 \cdot 10^{52}$  different double-stranded **siRNAs** that could be built.

One might think that it would be more reasonable to think in terms of possible **siRNAs** with a given base sequence, which would be our object of study in a real-life scenario once a target sequence within the target **mRNA** has been identified. In this case, our formula simplifies to

$$[m_B^2 m_S^2]^l$$

which under last example's conditions still yields approximately  $10^{40}$  possible combinations. For reference, to simulate a double-stranded, 21 nucleotide long **siRNA** chain in explicit water we would need approximately 8 hours of compute in a state of the art GPU.

It is also important to note that decomposing the chain in overlapping trimers from which we can extract relevant properties is a promising idea, but for this approach to succeed we must design an algorithm that is able to predict a global property from local ones. These global properties may be influenced by long-range dependencies that are not easily captured by local descriptors.

## 1.3 Purpose

The purpose of this thesis is to advance research on the field of therapeutic modified **siRNA**, taking steps towards a workflow which enables the efficient exploration of the modified **siRNA** chemical space. Although this is still very early-stage research, it can have an extremely positive impact down the line: access to modified **siRNA** drugs could be life-changing for millions of people.

## 1.4 Goals

This project has the following goals:

- Extract the unique modifications, nucleotides and trimers present in a curated dataset of publicly available siRNAs.
- Cross-reference these with available chemical modifications in the `modXNAparameters` catalogue.
- Perform high-throughput MD simulations of all the unique trimers present in the dataset which are parametrized by `modXNA`.
- Extract physical/dynamical features from the trimers' MD trajectories and create a dataset with the extracted properties for each trimer.
- Design a modular ML architecture to predict modified **siRNA** activity from local descriptors.
- Create an end-to-end automated pipeline that encompasses every step of this project and allows future practitioners to seamlessly add new **siRNA** molecules to the dataset, extract any new modifications, perform MD simulations for any previously unseen trimers and add their physical features to the trimer-level dataset, and train new ML models for **siRNA** activity prediction.

## 1.5 Research Methodology

The research conducted for this thesis follows two distinct methods: the mathematical analysis follows the mathematical method and thus relies on definitions, propositions and proofs; while the computational analysis relies on computational experiments, using statistical techniques to draw

insight from data and grounding decisions on both pre-existing literature and observations.

The detailed methodology followed by this thesis is covered in Chapter 4.

## 1.6 Delimitations

In this document, we focus on chemical modifications parametrised in ModXNA [4] to ensure MD compatibility. Considered siRNA sequences must be fully parametrised using the selected modification set. Trimers are treated as double-stranded and individually modifiable on each strand. To mitigate end-effects in MD, trimers are embedded in standardized pentamer constructs where positions 1 and 5 are fixed, unmodified base pairs chosen to minimize boundary artifacts.

An interesting extension would be to consider the role of the Argonaute protein that binds the guide strand of the siRNA complex and facilitates the bonding of the guide strand to the target sequence in the objective mRNA. Unfortunately, this thesis does not explore such path due to both time and compute limitations.

## 1.7 Structure of the thesis

Chapter 2 presents relevant background information on the biology of small interfering RNA, the statistical physics and molecular dynamics concepts used to simulate it, the elements of graph theory needed to formalise siRNA molecular graphs, and the machine learning methods on which this work builds. Chapter 3 develops the mathematical formulation of the research question, introduces the graph-of-graphs description of modified siRNA and analyses the behaviour of different families of ML models on siRNA molecular graphs. Chapter 4 presents the methodology and methods used to solve the problem, covering data curation and pre-processing, high-throughput molecular dynamics simulation of modified trimers, feature extraction, reliability assessment and the design of the machine learning models. Chapter 5 reports and discusses the results obtained at each stage of the pipeline, from the characterisation of the curated dataset to the evaluation of the proposed hierarchical models. Chapter 6 summarises the main findings of the thesis, identifies its main limitations and points at possible directions for future research.



# Chapter 2

## Background

This chapter provides the background material on which the rest of the thesis builds. Section 2.1 introduces the biology of small interfering RNA, covering the structure of nucleic acids and the role of siRNA in the RNA interference pathway. Section 2.2 reviews the concepts from statistical physics and molecular dynamics that underpin the simulation of modified siRNA trimers. Section 2.3 presents the elements of graph theory that will later be used to formalise siRNA molecular graphs. Finally, Section 2.4 reviews the machine learning methods relevant to this thesis.

### 2.1 An overview of small interfering Ribonucleic Acid (siRNA) for applied mathematicians

This section aims to provide a basic description of the biological function and importance of siRNAs. The section is structured as follows: first, we shortly review key concepts for the study of RNA and Deoxyribonucleic Acid (DNA). This is based on [7, Chapter 1]. Next, we provide a simplified explanation of siRNA function based on [8], [9]. We recommend the curious reader to consult these references for an in-depth dive on the biochemistry of nucleic acids and siRNA.

#### 2.1.1 Nucleic Acids

Both DNA and RNA are linear polymers composed of nucleotides. Each nucleotide consists of a phosphate group, a sugar and a nitrogenous base.

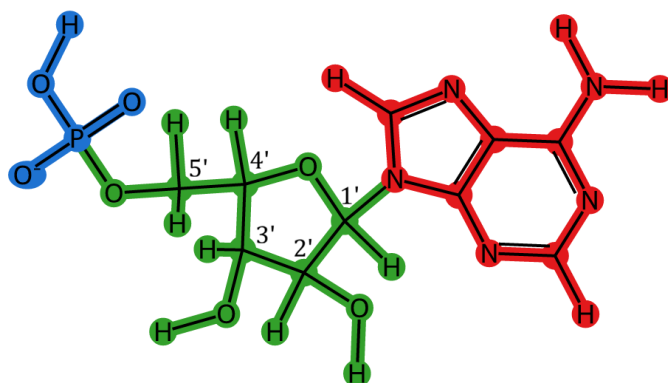


Figure 2.1: 2D representation of a **RNA** A nucleotide. The phosphate group is highlighted in blue, the sugar in green and the base (adenine) in red. The carbons in the sugar group are labelled with their standard notation.

These bases are famously named A (adenine), C (cytosine), G (guanine) and T(thymine) for **DNA**, while **RNA** substitutes T for U (uracil). An image of a **DNA** nucleotide with its three main components highlighted can be found in Figure 2.1

Apart from the presence of U nucleobases instead of T nucleobases, the main other difference between **DNA** and **RNA** is that the sugar in **RNA** has an extra hydroxyl group in the 2' position (see Figure 2.1). This is one of the reasons why **RNA** is generally more reactive and unstable than **DNA**. A visual overview of the differences between **RNA** and **DNA** nucleotides is presented in Figure 2.2.

**DNA** adopts a double-helix structure formed by two complementary strands, these two strands are bound together by the standard base pairings  $A - T$  and  $G - C$ . While it is common to have single-stranded **RNA**, it can also adopt a double-stranded configuration with the base pairings  $A - U$  and  $G - C$ . These base pairings could be described as a series of hydrogen bonds between two bases: the pairings  $A - T$  and  $A - U$  consist of two hydrogen bonds, while the pairing  $C - G$  consists of three. Each hydrogen bond has an energy of around  $1.5 \text{ kcal} \cdot \text{mol}^{-1}$ , and therefore the  $C - G$  pairing is observably more stable than the others.

Crucially, sequence-dependent interactions (i.e., interactions involving the nucleotides' bases) in both **RNA** and **DNA** are mediated not only through the

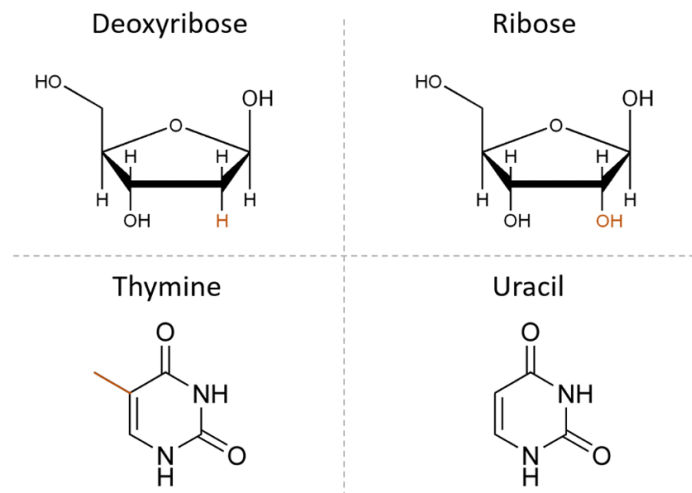


Figure 2.2: Differences between RNA (left column) and DNA (right column) nucleotides. Taken from [7, Figure 8]

hydrogen bonds in base pairings but also through the interactions between adjacent base pairs in the chain, called stacking. Please see Figure 2.3 for a visual overview on the double-helix structure, base pairings and base stacking.

Base pairing explains how **DNA** and **RNA** strands come together, but not how the nucleotides in a single strand bind one to another. The answer is straightforward: two nucleotides bind through a phosphodiester bond, which is a bond between the phosphorus in the phosphate fragment of one nucleotide and the oxygen in the hydroxyl group on the 3' position (see Figure 2.1 in the sugar fragment of the adjacent nucleotide. This induces a directionality in the strand: the 3' to 5' direction is given by reading the bases of the strand sequentially following the sense such that the phosphate group in nucleotide  $i$  is bound to the sugar of nucleotide  $i + 1$ , while the 5' to 3' direction is given by reading the bases of the strand in the opposite sense, i.e. such that the sugar group in nucleotide  $i$  is bound to the phosphate group in nucleotide  $i + 1$ . This directionality is not a mere convention, as it plays a role in the way two strands bind together. In a double-helix structure both strand have opposite directions, this is, if we read the the bases in one strand sequentially from 3' to 5' then we will be reading the bases of the complementary strand from 5' to 3'. This is represented in the top left section of 2.3.

Therefore, **RNA** and **DNA** duplexes are charged (with a negative charge on the phosphate group, see 2.1) and flexible polymers with sequence-dependent interactions. All these interactions give rise to a rich conformational landscape

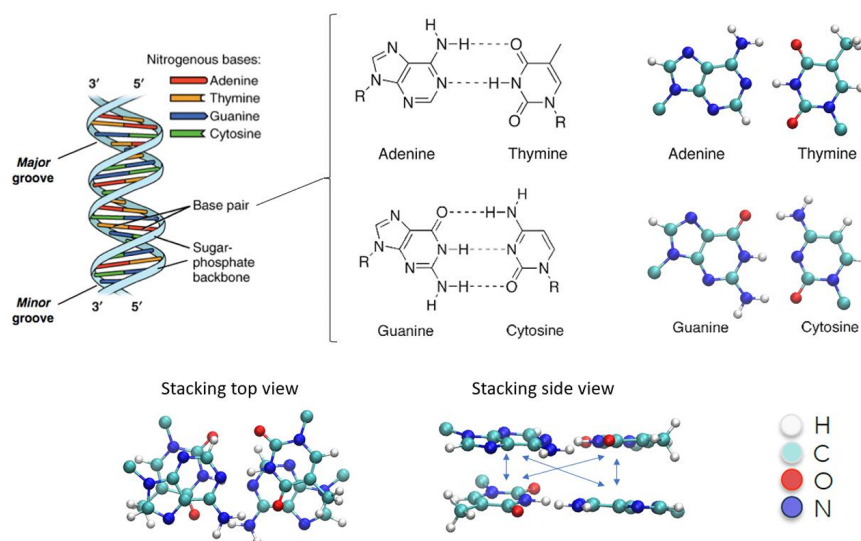


Figure 2.3: Overview of the double helix structure (top-left), hydrogen bonds in base pairing (top right) and base staking (bottom) on **DNA**. Taken from [7, Figure 2]

spanning multiple spatial and temporal scales, which poses a central challenge for atomistic simulation.

As a result, it is unsurprising that accurately capturing **RNA** and **DNA** structure is essential not only to simulate but also to understand their behaviour. Helical parameters enable a rigorous geometrical and mechanical description of the double-helix structure [10]. We will not go over their definition and computation in detail. For our purposes, it is sufficient to know that they are accurate 3D descriptors of **RNA** and **DNA** double-helix structures which have a direct impact on the properties and behaviour of the chains. For completeness, we include a visual list of helical parameters in Figure 2.4.

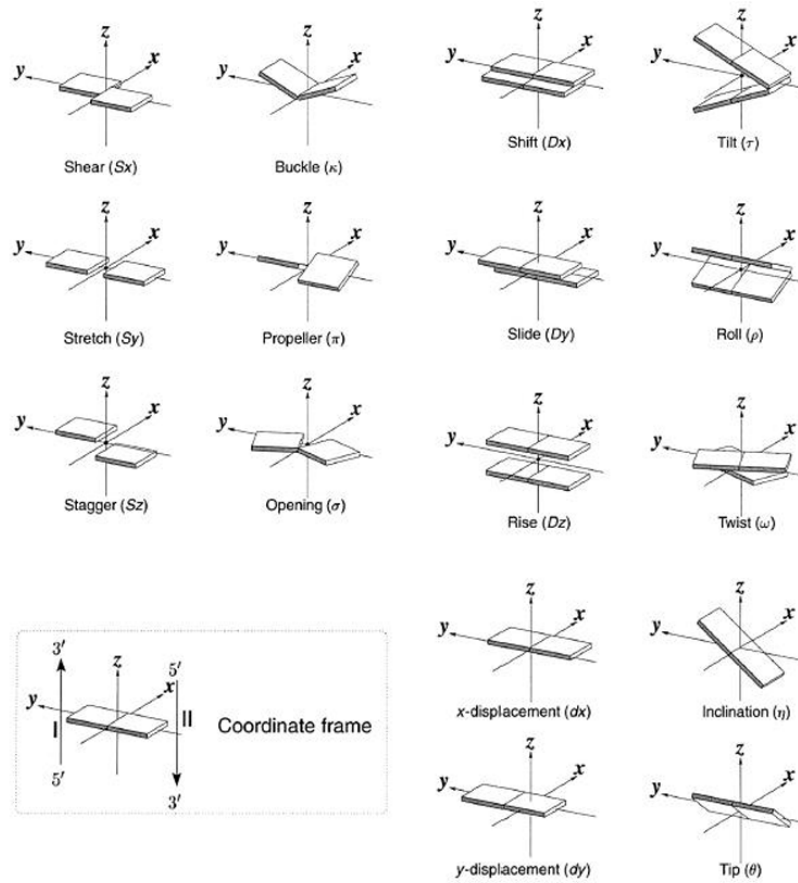


Figure 2.4: Helical parameters. Taken from [11, Figure 1]

## 2.1.2 The role of small interfering Ribonucleic Acids

In order to understand the role of **siRNA**, it is essential to look at the central dogma of biology. A schematic of the whole process is presented in Figure 2.5

The information encoded in the cell's genes (folded **DNA**) is copied into **mRNAs** (transcription step in 2.5), which is then read by ribosomes to synthesize the proteins (translation step in 2.5) through which the gene expresses itself.

As a result, any gene can theoretically be silenced by stopping the translation step of its **mRNA** (in engineering terms, by blocking the outward flow of information from the gene). This is precisely what **siRNA** is capable of achieving: the silencing of malign genes by stopping the translation of their **mRNA**.

The process through which **siRNA** silences genes by the translation of their **mRNA** is called the **RNA** interference pathway. Here is a simplified but nevertheless sufficient description of the process to understand the challenges behind **siRNA** activity prediction.

The process starts with a double-stranded **siRNA**, in which the 5' to 3' strand is the guide and the 3' to 5' strand is the passenger. The guide strand has the complementary sequence to that of a segment of the target **mRNA** to be neutralized, and therefore the passenger strand shares a segment of the sequence of the target **mRNA**. If this double stranded **siRNA** enters the cell and can avoid degradation for long enough, it will be loaded into a multi-protein complex known as **RNA-induced silencing complex (RISC)**, whose main functional component is the Argonaute enzyme [12]. Whilst bound to the enzyme, the two strands of the **siRNA** chain are dissociated: the passenger strand is discarded and degraded, while the guide strand remains bound. The guide strand then associates to the target sequence of the **mRNA** molecule via base pairing. Now, the Argonaute enzyme can cleave the **mRNA** strand, thereby preventing the translation of the targeted **mRNA** into protein.

Selecting an appropriate segment of sequence within the target **mRNA** is challenging. The selected segment could be present in other **mRNA** molecules in the cell that lead to the expression of vital proteins. The consequences of such a mistake could be fatal. This is known as off-targeting effects and it is the reason why **siRNA** design requires careful sequence optimization. The **siRNA** molecules this document deals with have been extracted from patents and publications and thus we assume that the preliminary work to measure off-target effects have been carried out as per the classic workflow in the development of **siRNA** [13].

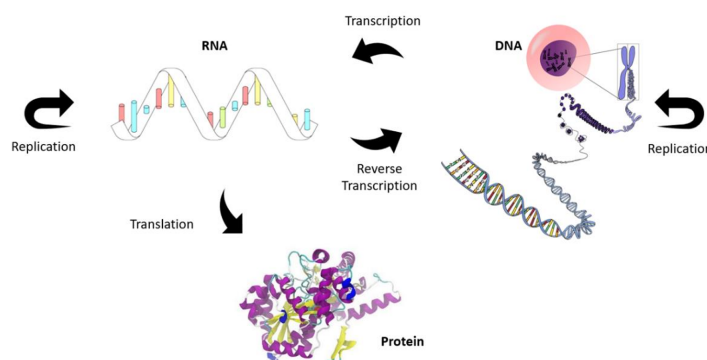


Figure 2.5: An overview of the central dogma of biology, representing information flow in eukaryote cells. Taken from [7, Figure 9]

The **RNA** interference pathway reveals a number of challenges beyond off-target minimization which underpin the purpose of this work. First, **siRNA** molecules must reach the inside of the cells. To do so, they have to traverse a collection of interfaces which we refer to as delivery barriers - the most notable being the cellular membrane. Chemists employ the use of conjugates, additional fragments bound to the siRNA duplex to facilitate the entry into the cell, these are beyond the scope of this project. The duplex must also avoid degradation once it reaches the inside of the cell, so the thermodynamic stability of the **siRNA** molecules plays a key role in this process. Finally, the crux of the pathway occurs when the guide strand of the **siRNA** molecule is loaded into the **RISC** complex and directs it towards the target sequence in the **mRNA** - meaning that the binding affinity to the RISC complex and to the target sequence are essential for the **siRNA** interference pathway's success.

The design of chemically modified **siRNAs** is the main approach to tackle all these challenges, but modifications may improve on one aspect yet deteriorate another. The resulting **siRNA** molecules must achieve stability without compromising **RISC** loading or target engagement. Additional factors include favouring guide strand incorporation, preventing passenger strand activity, and controlling duplex thermodynamics and secondary structure to maintain accessibility. In engineering terms, we may think of the design of chemically modified siRNA as a constrained optimization problem whose objective function is very difficult to evaluate unless experimentally measured. This measurement, called activity, looks at the amount of **mRNA** molecules present in a test cell before and after the **siRNA** is introduced and returns a value between 0 and 1. This value represents the percentage of target **mRNA** destroyed, with 0 meaning target **mRNA** remain unaltered by **siRNA**

introduction and 1 meaning all target mRNA present in the cell were silenced after siRNA introduction.

## 2.2 Statistical Physics and Molecular dynamics

This section reviews the background concepts in statistical physics and molecular dynamics that are used in this thesis for both the theoretical analysis of the RISC pathway and the double-stranded siRNA trimers simulation and feature extraction.

### 2.2.1 Statistical Ensembles

Statistical mechanics describes the macroscopic properties of a system as averages over the ensemble of possible configurations, i.e., as the expectancy of a probability distribution on phase space consistent with a set of thermodynamic constraints. The choice of ensemble depends on which quantities are held fixed.

The most common ensembles are the microcanonical, canonical, and isothermal-isobaric ensembles. The microcanonical ensemble describes an isolated system with fixed energy  $E$ , volume  $V$ , and number of particles  $N$ . Its probability measure is uniform over the hyper-surface of constant energy in phase space. While conceptually fundamental, the microcanonical ensemble is not practical for molecular dynamics simulations of biomolecular systems, where temperature and pressure control are often required.

The canonical ensemble describes a system in contact with a heat bath at fixed temperature  $T$ , volume  $V$ , and number of particles  $N$ . Thus, it is frequently also called the  $(N, V, T)$  ensemble. In this case, the probability density over phase space  $(q, p)$  is given by the Boltzmann distribution

$$\rho(q, p) = \frac{1}{Z} \exp(-\beta H(q, p)),$$

where  $\beta = (k_B T)^{-1}$ ,  $Z = \int \exp(-\beta H(q, p)) dq dp$  is called the partition function and  $H$  is the Hamiltonian. Given a system with  $N$  particles, the (classical) Hamiltonian is a function  $H : \mathbb{R}^{3N} \times \mathbb{R}^{3N} \rightarrow \mathbb{R}$  that rules the

evolution of the system through the system of ordinary differential equations

$$\begin{cases} \frac{dp}{dt} = -\frac{\partial H}{\partial q} \\ \frac{dq}{dt} = \frac{\partial H}{\partial p} \end{cases} \quad (2.1)$$

where  $q \in \mathbb{R}^{3N}$  are the positions of the particles and  $p \in \mathbb{R}^{3N}$  are the momenta of the particles.

In the canonical ensemble, observable averages are then computed as expectations with respect to the probability distribution  $\rho(q, p)$ .

Finally, the isothermal-isobaric ensemble extends the canonical ensemble by allowing volume fluctuations at fixed pressure  $P$ , leading to the  $(N, P, T)$  ensemble. This ensemble is often used for solvated biomolecular systems, since maintaining pressure near ambient conditions allows the system density and box volume to relax consistently with the surrounding environment. For this reason, both the canonical and isothermal-isobaric ensembles are relevant when simulating biomolecules under conditions intended to resemble physiological ones.

We must also mention the grand canonical (or macrocanonical) ensemble, used to describe systems in chemical equilibrium with a particle reservoir. However, its conditions are not relevant to the present work.

## 2.2.2 Molecular Dynamics Simulations

Molecular dynamics simulates the time evolution of a molecular system by numerically integrating the equations of motion (2.1) associated with a Hamiltonian. In the context of this work, the Hamiltonian  $H : \mathbb{R}^{3N} \times \mathbb{R}^{3N} \rightarrow \mathbb{R}$  always has the form

$$H = \frac{1}{2}p^t M^{-1} p + U(q; \lambda)$$

where:

- $M$  is a diagonal matrix whose  $i$ -th diagonal entry is the mass of the  $i$ -th particle in the system. It is usually referred to as the mass matrix.
- $U(q; \lambda)$  is a potential function, and  $\lambda \in \mathbb{R}^p$  is a parameter vector. Parametrising a certain system (or set of systems) corresponds to choosing the appropriate parameter vector to replicate their behaviour. The potential functions  $U(q; \lambda)$  we will use in this manuscript remain continuously differentiable on  $q$  as long as particles do not occupy the

same position, which we can safely assume will not happen thanks to strong repulsion forces when their positions grow too close. As a result, we can also safely assume our Hamiltonian is sufficiently regular for our purposes.

In the absence of any additional coupling, this leads to Hamiltonian dynamics and corresponds to the microcanonical setting.

In practice, biomolecular simulations are usually carried out with algorithms that regulate thermodynamic variables such as temperature and pressure, so as to sample the ensemble of interest. Temperature control is achieved through a thermostat, which mimics the effect of contact with a heat bath and allows sampling of the NVT and NPT ensembles. Likewise, pressure control is achieved through a barostat, which allows the simulation box volume to fluctuate and enables sampling of the isothermal-isobaric (NPT) ensemble. These couplings are essential in solvated systems, where both thermal fluctuations and density relaxation play an important role in reproducing realistic conditions.

Time integration in classical MD is performed with finite time-step numerical schemes such as the Verlet family of integrators. In this work we employ a standard leap-frog-type integration scheme. For the purposes of this thesis, it is enough to note that these methods update positions and momenta approximately over small time intervals and provide stable trajectories when an appropriately small time step is chosen.

In practice, molecular dynamics simulations are run in a finite size box with **Periodic Boundary Conditions (PBCs)** to mimic a bulk system in which the space is tiled with copies of the simulation box. This reduces finite-size and surface effects. Interactions are computed using the minimum image convention, which ensures that each particle interacts only with the closest periodic image of every other particle. Long-range electrostatic interactions, which are particularly relevant for nucleic acids such as **siRNA**, are typically treated using Ewald-based methods, such as Particle Mesh Ewald (PME), which allow for efficient and accurate computation in periodic systems.

### 2.2.3 Statistical Considerations for Molecular Dynamics Observables

We are interested in the equilibrium distribution of certain functions of the configuration or of the trajectory of the system which we refer to collectively as observables.

In its simplest form, a configuration-level observable is a function  $A : \mathbb{R}^{3N} \rightarrow \mathbb{R}$  that assigns a value  $A(q)$  to each of the possible position configurations of a system.

More generally, however, we are also interested in observables that depend on the trajectory as a whole rather than a single frame. Formally, given the set of all possible finite-sample trajectories from a Hamiltonian system

$$\mathcal{T} = \{(q(t_1), \dots, q(t_T)) \mid T \in \mathbb{N}_{>0}, t_1 < \dots < t_T \in \mathbb{R}\}$$

where, abusing the notation, we can write any single trajectory as a function  $q : \mathbb{R} \rightarrow \mathbb{R}^{3N}$ , we also use the term observable to refer to functions

$$A : \mathcal{T} \rightarrow \mathbb{R}.$$

The **Root Mean Square Fluctuation (RMSF)**, discussed in Section 4.4.4, is an example of a trajectory-based observable.

We use a number of statistical summaries to characterise the equilibrium distribution of the observables of interest. We will refer to these statistical summaries as features. Therefore, more formally, features are estimators computed from finite-sample trajectories. For example, given an observable  $A$ , its empirical mean over a finite-sample trajectory

$$\langle A \rangle_t := \frac{1}{T} \sum_{i=1}^T A(q(t_i)) \quad (2.2)$$

is a feature of the observable  $A$ .

In this manuscript, the term features often overrules the term observables. For example, hydrogen bond features refers to the hydrogen bond observable's features.

Molecular dynamics simulations with appropriate thermostat and barostat couplings are designed to sample the equilibrium distribution associated with the chosen ensemble. While well-studied for Langevin dynamics, this is a delicate topic when dealing with other thermostats and barostats (for further discussion on this, please see A.1). In principle, this allows the computation of expectation values of any configuration-based observable  $A : \mathbb{R}^{3N} \rightarrow \mathbb{R}$  as

$$\mathbb{E}[A] \approx \frac{1}{T} \sum_{i=1}^T A(q(t_i)) =: \langle A \rangle_t$$

where  $(q(t_1), \dots, q(t_T))$  are the position configurations sampled along the

trajectory after equilibration.

However, consecutive samples obtained from a molecular dynamics trajectory are generally correlated. This correlation affects the estimation of statistical uncertainties, as standard estimators assume independent samples. A common approach to address this issue is to estimate the integrated autocorrelation time and construct statistically independent samples via subsampling or block averaging [14, 15].

In the present work, uncertainty quantification is not performed from a single long trajectory. Instead, we generate multiple independent simulations (replicas) for each system, each initialized with different initial conditions. This allows us to treat the resulting observables as approximately independent samples, and thus directly estimate statistical quantities across replicas.

In the simplest setting, assume  $n_{\text{rep}}$  independent simulations of equal length  $T$ , and let  $A_r := \langle A \rangle_{t,r}$  denote the empirical mean of the observable  $A$  computed within run  $r \in \{1, \dots, n_{\text{rep}}\}$ , as defined earlier in (2.2). Treating the per-run means  $A_1, \dots, A_{n_{\text{rep}}}$  as approximately independent samples, one may compute their empirical mean and standard deviation as

$$\bar{A} = \frac{1}{n_{\text{rep}}} \sum_{r=1}^{n_{\text{rep}}} A_r,$$

$$\sigma_A^2 = \frac{1}{n_{\text{rep}} - 1} \sum_{r=1}^{n_{\text{rep}}} (A_r - \bar{A})^2,$$

and use these as features summarising the equilibrium distribution of  $A$ . This approach avoids the need for explicit estimation of autocorrelation times, while still providing a robust estimate of variability, provided that the simulations are sufficiently decorrelated through their initialization and equilibration procedures.

The specific aggregation formulas used in this work, which account for unequal frame retention across runs and for observables that are angular or distributional rather than scalar, are introduced in Section 4.4.5.

## 2.2.4 Existing methods

Let us briefly introduce the existing work and software that will enable our MD simulations. In particular, we will introduce GROMACS, Amber and modXNA.

#### 2.2.4.1 GROMACS

GROMACS is an open-source molecular dynamics engine designed for high-performance simulation of biomolecular systems [16], [17]. It provides efficient implementations of time integration schemes, constraint algorithms, and long-range interaction methods such as Particle Mesh Ewald. In this work, GROMACS is used as the simulation back-end, i.e., to perform the numerical integration of the equations of motion and generate trajectories from the parametrized systems; and also for trajectory post-processing and MD features extraction. Its efficiency and extensive validation make it a standard tool for large-scale MD simulations.

#### 2.2.4.2 AMBER force fields

The AMBER family of empirical force fields is widely used for the simulation of biomolecules, particularly proteins and nucleic acids [18], [19]. These force fields define the potential energy function  $U(q)$  appearing in the Hamiltonian, typically decomposed into bonded terms (bonds, angles, dihedrals) and non-bonded terms (van der Waals and electrostatic interactions). In the context of nucleic acids, AMBER force fields have been extensively calibrated to reproduce structural and thermodynamic properties. In this project, AMBER-compatible parameters are used to describe both canonical and modified nucleotides, where the latter require the use of the additional software provided by modXNA.

#### 2.2.4.3 ModXNA

modXNA is a framework designed to extend classical nucleic acid force fields to chemically modified systems by providing parameter sets and tools compatible with the AMBER ecosystem [4]. Its main purpose is to enable the simulation of non-canonical nucleotides - such as the modified nucleotides employed in siRNA design- which are not covered by standard force fields such as AMBER.

In practice, modXNA provides a library of parametrised modified residues together with utilities to construct the corresponding molecular topology files in a format that is amenable for integration with the AMBER format. This includes the definition of bonded interactions, partial charges, and atom types consistent with the underlying AMBER force field.

In this work, modXNA plays a central role, as it is used for the parametrization of the modified siRNA trimers to be simulated. The resulting

topologies are then exported to GROMACS using ParmEd [20], where the actual molecular dynamics simulations are performed. This workflow allows us to leverage established force field methodologies while extending them to the modified systems of interest and will be covered in detail in Section 4.3.

## 2.3 Graph Theory

Graphs are mathematical objects that enable the modelling of systems consisting of a collection of entities (nodes) and the relationships between them (edges). This versatility has boosted their widespread adoption in a variety of fields ranging from network science to molecular biology. Despite this apparent simplicity, graphs are remarkably difficult to study. Here, we introduce some of the key concepts in graph theory that will later prove essential to understanding elements of this thesis. First, let us give a mathematical definition of the concept of graph:

**Definition 1** (Undirected Graph). *An undirected graph  $G$  is a tuple  $(\mathcal{V}, \mathcal{E})$ , where  $\mathcal{V}$  is a set of nodes and  $\mathcal{E} \subseteq \{(u, v) \mid u, v \in \mathcal{V}\}$  is a set of edges. An edge defines an adjacency relationship between the two nodes that form it.*

Note that edges in an undirected graph do not have an intrinsic direction, that is, they allow the symmetric, bi-directional flow of information between the two nodes they connect. However, sometimes we will be interested in differentiating between the influence a node  $u$  has on a node  $v$  and vice-versa. Directed graphs enable this distinction:

**Definition 2** (Directed Graph). *A directed graph  $G$  is a tuple  $(\mathcal{V}, \mathcal{E})$ , where  $\mathcal{V}$  is defined as in an undirected graph and  $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$  is a set of directed edges. Here, each directed edge is an ordered pair  $(u \rightarrow v)$ . A directed edge defines a directed adjacency relationship from the first node to the second node.*

It is interesting to observe that nodes and edges have roles that are somewhat interchangeable: if we think of each edge as a "node", then the nodes it connects can be seen as defining adjacencies between the edges that contain that node. This is called the line graph:

**Definition 3** (Line Graph). *Let  $G = (\mathcal{V}, \mathcal{E})$ .*

*If  $G$  is undirected, its line graph  $G^* = (\mathcal{V}^*, \mathcal{E}^*)$  is the undirected graph defined by  $\mathcal{V}^* = \mathcal{E}$  and*

$$(e_1, e_2) \in \mathcal{E}^* \iff e_1 \neq e_2 \text{ and } e_1 \cap e_2 \neq \emptyset.$$

If  $G$  is directed, its line graph  $G^* = (\mathcal{U}^*, \mathcal{E}^*)$  is the directed graph defined by  $\mathcal{U}^* = \mathcal{E}$  and

$$(e_1 \rightarrow e_2) \in \mathcal{E}^* \iff e_1 = (u \rightarrow v), e_2 = (v \rightarrow w)$$

for some  $u, v, w \in \mathcal{U}$ .

Observe also that the structure of both directed and undirected graphs is completely invariant under relabelling of the nodes. Formally, we say graphs that only differ in the choice of node labels are isomorphic:

**Definition 4** (Graph Isomorphism). Two graphs  $G_1 = (\mathcal{U}_1, \mathcal{E}_1)$  and  $G_2 = (\mathcal{U}_2, \mathcal{E}_2)$  are isomorphic,  $G_1 \cong G_2$ , if and only if there exists a bijection  $\phi : \mathcal{U}_1 \rightarrow \mathcal{U}_2$  such that

$$(u, v) \in \mathcal{E}_1 \iff (\phi(u), \phi(v)) \in \mathcal{E}_2$$

in the undirected case, and

$$(u \rightarrow v) \in \mathcal{E}_1 \iff (\phi(u) \rightarrow \phi(v)) \in \mathcal{E}_2$$

in the directed case.

Note that we can use this relationship to identify smaller graphs contained within a larger one.

**Definition 5** (Subgraph).  $G_2 = (\mathcal{U}_2, \mathcal{E}_2)$  is a subgraph of  $G_1 = (\mathcal{U}_1, \mathcal{E}_1)$ , denoted  $G_2 \subseteq G_1$ , if and only if there exists a subset  $\mathcal{U}_H \subseteq \mathcal{U}_1$  such that the graph  $H = (\mathcal{U}_H, \mathcal{E}_H)$  with

$$\mathcal{E}_H = \{(u, v) \in \mathcal{E}_1 \mid u, v \in \mathcal{U}_H\}$$

in the undirected case, or

$$\mathcal{E}_H = \{(u \rightarrow v) \in \mathcal{E}_1 \mid u, v \in \mathcal{U}_H\}$$

in the directed case, is isomorphic to  $G_2$ , i.e.  $H \cong G_2$ .

Now that we have introduced the basic graph structure, let us build on the concept of node adjacency. For most of the rest of this section, we will work with undirected graphs only.

First, we introduce the concept of neighbourhoods:

**Definition 6** (Graph Neighbourhood). *Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$ , the neighbourhood of a node  $u \in \mathcal{V}$  is the set of nodes that share an edge with  $u$ , i.e.,*

$$N(u) = \{v \in \mathcal{V} \mid (u, v) \in \mathcal{E}\}.$$

Next, we can introduce the concepts of degree, degree matrix, adjacency matrix and normalized adjacency matrix:

**Definition 7** (Degree). *Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$ , the degree of a node  $u \in \mathcal{V}$  is  $d_u = |N(u)|$ . We denote the maximum degree of a graph  $G$  by  $\Delta(G) := \max_{u \in \mathcal{V}} d_u$ .*

**Definition 8** (Degree Matrix). *Given a graph  $G = (\mathcal{V}, \mathcal{E})$ , its degree matrix  $D \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$  is the diagonal matrix such that  $[D]_{u,u} = d_u$ .*

**Definition 9** (Adjacency Matrix). *Given a graph  $G = (\mathcal{V}, \mathcal{E})$ , its adjacency matrix  $A \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$  is defined as*

$$[A]_{u,v} = \begin{cases} 1 & \text{if } (u, v) \in \mathcal{E} \text{ or } (u \rightarrow v) \in \mathcal{E}, \\ 0 & \text{otherwise.} \end{cases}$$

**Definition 10** (Normalized Adjacency Matrix). *Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$  with no isolated nodes (i.e.,  $d_u > 0 \forall u \in \mathcal{V}$ ), its normalized adjacency matrix  $\hat{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$  is defined as  $\hat{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$ .*

**Lemma 11** (Spectral radius of normalized adjacency matrix). *If an undirected graph  $G = (\mathcal{V}, \mathcal{E})$  satisfies  $d_v > 0 \forall v \in \mathcal{V}$ , then its normalized adjacency matrix  $\hat{A}$  satisfies  $\rho(\hat{A}) = 1$ .*

*Proof.* Observe that  $D^{-1}$  is well defined and all rows of  $D^{-1}A$  sum to one, and thus  $\|D^{-1}A\|_\infty = 1$ . For any matrix norm consistent with a vector norm we have  $\rho(M) \leq \|M\|$  and thus  $\rho(D^{-1}A) \leq 1$ . Note that

$$\hat{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}} = D^{\frac{1}{2}} (D^{-1}A) D^{-\frac{1}{2}},$$

and thus  $\hat{A}$  and  $D^{-1}A$  share eigenvalues, which in particular means that  $\rho(\hat{A}) = \rho(D^{-1}A) \leq 1$ .

Further, taking  $v = D^{\frac{1}{2}} \mathbf{1}$  we see that

$$\hat{A}v = D^{-\frac{1}{2}} A D^{-\frac{1}{2}} D^{\frac{1}{2}} \mathbf{1} = D^{-\frac{1}{2}} A \mathbf{1} = D^{-\frac{1}{2}} D \mathbf{1} = D^{\frac{1}{2}} \mathbf{1} = v,$$

so  $\rho(\hat{A}) \geq 1$ .

We conclude that  $\rho(\hat{A}) = 1$ . □

Given a graph  $G = (\mathcal{V}, \mathcal{E})$ , we will sometimes refer to its extended normalized adjacency matrix as the normalized adjacency matrix of the graph  $G$  with self-loops, i.e., with an edge from each node to itself.

Intuitively, it is clear that we can travel from one node to another if there exists a sequence of intermediate neighbouring nodes between them. This can be formalised through the concepts of walks, paths and cycles in a graph:

**Definition 12** (Graph Walk). *Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$ , a walk of length  $l$  is a sequence of nodes  $(w_0, w_1, \dots, w_l)$  such that  $w_i \in \mathcal{V}$  for all  $i \in \{0, \dots, l\}$ ,  $(w_i, w_{i+1}) \in \mathcal{E}$  for all  $i \in \{0, \dots, l-1\}$ .*

Note that this definition allows the existence of walks that visit the same node twice. However, for any such walk, we can intuitively see that there must exist a shorter walk between its beginning and end nodes without repeating any nodes. Such a walk is called a path:

**Definition 13** (Graph Path and Cycle). *Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$ , a path  $p$  of length  $l$  is a walk  $(w_0, w_1, \dots, w_l)$  such that  $w_i \neq w_j$  for all distinct  $i, j \in \{0, \dots, l\}$ .*

*A cycle is a walk  $(w_0, w_1, \dots, w_{l-1}, w_l)$  with  $l \geq 1$  such that  $w_0 = w_l$  and  $w_0, w_1, \dots, w_{l-1}$  are pairwise distinct.*

Next, we rigorously prove our intuition that from any walk with repeated nodes we can extract a path:

**Lemma 14** (Path from walk with repeated nodes). *Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$  and a walk  $(u = w_0, w_1, \dots, w_l = v)$  with  $u \neq v$  and with at least one repeated node, there always exists a path  $(u = \tilde{w}_0, \dots, \tilde{w}_{\tilde{l}} = v)$  with  $\tilde{l} < l$ .*

*Proof.* Let  $(u = w_0, w_1, \dots, w_l = v)$  be a walk with at least one repeated node. Then there exist indices  $i < j$  such that  $w_i = w_j$ . Choose such a pair with  $j - i$  minimal. By minimality, there are no indices  $k$  such that  $i < k < j$  and  $w_k = w_i$ .

Consider the sequence  $(u = w_0, \dots, w_i, w_{j+1}, \dots, w_l = v)$  if  $j \neq l$ , or  $(u = w_0, \dots, w_i = v)$  if  $j = l$ .

We claim that this is still a walk. Indeed, all consecutive pairs before  $w_i$  and after  $w_{j+1}$  are unchanged, so they remain edges of  $G$ . This immediately proves our claim for the case  $j = l$ . If  $j \neq l$ , note that the only new consecutive pair is  $(w_i, w_{j+1})$  when  $j < l$ . Since the original sequence was a walk, we have  $(w_j, w_{j+1}) \in \mathcal{E}$ . But  $w_i = w_j$ , hence also  $(w_i, w_{j+1}) \in \mathcal{E}$ . Therefore the new sequence is a walk from  $u$  to  $v$  with length  $l' = l - (j - i) < l$  since  $j > i$ .

If this new walk still has repeated nodes, we repeat the same procedure. At each step the length decreases strictly, and since the length is a non-negative integer, after finitely many steps the process must terminate. The resulting walk has no repeated nodes, and is therefore a path. Since the length strictly decreases at each step, the final path  $(u = \tilde{w}_0, \dots, \tilde{w}_{\tilde{l}} = v)$  satisfies  $\tilde{l} < l$ .  $\square$

Observe that a path of length 0 is just a node. As a consequence of Lemma 14, any walk  $(w_0, w_1, \dots, w_l)$  with  $w_0 \neq w_l$  is either a path (if it has no repeated nodes) or can be reduced to a path of length  $l' < l$  (if it has repeated nodes). As a result, we can define distances in a graph using only the concept of paths:

**Definition 15** (Graph Distance). *Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$  and two nodes  $u, v \in \mathcal{V}$ , we define the distance between  $u$  and  $v$  as*

$$d(u, v) = \begin{cases} \min\{\text{length}(p) \mid p = (u, \dots, v) \text{ is a path in } G\} & \text{if there exists a path} \\ & \text{between } u \text{ and } v, \\ \infty & \text{otherwise.} \end{cases}$$

For the graph distance introduced in Definition 15 to be a distance in the mathematical sense, it needs to satisfy the usual properties of a distance metric, i.e. it needs to be symmetric, positive semi-definite (with 0 attained if and only if the input elements are identical) and satisfy the triangle inequality.

**Lemma 16** (Undirected Graph Distance is a Distance). *Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$ , the function  $d : \mathcal{V} \times \mathcal{V} \rightarrow \mathbb{N} \cup \{\infty\}$  defined in Definition 15 is a distance.*

*Proof.* First, observe that  $d(u, u) = 0 \forall u \in \mathcal{V}$  and  $d(u, v) > 0 \forall u \neq v \in \mathcal{V}$  because paths can only have length equal or greater than 0, and to go from any node to a distinct node we must take at least one step.

Second, note that in an undirected graph  $d(u, v) = d(v, u)$  because every path  $p = (u, \dots, v)$  can be traversed in the reverse direction  $\bar{p} = (v, \dots, u)$  and the length stays the same.

Finally, let  $u, v, w \in \mathcal{V}$ . If either  $d(u, w) = \infty$  or  $d(w, v) = \infty$ , then

$$d(u, v) \leq d(u, w) + d(w, v)$$

is immediate, since the right-hand side is  $\infty$ . This result is also immediate if  $d(u, v) = 0$  (i.e.,  $u = v$ ) as we have already seen that  $d(u, w) \geq 0$  and  $d(w, v) \geq 0$ .

Assume now that both  $d(u, w)$  and  $d(w, v)$  are finite and  $u \neq v$ . Then there exists a path  $p_1$  from  $u$  to  $w$  of length  $d(u, w)$  and a path  $p_2$  from  $w$  to  $v$  of length  $d(w, v)$ . Concatenating  $p_1$  and  $p_2$  we obtain a walk  $q = (u, \dots, w, \dots, v)$  of length  $d(u, w) + d(w, v)$ . There are now two options:

- The walk  $q$  has no repeated nodes and is therefore a path  $p_3 = q$ . Then, it holds that

$$\begin{aligned} d(u, v) &= \min\{\text{length}(p) \mid p = (u, \dots, v) \text{ is a path in } G\} \\ &\leq \text{length}(p_3) = d(u, w) + d(w, v) \end{aligned}$$

- The walk  $q$  has repeated nodes and therefore, by Lemma 14, we can extract a path  $p_4$  from  $u$  to  $v$  such that  $\text{length}(p_4) < \text{length}(q) = d(u, w) + d(w, v)$ . As a result, it holds that:

$$\begin{aligned} d(u, v) &= \min\{\text{length}(p) \mid p = (u, \dots, v) \text{ is a path in } G\} \\ &\leq \text{length}(p_4) < d(u, w) + d(w, v) \end{aligned}$$

We conclude that  $d(u, v) \leq d(u, w) + d(w, v) \forall u, v, w \in \mathcal{V}$ . □

Now that we have defined a distance, we will sometimes talk about geodesic paths, which are paths of length equal to the geodesic distance between two nodes:

**Definition 17** (Graph Geodesic Path). *Given  $G = (\mathcal{V}, \mathcal{E})$ , a path  $p$  between two nodes  $u, v \in \mathcal{V}$  is called a geodesic path if  $\text{length}(p) = d(u, v)$ .*

Given a distance, we can introduce the concept of balls and spheres in the graph, which will prove very useful:

**Definition 18** (Graph Ball and Sphere). *Given  $G = (\mathcal{V}, \mathcal{E})$ , the ball of radius  $r \in \mathbb{N}_{\geq 0}$  centred at the node  $u \in \mathcal{V}$  is defined as the set of nodes that lie within distance  $r$  of the node  $u$ , that is,*

$$B_r(u) = \{v \in \mathcal{V} \mid d(u, v) \leq r\}.$$

*The sphere of radius  $r$  centred at the node  $u$  is the set of nodes that lie at distance exactly  $r$  from the node  $u$ , that is,*

$$S_r(u) = \{v \in \mathcal{V} \mid d(u, v) = r\}.$$

In this thesis, we consider balls as graphs contained within a larger graph. Let us first define the notion of containment rigorously:

**Definition 19** (Containment).  $G_2 = (\mathcal{V}_2, \mathcal{E}_2)$  is contained in  $G_1 = (\mathcal{V}_1, \mathcal{E}_1)$ ,  $G_2 \preceq G_1$ , if and only if  $\mathcal{V}_2 \subseteq \mathcal{V}_1$  and  $\mathcal{E}_2 \subseteq \mathcal{E}_1$ .

Thinking about graph balls as graphs contained in a larger graph is convenient because it allows us to reason about how a given graph "looks like" locally around a certain node.

**Definition 20** (Ball as graph). Given  $G = (\mathcal{V}, \mathcal{E})$ , the ball of radius  $r \in \mathbb{N}_{\geq 0}$  centred at the node  $u \in \mathcal{V}$  is a graph  $\mathcal{B}_r(u) \preceq G$  with nodes  $\mathcal{V}(\mathcal{B}_r(u)) = \mathcal{B}_r(u) = \{v \in \mathcal{V} \mid d(u, v) \leq r\}$  and edges  $\mathcal{E}(\mathcal{B}_r(u)) = \{(v, w) \in \mathcal{E} \mid v, w \in \mathcal{B}_r(u), \min\{d(u, v), d(u, w)\} < r\}$

Observe that this definition ensures that, given two nodes  $v, w \in \mathcal{B}_r(u)$  such that  $(v, w) \in \mathcal{E}$ ,  $(v, w) \notin \mathcal{E}(\mathcal{B}_r(u))$ . This criteria is appropriate for reasoning about information propagation in graphs: if nodes  $v$  and  $w$  each have a message and transmit it to its neighbours, who iteratively transmit it to their neighbours, after  $r$  steps of this message-passing procedure the message has arrived to  $u$ , but the message  $u$  receives from  $v$  has not gone through  $w$  and the message  $u$  receives from  $w$  has not gone through  $v$ . If we now imagine each node slightly altering the message before transmitting it again, it is clear that the  $(v, w)$  edge has played no role in the messages arriving to  $u$  after  $r$  steps.

Let us prove a simple but crucial property for graph balls:

**Lemma 21** (Distance equality for graph balls). Let  $G = (\mathcal{V}, \mathcal{E})$ , and consider the ball of radius  $r \in \mathbb{N}_{\geq 0}$  centred at the node  $u \in \mathcal{V}$ ,  $\mathcal{B}_r(u) \preceq G$ .

Then,

$$d(u, a) = d^{\mathcal{B}_r(u)}(u, a) \quad \forall a \in \mathcal{B}_r(u),$$

where  $d^{\mathcal{B}_r(u)}(\cdot, \cdot)$  denotes distances in the graph  $\mathcal{B}_r(u)$ .

*Proof.* It is clear that  $d(u, a) \leq d^{\mathcal{B}_r(u)}(u, a)$  because  $\mathcal{B}_r(u) \preceq G$  and hence any edge in  $\mathcal{E}(\mathcal{B}_r(u))$  is also an edge in  $\mathcal{E}$ .

Consider now a node  $a \in \mathcal{B}_r(u)$  such that  $d(u, a) = l \leq r$  and take a geodesic path  $p = (u = w_0, \dots, w_l = a)$  in  $G$ . Then, for every  $i \in \{0, \dots, l-1\}$ , it holds that

- $d(w_i, u) = i < l \leq r \implies w_i \in \mathcal{B}_r(u)$
- $d(w_{i+1}, u) = i+1 \leq l \leq r \implies w_{i+1} \in \mathcal{B}_r(u)$

$$\bullet \min\{d(u, w_i), d(u, w_{i+1})\} = \min\{i, i+1\} = i < l \leq r$$

which implies  $(w_i, w_{i+1}) \in \mathcal{E}(\mathcal{B}_r(u))$ . This means that  $p$  is included in  $\mathcal{B}_r(u)$ , and therefore  $d^{\mathcal{B}_r(u)}(u, a) \leq l(p) = d(u, a)$ .

Putting both inequalities together, we have  $d(u, a) = d^{\mathcal{B}_r(u)}(u, a)$  for all  $a \in \mathcal{B}_r(u)$ .  $\square$

We may extend this idea to think of unions of balls as a graph:

**Definition 22** (Union of graph balls as graph). *Given  $G = (\mathcal{V}, \mathcal{E})$  and a set of nodes  $W \subseteq \mathcal{V}$ , the union of balls of radiuses  $\{r_w \geq 0\}_{w \in W}$  centered at the nodes  $w \in W$  is a graph  $\mathcal{B}(\{(w, r_w)\}_{w \in W}) \preceq G$  with nodes*

$$\mathcal{V}(\mathcal{B}(\{(w, r_w)\}_{w \in W})) = \bigcup_{w \in W} B_{r_w}(w)$$

and edges

$$\begin{aligned} \mathcal{E}(\mathcal{B}(\{(w, r_w)\}_{w \in W})) = \\ \{(a, b) \in \mathcal{E} \mid \exists w \in W \text{ s.t. } \min\{d(w, a), d(w, b)\} < r_w\} \end{aligned}$$

We are now ready to introduce two of the most well-known, widely used algorithms in graph theory: **Breadth First Search (BFS)** and **Depth First Search (DFS)**.

**2.3.0.0.1 BFS** **BFS** explores a graph layer by layer starting from a source node. Given an undirected graph  $G = (\mathcal{V}, \mathcal{E})$  and a source node  $u \in \mathcal{V}$ , one may describe its action through a sequence of sets  $(L_0, L_1, \dots)$  such that

$$L_0 = \{u\}, \quad L_{k+1} = \bigcup_{v \in L_k} N(v) \setminus \bigcup_{i=0}^k L_i.$$

That is, at each step  $k$ , the algorithm visits all previously unvisited neighbours of the nodes in  $L_k$ . Typically, the process terminates when  $L_k = \emptyset$  for some  $k$ .

Observe that **BFS** induces a partition of the connected component of  $u$  into layers corresponding exactly to the spheres defined above, i.e.  $L_k = S_k(u)$  for all  $k \in \mathbb{N}_{\geq 0}$ . In particular, **BFS** truncated at layer  $K$  provides an efficient procedure to compute the distance  $d(u, v)$  for all nodes  $v$  reachable from  $u$  that satisfy  $d(u, v) \leq K$ , and can also provide geodesic paths to any such node.

**2.3.0.0.2 DFS** **DFS**, in contrast, explores the graph by following paths as deeply as possible before backtracking. Starting from a source node  $u$ , it repeatedly visits an unvisited neighbour of the current node whenever one exists, and otherwise returns to the most recent node that still has unvisited neighbours. The process terminates when all nodes reachable from  $u$  have been visited.

In contrast to **BFS**, **DFS** does not explore the graph in order of increasing distance from the source node. Instead, it prioritises depth over breadth, traversing paths as far as possible before backtracking. As a consequence, **DFS** is particularly well suited for tasks that rely on the global structure of paths in the graph, such as detecting cycles or constructing spanning trees, whereas **BFS** is naturally adapted to problems involving distances and neighbourhood structure.

### 2.3.1 Molecular Graphs and RDKit

Molecular structures can be represented as graphs, where atoms correspond to nodes and chemical bonds to edges.

In many applications, and in particular in machine learning, one equips the nodes and edges of a graph with vector representations encoding relevant information.

**Definition 23** (Graph with Node and Edge Features). *A graph with node and edge features is a graph  $G = (\mathcal{V}, \mathcal{E})$  where each node  $v \in \mathcal{V}$  has an associated feature vector  $x_v \in \mathbb{R}^{d_{\text{nodes}}}$  and each edge  $e \in \mathcal{E}$  has a associated feature vector  $y_e \in \mathbb{R}^{d_{\text{edges}}}$ .*

For molecular graphs, these feature maps typically encode atom and bond properties. Another possible approach for molecular representation is to build a molecular graph and draw edges between those atoms whose 3D positions are within a certain radius of one another. However, a single 3D representation is often-times not representative enough when dealing with flexible, dynamical molecules [21].

Given this new object, we now need to extend our previous definition of containment to the case of graph with features by requiring that features are preserved:

**Definition 24** (Containment for graphs with features). *The graph  $G_2 = (\mathcal{V}_2, \mathcal{E}_2)$  with features  $\{x_v^2\}_{v \in \mathcal{V}_2}$  and  $\{y_e^2\}_{e \in \mathcal{E}_2}$  is contained in the graph  $G_1 = (\mathcal{V}_1, \mathcal{E}_1)$  with features  $\{x_v^1\}_{v \in \mathcal{V}_1}$  and  $\{y_e^1\}_{e \in \mathcal{E}_1}$ ,  $G_2 \preceq G_1$ , if and only if*

- $\mathcal{V}_2 \subseteq \mathcal{V}_1$  and  $x_v^2 = x_v^1$  for all  $v \in \mathcal{V}_2$ .
- $\mathcal{E}_2 \subseteq \mathcal{E}_1$  and  $y_e^2 = y_e^1$  for all  $e \in \mathcal{E}_2$ .

The concept of isomorphism also needs to be extended. For two graphs with features to be isomorphic, an isomorphism between the two underlying graph objects is not enough: features should also match under the isomorphism.

**Definition 25** (Graph with features (rooted) Isomorphism). *Two graphs with features,  $G_1 = (\mathcal{V}_1, \mathcal{E}_1)$  with features  $\{x_v^1\}_{v \in \mathcal{V}_1}$  and  $\{y_e^1\}_{e \in \mathcal{E}_1}$  and  $G_2 = (\mathcal{V}_2, \mathcal{E}_2)$  with features  $\{x_v^2\}_{v \in \mathcal{V}_2}$  and  $\{y_e^2\}_{e \in \mathcal{E}_2}$ , are isomorphic,  $G_1 \cong G_2$ , if and only if there exists a bijection  $\phi : \mathcal{V}_1 \rightarrow \mathcal{V}_2$  such that*

- $(u, v) \in \mathcal{E}_1 \iff (\phi(u), \phi(v)) \in \mathcal{E}_2$
- $x_u^1 = x_{\phi(u)}^2$  for all  $u \in \mathcal{V}_1$
- $y_{(u,v)}^1 = y_{(\phi(u), \phi(v))}^2$  for all  $(u, v) \in \mathcal{E}_1$

*Given  $W_1 \subseteq \mathcal{V}_1$  and  $W_2 \subseteq \mathcal{V}_2$ , there is a rooted isomorphism between the rooted graphs with features  $(G_1, W_1)$  and  $(G_2, W_2)$ ,  $(G_1, W_1) \cong (G_2, W_2)$ , if and only if there exists a bijection  $\phi : \mathcal{V}_1 \rightarrow \mathcal{V}_2$  such that*

- $\phi$  gives an isomorphism between  $G_1$  and  $G_2$ ,  $G_1 \cong G_2$
- $\phi(W_1) = W_2$

Rooted isomorphisms are useful because they guarantee stronger structural similarity. For example, an isomorphism between two graphs rooted on a pair of nodes preserves distances to the roots:

**Lemma 26** (Rooted isomorphism preserves distance to the root). *Let  $G_1 = (\mathcal{V}_1, \mathcal{E}_1)$  and  $G_2 = (\mathcal{V}_2, \mathcal{E}_2)$  be two graphs (with features). Let  $u \in \mathcal{V}_1$  and  $v \in \mathcal{V}_2$  and suppose that  $\phi : \mathcal{V}_1 \rightarrow \mathcal{V}_2$  is a bijection that gives  $(G_1, u) \cong (G_2, v)$ .*

*Then, for any  $a \in \mathcal{V}_1$  it holds that  $d^{G_1}(u, a) = d^{G_2}(v, \phi(a))$*

*Proof.* Let  $p = (u = w_0, \dots, w_l = a)$  a path in  $G_1$  for which  $d(u, a) = l$  is attained. Then, observe that  $(\phi(w_i), \phi(w_{i+1})) \in \mathcal{E}_2$  for all  $i \in \{0, \dots, l-1\}$  and thus  $\phi(p) := (v = \phi(w_0), \dots, \phi(w_l) = \phi(a))$  is a path in  $G_2$ , which guarantees  $d(v, \phi(a)) \leq l$ .

Next, we finish the proof with an argument by contradiction. Suppose there exists a path  $q = (v = w'_0, \dots, w'_k = \phi(a))$  with  $k < l$ . The same argument

as before gives us  $(\phi^{-1}(w'_i), \phi^{-1}(w'_{i+1})) \in \mathcal{E}_1$ , and therefore  $\phi^{-1}(q) = (u = \phi^{-1}(w'_0), \dots, \phi^{-1}(w'_k) = a)$  is a path in  $G_1$  which would imply  $d(u, a) = k < l$  which is impossible because  $d(u, a) = l$ . Thus, there cannot exist such a path  $q$  in  $G_2$ . We conclude  $d(v, \phi(a)) = d(u, a)$ .  $\square$

In this manuscript, we will use a result which is a direct consequence of the lemma we just proved: an isomorphism between two graphs rooted on a pair of nodes sends balls around the node in the first graph to balls around the node in the second graph:

**Lemma 27** (Rooted isomorphism sends balls to balls). *Let  $G_1 = (\mathcal{U}_1, \mathcal{E}_1)$  and  $G_2 = (\mathcal{U}_2, \mathcal{E}_2)$  be two graphs (with features). Let  $u \in \mathcal{U}_1$  and  $v \in \mathcal{U}_2$  and suppose that  $(G_1, u) \cong (G_2, v)$ .*

*Then, for any  $r \in \mathbb{N}_{\geq 0}$ , it holds that*

$$(\mathcal{B}_r^{G_1}(u), u) \cong (\mathcal{B}_r^{G_2}(v), v)$$

*Proof.* Let  $\phi : \mathcal{U}_1 \rightarrow \mathcal{U}_2$  be the bijection that achieves the rooted isomorphism and let  $a \in \mathcal{U}_1$  such that  $d(u, a) = k$ . Then, by Lemma 26,  $d(v, \phi(a)) = k$ . This implies that  $\phi(a) \in \mathcal{B}_r^{G_2}(v) \iff a \in \mathcal{B}_r^{G_1}(u)$ .

Consider now  $b \in \mathcal{U}_1$  such that  $d(u, b) = l$  and  $(a, b) \in \mathcal{E}_1$ . Then,  $(\phi(a), \phi(b)) \in \mathcal{E}_2$  and once again by Lemma 26,  $d(v, \phi(b)) = l$ .

If  $a, b \notin \mathcal{B}_r^{G_1}(u)$  ( $k, l > r$ ) we also have  $\phi(a), \phi(b) \notin \mathcal{B}_r^{G_2}(v)$  and  $(a, b) \notin \mathcal{E}(\mathcal{B}_r^{G_1}(u))$ ,  $(\phi(a), \phi(b)) \notin \mathcal{E}(\mathcal{B}_r^{G_2}(v))$ .

Let us now study the situation where at least one of  $a, b$  is included in the ball of radius  $r$  around  $u$ . WLOG, suppose  $k \leq r$  (i.e.,  $a \in \mathcal{B}_r^{G_1}(u)$ ,  $\phi(a) \in \mathcal{B}_r^{G_2}(v)$ ). There are three possible cases:

1.  $l > r$  (thus, necessarily  $k = r, l = r + 1$ ). In that case,  $b \notin \mathcal{B}_r^{G_1}(u)$ ,  $(a, b) \notin \mathcal{E}(\mathcal{B}_r^{G_1}(u))$ ,  $\phi(b) \notin \mathcal{B}_r^{G_2}(v)$  and  $(\phi(a), \phi(b)) \notin \mathcal{E}(\mathcal{B}_r^{G_2}(v))$ .
2.  $\min\{l, k\} < r$  (thus, necessarily  $b \in \mathcal{B}_r^{G_1}(u) \Rightarrow \phi(b) \in \mathcal{B}_r^{G_2}(v)$ ). In that case,  $(a, b) \in \mathcal{E}(\mathcal{B}_r^{G_1}(u))$  and also  $(\phi(a), \phi(b)) \in \mathcal{E}(\mathcal{B}_r^{G_2}(v))$  (see Definition 20).
3.  $l = k = r$ . In that case,  $(a, b) \notin \mathcal{E}(\mathcal{B}_r^{G_1}(u))$  and  $(\phi(a), \phi(b)) \notin \mathcal{E}(\mathcal{B}_r^{G_2}(v))$ ; even though  $a, b \in \mathcal{B}_r^{G_1}(u)$  and  $\phi(a), \phi(b) \in \mathcal{B}_r^{G_2}(v)$  (see Definition 20).

Therefore,  $(a, b) \in \mathcal{E}(\mathcal{B}_r^{G_1}(u)) \iff (\phi(a), \phi(b)) \in \mathcal{E}(\mathcal{B}_r^{G_2}(v))$ .

We conclude that  $(\mathcal{B}_r^{G_1}(u), u) \cong (\mathcal{B}_r^{G_2}(v), v)$ .  $\square$

In this work, molecular graph operations are mainly performed using RDKit, an open-source cheminformatics toolkit [22]. RDKit provides utilities for manipulating molecular graphs and exporting them to formats compatible with downstream molecular simulation tools.

Despite their suitability for molecule representation, graphs are not amenable to efficient storage, search and retrieval – all essential tasks in chemo/bioinformatics. Fortunately, molecular graphs can be encoded as strings. One of the most widely used standards for this purpose is **Simplified Molecular Input Line Entry System (SMILES)**.

A **SMILES** representation of a molecule is a string over a predefined alphabet that encodes the atoms, bonds, branching structure, ring closures and, when specified, stereochemical information of its molecular graph. A **SMILES** string can be obtained by traversing the molecular graph according to a prescribed convention. A canonical **SMILES** representation is a **SMILES** string obtained through a deterministic procedure that assigns the same string to all graph-isomorphic representations of the same molecule within a given canonicalization convention.

Another standard molecular identifier is the **IUPAC International Chemical Identifier (InChI)**, together with its hashed version, the **IUPAC International Chemical Identifier Key (InChIKey)**. The **InChI** is a text identifier obtained through a standardized procedure designed for universality and interoperability. Since full **InChI** strings can be relatively long, they are often converted through hashing into a shorter fixed-length representation, the **InChIKey**.

It is common to employ **SMILES** as a readable, structural string representation, while **InChIKey** is mainly an identifier for indexing and retrieval. However, such text-based representations are often not enough for **ML** tasks such as clustering or model input, which require vectorial embeddings.

Molecular fingerprints address this need by mapping a molecule to a finite-dimensional binary or count vector whose components encode the presence of certain structural patterns. Depending on the construction procedure, these patterns may correspond to predefined substructures, topological paths, circular neighbourhoods around atoms, or other molecular descriptors:

**Definition 28** (Molecular Fingerprint). *A molecular fingerprint is a function*

$$f : \mathcal{G} \rightarrow \mathbf{Y}$$

where  $\mathcal{G}$  denotes a set of molecular graphs and  $\mathbf{Y}$  is typically a finite-

dimensional vector space such as  $\{0, 1\}^d$  or  $\mathbb{N}^d$ .

A particularly important family of fingerprints are circular fingerprints, with the Morgan fingerprint being the best-known example.

**Definition 29** (Circular Fingerprint). A circular fingerprint of radius  $r$  is a molecular fingerprint  $f : \mathcal{G} \rightarrow \{0, 1\}^d$  obtained through the following procedure for any graph with node and edge features  $G = (\mathcal{V}, \mathcal{E}) \in \mathcal{G}$ :

1. Initialize labels  $x_v^{(0)}$  for all  $v \in \mathcal{V}$  and edge labels  $y_e$  for all  $e \in \mathcal{E}$ . Initialize the result  $f(G)$  as a vector of zeros.
2. For  $\ell \in \{1, \dots, r\}$ , define

$$x_v^{(\ell)} = \text{Hash}(x_v^{(\ell-1)}, \{\{x_u^{(\ell-1)}, y_{(u,v)}\}_{u \in N(v)}\})$$

for all  $v \in \mathcal{V}$ .

3. For all  $v \in \mathcal{V}$ , set

$$f(G)_{\text{Hash}(x_v^{(r)}) \pmod{d}} = 1.$$

Note that hash functions used in molecular fingerprints need to be engineered to capture relevant structural and chemical information of the molecular graph.

**Definition 30** (Tanimoto Similarity). Given two binary molecular fingerprints  $x, y \in \{0, 1\}^d$ , their Tanimoto similarity is defined as

$$T(x, y) = \frac{\sum_{i=1}^d x_i y_i}{\sum_{i=1}^d x_i + \sum_{i=1}^d y_i - \sum_{i=1}^d x_i y_i}.$$

**Proposition 31** (Tanimoto distance).  $d_T(x, y) := 1 - T(x, y)$  is a distance on  $\{0, 1\}^d$ .

*Proof.* First, observe that  $T(x, x) = \frac{|\{i \mid x_i=1\}|}{|\{i \mid x_i=1\}|} = 1$  and thus  $d_T(x, x) = 0$  for all  $x \in \{0, 1\}^d$ .

Further, note that for any  $x, y \in \{0, 1\}^d$  it holds that  $2x_i y_i \leq x_i + y_i$ : if both  $x_i$  and  $y_i$  are 0 we get  $0 = 0$ , if they are both 1 we get  $2 = 2$ , and if they are different we get  $0 \leq 1$ . Thus, we can bound  $T(x, y) \leq \frac{\sum_{i=1}^d x_i y_i}{2 \sum_{i=1}^d x_i y_i - \sum_{i=1}^d x_i y_i} = 1$ , which ensures  $d_T(x, y) \geq 0$  for all  $x, y \in \{0, 1\}^d$ .

Symmetry is trivially given by the commutative nature of multiplication and addition,  $d_T(x, y) = d_T(y, x)$  for all  $x, y \in \{0, 1\}^d$ .

Finally, given  $x, y, z \in \{0, 1\}^d$ , observe that  $d_T(x, z) + d_T(z, y) = 2 - T(x, z) - T(z, y)$  and thus proving the statement  $d_T(x, y) \leq d_T(x, z) + d_T(z, y)$  is equivalent to proving that  $T(x, z) + T(z, y) \leq 1 + T(x, y)$ . The proof of this statement is more convoluted and does not include any techniques relevant to this thesis, so we will not include it here. It may be found in [23].  $\square$

In RDKit, these representations can all be generated from the same underlying molecular graph object. This makes it possible to move seamlessly between graph-based representations, string encodings and fixed-dimensional descriptors depending on the requirements of the downstream task.

## 2.4 Machine Learning

To start, it is important to consider some foundational algorithms in ML that provide grounded, easy-to-implement baselines that allow us to reason on more advanced methods.

### 2.4.1 Principal Component Analysis

Principal Component Analysis (PCA) [24], [25] is a classical linear dimensionality reduction method that seeks a low-dimensional representation of the data that preserves as much information as possible. Given a centered data matrix  $X \in \mathbb{R}^{d \times n}$  (with  $n$  being the number of data points and  $d$  being their dimension/number of features), PCA finds an orthogonal projection onto a  $k$ -dimensional subspace, with  $k < d$ , such that the reconstruction error of the data after projection is minimised. The input data matrix can be easily centered by taking  $X_{\text{centered}} = X_{\text{un-centered}} - \frac{1}{n} X_{\text{un-centered}} \mathbf{1}_n \mathbf{1}_n^t$ . Mathematically, PCA solves the minimization problem

$$\operatorname{argmin}_{W_k \in \mathbb{R}^{d \times k} \mid W_k^t W_k = I} \mathbb{E}_x [\|x - W_k W_k^t x\|_2^2] \quad (2.3)$$

where  $x \in \mathbb{R}^d$  are the input data points, corresponding to the columns of  $X$ , and  $\mathbb{E}_x$  denotes the empirical average, i.e., the mean across the columns of  $X$ . Thus, PCA identifies the directions along which the data can be projected and then approximately reconstructed with the smallest possible squared error among all rank- $k$  linear approximations.

In practice, PCA can be computed through the singular value decomposition of the centered data matrix:

$$X = U\Sigma V^t$$

where the columns of  $V \in \mathbb{R}^{n \times n}$  are the right singular vectors, the columns of  $U \in \mathbb{R}^{d \times d}$  are the principal directions and the singular values  $\sigma_i$  in

$$\Sigma = \begin{cases} \begin{bmatrix} \text{diag}(\sigma_1, \dots, \sigma_n) \\ \mathbf{0}_{(d-n) \times n} \end{bmatrix} \in \mathbb{R}^{d \times n} & \text{if } d > n \\ \begin{bmatrix} \text{diag}(\sigma_1, \dots, \sigma_d) & \mathbf{0}_{d \times (n-d)} \end{bmatrix} \in \mathbb{R}^{d \times n} & \text{if } d \leq n \end{cases}$$

determine their importance. The solution to the minimization problem in (2.3) is given by  $U_k \in \mathbb{R}^{d \times k}$ , where  $U_k$  are the columns of  $U$  – i.e., the principal directions – that correspond to the  $k$  largest singular values and span the optimal  $k$ -dimensional subspace.

The variance captured in this optimal  $k$ -dimensional linear subspace is given by  $\sum_{i=1}^k \sigma_i^2$  (multiplied by a factor of  $\frac{1}{n}$  if we consider the empirical mean over the data points), while the optimal reconstruction error  $\mathbb{E}_x[\|x - U_k U_k^t x\|_2^2] = \frac{1}{n} \sum_{i=k+1}^r \sigma_i^2$  – where  $r = \min\{n, d\}$  – corresponds to variance left unexplained by the optimal projection  $U_k U_k^t$ . Since the singular values are ordered by decreasing magnitude, each additional principal dimension considered contributes less than the previous one to the explained variance, which makes PCA particularly useful for identifying low-dimensional structure and constructing simple, interpretable baselines.

## 2.4.2 Clustering

Given a certain similarity metric, some data points can be extremely similar and others radically different. Understanding the different classes/groups on one's data under a relevant similarity metric can be crucial for downstream tasks (for example, if a model is trained with highly similar data, it will be very accurate for other data within that class but lack the ability to generalize to others) and is sometimes the main goal in some **ML** problems.

One way to classify data is through supervised learning, training the model to label data based on examples. An alternative approach is to employ unsupervised learning methods to try and unveil the latent structure in data. Clustering refers to a family of unsupervised learning methods with this precise objective: to partition a dataset into groups of similar observations,

called clusters, so that points within the same cluster are more similar to one another than to points in different clusters.

### 2.4.2.1 K-Medoids

The k-medoids algorithm is a partition-based clustering method which generalizes the k-means algorithm to work with any kind of distance metric, not only the Euclidean metric, and forces the cluster centres to be actual data points in the dataset. Further, it is more robust to outliers as it does not need to compute means. Given a dataset  $\{x_i\}_{i=1}^n$  where  $x_i \in \mathcal{S}$  for some set  $\mathcal{S}$  and a dissimilarity measure  $d(\cdot, \cdot) : \mathcal{S} \times \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ , the goal is to select a set of medoids  $\{m_1, \dots, m_k\} \subset \{x_1, \dots, x_n\}$  that minimises the total within-cluster dissimilarity:

$$\operatorname{argmin}_{\{m_1, \dots, m_k\}} \sum_{i=1}^n \min_{j \in \{1, \dots, k\}} d(x_i, m_j)$$

Each point is assigned to the cluster given by its nearest medoid, which is implicitly encoded in this equation. A common implementation is the Partitioning Around Medoids (PAM) algorithm [26], which has complexity of  $O(k(n-k)^2)$  per step. While FastPAM [27] brought the per step complexity down to  $O(n^2)$ , it is still significantly slower than the k-means algorithm - whose per step complexity is  $O(n)$ .

### 2.4.3 Random Forest

Decision trees are a class of non-parametric models that recursively partition the input space  $\mathbb{R}^{d_{\text{input}}}$  into disjoint regions  $\{R_k\}_{k=1}^K$  and assign a constant prediction to each region. Formally, a tree defines a function

$$f(x) = \sum_{k=1}^K c_k \mathbf{1}_{\{x \in R_k\}}$$

where  $c_k \in \mathbb{R}^{d_{\text{output}}}$ .

Random forests improve upon single decision trees by constructing an ensemble  $\{f^{(m)}\}_{m=1}^M$  of trees trained on bootstrapped samples of the data and random subsets of features. The final prediction is obtained by averaging:

$$f_{\text{RF}}(x) = \frac{1}{M} \sum_{m=1}^M f^{(m)}(x)$$

This reduces variance and improves generalization, making random forests strong baseline models in many applications.

#### 2.4.4 XGBoost

Extreme Gradient Boosting (XGBoost [28]) is a scalable implementation of gradient boosting for decision trees. Instead of training trees independently, boosting builds them sequentially, with each new tree attempting to correct the residual errors of the previous ensemble.

Given a loss function  $\mathcal{L}$ , the model is built as an additive expansion

$$f^{(t)}(x) = f^{(t-1)}(x) + h_t(x)$$

where  $h_t$  is a decision tree fitted to the negative gradient of the loss with respect to the current predictions. In practice, a second-order approximation of the loss is used, leading to efficient optimization and regularization of tree complexity. XGBoost incorporates additional features such as shrinkage, column subsampling, and explicit regularization, which contribute to its strong empirical performance.

An additional practical advantage of tree-based boosting models is that they naturally provide a measure of feature importance. Since each internal split in a decision tree is performed on one particular input feature, the contribution of that feature can be quantified from the extent to which its use improves the objective function across the ensemble. In the gain-based formulation, which is one of the standard importance measures in XGBoost, the importance of a feature  $f$  is obtained by summing the reduction in the regularized training loss over all tree nodes that split on  $f$ . If  $\mathcal{T}$  denotes the set of trees in the ensemble and  $\mathcal{N}_f(t)$  the set of internal nodes of tree  $t$  that split on feature  $f$ , then this can be written as

$$I_{\text{gain}}(f) = \sum_{t \in \mathcal{T}} \sum_{n \in \mathcal{N}_f(t)} \Delta \mathcal{L}_{t,n},$$

where  $\Delta \mathcal{L}_{t,n}$  denotes the decrease in the regularized objective achieved by the split at node  $n$ . Thus, features that are repeatedly selected for highly beneficial splits receive larger importance scores. These scores may be analysed either at the individual-feature level or after aggregation across related input coordinates, which is particularly useful when several coordinates correspond to the same chemical descriptor family or to the same positional block in the input representation.

### 2.4.5 Neural Networks

A deep neural network consists of a sequence of layers, each of which applies an affine transformation followed by an element-wise non-linear activation function  $\tau : \mathbb{R} \rightarrow \mathbb{R}$ . For layer  $l$ , the output is

$$x_{l+1} = \tau(W_l x_l + b_l), \quad (2.4)$$

where  $W_l \in \mathbb{R}^{d_{l+1} \times d_l}$  and  $b_l \in \mathbb{R}^{d_{l+1}}$  are learnable parameters. In the case of supervised learning, the loss function is typically  $\mathcal{L} : \mathbb{R}^{d_{\text{output}}} \times \mathbb{R}^{d_{\text{output}}} \rightarrow \mathbb{R}$ . The necessary inputs to the loss function are the neural network's predictions and the true values for the training set.

It is sometimes useful to drop the non-linear activation function in the last layer. Some other features commonly added to the standard layer architecture to regularize and/or stabilize training are layer normalization and dropout.

Layer normalization [29] stabilizes training by normalizing the activations across the feature dimension. Given  $x_l \in \mathbb{R}^{d_l}$ , we define

$$\text{LayerNorm}(x_l) = \frac{x_l - \mu(x_l)}{\sigma(x_l)} \odot \gamma + \beta$$

where  $\mu(x_l)$  and  $\sigma(x_l)$  are the empirical mean and standard deviation of the components of  $x_l$ , and  $\gamma, \beta \in \mathbb{R}^{d_l}$  are learnable parameters.

Dropout is a regularization technique that randomly sets a subset of entries of the vector  $x_l$  to zero during training. Given a dropout rate  $p \in (0, 1)$ , we define a random mask  $m \in \{0, 1\}^{d_l}$  with independent Bernoulli( $1 - p$ ) entries and compute

$$\tilde{x}_l = m \odot x_l$$

This reduces co-adaptation between neurons, meaning that a neuron cannot rely too strongly on specific other neurons being present. Instead, units are encouraged to learn features that remain useful across many randomly perturbed subnetworks, which can improve generalization [30].

Note that **Multi Layer Perceptron (MLP)** and **Feed Forward Network (FFN)** are just alternative names for the same object, a standard deep neural network. Here, we use these names interchangeably, following the original notation of referenced publications.

### 2.4.5.1 Optimizers

Training neural networks amounts to minimizing the loss function  $\mathcal{L}$  over the parameter space. A basic optimization method is gradient descent, in which parameters are updated using the gradient of the loss over the entire training set. In practice, however, neural networks are typically trained with stochastic gradient descent (SGD) [31], which replaces the full gradient by a noisy estimate computed from a single example or, more commonly, from a mini-batch. The update rule is

$$\theta_{t+1} = \theta_t - \eta \nabla \mathcal{L}_t(\theta_t)$$

where  $\eta > 0$  is the learning rate and  $\nabla \mathcal{L}_t$  is the gradient of the mini-batch loss at iteration  $t$ .

In practice, more sophisticated optimizers are often used. Adaptive methods such as Adam [32] introduce momentum and per-parameter learning rate scaling based on first and second moments of the gradients. More recently, optimizers such as Muon [33] have been proposed and are gaining popularity.

A common feature of these methods is that they do not attempt to explicitly capture the potentially rugged geometry of the loss landscape induced by the data manifold. Instead, they introduce an implicit smoothing which biases the optimization trajectory towards flatter and more stable regions of the parameter space, often leading to better generalization.

## 2.4.6 Convolutional Neural Networks

**Convolutional Neural Networks (CNNs)** [34, 35, 36] are a class of neural networks that learn a kernel/filter which is slid across the spatial dimensions of the input, computing a local weighted sum at each position. This kernel is the same at every position (we say its weights are shared), which lowers the parameter count with respect to a fully connected layer  $W \in \mathbb{R}^{d_{out} \times d_{in}}$  that connects every input unit to every output unit.

Consider an input  $X \in \mathbb{R}^{C_{in} \times d_1 \times \dots \times d_D}$ , where  $D$  is the number of "spatial" dimensions of the input (number of dimensions the model slides across) and  $C_{in}$  is the number of input channels. A  $D$ -dimensional convolutional layer defines kernels  $K^{(c)} \in \mathbb{R}^{C_{in} \times k_1 \times \dots \times k_D}$  for  $c \in \{1, \dots, C_{out}\}$ . The output  $Y \in \mathbb{R}^{C_{out} \times d'_1 \times \dots \times d'_D}$  is

$$[Y]_{c, i_1, \dots, i_D} = \sum_{c'=1}^{C_{in}} \sum_{p_1=0}^{k_1-1} \dots \sum_{p_D=0}^{k_D-1} [K^{(c)}]_{c', p_1, \dots, p_D} \cdot [X]_{c', i_1 \cdot s_1 + p_1, \dots, i_D \cdot s_D + p_D} + b^{(c)}$$

where  $s_1, \dots, s_D \in \mathbb{N}_{>0}$  are the strides of the sliding window in the different dimensions and  $b^{(c)} \in \mathbb{R}^{d_f}$ . Note that the kernel spans all input channels but slides across the  $D$  "spatial" dimensions. For example, a 2D convolution with three channels would be used to process images (where the three channels represent the RGB colour channels).

This can be viewed as a structured analogue of the standard feed-forward layer introduced previously (see (2.4) in which the dense linear map is replaced by a convolution. Observe that convolution is still a linear operation, but one that is local and uses shared weights. The bias term remains, and - as in standard deep neural networks - the convolution is typically followed by a non-linear activation function and may also be followed by layer normalization and dropout. Stacking such layers yields a deep convolutional neural network.

Pooling in **CNNs** aims to aggregate information across the spatial dimensions and is applied through a sliding window, just like the learned kernel. For example, given  $X \in \mathbb{R}^{C_{\text{in}} \times d_1 \times d_2}$ , max pooling in a 2D **CNN** with a  $k_1 \times k_2$  window and stride  $s_1, s_2$  can be written down as

$$[Y]_{c,i,j} = \max_{\substack{0 \leq p < k_1 \\ 0 \leq q < k_2}} [X]_{c, i \cdot s_1 + p, j \cdot s_2 + q}$$

Typically, pooling is applied independently to each channel, preserving the number of channels. Furthermore, both convolutional pooling and convolution layers reduce the spatial extent by a factor of approximately the shift in each dimension (with floor division when the spatial size is not exactly divisible by the shift).

**CNNs** are well-known for their translation equivariance: if the input is shifted in the spatial dimensions, the output shifts by the same amount, meaning the network detects the same local patterns regardless of their position. These properties make CNNs particularly effective when the relevant structure in the data is local and can appear anywhere in the input domain.

## 2.4.7 Message Passing Graph Neural Networks

**Message Passing Neural Networks (MPNNs)** are a class of **Graph Neural Networks (GNNs)** that learn node representations by iteratively exchanging and aggregating information between neighbouring nodes through a series of learnable message and update functions, capturing the structural and feature-based dependencies in graph data.

Given a graph  $G = (\mathcal{U}, \mathcal{E})$  with node features  $\{x_u\}_{u \in \mathcal{U}}$  (see Definition 23),

if we fix a node indexing we can build a matrix  $X \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{nodes}}}$ . A **MPNN** iteratively updates the input node features through a sequence of layers. Each layer  $\ell$  takes as input a node feature matrix  $X_\ell \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$  and outputs another node feature matrix  $X_{\ell+1} \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ . Note that if the desired  $d_{\text{model}}$  does not match the initial  $d_{\text{nodes}}$ , then an input transformation (such as a **MLP** or a simpler learned linear map) is required to obtain  $X_0 \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$  from  $X \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{nodes}}}$ .

Each layer is structured as follows. Let  $X_\ell \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$  be the features of the graph nodes at layer  $\ell$ , where  $[X_\ell]_i \in \mathbb{R}^{d_{\text{model}}}$  are the features of node  $i$  at layer  $\ell$ . We define  $M_\ell \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$  as the messages passed at layer  $\ell$ . Thus,  $[M_\ell]_i \in \mathbb{R}^{d_{\text{model}}}$  are the messages passed to node  $i$  at layer  $\ell$ , this is:

$$[M_\ell]_i = A_\ell([X_\ell]_i, \{[X_\ell]_j \mid j \in N(i)\}) \quad (2.5)$$

where  $A_\ell : \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{model}}}$  is the aggregation function at layer  $\ell$ , which can either be pre-specified or learned. A common approach is to take

$$[M_\ell]_i = \sum_{j=1}^{|\mathcal{V}|} [\hat{A}]_{i,j} \psi_\ell([X_\ell]_i, [X_\ell]_j) \quad (2.6)$$

where  $\hat{A}$  is the normalised augmented adjacency matrix of  $G$  and  $\psi_\ell : \mathbb{R}^{d_{\text{model}}} \times \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{model}}}$  is a function that may be learned [37]. Observe that if we take  $\psi_\ell(a, b) = b$  we can simplify the aggregation update to  $M_\ell = \hat{A}X_\ell$ .

The last step of each layer applies an update function  $U_\ell$ :

$$[X_{\ell+1}]_i = U_\ell([X_\ell]_i, [M_\ell]_i) \quad (2.7)$$

where examples of possible  $U_\ell$  implementations include a simple addition followed by a non-linear activation function or a **MLP** on the concatenated inputs.

If it is our objective to predict global properties of the graph, we apply a pooling function  $P : \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{output}}}$  to the output node feature matrix of the message-passing layers stack to obtain a feature vector for the whole graph. Subsequently, a readout function  $R : \mathbb{R}^{d_{\text{output}}} \rightarrow \mathbb{R}^{d_{\text{prop}}}$  predicts the objective properties from the whole-graph feature vector, where  $d_{\text{prop}}$  is the dimension of the global graph properties we want to predict.

**MPNNs** have become a popular paradigm for learning on graphs in recent years. The message-passing architecture is very flexible, with some of its

most popular implementations being diffusion-based [38, 39, 40, 41]. Despite the multiple advances made in the field, **MPNNs** face two key limitations: over-smoothing [42, 43], where repeated aggregation causes node features to become indistinguishable, and over-squashing [44, 45], where long-range information is compressed through limited-capacity paths, leading to poor modelling of distant dependencies. Together, these issues restrict **MPNNs** expressivity.

**2.4.7.0.1 Over-squashing.** Topping *et al.* [46] proposes to quantify the over-squashing of the information shared from node  $u \in \mathcal{V}$  to node  $v \in \mathcal{V}$  as the partial derivative of the embedding vector of node  $v$  when the information from node  $u$  reaches it for the first time with respect to the input features of node  $u$ :

**Definition 32.** *The over-squashing of the information shared from node  $u \in \mathcal{V}$  to node  $v \in \mathcal{V}$  is quantified as  $\frac{\partial [X_r]_v}{\partial x_u}$  where  $r = d(u, v)$ , i.e.  $v \in S_r(u)$ .*

[46, Lemma 1] gives an upper bound to this quantity for any message passing architecture that uses the aggregation rule defined in (2.6). We include it here with adapted notation for convenience:

**Lemma 33.** *Assume an **MPNN** with aggregation as defined in (2.6) and an update function  $U$  as defined in (2.7). Let  $u, v \in \mathcal{V}$  such that  $d(u, v) = r$ . If  $|\nabla U_\ell| \leq \alpha$  and  $|\nabla \psi_\ell| \leq \beta \forall \ell \in \{0, \dots, r-1\}$ , then*

$$\left| \frac{\partial [X_r]_v}{\partial x_u} \right| \leq (\alpha\beta)^r [\hat{A}^r]_{v,u}$$

**2.4.7.0.2 Over-smoothing** Meanwhile, over-smoothing occurs because each layer aggregates and mixes the embeddings from neighbouring nodes, and repeated application of this process acts like a diffusion or low-pass filter on the graph, suppressing informative high-frequency components. As a result, over-smoothing grows with the number of message-passing steps, leading to a loss of expressive power in deep graph neural networks [42], [43].

### 2.4.7.1 Chemprop

Chemprop [47] is a popular software package that implements a **Directed Message Passing Neural Network (D-MPNN)**, an approach which has achieved remarkable success at molecular property prediction. Chemprop

separates each edge into two directed edges and performs message passing on the directed line graph.

Suppose we have a graph  $G = (\mathcal{U}, \mathcal{E})$  and let  $X_0 \in \mathbb{R}^{|\mathcal{U}| \times d_{\text{nodes}}}$  (which corresponds to the embedding matrix of a **MPNN** and  $X_0^* \in \mathbb{R}^{|\mathcal{E}| \times d_{\text{edges}}}$ . From now on, for every edge embedding matrix we use the edge notation as a subscript to denote the row of the matrix that contains the embedding for that edge. For example, given  $u, v \in \mathcal{U}$  such that  $(u, v) \in \mathcal{E}$ ,  $[X_0^*]_{(u,v)}$  denotes the row of  $X_0^*$  that contains the initial embedding for the edge  $(u, v)$ .

Next, we build the directed line graph embedding matrix  $H_0 \in \mathbb{R}^{2|\mathcal{E}| \times d_{\text{model}}}$ , with  $[H_0]_{u \rightarrow v} = \tau(W_i \text{Concat}(x_u, e_{(u,v)})) \forall u, v \in \mathcal{U}$  such that  $(u, v) \in \mathcal{E}$ , where  $\tau : \mathbb{R} \rightarrow \mathbb{R}$  is a non-linear activation function applied element-wise and  $W_i \in \mathbb{R}^{d_{\text{model}} \times (d_{\text{nodes}} + d_{\text{edges}})}$  is a matrix of weights.

The message passing updates on the directed edges are then performed as

$$[H_{\ell+1}]_{u \rightarrow v} = \tau \left( [H_0]_{u \rightarrow v} + W_\ell^t \sum_{w \in N(u) \setminus v} [H_\ell]_{w \rightarrow u} \right) \quad (2.8)$$

where  $W_\ell \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$  is a weight matrix.

The last steps consist on obtaining a final embedding for each node  $[X_f]_u = \tau(W_o \text{Concat}(x_u, \sum_{v \in N(u)} [H_L]_{v \rightarrow u}))$  - where  $W_o \in \mathbb{R}^{d_{\text{output}} \times d_{\text{model}}}$  and aggregating these into a single molecular embedding. By default, the aggregation is done by averaging the atom embeddings (i.e., taking  $\mathbf{1}^t X_f$  with  $\mathbf{1} = [1 \dots 1] \in \mathbb{R}^{|\mathcal{U}|}$ ) and (optionally) concatenating with some input molecular features  $x_m$ .

Finally, the obtained molecular embedding is fed to a **FFN** with output dimension the dimension of the properties that are being predicted.

## 2.4.8 Transformers

Transformers are neural network architectures designed to process inputs represented as sequences of discrete elements, called *tokens*. A token may correspond to a word in natural language, an amino acid in a protein sequence, or, more generally, any individual element of a structured input. Each token is first mapped to a vector representation, or embedding, and the transformer then updates these embeddings by allowing every token to selectively incorporate information from other tokens in the sequence. This makes transformers particularly suitable for modelling long-range dependencies.

The original transformer architecture was introduced by Vaswani *et al.* [48] for sequence-to-sequence tasks such as machine translation. It consists of an

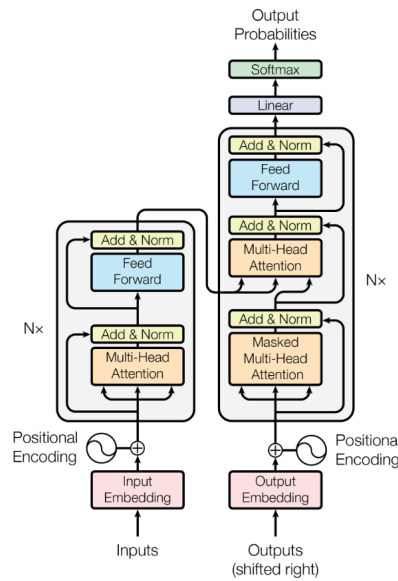


Figure 2.6: Original transformer architecture - Taken from [48, Figure 1]. The encoder architecture corresponds to the left stack and the decoder architecture corresponds to the right stack.

*encoder* and a *decoder* (see Figure 2.6). The encoder receives the full input token sequence and produces contextualized embeddings, in which each token representation accounts for the other input tokens. The decoder then uses these encoder representations together with the previously generated output tokens to predict the next output token. Repeating this autoregressive procedure generates an output sequence.

For the tasks considered in this thesis, the objective is not to generate an output sequence or predict a next token. Instead, the aim is to predict a global property of an entire modified **siRNA** duplex, namely its activity. Consequently, only the encoder component is required. The contextualized token embeddings produced by the encoder are aggregated into a single fixed-dimensional representation of the full input. A readout function, typically a pooling operation followed by a small **MLP**, then maps this global representation to the final scalar prediction.

Thus, in what follows, we focus on the two central components required for encoder-based transformer models: multi-head attention, which allows tokens to exchange information, and positional embeddings, which provide information about the order or location of tokens in the input sequence.

### 2.4.8.1 Scaled Dot-Product Attention

Attention is a mechanism that allows each input token to selectively aggregate information from other tokens in the sequence. More precisely, it constructs a new embedding for each token as a weighted combination of token representations, where the weights depend on the relevance of the other tokens in the context of the token being updated.

Let  $E \in \mathbb{R}^{d_i \times d_{\text{model}}}$  be the input embedding matrix, where  $d_i$  is the number of input tokens and each row of  $E$  is a  $d_{\text{model}}$ -dimensional token embedding. Attention uses three learned linear maps to transform the input embeddings into a query matrix  $Q \in \mathbb{R}^{d_i \times d_k}$ , a key matrix  $K \in \mathbb{R}^{d_i \times d_k}$ , and a value matrix  $V \in \mathbb{R}^{d_i \times d_v}$ .

The terms *query projection*, *key projection*, and *value projection* are standard in the machine learning literature. However, these maps are not projections in the strict mathematical sense, as they are not required to be orthogonal or idempotent. Rather, they are general learned linear mappings from the input embedding space to the query, key, and value spaces.

The query vector of a token represents the information that the token seeks from the sequence, while the key vectors determine the relevance of each token to that query. The value vectors contain the information that is ultimately aggregated. The compatibility between the query of one token and the keys of all tokens is measured through their scaled dot products. Scaled dot-product attention is defined as

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^t}{\sqrt{d_k}} \right) V \in \mathbb{R}^{d_i \times d_v}, \quad (2.9)$$

where the softmax function is applied independently to each row. In particular, the  $j$ -th row of  $\text{softmax} \left( \frac{QK^t}{\sqrt{d_k}} \right)$  contains the weights assigned by token  $j$  to all input tokens. Thus, the output embedding of token  $j$  is a weighted combination of the value vectors of all tokens.

The normalization by  $\sqrt{d_k}$  prevents the query-key dot products from becoming excessively large as  $d_k$  increases. As noted in [48], if two  $d_k$ -dimensional random vectors have independent entries with mean zero and variance one, then their dot product has variance  $d_k$ . Without normalization, the softmax function can become overly concentrated, resulting in small gradients and less stable optimization.

### 2.4.8.2 Multi-Head Attention

A single attention operation learns one set of query, key, and value mappings, and therefore one notion of relevance between tokens. Multi-head attention applies several attention operations in parallel, allowing the model to learn different types of relationships between the input tokens.

Let  $h$  be the number of attention heads. For each head  $i \in \{1, \dots, h\}$ , let  $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$  and  $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$  be learned weight matrices. The output of head  $i$  is computed as

$$\begin{aligned} \text{head}_i(E) &= \text{Attention}(Q_i, K_i, V_i) \in \mathbb{R}^{d_i \times d_v}, \\ Q_i &= EW_i^Q, \quad K_i = EW_i^K, \quad V_i = EW_i^V. \end{aligned} \quad (2.10)$$

The outputs of the different heads are concatenated along the feature dimension and transformed through a final learned linear map:

$$\text{MultiHead}(E) = \text{Concat}(\text{head}_1(E), \dots, \text{head}_h(E)) W^O \in \mathbb{R}^{d_i \times d_{\text{model}}}, \quad (2.11)$$

where  $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ . Therefore, multi-head attention combines several learned views of dependencies between tokens into one updated embedding for each token.

### 2.4.8.3 Positional Embedding

Multi-head attention does not naturally consider the position of each token in the sequence. Thus, a positional embedding is any sort of modification of the original token embeddings to include information about the token's position in the sequence. For example, Vaswani *et al.* [48] adds the positional embedding vector  $PE(\text{token}) \in \mathbb{R}^{d_{\text{model}}}$  to the original embedding of each token, defined as

$$[PE(\text{token})]_i = \begin{cases} \sin\left(\frac{\text{pos}_{\text{token}}}{10000} \frac{i}{d_{\text{model}}}\right) & \text{if } i \text{ even} \\ \cos\left(\frac{\text{pos}_{\text{token}}}{10000} \frac{i-1}{d_{\text{model}}}\right) & \text{if } i \text{ odd} \end{cases} \quad \text{for } i \in \{1, \dots, d_{\text{model}}\}$$

### 2.4.9 Graph Attention Transformers

The observation that multi-head attention is blind to the token's positions naturally leads to the question of whether transformers can be used with graph-structured data while preserving the graph's inductive bias. This is particularly

desirable given the challenge that over-squashing represents for traditional **MPNNs**.

Graph Attention Networks [49] were a first attempt to tackle this challenge, but their attention mechanism was limited to the message aggregation between neighbouring nodes. Nowadays, most modern **GNNs** favour a simpler message aggregation mechanism such as averaging. Furthermore, having attention limited to message aggregation from neighbouring nodes means this approach is just a type of **MPNN** and thus suffers from their same limitations.

A more recent approach that goes beyond **MPNNs** is Edge Set Attention [50]. To the best of our knowledge, Edge Set Attention is the architecture that better answers our question of whether transformers can be extended to graphs while preserving inductive bias. Edge Set Attention views graphs as a set of edges (this is, each edge is a token) and performs a series of attention operations that alternate masked attention between neighbouring edges in the line graph and full attention over all the edges in the graph.

Given a graph  $G = (\mathcal{V}, \mathcal{E})$ , the procedure is the following: first, we obtain an embedding for each edge  $e \in \mathcal{E}$  by combining the information of the edge and the two nodes that compose it (typically, concatenating their embeddings on the original graph). Next, we build the input matrix  $E \in \mathbb{R}^{|\mathcal{E}| \times d_{model}}$ , where  $d_{model}$  is the dimension of each resulting edge embedding. Then, (Multi-Head) Self Attention is performed just as described in Equations (2.10) and (2.9). (Multi-Head) Masked Attention follows a similar procedure, the only difference being that a matrix  $M \in \mathbb{R}^{|\mathcal{E}| \times |\mathcal{E}|}$  is added to the normalised query-key matrix product:

$$\text{MaskedAttention}(Q, K, M, V; G) = \text{softmax} \left( \frac{QK^t}{\sqrt{d_k}} + M \right) V \in \mathbb{R}^{d_i \times d_v} \quad (2.12)$$

where the masking matrix  $M = M(G)$  satisfies

$$[M]_{i,j} = \begin{cases} 0 & \text{if } (i, j) \in \mathcal{E} \\ -\infty & \text{otherwise} \end{cases} \quad (2.13)$$

The detailed architecture is outlined in 2.7

## 2.4.10 Cm-siRPred

Cm-siRPred [5] is, to the best of our knowledge, the most advanced model for data-driven **siRNA** activity prediction. This model extracts three different

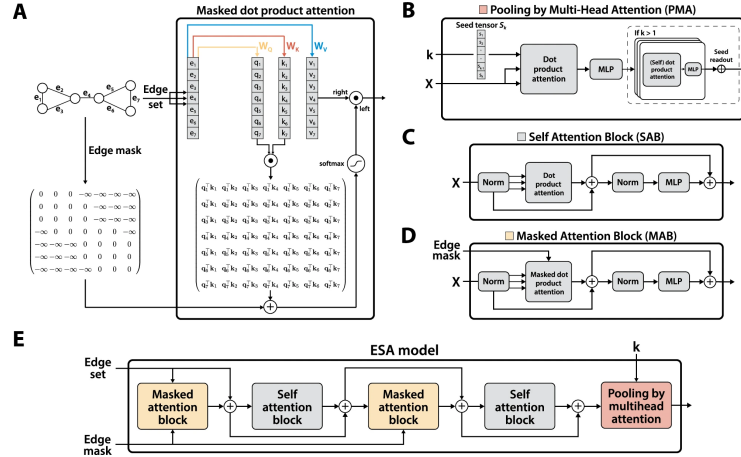


Figure 2.7: Original Edge Set Transformer architecture - Taken from [50, Figure 4]

feature matrices from each **siRNA** duplex: one-hot sequence encoding, nucleotide molecular fingerprints, and nucleotide physicochemical properties. A cross-attention mechanism and integrates them through a cross-attention mechanism followed by a convolutional neural network.

Let  $l$  denote the number of nucleotide positions in the representation. The nucleotides in the guide strand and passenger strand are concatenated, yielding  $l = 50$  for both the original paper and this thesis. Let  $F_o \in \mathbb{R}^{l \times d_o}$  be the one-hot embedding feature matrix,  $F_m \in \mathbb{R}^{l \times d_m}$  the molecular fingerprints feature matrix and  $F_p \in \mathbb{R}^{l \times d_p}$  the physicochemical properties feature matrix; where  $d_o$ ,  $d_m$ , and  $d_p$  are the respective feature dimensions. To apply the cross-attention mechanism, each view must first be projected to a shared embedding space  $\mathbb{R}^{l \times d}$ . This is achieved through a linear embedding layer  $X_i = F_i W_i^E$  for  $i \in \{o, m, p\}$ , where  $W_o^E \in \mathbb{R}^{d_o \times d}$ ,  $W_m^E \in \mathbb{R}^{d_m \times d}$ ,  $W_p^E \in \mathbb{R}^{d_p \times d}$  are learnable projection matrices and  $d$  is the shared embedding dimension.

Now, information is exchanged between the three different types of features by allowing each feature to attend to the others. Given two feature matrices  $X_i, X_j \in \mathbb{R}^{l \times d}$ , cross-attention is defined as

$$\text{CrossAttention}(X_i, X_j) = \text{Concat}(Z_{ij}^1, \dots, Z_{ij}^h) W^O$$

where  $h$  is the number of attention heads,  $W^O \in \mathbb{R}^{d \times d}$  is a learnable output projection, and each head computes  $Z_{ij}^k = \text{Attention}(Q_j^k, K_i^k, V_i^k)$  with  $Q_j^k = X_j W_k^Q$ ,  $K_i^k = X_i W_k^K$ ,  $V_i^k = X_i W_k^V$ , where  $W_k^Q, W_k^K, W_k^V \in \mathbb{R}^{d \times d_m}$  and

$$d_m = d/h.$$

Note that the queries are derived from view  $j$  while the keys and values are derived from view  $i$ . This allows view  $j$  to selectively attend to informative features in view  $i$ . However, this mechanism performs no intra-view attention: nucleotide positions within the same view do not attend to each other.

Let  $\tilde{X}_o, \tilde{X}_m, \tilde{X}_p \in \mathbb{R}^{l \times d}$  be the resulting feature matrices after the cross-attention module. These are stacked as channels to form a tensor  $\mathcal{X} \in \mathbb{R}^{3 \times l \times d}$ , which is processed by a two-layer convolutional network. Each layer consists of a 2D convolution with  $3 \times 3$  kernels (stride 1, padding 1), a ReLU activation, and  $2 \times 2$  max pooling:

$$\mathcal{X}_c^{(n)} = \text{MaxPool}_{2 \times 2} \left( \text{ReLU} \left( \text{Conv}_{3 \times 3}^{(n)} (\mathcal{X}_c^{(n-1)}) \right) \right)$$

The first convolutional layer maps 3 input channels to 16 feature maps, and the second expands from 16 to 32 feature maps. The convolutions with padding preserve spatial dimensions, while the max pooling operations progressively downsample the spatial extent.

Finally, the output of the CNN is flattened into a vector  $z_{\text{cnn}} \in \mathbb{R}^{d_{\text{flat}}}$ , which is passed through a fully connected network for final prediction:

$$\hat{y} = \sigma \left( W_2 \cdot \text{Dropout}_p \left( \text{ReLU} (W_1 z_{\text{cnn}} + b_1) \right) + b_2 \right)$$

where  $W_1 \in \mathbb{R}^{128 \times d_{\text{flat}}}$ ,  $W_2 \in \mathbb{R}^{1 \times 128}$ ,  $p = 0.5$  is the dropout rate, and  $\sigma$  is the sigmoid function that constrains the output to the interval  $(0, 1)$ .

#### 2.4.10.1 Hierarchical and Multi-Level ML Models

Hierarchical (or multi-level) algorithms play an important role in the computational sciences because they allow for the systematic integration of different scales in a "divide and conquer" approach. The most well-known multi-level model is perhaps the multi-grid approach to numerically solve **Partial Differential Equations (PDEs)** [51].

While hierarchical or multi-level **GNNs** have been studied in graph representation learning and have seen some use in molecular machine learning, they have been less prominent in bio-related applications than other methods such as transformers or Chemprop. These methods construct coarser graph representations through pooling or graph coarsening, aiming to capture dependencies across multiple scales. A classical example is DiffPool [52], which learns soft cluster assignments and builds a hierarchy of graphs during training. More recently, [53] proposed a hierarchical **GNN** for binding affinity

in protein complexes prediction.

Related multi-scale ideas have also proved attractive in scientific machine learning, particularly in computational mechanics and computational fluid dynamics [54, 55].

However, to the best of our knowledge, the naturally hierarchical structure of modified **siRNA** has not been exploited through multi-level methods in the literature.



## Chapter 3

# Theoretical Analysis of ML models on modified siRNA graphs

This chapter focuses on the mathematical description of the research statement in Section 3.1 and, subsequently, the hierarchical graph structure of modified siRNA in Section 3.2 and the mathematical analysis of different types of ML models applied to siRNA in Section 3.3.

### 3.1 Mathematical Formulation of the Research Question

In order to build a successful ML model for siRNA activity prediction, it is convenient to first formulate the problem in mathematical terms to understand the mapping that we are aiming to approximate.

Formally, let  $\mathcal{G}_N$  be the set of the molecular graphs of all possible double-strands of up to  $N$  (modified) nucleotides and consider a mapping  $M_N : \mathcal{G}_N \rightarrow \{\Gamma_{N,g} \mid g \in \mathcal{G}_N\}$  that maps each molecular graph to its corresponding configuration space. Ideally, each of these configuration spaces would include solvent and cellular environment, but we will only consider the solvent. We assume we have access to  $x$  double-strands of  $N$  modified nucleotides, whose configuration spaces are  $\{\Gamma_{N,g_1}, \dots, \Gamma_{N,g_x}\}$ . Let  $P_N : \{\Gamma_{N,g} \mid g \in \mathcal{G}_N\} \rightarrow \{L^1(\Gamma_{N,g}) \mid g \in \mathcal{G}_N\}$  a mapping that gives us the probability density function (assuming, for example, a canonical ensemble) associated to any configuration space of a double strand of  $N$  modified nucleotides, this is,  $P_N(\Gamma_N) = p_{\Gamma_N} : \Gamma_N \rightarrow \mathbb{R}_{\geq 0}$ . Finally, assume there exists a mapping  $\tilde{F} : \bigsqcup_{g \in \mathcal{G}_N} L^1(\Gamma_{N,g}) \rightarrow \mathbb{R}$  whose output is the resulting siRNA activity (*mRNA knockdown*). Clearly,

$\tilde{F}$  and  $P_N$  are inaccessible to us because we do not have the sampling capacity to probe them. We are interested in reconstructing the full mapping

$$F = \tilde{F} \circ P_N \circ M_N$$

for which we have experimental evaluations at the data-points  $g_1, \dots, g_x$ .

This project aims to tackle this problem by approximating the full mapping  $F$  through a combination of modern machine learning techniques and physics-based (MD) simulation. To do this, let us consider  $\mathcal{G}_3$  the set of the molecular graphs of all possible double-stranded (modified) trimers and a mapping  $M_3 : \mathcal{G}_3 \rightarrow \{\Gamma_{3,g} \mid g \in \mathcal{G}_3\}$  that maps each molecular graph to its corresponding configuration space. We assume we have access to  $y$  double-stranded modified trimers, whose configuration spaces are  $\{\Gamma_{3,g_1}, \dots, \Gamma_{3,g_y}\}$ . Let  $P_3 : \{\Gamma_{3,g} \mid g \in \mathcal{G}_3\} \rightarrow \{L^1(\Gamma_{N,g}) \mid g \in \mathcal{G}_N\}$  be a mapping that gives us the probability density function (assuming, for example, a canonical ensemble) associated to any configuration space of a double-stranded modified trimer, this is,  $P_3(\Gamma_3) = p_{\Gamma_3} : \Gamma_3 \rightarrow \mathbb{R}_{\geq 0}$ .

We are going to decompose every double-strand of  $N$  modified nucleotides in overlapping trimers (consecutive overlapping trimers have two nucleotides in common). Thus, we obtain  $N - 2$  overlapping trimers from each double-strand of  $N$  modified nucleotides (some of which may even be empty graphs, as not every **siRNA** molecule in  $\mathcal{G}_N$  has  $N$  nucleotides in each strand) through the mapping  $D_T(N) : \mathcal{G}_N \rightarrow (\mathcal{G}_3)^{N-2}$ , which we can construct explicitly. Observe that the mapping  $P_3$  is much more accessible to us than  $P_N$ , as we can afford to run **MD** simulation of relatively large numbers of possible trimers.

Finally, we have a functional  $G : \bigsqcup_{g \in \mathcal{G}_3} L^1(\Gamma_{3,g}) \rightarrow \mathbb{R}^k$  that gives us the observables of interest for a given trimer (where  $k$  denotes the sum of the dimensions of these observables). Our objective is then to learn a function  $H : \prod_{j=1}^{N-2} \mathbb{R}^k \rightarrow \mathbb{R}$  that minimizes

$$\|F - H \circ (G, \dots, G) \circ (P_3, \dots, P_3) \circ (M_3, \dots, M_3) \circ D_T\| \quad (3.1)$$

For convenience, let us introduce the notation  $\mathcal{G}_1^*$  to refer to the set of all possible modified nucleotides and  $D(M) : \mathcal{G}_M \rightarrow (\mathcal{G}_1^*)^{2M}$  the map that extracts all the nucleotides in a double-stranded **siRNA** of up to  $M$  nucleotides in each strand. This will be useful to formalise the implemented ML models.

## 3.2 The graph of graphs formulation

To be able to reason about the molecular graphs given by our modified, double stranded **siRNA**, it is helpful to define a hierarchical structure in which, for example, nucleotides are a sub-graph partition of the whole molecular graph. Note that this hierarchical partition can have multiple layers: whereby, the backbone, sugar and base fragments of each nucleotide can also be thought of as a sub-graph partition of each of the nucleotide graphs, which at the same time are a sub-graph partition of the whole molecular graph. We will refer to this concept as a graph of graphs.

**Definition 34** (Graph of graphs). *A graph of graphs is a tuple of tuples  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{\text{CG}}, \mathcal{E}^{\text{CG}}))$  where  $G = (\mathcal{U}, \mathcal{E})$  is the underlying graph,  $\mathcal{U}^{\text{CG}} = \{G_1 = (\mathcal{U}_1, \mathcal{E}_1) \dots, G_n = (\mathcal{U}_n, \mathcal{E}_n)\}$  is a set of graphs, called graph nodes, such that  $G_i$  is a sub-graph of  $G$ ,  $G_i \subseteq G$ , for all  $i \in \{1, \dots, n\}$  and  $\bigsqcup_{i=1}^n \mathcal{U}_i = \mathcal{U}$ ; and  $\mathcal{E}^{\text{CG}} = \{(G_i, G_j) \mid \exists e = (u, v) \in \mathcal{E} \text{ where } u \in \mathcal{U}_i, v \in \mathcal{U}_j\} \subseteq \mathcal{U}^{\text{CG}} \times \mathcal{U}^{\text{CG}}$ .*

Figure 3.1 depicts an example graph of graphs over an underlying graph  $G$ .

In order to facilitate dealing with graph of graph structures, we can introduce the concept of coarse grained interfaces and coarse grained paths.

**Definition 35** (Coarse grained Interface). *Given a graph of graphs  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{\text{CG}}, \mathcal{E}^{\text{CG}}))$  and two graphs  $G_i, G_j \in \mathcal{U}^{\text{CG}}$ , the coarse grained  $G_i$ -interface to  $G_j$  is defined as the set of nodes in  $G_i$  that have neighbours in  $G_j$ , i.e.*

$$\partial_{G_i}(G_j) = \{u \in \mathcal{U}_i \mid \exists v \in \mathcal{U}_j \text{ st } (u, v) \in \mathcal{E}\}.$$

*The coarse grained  $G_i$ -interface is then defined as the union of its interfaces with all its neighbouring graph nodes in the coarse grained graph,*

$$\partial_{G_i} = \bigcup_{j \mid (G_i, G_j) \in \mathcal{E}^{\text{CG}}} \partial_{G_i}(G_j)$$

**Definition 36** (Coarse grained Walk). *Given a graph of graphs  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{\text{CG}}, \mathcal{E}^{\text{CG}}))$  with an underlying graph  $G = (\mathcal{U}, \mathcal{E})$  and a path  $p$  of length  $l$ ,  $p = (w_0, \dots, w_l)$  on  $G$  (where paths in the whole graph are defined just as in 13), the coarse grained walk  $p^{\text{CG}}$  associated to  $p$  is the sequence of sub-graphs visited by the path  $p$ .*

*Formally, the coarse grained walk associated to a path is given by the sequence of subgraphs obtained by partitioning the path  $p$  in sequences of*

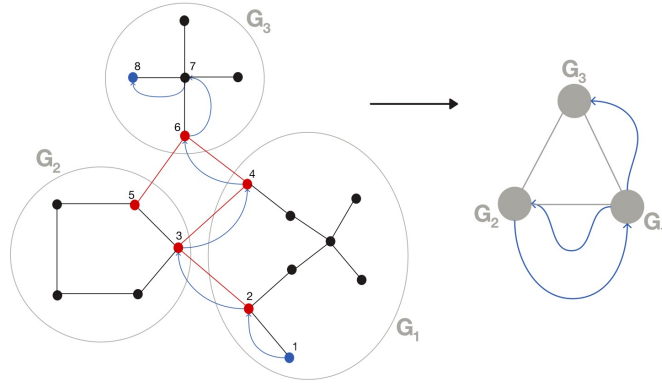


Figure 3.1: Example graph of graphs structure over an underlying, standard graph (left) and its corresponding coarse-grained graph (right).

The nodes in the coarse grained interfaces and the edges between them are highlighted in red. Observe that  $\partial_{G_1}(G_2) = \{2, 4\}$ ,  $\partial_{G_1}(G_3) = \{4\}$ ,  $\partial_{G_2}(G_1) = \{3\}$ ,  $\partial_{G_2}(G_3) = \{5\}$ ,  $\partial_{G_3}(G_1) = \partial_{G_3}(G_2) = \{6\}$ .

The blue arrows on the underlying graph (left) depict a path  $p = (1, 2, 3, 4, 6, 7, 8)$  with length  $l(p) = 6$ . The blue arrows on the coarse-grained graph (right) depict the coarse-grained walk  $p^{CG} = (G_1, G_2, G_1, G_3)$  associated to it, with coarse-grained length  $c(p) = 3$ .

consecutive nodes on the same subgraph, i.e., let  $(w_0, \dots, w_{p_1}) \in \mathcal{U}_{k_0}$ ,  $(w_{p_1+1}, \dots, w_{p_2}) \in \mathcal{U}_{k_1}$ , ...,  $(w_{p_{k_c-1}+1}, \dots, w_{p_k} = w_l) \in \mathcal{U}_{k_c}$  with  $p_{i+1} \geq p_i$  and  $p_1 \geq 0$ . Then, the coarse grained path  $p^{CG}$  associated to  $p$  is  $p^{CG} = (G_{k_0}, \dots, G_{k_c})$ , and we say it has coarse-grained length  $c$ .

From now on, we write  $l(p)$  to denote the length of a path and  $c(p)$  to denote its coarse-grained length. Note that, trivially,  $c(p) \leq l(p)$  for any path  $p$  in a graph of graphs.

A simple, convenient property of a graph of graphs is that the distance between a node in a given graph node  $G_i$  and another node is always bounded below:

**Lemma 37** (Inter graph node distance is bounded below). *Let  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{CG}, \mathcal{E}^{CG}))$  where  $G = (\mathcal{U}, \mathcal{E})$  is the underlying graph,  $\mathcal{U}^{CG} = \{G_1 = (\mathcal{U}_1, \mathcal{E}_1) \dots, G_n = (\mathcal{U}_n, \mathcal{E}_n)\}$ .*

*Let  $d(\cdot, \cdot)$  denote the distance in the underlying graph  $G$ , and  $d^{G_i}(\cdot, \cdot)$  denote the distance in a graph node  $G_i \in \mathcal{U}^{CG}$ .*

Given two nodes  $u \in \mathcal{U}_i$  and  $v \in \mathcal{U} \setminus \mathcal{U}_i$ , it holds that

$$d(u, v) \geq 1 + d^{G_i}(u, \partial_{G_i}).$$

*Proof.* Let  $p = (v = w_0, \dots, w_l = u)$  a path for which  $d(u, v) = l$  is achieved. Then, there must exist  $k \in \{1, \dots, l\}$  such that  $w_k \in \partial_{G_i}$  and  $w_m \in \mathcal{U}_i$  for all  $m \in \{k, \dots, l\}$ .

In particular, this means that the path  $p' = (w_k, \dots, u)$  is completely contained in  $G_i$ , and therefore  $l(p') \geq d^{G_i}(u, \partial_{G_i})$  as  $w_k \in \partial_{G_i}$ .

Note that node  $w_{k-1}$  in  $p$  cannot belong to  $G_i$ , i.e, we need to spend at least one step to go into  $G_i$ . We conclude that

$$d(u, v) = l(p) \geq 1 + l(p') \geq 1 + d^{G_i}(u, \partial_{G_i})$$

□

In this thesis, we will mainly focus on graphs of graphs that consist on a series of graphs in a straight line. We call these lines of graphs:

**Definition 38** (Line of graphs). A line of graphs is a graph of graphs  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{CG}, \mathcal{E}^{CG}))$  where  $\mathcal{U}^{CG} = \{G_1 = (\mathcal{U}_1, \mathcal{E}_1) \dots, G_n = (\mathcal{U}_n, \mathcal{E}_n)\}$  and  $(G_i, G_j) \in \mathcal{E}^{CG} \iff i = j \pm 1$ . As a result, we denote the interfaces of a graph  $G_i \in \mathcal{U}^{CG}$  as  $\partial_i^+ := \partial_{G_i}(G_{i+1})$  and  $\partial_i^- := \partial_{G_i}(G_{i-1})$ .

For the case of a line of graphs, we can obtain a small refinement of the bound in Lemma 37:

**Lemma 39** (Inter graph node distance is bounded below, line of graphs case). Let  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{CG}, \mathcal{E}^{CG}))$  a line of graphs where  $G = (\mathcal{U}, \mathcal{E})$  is the underlying graph,  $\mathcal{U}^{CG} = \{G_1 = (\mathcal{U}_1, \mathcal{E}_1) \dots, G_n = (\mathcal{U}_n, \mathcal{E}_n)\}$ .

Let  $d(\cdot, \cdot)$  denote the distance in the underlying graph  $G$ , and  $d^{G_i}(\cdot, \cdot)$  denote the distance in a graph node  $G_i \in \mathcal{U}^{CG}$ .

Given two nodes  $u \in \mathcal{U}_i$  and  $v \in \mathcal{U}_j$  such that  $d(u, v) \leq 2 + d^{G_i}(\partial_i^-, \partial_i^+)$  (if  $i \in \{1, l\}$ , then we set  $d^{G_{1/l}}(\partial_{1/l}^-, \partial_{1/l}^+) = \infty$  as one of the interfaces does not exist) it holds that

$$d(u, v) \geq \begin{cases} 1 + d^{G_i}(u, \partial_i^-) & \text{if } j < i \\ 1 + d^{G_i}(u, \partial_i^+) & \text{if } j > i \end{cases} \geq 1 + d^{G_i}(u, \partial_i).$$

*Proof.* Let us study the case  $j < i$  first, as the case  $j > i$  is entirely symmetrical.

Assume  $j < i$ . Let  $p = (v = w_0, \dots, w_l = u)$  a path for which  $d(u, v) = l$  is achieved. Then, there must exist  $k \in \{1, \dots, l\}$  such that  $w_{k-1}$  is the last node

in  $p$  contained in a block strictly below  $i$ , this is,  $w_k \in \partial_i^-$  and  $w_m \in \bigcup_{a \geq i} \mathcal{U}_a$  for all  $m \in \{k, \dots, l\}$ .

Assume that there exists  $m^* \in \{k+1, \dots, l-1\}$  such that  $w_{m^*} \in \mathcal{U}_{i+1}$ . Then, as  $u \in \mathcal{U}_i$ , it holds that there exist  $n, n'$  fulfilling  $k \leq n < m^* < n' \leq l$  such that  $w_n, w_{n'} \in \partial_i^+$ . Take  $n$  to be the first index for which the path visits  $\partial_i^+$  after (and including)  $k$ , and  $n'$  to be the last index for which the path visits  $\partial_i^+$  after  $m^*$ .

Consider  $q = (w_n, \dots, w_{m^*}, \dots, w_{n'})$ , clearly  $l(q) \geq 2$ . Consider also  $q' = (w_k, \dots, w_n)$ . Observe that  $q'$  is contained in  $G_i$ :  $w_k \in \partial_i^- \subseteq G_i$  and we know that  $w_m \in \bigcup_{a \geq i} \mathcal{U}_a$  for all  $m \geq k$  but  $n$  is the first index such that  $n > k$  and  $w_n \in \partial_i^+ \subseteq G_i$ , which must be traversed to reach graph nodes  $G_a$  with  $a > i$ . Therefore, it must hold that  $w_m \in G_i$  for all  $m \in \{k, \dots, n\}$ .

Further,  $w_k \in \partial_i^-$  and  $w_n \in \partial_i^+$ , and therefore  $l(q') \geq d^{G_i}(\partial_i^-, \partial_i^+)$ . Then, it holds that:

$$l(p) > l(q' \| q) = l(q) + l(q') \geq 2 + d^{G_i}(\partial_i^-, \partial_i^+) \geq d(u, v)$$

where  $q' \| q = (w_k, \dots, w_n, \dots, w_{m^*}, \dots, w_{n'})$  denotes the concatenation of paths  $q$  and  $q'$ . Observe that  $l(p) > l(q' \| q)$  is guaranteed because we cannot have  $k = 0$ .

This is a contradiction because  $l = l(p) = d(u, v)$ ! We conclude that there cannot exist  $m^* \in \{k+1, \dots, l-1\}$  such that  $w_{m^*} \in \mathcal{U}_{i+1}$ . Clearly, this rules out the possibility to have  $w_{m^*} \in \mathcal{U}_{i+a}$  for any  $a \in \mathbb{N}_{>0}$ , because for  $p$  to reach  $G_{i+a}$  with  $a \geq 1$  it must go through  $G_{i+1}$  first.

We conclude that  $w_m \in \mathcal{U}_i$  for all  $m \in \{k, \dots, l\}$ . Therefore, the path  $p' = (w_k, \dots, w_l = u)$  is contained in  $G_i$ . As  $v = w_0 \notin \mathcal{U}_i$ , it holds that:

$$d(u, v) = l(p) \geq 1 + l(p') \geq 1 + d^{G_i}(u, \partial_i^-)$$

For the case  $j > i$ , the equivalent argument gives us

$$d(u, v) \geq 1 + d^{G_i}(u, \partial_i^+)$$

□

### 3.3 Analysis of graph-based ML methods on siRNA molecular graphs

Having introduced the graph of graphs formulation in Definition 34, we are now ready to study how does the structure of siRNA molecular graphs influence the performance of graph-based ML methods.

#### 3.3.1 The limitations of standard Message Passing Neural Networks on siRNA molecular graphs

This section leverages the concept of graph of graphs in Definition 34 to prove a number of theoretical results which showcase the limitations of standard MPNNs, highlight the role of over-squashing as introduced in Definition 32, inform the construction of siRNA molecular graphs and point towards potential solutions.

##### 3.3.1.1 The importance of hydrogen bonds and stacking interactions

First, we begin by highlighting the importance of including hydrogen bonds and stacking interactions between consecutive bases as edges in any siRNA molecular graph. Hydrogen bonds and stacking interactions play a key role in any double stranded nucleic acid and thus it seems essential to include them in our model to preserve the right inductive bias grounded in the biochemistry of the molecules we are dealing with. However, there is also a mathematical argument for the inclusion of hydrogen bonds and stacking interactions as edges that gives quantitative results on the consequences of failing to do so for the case of a standard MPNN.

First, we note that the (modified) nucleotides in a siRNA strand can be thought of as graphs in a line of graphs, which would in this case correspond to the strand. Further, observe that given a set of (modified) nucleotides they may be arranged interchangeably to form a strand. We formalize this intuition with the concept of interchangeable graph nodes:

**Definition 40** (Interchangeable Graph Nodes). *We say that a set of graphs  $G_1, \dots, G_l$  (with features) is a set of interchangeable graph nodes (with features) for a line of graphs if:*

- *for every  $G_i \in \{G_1 \dots, G_l\}$  there exist interfaces  $\partial_i^+$ ,  $\partial_i^-$  that are fixed and thus do not depend on its assigned position on the line of graphs*

- *there exist  $s^+, s^-$  such that  $|\partial_i^+| = s^+$  and  $|\partial_i^-| = s^-$  for all  $i \in \{1, \dots, l\}$*
- *for every  $G_i \in \{G_1, \dots, G_l\}$  there exist bijective maps  $o_i^+ : \partial_i^+ \rightarrow \{1, \dots, s^+\}$  and  $o_i^- : \partial_i^- \rightarrow \{1, \dots, s^-\}$  that are fixed and thus do not depend on its assigned position on the line of graphs*

*and there exists a map  $R : \{1, \dots, s^+\} \rightarrow \{M \mid M \subseteq \{1, \dots, s^-\}\}$  such that  $R(k) \neq \emptyset$  for all  $k \in \{1, \dots, s^+\}$  and  $\bigcup_{k \in \{1, \dots, s^+\}} R(k) = \{1, \dots, s^-\}$ , called a rule, which dictates how to connect them: if  $G_i$  and  $G_j$  are consecutive graph nodes ( $\partial_i^+ \rightarrow \partial_j^-$  orientation) in the line of graphs, then the edges between them are  $\bigcup_{u \in \partial_i^+} \{(u, v) \mid v \in (o_j^-)^{-1}(R(o_i^+(u)))\}$ .*

Observe that, even though all interchangeable graph nodes have fixed, designated interfaces, those graph nodes that are sent to the first and last positions will not have one of their interfaces (i.e., one of their interfaces is not "active").

Using the concept of interchangeable graph nodes for a line of graphs, we can prove a result regarding the permutation invariance of certain functions over a graph with the line of graphs structure:

**Lemma 41.** *Let  $G_1, \dots, G_l$  be a set of interchangeable graph nodes with features for a line of graphs with a rule  $R$  such that the features of the edges defined by  $R$  are fixed and assigned according to*

$$\{c_{p,q;R} \mid p \in \{1, \dots, s^+\}, q \in R(p)\}. \quad (3.2)$$

*That is, if  $(u, v)$  satisfies  $u \in \partial_i^+$  and  $v \in (o_j^-)^{-1} \circ R \circ o_i^+(u)$  for some  $i, j$  such that  $G_i, G_j$  are consecutive graph nodes in the line of graphs, then*

$$y_{(u,v)} = c_{o_i^+(u), o_j^-(v); R}.$$

*Let  $\sigma : \{1, \dots, l\} \rightarrow \{1, \dots, l\}$  be a permutation of these building blocks such that it keeps 1 and  $l$  fixed, and let  $H(\sigma, R) = ((\mathcal{V}, \mathcal{E}), (\mathcal{V}^{\text{CG}}, \mathcal{E}^{\text{CG}}))$  be the resulting line of graphs, where  $G(\sigma, R) = (\mathcal{V}, \mathcal{E})$  is the underlying graph,  $\mathcal{V}^{\text{CG}} = \{G_1, \dots, G_l\}$ ,  $(G_{\sigma^{-1}(i)}, G_{\sigma^{-1}(j)}) \in \mathcal{E}^{\text{CG}} \iff i = j \pm 1$ ,*

$$\mathcal{U} = \bigsqcup_{i \in \{1, \dots, l\}} \mathcal{U}(G_{\sigma^{-1}(i)}) \text{ and}$$

$$\mathcal{E} = \bigsqcup_{i \in \{1, \dots, l\}} \mathcal{E}(G_{\sigma^{-1}(i)}) \\ \bigsqcup \left( \bigsqcup_{j \in \{1, \dots, l-1\}} \bigcup_{s \in \partial_{\sigma^{-1}(j)}^+} \{(s, v) \mid v \in (o_{\sigma^{-1}(j+1)}^-)^{-1} \circ R \circ o_{\sigma^{-1}(j)}^+(s)\} \right)$$

If there exists  $r \in \mathbb{N}_{>0}$  and graphs with features  $C^\pm$  with node subsets  $I^\pm$ , where  $|I^\pm| = s^\pm$  are indexed by bijections  $n^\pm : I^\pm \rightarrow \{1, \dots, s^\pm\}$ , such that for any  $i \in \{1, \dots, l\}$  it holds that:

- There exists a bijection  $\phi_i^+ : B^{G_i}(\{(u, r-1)\}_{u \in \partial_i^+}) \rightarrow \mathcal{U}(C^+)$  satisfying  $\phi_i^+ \circ (o_i^+)^{-1}(k) = (n^+)^{-1}(k)$  for any  $k \in \{1, \dots, s^+\}$  which gives

$$(\mathcal{B}^{G_i}(\{(u, r-1)\}_{u \in \partial_i^+}), \partial_i^+) \cong (C^+, I^+)$$

- There exists a bijection  $\phi_i^- : B^{G_i}(\{(v, r-1)\}_{v \in \partial_i^-}) \rightarrow \mathcal{U}(C^-)$  satisfying  $\phi_i^- \circ (o_i^-)^{-1}(k) = (n^-)^{-1}(k)$  for any  $k \in \{1, \dots, s^-\}$  which gives

$$(\mathcal{B}^{G_i}(\{(v, r-1)\}_{v \in \partial_i^-}), \partial_i^-) \cong (C^-, I^-)$$

- $r < 2 + \min_{i \in \{1, \dots, l\}} d^{G_i}(\partial_i^+, \partial_i^-)$

then any function  $f_G(u)$  determined solely by the isomorphism class of the rooted radius- $r$  ball  $(\mathcal{B}_r(u), u)$  as a graph with node and edge features is invariant to the choice of permutation  $\sigma$  with fixed ends  $1, l$ .

*Proof.* For convenience, define  $R^{-1} : \{1, \dots, s^-\} \rightarrow \{M \mid M \subseteq \{1, \dots, s^+\}\}$  in the following way:

$$\text{Given } j \in \{1, \dots, s^-\}, \quad R^{-1}(j) := \{k \in \{1, \dots, s^+\} \mid j \in R(k)\}$$

Observe also that for each  $i \in \{1, \dots, l\}$  it holds that

$$\phi_{i|\partial_i^\pm}^\pm = (n^\pm)^{-1} \circ o_i^\pm \tag{3.3}$$

Let us now tackle the proof. For any node  $u \in G_{i^*}$  with  $i^* \neq 1, l$ , assume that given a permutation  $\sigma$  with fixed ends  $1, l$  and the resulting line of graphs  $G(\sigma, R)$  the graph  $G_{i^*}$  ends up in position  $i$ , i.e.  $\sigma^{-1}(i) = i^*$ .

Let us define the extended graph  $G'_{i^*}(\sigma)$  by "attaching" all the balls of radius  $r - 1$  in the graphs  $\sigma^{-1}(i \pm 1)$  to the interfaces of  $G_{i^*}$ :

$$\begin{aligned} \mathcal{U}(G'_{i^*}(\sigma)) = & \mathcal{U}(G_{i^*}) \\ & \bigsqcup \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i+1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i+1)}^-})\right) \\ & \bigsqcup \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i-1)}}(\{(w, r-1)\}_{w \in \partial_{\sigma^{-1}(i-1)}^+})\right) \end{aligned}$$

and

$$\begin{aligned} \mathcal{E}(G'_{i^*}(\sigma)) = & \mathcal{E}(G_{i^*}) \\ & \bigsqcup \mathcal{E}\left(\mathcal{B}^{G_{\sigma^{-1}(i+1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i+1)}^-})\right) \\ & \bigsqcup \{(s, v) \mid s \in \partial_{i^*}^+, v \in (o_{\sigma^{-1}(i+1)}^-)^{-1} \circ R \circ o_{i^*}^+(s)\} \\ & \bigsqcup \mathcal{E}\left(\mathcal{B}^{G_{\sigma^{-1}(i-1)}}(\{(w, r-1)\}_{w \in \partial_{\sigma^{-1}(i-1)}^+})\right) \\ & \bigsqcup \{(t, w) \mid t \in \partial_{i^*}^-, w \in (o_{\sigma^{-1}(i-1)}^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(t)\} \end{aligned}$$

where we recall that the features of the edges in  $\{(s, v) \mid s \in \partial_{i^*}^+, v \in (o_{\sigma^{-1}(i+1)}^-)^{-1} \circ R \circ o_{i^*}^+(s)\}$  are given by  $c_{o_{i^*}^+(s), o_{\sigma^{-1}(i+1)}^-(v); R}$  and the features of the edges in  $\{(t, w) \mid t \in \partial_{i^*}^-, w \in (o_{\sigma^{-1}(i-1)}^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(t)\}$  are given by  $c_{o_{\sigma^{-1}(i-1)}^+(w), o_{i^*}^-(t); R}$  by assumption.

Next, let us define the generalized extended graph  $\hat{G}_{i^*}$  by "attaching" the graphs  $C^\pm$  to the interfaces of  $G_{i^*}$ :

$$\mathcal{U}(\hat{G}_{i^*}) = \mathcal{U}(G_{i^*}) \bigsqcup \mathcal{U}(C^-) \bigsqcup \mathcal{U}(C^+)$$

and

$$\begin{aligned} \mathcal{E}(\hat{G}_{i^*}) = & \mathcal{E}(G_{i^*}) \\ & \bigsqcup \mathcal{E}(C^-) \\ & \bigsqcup \{(s, v) \mid s \in \partial_{i^*}^+, v \in (n^-)^{-1} \circ R \circ o_{i^*}^+(s)\} \\ & \bigsqcup \mathcal{E}(C^+) \\ & \bigsqcup \{(t, w) \mid t \in \partial_{i^*}^-, w \in (n^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(t)\} \end{aligned}$$

and we set the features of the edges in  $\{(s, v) \mid s \in \partial_{i^*}^+, v \in (n^-)^{-1} \circ R \circ o_{i^*}^+(s)\}$  to be  $c_{o_{i^*}^+(s), n^-(v); R}$  and the features of the edges in  $\{(t, w) \mid t \in \partial_{i^*}^-, w \in (n^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(t)\}$  to be  $c_{n^+(w), o_{i^*}^-(t); R}$ .

Observe that  $I^+$  is acting as the interface of  $C^+$  and  $I^-$  is acting as the interface of  $C^-$  (by the indexing bijections  $n^\pm$ ). This is well defined because  $|I^\pm| = |\partial_{i^*}^\pm| = s^\pm$ .

We are going to prove that

$$(G'_{i^*}(\sigma), a) \cong (\hat{G}_{i^*}, a)$$

for any  $a \in \mathcal{U}(G_{i^*})$ .

Let us define  $\Phi_\sigma : \mathcal{U}(G'_{i^*}(\sigma)) \rightarrow \mathcal{U}(\hat{G}_{i^*})$  as

$$\Phi_\sigma(a) = \begin{cases} a & \text{if } a \in \mathcal{U}(G_{i^*}) \\ \phi_{\sigma^{-1}(i+1)}^-(a) & \text{if } a \in \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i+1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i+1)}^-})\right) \\ \phi_{\sigma^{-1}(i-1)}^+(a) & \text{if } a \in \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i-1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i-1)}^+})\right) \end{cases}$$

which is a well-defined bijection and preserves node features because, by assumption,  $(\mathcal{B}^{G_{\sigma^{-1}(i+1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i+1)}^-}), \partial_{\sigma^{-1}(i+1)}^-) \cong (C^-, I^-)$  and also  $(\mathcal{B}^{G_{\sigma^{-1}(i-1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i-1)}^+}), \partial_{\sigma^{-1}(i-1)}^+) \cong (C^+, I^+)$ .

Observe that  $\Phi_\sigma(v) = v$  for all  $v \in \mathcal{U}(G_{i^*})$ ,  $\Phi_\sigma(\partial_{\sigma^{-1}(i+1)}^-) = I^-$  and  $\Phi_\sigma(\partial_{\sigma^{-1}(i-1)}^+) = I^+$ .

Let  $a, b \in \mathcal{U}(G'_{i^*}(\sigma))$ . If either  $a, b \in \mathcal{U}(G_{i^*})$ ,  $a, b \in \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i+1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i+1)}^-})\right)$  or  $a, b \in \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i-1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i-1)}^+})\right)$ , it is clear that  $(a, b) \in$

$\mathcal{E}(G'_{i^*}(\sigma)) \iff (\Phi_\sigma(a), \Phi_\sigma(b)) \in \mathcal{E}(\hat{G}_{i^*})$  and edge features are preserved by the assumed isomorphisms.

If  $a \in \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i+1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i+1)}^-})\right) \iff \Phi_\sigma(a) \in \mathcal{U}(C^-)$  and  $b \in \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i-1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i-1)}^+})\right) \iff \Phi_\sigma(b) \in \mathcal{U}(C^+)$ , then clearly  $a, b$  do not form an edge and neither do  $\Phi_\sigma(a)$  and  $\Phi_\sigma(b)$ .

If  $a \in \mathcal{U}\left(\mathcal{B}^{G_{\sigma^{-1}(i \pm 1)}}(\{(v, r-1)\}_{v \in \partial_{\sigma^{-1}(i \pm 1)}^\mp})\right) \iff \Phi_\sigma(a) \in \mathcal{U}(C^\mp)$  and  $b = \Phi_\sigma(b) \in \partial_{i^*}^\mp$ , then clearly  $a, b$  do not form an edge and neither do  $\Phi_\sigma(a)$  and  $\Phi_\sigma(b)$ .

There are also no edges if  $a = \Phi_\sigma(a) \in G_{i^*} \setminus \partial_{i^*}^+ \setminus \partial_{i^*}^-$  and  $b \in G'_{i^*}(\sigma) \setminus G_{i^*} \iff \Phi_\sigma(b) \in \hat{G}_{i^*} \setminus G_{i^*}$ . Finally, observe that:

1. Let  $(a, b) \in \{(s, v) \mid s \in \partial_{i^*}^+, v \in (o_{\sigma^{-1}(i+1)}^-)^{-1} \circ R \circ o_{i^*}^+(s)\}$  and, WLOG, let  $a \in \partial_{i^*}^+$ . Then, by the definition of  $\Phi_\sigma$  and (3.3) it holds that  $(\Phi_\sigma(a), \Phi_\sigma(b)) = (a, \phi_{\sigma^{-1}(i+1)}^-(b)) = (a, (n^-)^{-1} \circ o_{\sigma^{-1}(i+1)}^-(b))$ .

Therefore,  $b \in (o_{\sigma^{-1}(i+1)}^-)^{-1} \circ R \circ o_{i^*}^+(a)$  if and only if

$$\Phi_\sigma(b) \in (n^-)^{-1} \circ o_{\sigma^{-1}(i+1)}^- \circ (o_{\sigma^{-1}(i+1)}^-)^{-1} \circ R \circ o_{i^*}^+(a) = (n^-)^{-1} \circ R \circ o_{i^*}^+(a),$$

this is, if and only if

$$(\Phi_\sigma(a), \Phi_\sigma(b)) \in \{(s, v) \mid s \in \partial_{i^*}^+, v \in (n^-)^{-1} \circ R \circ o_{i^*}^+(s)\}.$$

Further, note that we can also write  $o_{\sigma^{-1}(i+1)}^-(b) = n^- \circ \phi_{\sigma^{-1}(i+1)}^-(b) = n^- \circ \Phi_\sigma(b)$  as a consequence of (3.3) (obtained by applying  $n^-$  on the left on both sides of the equality). Thus

$$\mathcal{Y}_{(a,b)} = c_{o_{i^*}^+(a), o_{\sigma^{-1}(i+1)}^-(b); R} = c_{o_{i^*}^+(a), n^-(\Phi_\sigma(b)); R} = \mathcal{Y}_{(\Phi_\sigma(a), \Phi_\sigma(b))}$$

2. Let  $(a, b) \in \{(t, w) \mid t \in \partial_{i^*}^-, w \in (o_{\sigma^{-1}(i-1)}^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(t)\}$  and, WLOG, let  $a \in \partial_{i^*}^-$ . Then, by the definition of  $\Phi_\sigma$  and (3.3) it holds that  $(\Phi_\sigma(a), \Phi_\sigma(b)) = (a, \phi_{\sigma^{-1}(i-1)}^+(b)) = (a, (n^+)^{-1} \circ o_{\sigma^{-1}(i-1)}^+(b))$ .

Therefore,  $b \in (o_{\sigma^{-1}(i-1)}^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(a)$  if and only if

$$\Phi_\sigma(b) \in (n^+)^{-1} \circ o_{\sigma^{-1}(i-1)}^+ \circ (o_{\sigma^{-1}(i-1)}^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(a) = (n^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(a),$$

this is, if and only if

$$(\Phi_\sigma(a), \Phi_\sigma(b)) \in \{(t, w) \mid t \in \partial_{i^*}^-, w \in (n^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(t)\}.$$

Further, note that we can also write  $o_{\sigma^{-1}(i-1)}^+(b) = n^+ \circ \phi_{\sigma^{-1}(i-1)}^+(b) = n^+ \circ \Phi_\sigma(b)$  as a consequence of (3.3) (obtained by applying  $n^+$  on the left on both sides of the equality). Thus

$$y_{(a,b)} = c_{o_{\sigma^{-1}(i-1)}^+(b), o_{i^*}^-(a); R} = c_{n^+(\Phi_\sigma(b)), o_{i^*}^-(a); R} = y_{(\Phi_\sigma(a), \Phi_\sigma(b))}$$

We conclude that  $(a, b) \in \mathcal{E}(G'_{i^*}(\sigma)) \iff (\Phi_\sigma(a), \Phi_\sigma(b)) \in \mathcal{E}(\hat{G}_{i^*})$ , and  $\Phi_\sigma$  preserves edge features.

This concludes the proof that  $(G'_{i^*}(\sigma), a) \cong (\hat{G}_{i^*}, a)$  for any  $a \in \mathcal{U}(G_{i^*})$ .

Consider now  $\mathcal{B}_r(u)$  (if we do not use a superscript, we are referring to the ball in the underlying graph  $G(\sigma, R)$  of the whole line graph  $H(\sigma, R)$ ). We claim that  $\mathcal{B}_r(u) \leq G'_{i^*}(\sigma)$ .

First, let us prove that  $\mathcal{B}_r(u) \subseteq \mathcal{U}(G'_{i^*}(\sigma))$  by contradiction: suppose there is a node  $a$  such that  $a \in \mathcal{B}_r(u)$  but  $a \notin \mathcal{U}(G'_{i^*}(\sigma))$ . It is clear that  $a$  cannot be in  $G_{i^*}$  (as  $\mathcal{U}(G_{i^*}) \subseteq \mathcal{U}(G'_{i^*}(\sigma))$ ). Observe that  $a$  cannot lie beyond  $G_{\sigma^{-1}(i+1)}, G_{\sigma^{-1}(i-1)}$  either: for this to happen, any path from  $u$  to  $a$  would have to cross two graph interfaces and either  $G_{\sigma^{-1}(i+1)}$  or  $G_{\sigma^{-1}(i-1)}$ , so

$$d(u, a) \geq 2 + \min_{j \in \{\sigma^{-1}(i-1), \sigma^{-1}(i+1)\}} d^{G_j}(\partial_j^+, \partial_j^-) \geq 2 + \min_{j \in \{1, \dots, l\}} d^{G_j}(\partial_j^+, \partial_j^-) > r,$$

by assumption, which contradicts the initial claim that  $a \in \mathcal{B}_r(u)$ .

Therefore, the only possibility left is to have  $a \in G_{\sigma^{-1}(i \pm 1)}$  but  $d^{G_{\sigma^{-1}(i \pm 1)}}(a, \partial_{\sigma^{-1}(i \pm 1)}^\mp) > r - 1$ . Observe that

$$d(u, a) \leq r < 2 + \min_{j \in \{1, \dots, l\}} d^{G_j}(\partial_j^-, \partial_j^+) \leq 2 + d^{G_{\sigma^{-1}(i \pm 1)}}(\partial_{\sigma^{-1}(i \pm 1)}^-, \partial_{\sigma^{-1}(i \pm 1)}^+) \quad (3.4)$$

by assumption, and therefore we can apply Lemma 39 to get that

$$d(u, a) \geq 1 + d^{G_{\sigma^{-1}(i \pm 1)}}(a, \partial_{\sigma^{-1}(i \pm 1)}^\mp) > 1 + r - 1 = r,$$

which again contradicts the initial claim that  $a \in B_r(u)$ . We conclude that

$$B_r(u) \subseteq \mathcal{U}(G'_{i^*}(\sigma))$$

Next, we prove that  $\mathcal{E}(B_r(u)) \subseteq \mathcal{E}(G'_{i^*}(\sigma))$ . There are three possible cases:

1.  $(a, b) \in \mathcal{E}(B_r(u)) \cap \mathcal{E}(G_{i^*})$ . Then, clearly  $(a, b) \in \mathcal{E}(G_{i^*}) \subseteq \mathcal{E}(G'_{i^*}(\sigma))$ .
2.  $(a, b) \in \mathcal{E}(B_r(u))$  is an edge between graph nodes. Then,

$$(a, b) \in \{(s, v) \mid s \in \partial_{i^*}^+, v \in (o_{\sigma^{-1}(i+1)}^-)^{-1} \circ R \circ o_{i^*}^+(s)\} \\ \sqcup \{(t, w) \mid t \in \partial_{i^*}^-, w \in (o_{\sigma^{-1}(i-1)}^+)^{-1} \circ R^{-1} \circ o_{i^*}^-(t)\}$$

3.  $(a, b) \in \mathcal{E}(B_r(u)) \cap \mathcal{E}(G_{\sigma^{-1}(i \pm 1)})$ . We already know  $a, b \in \mathcal{U}(G'_{i^*}(\sigma))$ . Suppose  $(a, b) \notin \mathcal{E}(G'_{i^*}(\sigma))$ . Then, it must hold that there does not exist any  $v \in \partial_{\sigma^{-1}(i \pm 1)}^\mp$  such that  $\min\{d^{G_{\sigma^{-1}(i \pm 1)}}(a, v), d^{G_{\sigma^{-1}(i \pm 1)}}(b, v)\} < r - 1$ .

Observe that  $\min\{d(a, u), d(b, u)\} < r$  (as  $(a, b) \in \mathcal{E}(B_r(u))$ ). WLOG, suppose  $a < r$ . As (3.4) also holds here, by Lemma 39 we have that

$$d(a, u) \geq 1 + d^{G_{\sigma^{-1}(i \pm 1)}}(a, \partial_{\sigma^{-1}(i \pm 1)}^\mp) \geq 1 + r - 1 = r$$

because we have assumed  $\min\{d^{G_{\sigma^{-1}(i \pm 1)}}(a, w), d^{G_{\sigma^{-1}(i \pm 1)}}(b, w)\} \geq r - 1$ . However, this is impossible, as we started from  $d(a, u) < r$ !

Therefore, it must hold that  $(a, b) \in \mathcal{E}(G'_{i^*}(\sigma))$ .

We have therefore seen that  $B_r(u) \preceq G'_{i^*}(\sigma)$ , this is,  $B_r(u)$  is contained in  $G'_{i^*}(\sigma)$ .

This gives us the following containment chain:

$$B_r(u) \preceq G'_{i^*}(\sigma) \preceq G(\sigma, R)$$

and therefore  $d(u, a) \leq d^{G'_{i^*}(\sigma)}(u, a) \leq d^{B_r(u)}(u, a)$  for all  $a \in B_r(u)$  (this inequality in fact holds true for any two nodes in  $B_r(u)$ ).

However, from Lemma 21 we know that  $d^{\mathcal{B}_r(u)}(u, a) = d(u, a)$  for all  $a \in \mathcal{B}_r(u)$ , which allows us to conclude

$$d(u, a) = d^{G'_{i^*}(\sigma)}(u, a) = d^{\mathcal{B}_r(u)}(u, a) \quad \forall a \in \mathcal{B}_r(u).$$

This ensures  $\mathcal{B}_r(u) = \mathcal{B}_r^{G'_{i^*}(\sigma)}(u)$ .

Further,  $\mathcal{E}(\mathcal{B}_r(u)) \subseteq \mathcal{E}(G'_{i^*}(\sigma))$  and whether an edge  $(a, b) \in \mathcal{E}$  is an edge of  $\mathcal{B}_r(u)$  is uniquely determined by the distances of the participating nodes to  $u$ :

$$\begin{aligned} (a, b) \in \mathcal{E}(\mathcal{B}_r(u)) &\iff \min\{d(u, a), d(u, b)\} < r \\ &\iff \min\{d^{G'_{i^*}(\sigma)}(u, a), d^{G'_{i^*}(\sigma)}(u, b)\} < r \iff (a, b) \in \mathcal{E}(\mathcal{B}_r^{G'_{i^*}(\sigma)}(u)) \end{aligned}$$

for  $(a, b) \in \mathcal{E}$ .

We conclude that  $\mathcal{B}_r(u) = \mathcal{B}_r^{G'_{i^*}(\sigma)}(u)$ .

Finally, Lemma 27 states that

$$(\mathcal{B}_r(u), u) = (\mathcal{B}_r^{G'_{i^*}(\sigma)}(u), u) \cong (\mathcal{B}_r^{\hat{G}_{i^*}}(u), u)$$

where  $\mathcal{B}_r^{\hat{G}_{i^*}}(u)$  does not depend on the permutation  $\sigma$ .

For a node in  $G_1$  or  $G_l$ , the argument is exactly the same with the exception that the constructed graphs  $G'_1(\sigma), \hat{G}_1$  would only take the union of balls around the interface  $\partial_{\sigma^{-1}(2)}^-$  and  $C^-$ , respectively; and  $G'_l(\sigma), \hat{G}_l$  would only take the union of balls around the interface  $\partial_{\sigma^{-1}(l-1)}^+$  and  $C^+$ , respectively.

We conclude that  $f$  yields the same results no matter the fixed-ends permutation used.  $\square$

Notice that the functions that are proven to be invariant under the appropriate conditions by Lemma 41 include the message passing block of a MPNN with  $r$  layers or molecular fingerprints with radius  $r$ . Interestingly, both of these function instances have in common the fact that increasing the parameter  $r$  can have negative effects on their behaviour: on MPNNs this is caused by over-smoothing, while in molecular fingerprints it is caused by colliding hashes.

However, the lower the parameter  $r$  in these functions the easier it is for the conditions on Lemma 41 to be verified. In particular, having  $r \leq 5$  and a chain of nucleotides which satisfy the following conditions:

- all nucleotides share the same backbone

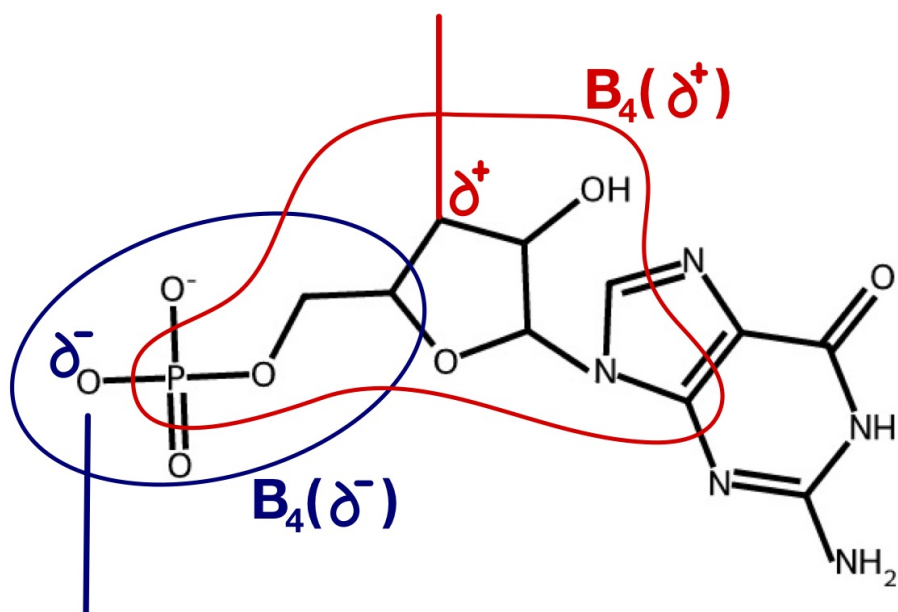


Figure 3.2: Ball of radius 4 around the generic  $\delta^+$  (in red) and  $\delta^-$  (in blue) interfaces of a **RNA** G nucleotide in a strand with direction 5' to 3'. Any nucleotide with the same backbone and sugar but an arbitrary base – as long as it has a Nitrogen which binds it to the sugar and has two Carbon neighbours in the base – would yield the same balls.

- all nucleotides share the same sugar
- all nucleotides have a base such that the Nitrogen which binds the base to the sugar has two Carbon neighbours in the base. This condition is not particularly restrictive: for example, it holds for all standard bases A, C, G, U, T.

is enough for the conditions in Lemma 41 to hold, as the balls of radius 4 at the interfaces only encompass the backbone and sugar fragments of every nucleotide, and only go two steps into the base - see Figure 3.2. As a result, by Lemma 41 it holds that the message passing block of a **MPNN** with 5 layers or a molecular fingerprint with radius 5 on a nucleic acid strand with fixed ends would yield the same representation for any node in the chain's inner

nucleotides irrespective of the order in which these appear! This is problematic because both **MPNNs** and molecular fingerprints have a final pooling layer that is invariant to permutations of the node labels, which implies that, for example, they would yield the same result for two strands *ACGAGA* and *AGGCAA* - but we know those two strands may have completely different properties and biological function!

We have established that Lemma 41 issues a word of caution against the naive use of standard **MPNNs** or molecular fingerprints on separate nucleic acid strands. However, this result does not hold if the backbone and sugar fragments used along the strand show sufficient variability, a characteristic we expect from chemically modified **siRNA**. If we also include hydrogen bonds and consider each nucleotide pair as an interchangeable graph node in the chain, then having both strands share the same backbone and sugar fragments (each strand could have their own, but they would need to be shared among the whole strand) seems even more unlikely. Furthermore, one could accept the drawbacks of increasing the number of message-passing layers to ensure the conditions in Lemma 41 do not hold.

So, is there no reason to worry after all? Yes there is. The problem here is that there are further issues with the use of standard **MPNNs** on double stranded **siRNA** molecular graphs which these proposed solutions do not address. The most relevant of these issues is over-squashing, which we would intuitively expect to become quite severe as the length of the chain increases and the information of more and more atoms belonging to different nucleotides gets compressed into a finite-length vector. We are going to prove that, for standard **MPNNs**, over-squashing is indeed exponential on the distance between nodes. To do so, we first need some theoretical results regarding graphs of graphs.

As stated in Lemma 33, over-squashing in standard **MPNNs** between two nodes at a distance  $k$  is bounded by the  $k$ -th power of the normalized adjacency matrix. Given two nodes  $u, v$  such that  $d(u, v) = k$ , the  $(u, v)$ -entry of the  $k$ -th power of the normalized adjacency matrix is given by

$$[\hat{A}^k]_{u,v} = \sum_{p=(w_0, \dots, w_{k-1}) \mid w_0=u, w_{k-1}=v} \prod_{r=1}^{k-1} \frac{1}{\sqrt{d_{w_{r-1}} d_{w_r}}} \quad (3.5)$$

$$= \frac{1}{\sqrt{d_u d_v}} \sum_{p=(w_0, \dots, w_{k-1}) \mid w_0=u, w_{k-1}=v} \prod_{r=1}^{k-2} \frac{1}{d_{w_r}} \quad (3.6)$$

In a normal graph, we can upper bound the number of geodesic paths between

two nodes at a distance  $k$  by  $\Delta(\Delta - 1)^{k-2}$ , where  $\Delta$  is the maximum degree of a graph (simply using the fact that a geodesic path cannot go back to the node it came from at any step). If we use this estimate together with the observation that the degree of any node in a path that is not one of the extremes must have degree greater or equal than 2, we can bound Equation (3.6) by

$$[\hat{A}^k]_{u,v} \leq \frac{1}{\sqrt{d_u d_v}} \cdot n_g(u, v) \cdot 2^{-k+2} \leq \frac{1}{\sqrt{d_u d_v}} \cdot \Delta(\Delta - 1)^{k-2} \cdot 2^{-k+2} \quad (3.7)$$

where  $n_g(u, v)$  denotes the number of geodesic paths between nodes  $u$  and  $v$ .

In many cases, this bound is extremely loose. We want to take advantage of the graph of graphs structure to give a tighter bound that can in turn help us bound the powers of the normalized adjacency matrix.

In order to derive such a bound, let us first introduce the following notation for a graph of graphs  $H = ((\mathcal{V}, \mathcal{E}), (\mathcal{V}^{CG}, \mathcal{E}^{CG}))$ :

- $n_g(u, v) = \# \text{geodesic paths between } u, v$
- Given  $w, y \in G_i$ ,

$$n_g^{G_i}(w, y) = \# \text{geodesic paths in } G_i \text{ between } w, y$$

- $M_g$  is an upper bound on the number of geodesic paths traversing any graph node on the graph of graphs  $H$ ,

$$\begin{aligned} M_g = & \max_{i \in \{1, \dots, N\}} \max_{j, k \mid (G_i, G_j), (G_i, G_k) \in \mathcal{E}^{CG}} \\ & |\partial_{G_i}(G_j) \cap \partial_{G_i}(G_k)| \cdot \mathbb{1}(j \neq k) + \sum_{w \in \partial_{G_i}(G_j), y \in \partial_{G_i}(G_k) \mid w \neq y} n_g^{G_i}(w, y) \\ & + |\{w_0 \in \partial_{G_i}(G_j) \mid \exists w_1 \neq w_2 \in \partial_{G_j}(G_i) \cap N(w_0)\}| \cdot \mathbb{1}(j = k) \end{aligned}$$

where we admit  $j = k$ .

- $\partial_{G_i}$  is the union of all interfaces of the coarse-grained node  $G_i$ ,

$$\partial_{G_i} = \bigcup_{j \mid (G_i, G_j) \in \mathcal{E}^{CG}} \partial_{G_i}(G_j) \quad \forall i \in \{1, \dots, N\}$$

- Given  $k, m, n$  such that  $k \neq m, k \neq n$ , the set  $\mathcal{D}(k, m, n)$  is defined as

$$\mathcal{D}(k, m, n) = \begin{cases} \partial_{G_k}(G_m) \cap \partial_{G_k}(G_n) & \text{if } m \neq n \\ \{w_0 \in \partial_{G_k}(G_m) \mid \exists w_1 \neq w_2 \in \partial_{G_m}(G_k) \cap N(w_0)\} & \text{if } m = n \end{cases}$$

- $g_{min}$  is the minimum length that a geodesic path may "spend" on any intermediary coarse-grained node of a graph of graphs:

$$g_{min} = \begin{cases} 0 & \text{if } \exists k, m, n \text{ with } k \neq m, k \neq n \text{ st } \mathcal{D}(k, m, n) \neq \emptyset \\ \min_{i \in \{1, \dots, N\}} \min_{w, y \in \partial_{G_i} \mid w \neq y} d_{G_i}(w, y) & \text{otherwise} \end{cases}$$

- $g_{max}$  is the maximum length that a geodesic path may "spend" on any intermediary coarse-grained node of a graph of graphs:

$$g_{max} = \max_{i \in \{1, \dots, N\}} \max_{w, y \in \partial_{G_i} \mid w \neq y} d_{G_i}(w, y)$$

We are now ready to prove the following Lemma:

**Lemma 42.** Let  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{CG}, \mathcal{E}^{CG}))$  be a graph of graphs where  $G = (\mathcal{U}, \mathcal{E})$  is the underlying graph,  $\mathcal{U}^{CG} = \{G_1, \dots, G_N\}$ , and let  $u \in \mathcal{U}_{i^*}$  and  $v \in \mathcal{U}_{j^*}$  be two nodes such that  $d(u, v) = k$ .

Let  $\Delta_{CG}$  be the maximum degree of the coarse grained graph  $(\mathcal{U}^{CG}, \mathcal{E}^{CG})$ , and assume  $\Delta_{CG} > 1$ .

Then, there exist functions  $U(u, v)$  and  $L(u, v)$  such that

$$n_g(u, v) \leq \left( \max_{k \mid (G_{i^*}, G_k) \in \mathcal{E}^{CG}} \sum_{w \in \partial_{G_{i^*}}(G_k)} n_g^{G_{i^*}}(u, w) \right) \quad (3.8)$$

$$\cdot \left( \max_{k \mid (G_{j^*}, G_k) \in \mathcal{E}^{CG}} \sum_{w \in \partial_{G_{j^*}}(G_k)} n_g^{G_{j^*}}(v, w) \right) \quad (3.9)$$

$$\cdot \frac{(M_g \Delta_{CG})^{L(u, v)-1} - (M_g \Delta_{CG})^{U(u, v)}}{1 - M_g \Delta_{CG}}, \quad (3.10)$$

where  $L(u, v)$  is given by

$$L(u, v) = \lceil \max\{0, \frac{k + g_{max} - \max_{w \in \partial_{G_{i^*}}} d(u, w) - \max_{w \in \partial_{G_{j^*}}} d(v, w)}{g_{max} + 1}\} \rceil$$

and  $U(u, v)$  is given by

$$U(u, v) = \lfloor \frac{k + g_{\min} - d(u, \partial G_{i^*}) - d(v, \partial G_{j^*})}{g_{\min} + 1} \rfloor.$$

*Proof.* Recall that for any geodesic path  $p$  between  $u, v$  we can obtain its associated coarse-grained walk  $p^{\text{CG}} = (G_{p_0}, \dots, G_{p_l})$  where  $G_{p_0} = G_{i^*}$  and  $G_{p_l} = G_{j^*}$ . Crucially, note that the section of path  $p$  in graph  $G_k = G_{p_s} \in p^{\text{CG}}$  must have been a geodesic path between the first and last nodes it visited on  $G_k$ . Thus, we can derive that the number of geodesic paths with an associated coarse-grained walk of coarse-grained length  $l$  ( $G_{p_0} = G_{i^*}, \dots, G_{p_l} = G_{j^*}$ ) is bounded above by

$$\left( \max_{k \mid (G_{i^*}, G_k) \in \mathcal{E}^{\text{CG}}} \sum_{w \in \partial_{G_{i^*}}(G_k)} n_g^{G_{i^*}}(u, w) \right) \cdot M_g^{l-1} \quad (3.11)$$

$$\cdot \left( \max_{k \mid (G_{j^*}, G_k) \in \mathcal{E}^{\text{CG}}} \sum_{w \in \partial_{G_{j^*}}(G_k)} n_g^{G_{j^*}}(v, w) \right) \quad (3.12)$$

Note that if we were given a coarse grained path of coarse-grained length  $l$  associated to a geodesic path between the two nodes  $u, v$  and we did not have access to  $d(u, v) = k$ , we could bound this distance (i.e., the length of the geodesic path) as

$$\begin{aligned} l + d(u, \partial G_{i^*}) + d(v, \partial G_{j^*}) + (l-1)g_{\min} &\leq k \\ &\leq l + \max_{w \in \partial_{G_{i^*}}} d(u, w) + \max_{w \in \partial_{G_{j^*}}} d(v, w) + (l-1)g_{\max} \end{aligned}$$

because  $g_{\min}$  and  $g_{\max}$  give us the minimum and maximum length that a geodesic path traversing a graph in the graphs of graphs may have, and  $l$  is the number of inter-graph edges of the path.

Thus, we can rework this bounds to obtain a bound for the possible lengths  $l$  that a coarse-grained walk associated to a geodesic path between  $u$  and  $v$  can

have:

$$\begin{aligned} & \max\{0, \frac{k + g_{\max} - \max_{w \in \partial G_{i^*}} d(u, w) - \max_{w \in \partial G_{j^*}} d(v, w)}{g_{\max} + 1}\} \\ & \leq l \leq \\ & \frac{k + g_{\min} - d(u, \partial G_{i^*}) - d(v, \partial G_{j^*})}{g_{\min} + 1} \end{aligned}$$

Let

$$U(u, v) = \lfloor \frac{k + g_{\min} - d(u, \partial G_{i^*}) - d(v, \partial G_{j^*})}{g_{\min} + 1} \rfloor \quad (3.13)$$

$$L(u, v) = \lceil \max\{0, \frac{k + g_{\max} - \max_{w \in \partial G_{i^*}} d(u, w) - \max_{w \in \partial G_{j^*}} d(v, w)}{g_{\max} + 1}\} \rceil \quad (3.14)$$

Then, the number of geodesic paths can be upper bounded by

$$\begin{aligned} & \sum_{l=L(u,v)}^{U(u,v)} \Delta_{CG}^{l-1} \cdot \left( \max_{k \mid (G_{i^*}, G_k) \in \mathcal{E}^{CG}} \sum_{w \in \partial_{G_{i^*}}(G_k)} n_g^{G_{i^*}}(u, w) \right) \cdot M_g^{l-1} \\ & \cdot \left( \max_{k \mid (G_{j^*}, G_k) \in \mathcal{E}^{CG}} \sum_{w \in \partial_{G_{j^*}}(G_k)} n_g^{G_{j^*}}(v, w) \right) \end{aligned}$$

because for every  $G_k$  with  $k \neq 0, l$  in a coarse grained path of coarse grained length  $l$  we have at most  $\Delta_{CG}$  options to choose (recall that, by assumption,  $\Delta_{CG} > 1$ ), and for every possible resulting coarse-grained walk the maximum number of geodesic paths that it could be associated to is bounded by (3.11).

We can simplify this summation to obtain

$$\begin{aligned} n_g(u, v) & \leq \left( \max_{k \mid (G_{i^*}, G_k) \in \mathcal{E}^{CG}} \sum_{w \in \partial_{G_{i^*}}(G_k)} n_g^{G_{i^*}}(u, w) \right) \\ & \cdot \left( \max_{k \mid (G_{j^*}, G_k) \in \mathcal{E}^{CG}} \sum_{w \in \partial_{G_{j^*}}(G_k)} n_g^{G_{j^*}}(v, w) \right) \\ & \cdot \frac{(M_g \Delta_{CG})^{L(u,v)-1} - (M_g \Delta_{CG})^{U(u,v)}}{1 - M_g \Delta_{CG}} \end{aligned}$$

□

While we are not aware of this exact upper bound in the literature, the proof follows a standard decomposition principle: geodesics in the full graph induce walks in a coarse graph of components (graph of graphs here), and each traversal contributes a bounded number of internal geodesic realizations through boundary vertices.

Let us compare the predictions of this formula with the standard  $\Delta(\Delta - 1)^{k-2}$  bound for a double stranded **siRNA** molecule, where each nucleotide pair is a graph in the graph of graphs and hydrogen atoms have been removed (standard practice). For simplicity, suppose we have unmodified nucleotides only. Then,  $g_{min} = 5$  (achieved between the oxygen and the carbon in  $\partial^+$  and  $\partial^-$  of each nucleotide),  $g_{max} = 21$  (achieved between the two oxygen atoms which belong to  $\partial^-$  for the nucleotide in the guide strand and  $\partial^+$  for the nucleotide in the passenger strand),  $M_g = 3$  (achieved for a C – G pair, where there are three possible geodesic paths crossing the C-G base pair which result in 3 possible geodesic paths between any two atoms in  $\partial^+ \cup \partial^-$  such that each atom belongs to a different nucleotide in the base pair) and  $\Delta_{CG} = 2$  because we have a line of graphs. Figure 3.3 illustrates how these constants are obtained.

Suppose we have  $u, v$  such that  $d(u, v) = k$ . The maximum degree is achieved, for example, in the phosphorus atom in the phosphate fragment, which has degree 4. Then, the standard bound for the number of geodesics between two nodes gives us the bound  $4 \cdot 3^{k-2}$  for the number of geodesic paths. If we were to use this to bound  $[\hat{A}^k]_{u,v}$  with formula (3.6), using that the degree of every node in a geodesic path is bigger or equal than two (see (3.7)) we would get that  $[\hat{A}^k]_{u,v} \leq \frac{4}{\sqrt{d_u d_v}} (\frac{3}{2})^{k-2}$ , which allows exponential growth with  $k$ . However, we know this is impossible because the normalized adjacency satisfies  $\rho(\hat{A}) = 1$ .

Let us use the formula we have just derived in Lemma 42 instead. For simplicity, assume both  $u$  and  $v$  sit at the interface of their respective nucleotide pairs. Then,  $L(u, v) = \lceil \frac{k-21}{22} \rceil$  and  $U(u, v) = \lfloor \frac{k+5}{6} \rfloor$ . and the number of geodesic paths between  $u, v$  is bound by  $9 \cdot \frac{6^{U(u,v)} - 6^{L(u,v)-1}}{5} \leq \frac{9}{5} \cdot 6^{\frac{5}{6} \frac{k}{6}} \leq \frac{9}{5} \cdot \frac{9}{2} \cdot 2^{\frac{k}{2}} \leq 9 \cdot 2^{\frac{k}{2}}$ . Then, taking bounds in (3.6) we get that

$$[\hat{A}^k]_{u,v} \leq \frac{9}{\sqrt{d_u d_v}} 2^{\frac{k}{2} - k + 2} \leq \frac{36}{\sqrt{d_u d_v}} 2^{-\frac{k}{2}}$$

which shows, even after some loose bounds, that the over-squashing suffered

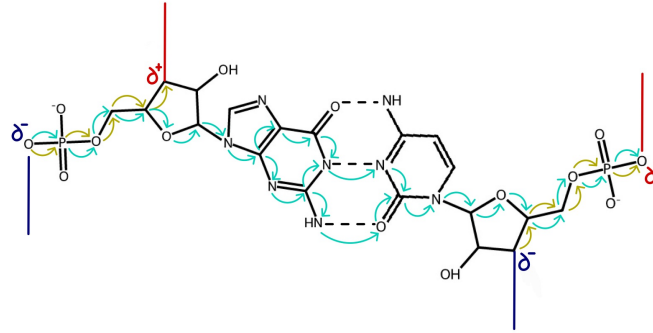


Figure 3.3: G-C base pair with generic  $\partial^+$  (in red) and  $\partial^-$  (in blue) interfaces. Hydrogen bonds between the two nucleotides are included as edges (dashed lines). Hydrogen atoms are implicit and not included as graph nodes. The golden arrows showcase the two paths for which  $g_{\min} = 5$  is obtained. The cyan arrows showcase the three paths for which  $g_{\max} = 21$  is obtained. These three paths also reveal the three possible geodesic crossings from one nucleotide in the base pair to the other; which dictate the maximum number of geodesic paths between any two nodes in  $\partial^+ \cup \partial^-$ , i.e.  $M_g = 3$ . See Lemma 42 for the definition of  $g_{\min}$ ,  $g_{\max}$  and  $M_g$

by a standard MPNN in a double stranded siRNA molecular graph with added edges to represent hydrogen bonds is exponential.

We now have all the elements to mathematically justify the importance of the inclusion of both hydrogen bonds and stacking interactions between consecutive bases as edges in our molecular graph. Lemma 41 highlights the pitfalls of considering strands separately, and even though its conditions are harder to verify once we include hydrogen bonds between the two chains, it remains a warning sign that something is missing. It is clear that two nodes  $u, v$  each in one of the strands would have no influence on each other, i.e.  $\frac{\partial [X_r]_v}{\partial x_u} = 0$  and vice versa for all  $r \in \{0, \dots, L\}$  where  $L$  is the number of layers. However, Lemma 42 reveals that a double stranded siRNA molecular graph including hydrogen bonds would still show exponential over-squashing.

Adding edges to represent stacking interactions between consecutive bases partially solves these problems. Once edges representing stacking interactions are included, the conditions in Lemma 41 cannot hold because the balls around some of the nodes in each graph's interface (here, graph = nucleotide

pair) include the nucleotide pair's bases, which must be different along the chain unless every base in the chain is the same (in which case permutation invariance is unavoidable). Furthermore, adding edges to represent stacking interactions between consecutive bases can push  $g_{min} = 0$  (for example, we can achieve this by drawing an edge between all the atoms in consecutive bases participating in a hydrogen bond), which renders the bound given by Lemma 42 useless and facilitates inter-nucleotide message-passing.

It is therefore safe to conclude that we have mathematically grounded reasons to impose the use of edges that represent hydrogen bonds and stacking interaction between consecutive bases in **siRNA** molecular graphs.

### 3.3.1.2 Further results on over-squashing in a standard Message Passing Neural Network

The inclusion of stacking interactions between consecutive bases as edges in **siRNA** molecular graphs mitigates over-squashing in standard **MPNNs**, but it does not solve the problem entirely. We are going to see that over-squashing remains an issue for standard **MPNNs** even after the inclusion of stacking interactions.

To do this, let us first introduce the concept of a highway in a graph of graphs.

**Definition 43.** Given a graph of graphs  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{CG}, \mathcal{E}^{CG}))$  with  $\mathcal{U}^{CG} = \{G_1, \dots, G_N\}$  where the coarse-grained graph  $(\mathcal{U}^{CG}, \mathcal{E}^{CG})$  is connected, we say that a sub-graph  $W$  of the underlying graph  $G = (\mathcal{U}, \mathcal{E})$  is a highway if

- For all  $G_i \in \mathcal{U}^{CG}$ , there exists  $u \in W \cap G_i$
- For all  $G_i \in \mathcal{U}^{CG}$  and for all  $G_j \in N^{CG}(G_i)$  it holds that  $\forall u \in G_i \cap W, \forall v \in G_j \cap W$  we have  $(u, v) \in \mathcal{E}$

Observe that connecting all the nodes participating in hydrogen bonds in one nucleotide pair to all the nodes participating in hydrogen bonds in the adjacent node pairs creates a highway in the line of graphs that is our **siRNA** molecular graph, as illustrated in Figure 3.4.

This is essentially the best possible case for over-squashing that still preserves physical inductive bias (i.e., we are not indiscriminately drawing edges between nodes), as once messages reach this highway they only take one step to travel between consecutive nucleotide pairs. However, it still yields exponential over-squashing on the coarse-grained distance:

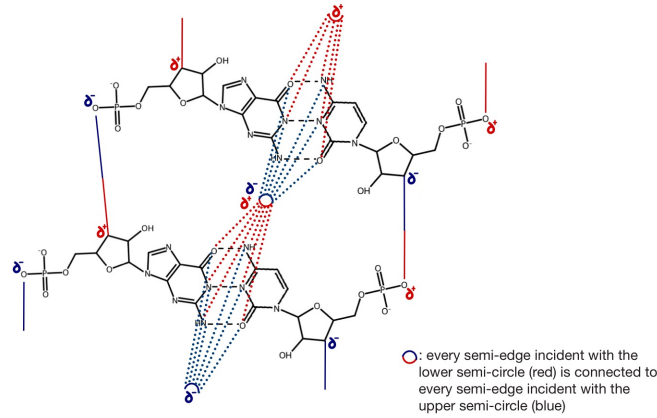


Figure 3.4: Molecular highway on two consecutive G-C base pairs. Hydrogen bonds between the two nucleotides in each base pair are included as edges (dashed lines). Hydrogen atoms are implicit and not included as graph nodes. Stacking interactions are also included as edges (dotted lines). Interfaces with the previous nucleotide pair appear in blue, while interfaces with the next nucleotide pair appear in red.

**Lemma 44.** Suppose we have a line of graphs  $H = ((\mathcal{U}, \mathcal{E}), (\mathcal{U}^{CG}, \mathcal{E}^{CG}))$  with  $\mathcal{U}^{CG} = \{G_1, \dots, G_N\}$  and  $(G_k, G_l) \in \mathcal{E}^{CG} \iff k = l \pm 1$ . Suppose there exists a highway  $W \subseteq H$ .

Let  $u \in G_i \cap H$  and  $v \in G_j \cap H$  with  $j > i$ . Then,  $d(u, v) = j - i$  and

$$n_g(u, v) \leq \left( \max_{G_k \in \mathcal{U}^{CG}} |G_k \cap H| \right)^{j-i-1} + |\tilde{h}(u, v)|$$

where  $\tilde{h}(u, v) = \{p = (u = w_0, \dots, v = w_{j-i}) \subseteq G \mid \exists k \in \{1, j-i-1\} \text{ st } w_k \notin W\}$ .

*Proof.* It is clear that the distance between  $u$  and  $v$  must be equal or greater than  $j - i$  because we need to traverse at least one edge to go from graph  $G_k$  to graph  $G_{k+1}$ . As we can jump from  $G_k$  to  $G_{k+1}$  using one edge only through the highway  $H$ , we conclude  $d(u, v) = j - i$ . This means that any geodesic path between  $u$  and  $v$  has to move from  $G_k$  to  $G_{k+1}$  at every step.

We are left to bound the number of possible geodesic paths of this length between the two nodes. First, note that the number of geodesic paths that are not contained in the highway is given by the number of paths in the set  $\tilde{h}(u, v) = \{p = (u = w_0, \dots, v = w_{j-i-1}) \subseteq G \mid \exists k \in \{1, j-i-2\} \text{ st } w_k \notin$

$H\}$ . Second, observe that, in constructing a path contained in the highway, at every step we can choose between as many options as nodes are contained in the intersection between the highway and the next graph; i.e., if we are at graph  $G_k$ , we can choose between  $|G_{k+1} \cap H|$  nodes for the next step. We can conclude that

$$n_g(u, v) \leq \left( \max_{G_k \in \mathcal{U}^{CG}} |G_k \cap H| \right)^{j-i-1} + |\tilde{h}(u, v)|$$

□

Let us apply 44 to the case of a **siRNA** molecular graph with a highway given by all the nodes participating in hydrogen bonds. Suppose we take two nodes  $u, v$  where  $u$  is a node participating in a hydrogen bond in the nucleotide pair  $i$  and  $v$  is a node participating in a hydrogen bond in the nucleotide pair  $j$  with  $j > i$ . Here, there are no geodesic paths from  $u$  to  $v$  which are not contained in the highway. Note that  $\max_{G_k \in \mathcal{U}^{CG}} |G_k \cap H| = 6$  which is the number of atoms in a C-G base pair that participate in a hydrogen bond, and thus  $n_g(u, v) \leq 6^{j-i-1}$ .

Next, observe that the minimum degree of a node in this highway is 10: every node involved in a hydrogen bond originally has at least two edges (the hydrogen bond and one bond to another atom in its own base) and then gets added at least 8 more edges that correspond to the stacking interactions for the case when both the base pairs above and below are  $A - T$  or  $A - U$  (which have 4 atoms each participating in the hydrogen bonds). Thus, taking bounds in (3.6) we get that

$$[\hat{A}^{j-i}]_{u,v} \leq \frac{1}{\sqrt{d_u d_v}} \frac{6^{j-i-1}}{10^{j-i-2}} = \frac{10}{\sqrt{d_u d_v}} \left(\frac{3}{5}\right)^{j-i-2} \quad (3.15)$$

In a double stranded **siRNA** chain with 21 nucleotide pairs, this means that the over-squashing between two nodes participating in hydrogen bonds in the first and last pair is dictated by  $\frac{10}{6} \left(\frac{3}{5}\right)^{18} \approx 1,7 \times 10^{-4}$  (assuming  $d_u = d_v = 6$ , which is the minimum degree those nodes can have).

As a result, standard **MPNN** struggle to capture long-range interactions even if one includes hydrogen bonds and stacking interactions and takes a sufficiently large number of message-passing layers.

One possible solution would be to add even more edges that allow messages to travel even faster across nucleotide pairs (for example, adding edges between nucleotide pair  $i$  and nucleotide pair  $i + k$ ). Graph rewiring

methods ([46], [56]) provide a principled way to implement such shortcuts. However, in the context of siRNA molecular graphs, these approaches can weaken important inductive biases: the added edges are not necessarily grounded in known biochemical interactions, may lead to excessive mixing of local and global signals, and reduce interpretability of the resulting representations.

These last two points are especially relevant to us because

- we are looking for a method that retains **MPNNs**' ability to capture local interactions while still being able to separately identify long-range interactions and integrate them in a principled manner
- we want a representation of the double-stranded **siRNA** molecules that is amenable to the integration of the extracted trimer properties, enabling a physics-assisted approach

### 3.3.2 Theoretical Analysis of Chemprop

Recall that Chemprop's update at the message passing step is given by (2.8). We would like to rewrite this equation in matrix form in order to understand its behaviour on the graph as a whole. Note that if we define the directed line graph pseudo-adjacency matrix  $B \in \mathbb{R}^{2|\mathcal{E}| \times 2|\mathcal{E}|}$  as

$$[B]_{u \rightarrow v, w \rightarrow y} = \begin{cases} 1 & \text{if } v = w \text{ and } y \neq u \\ 0 & \text{otherwise} \end{cases} \quad (3.16)$$

then - provided we keep a set ordering of the directed edges - we can write Chemprop's update as

$$H_{l+1} = \tau(H_0 + BH_lW_l^t) \quad (3.17)$$

The issue is that  $B$  is not a normalized graph shift operator like, for example, the normalized adjacency matrix used in traditional **MPNNs**. To the best of our knowledge, the literature on **GNN** theoretical analysis has not addressed the behaviour of **GNN** based on un-normalized graph shift operators such as  $B$ .

Lemma 33 can be extended to un-normalised graph shift operators, but in that case it loses all its predictive power because the powers of un-normalised graph shift operators can (and do) explode. Thus, we will not include the proof here.

A possible approach would be to capture over-squashing in terms of the ratio of the sensitivity of an embedding with respect to the initial embedding of a particular node and the sum of the sensitivities of the same embedding with respect to the initial embeddings of all nodes in the graph: let  $S^r(v, u^*) := \left| \frac{\partial [X^r]_v}{\partial x_{u^*}} \right|$  be the unweighted sensitivity of node  $v$ 's embedding in layer  $r$  to node  $u^*$ 's initial embedding as defined in 33, then we can define the weighted sensitivity of node  $v$ 's embedding in layer  $r$  to node  $u^*$ 's initial embedding as:

$$\bar{S}^r(v, u^*) := \frac{S^r(v, u^*)}{\sum_{u \in \mathcal{V}} S^r(v, u)}$$

The problem with this definition is that we do not have lower bounds for the sensitivities that appear in the denominator, only upper bounds.

Although we cannot prove that an architecture like Chemprop will suffer from over-squashing, there are other potential issues that may arise. First, using an un-normalized graph shift operator means that the sensitivity of an atom embedding with respect to its own original embedding could grow exponentially with the number of message-passing steps. This is not guaranteed due to the use of non-linearities after each message-passing step, but it is still a worrying possibility.

Secondly, a large number of message passing steps has been associated with over-smoothing [57, 42], although recent work has highlighted possible mitigation strategies [58].

These issues are of course circumvented by the Edge Set Transformer architecture [50] while preserving the graph structure inductive bias. However, neither ChemProp [47] nor the Edge Set Transformer can seamlessly incorporate the extracted trimer properties. Thus, although they are interesting base architectures, we must move beyond them or incorporate them in a larger, hierarchical model.

### 3.3.3 A hybrid hierarchical machine learning framework

Hierarchical or multilevel machine learning architectures typically proceed in two stages: representations are first updated through interactions within a given level and are then aggregated to form representations at the next level. In our setting, it is important to distinguish between three different ingredients. First, each level transition carries a fixed incidence structure specifying which lower-level elements may contribute to each higher-level

element. Second, the corresponding cross-level aggregation maps may either be prescribed or learned. Third, each level is equipped with a within-level interaction structure which determines how elements at that level may exchange information through the propagator and at aggregation.

Let the hierarchy consist of levels  $0, \dots, m$ , where level 0 is the base level. For each level  $i \in \{0, \dots, m\}$ , let  $s_i$  denote the number of elements at that level, and let

$$H^{(i)} = (h_1^{(i)}, \dots, h_{s_i}^{(i)}) \in (\mathbb{R}^{d_i})^{s_i}$$

be the collection of level- $i$  embeddings, where each  $h_k^{(i)} \in \mathbb{R}^{d_i}$ .

For each level  $i \in \{0, \dots, m-1\}$ , let  $I^{(i)}$  denote the within-level interaction structure at level  $i$ , and let

$$g_i^{I^{(i)}} : (\mathbb{R}^{d_i})^{s_i} \rightarrow (\mathbb{R}^{\tilde{d}_i})^{s_i}$$

be a continuously differentiable propagator compatible with that structure. We write

$$\tilde{H}^{(i)} = g_i^{I^{(i)}}(H^{(i)})$$

and denote by  $\tilde{h}_k^{(i)}$  the  $k$ -th component of  $\tilde{H}^{(i)}$ .

For each level transition  $i-1 \rightarrow i$  and each index  $j \in \{1, \dots, s_i\}$ , let

$$A_i(j) \subseteq \{1, \dots, s_{i-1}\}$$

denote the incidence set of the  $j$ -th level- $i$  element, that is, the set of level- $(i-1)$  indices that are permitted to contribute to it. The family  $\{A_i(j)\}_{j=1}^{s_i}$  is regarded as fixed by the representation design.

Let

$$p_{i,j}^{I^{(i)}} : \prod_{r \in A_i(j)} \mathbb{R}^{\tilde{d}_{i-1}} \rightarrow \mathbb{R}^{d_i}$$

be a continuously differentiable cross-level aggregation map. Then the embedding of the  $j$ -th element at level  $i$  is given by

$$h_j^{(i)} = p_{i,j}^{I^{(i)}}((\tilde{h}_r^{(i-1)})_{r \in A_i(j)}). \quad (3.18)$$

Now let  $F : (\mathbb{R}^{d_m})^{s_m} \rightarrow \mathbb{R}$  be a continuously differentiable scalar-valued function of the top-level embeddings. For any  $k \in \{1, \dots, s_{m-1}\}$ , the chain rule gives

$$\nabla_{h_k^{(m-1)}} F = \sum_{j=1}^{s_m} \left( \frac{\partial h_j^{(m)}}{\partial h_k^{(m-1)}} \right)^\top \nabla_{h_j^{(m)}} F$$

where

$$\frac{\partial h_j^{(m)}}{\partial h_k^{(m-1)}} = \sum_{r \in A_m(j)} \frac{\partial h_j^{(m)}}{\partial \tilde{h}_r^{(m-1)}} \frac{\partial \tilde{h}_r^{(m-1)}}{\partial h_k^{(m-1)}}.$$

This formulation is agnostic to the geometric structure used within each level. Further, it is compatible with the notion of cross-sensitivity we introduced for **GNNs** as the derivative of the embedding at one particular node and layer with respect to the initial embedding of a different node. For example, if the base level is the molecular graph of an **siRNA**, two atoms that do not interact within the receptive field of a message-passing **GNN** may exhibit no cross-sensitivity after propagation at the atomic level, or they may suffer from over-squashing even if the number of message passing steps is large enough for both nodes to share information. However, if both atoms contribute (through cross-level aggregation) to higher-level objects that interact through their level propagator maps, then cross-sensitivity may arise at the next level even when it is absent at the lower one.

To make the previous remark more explicit, fix two base-level elements  $u, v \in \{1, \dots, s_0\}$ . Let  $r_i(u)$  denote an index of a level- $i$  element to which  $u$  contributes and let  $r_i(v)$  denote an index of a level- $i$  element to which  $v$  contributes.

Suppose that  $i$  is the first level at which a level- $i$  element influenced by  $v$  affects the propagated representation  $\tilde{h}_{r_i(u)}^{(i)}$  through the propagator  $g_i^{I^{(i)}}$ . In particular, this entails that, at every level  $l < i$ , no element to which  $u$  contributes at level  $l$  interacts through  $g_l^{I^{(l)}}$  with an element to which  $v$  contributes at the same level. Let us define, for each level  $l$ , the sensitivities

$$D_j^{(l)}(v) := \frac{\partial h_j^{(l)}}{\partial h_v^{(0)}}, \quad \tilde{D}_a^{(l)}(v) := \frac{\partial \tilde{h}_a^{(l)}}{\partial h_v^{(0)}}.$$

At the base level, these satisfy

$$D_j^{(0)}(v) = \begin{cases} I, & j = v, \\ 0, & j \neq v, \end{cases}$$

where  $I$  denotes the identity matrix of size  $d_0 \times d_0$ .

The propagation step at level  $l$  gives

$$\tilde{D}_a^{(l)}(v) = \sum_{b=1}^{s_l} \frac{\partial \tilde{h}_a^{(l)}}{\partial h_b^{(l)}} D_b^{(l)}(v)$$

while the cross-level aggregation step from level  $l - 1$  to level  $l$  gives

$$D_j^{(l)}(v) = \sum_{k \in A_l(j)} \frac{\partial h_j^{(l)}}{\partial \tilde{h}_k^{(l-1)}} \tilde{D}_k^{(l-1)}(v).$$

Starting from the level- $i$  element  $r_i(u)$  which is, by assumption, the first ancestor of  $u$  which interacts through a propagation map with an ancestor of  $v$ , one obtains

$$\frac{\partial \tilde{h}_{r_i(u)}^{(i)}}{\partial h_v^{(0)}} = \sum_{j=1}^{s_i} \frac{\partial \tilde{h}_{r_i(u)}^{(i)}}{\partial h_j^{(i)}} D_j^{(i)}(v)$$

which we can expand recursively using the previously derived expressions. For example, expanding one level downwards yields

$$\frac{\partial \tilde{h}_{r_i(u)}^{(i)}}{\partial h_v^{(0)}} = \sum_{j=1}^{s_i} \frac{\partial \tilde{h}_{r_i(u)}^{(i)}}{\partial h_j^{(i)}} \left( \sum_{k \in A_i(j)} \frac{\partial h_j^{(i)}}{\partial \tilde{h}_k^{(i-1)}} \tilde{D}_k^{(i-1)}(v) \right)$$

and recursively

$$\tilde{D}_k^{(i-1)}(v) = \sum_{\tilde{k}=1}^{s_{i-1}} \frac{\partial \tilde{h}_k^{(i-1)}}{\partial h_{\tilde{k}}^{(i-1)}} D_{\tilde{k}}^{(i-1)}(v).$$

This recursive structure shows that cross-sensitivity between the branches of  $u$  and  $v$  can emerge through the combined effect of within-level propagation and hierarchical aggregation. In particular, it may happen that a higher-level representation on the branch of  $u$ , such as  $h_{r_i(u)}^{(l)}$ , has a non-zero Jacobian with respect to  $h_v^{(0)}$  already for some  $l < i$ , even though level  $i$  is the first level at which the corresponding ancestor branches interact directly through a propagator map.

There is, however, an important class of hierarchical models for which this "anticipated" sensitivity cannot occur. For each level  $l \in \{0, \dots, m-1\}$  and each index  $j \in \{1, \dots, s_l\}$ , define the aggregation-neighbourhood

$$C_l(j) := \bigcup_{n \mid j \in A_{l+1}(n)} A_{l+1}(n).$$

Thus,  $C_l(j)$  is the set of all level- $l$  elements that co-contribute with  $j$  to at least one common parent at level  $l + 1$ .

We then consider the class of hierarchical models satisfying

$$\frac{\partial \tilde{h}_j^{(l)}}{\partial h_k^{(l)}} = 0 \quad \text{for all } k \notin C_l(j)$$

for every level  $l$  and every index  $j$ . In other words, within-level propagation is restricted to elements that share at least one admissible aggregation context at the next level.

Under this assumption, the propagation step at level  $l$  becomes

$$\tilde{D}_a^{(l)}(v) = \sum_{b \in C_l(a)} \frac{\partial \tilde{h}_a^{(l)}}{\partial h_b^{(l)}} D_b^{(l)}(v)$$

while the cross-level aggregation step from level  $l - 1$  to level  $l$  remains

$$D_j^{(l)}(v) = \sum_{k \in A_l(j)} \frac{\partial h_j^{(l)}}{\partial \tilde{h}_k^{(l-1)}} \tilde{D}_k^{(l-1)}(v).$$

Hence,

$$\frac{\partial \tilde{h}_{r_i(u)}}{\partial h_v^{(0)}} = \sum_{j \in C_i(r_i(u))} \frac{\partial \tilde{h}_{r_i(u)}}{\partial h_j^{(i)}} D_j^{(i)}(v)$$

and recursively each term  $D_j^{(i)}(v)$  depends only on sensitivities propagated through the corresponding sequence of aggregation-neighbourhoods.

It follows that cross-sensitivity between the branches of  $u$  and  $v$  cannot arise before the first level at which those branches become linked through the aggregation-neighbourhood relation. In particular, if  $i$  denotes the first level at which this occurs, then

$$\frac{\partial \tilde{h}_{r_l(u)}}{\partial h_v^{(0)}} = 0 \quad \text{for all } l < i.$$

Because this class of models prevents cross-sensitivity from leaking across distinct branches before they are explicitly brought into a common aggregation context by the hierarchy, we refer to them as *airtight hierarchical models*.

**Definition 45** (Airtight hierarchical model). *A hierarchical model is said to*

be airtight if, for every level  $l \in \{0, \dots, m-1\}$  and every index  $j \in \{1, \dots, s_l\}$ ,

$$\frac{\partial \tilde{h}_j^{(l)}}{\partial h_k^{(l)}} = 0 \quad \text{for all } k \notin C_l(j),$$

where

$$C_l(j) := \bigcup_{n: j \in A_{l+1}(n)} A_{l+1}(n)$$

is the set of level- $l$  elements that co-contribute with  $j$  to at least one common parent at level  $l+1$ .

This distinction is of practical interest because airtight architectures induce substantially smaller computational graphs than general hierarchical models. It is therefore natural to ask whether this restriction can be imposed without loss of predictive performance. If airtight models perform comparably to unconstrained hierarchical architectures, this would suggest that aggregation-aligned interactions capture most of the relevant signal while offering a more efficient and structured inductive bias. On the other hand, a marked performance gap would indicate that cross-branch interactions prior to explicit aggregation carry important task-relevant information.

Motivated by this perspective, we consider a hierarchical model with the following levels: individual atoms, nucleotides, trimers, and the full **siRNA** molecule. This enables the incorporation of trimer-level information directly into the propagator defined at the trimer level. The framework is modular, and different architectural instantiations may be obtained through different choices of initial embeddings, propagators, and pooling maps. Moreover, a specific instantiation need not use all levels; for example, one may start directly at the nucleotide level, using a molecular fingerprint as the initial nucleotide embedding. Figure 3.5 illustrates this general approach. In this work, due to time limitations, we restrict ourselves to the study of airtight hierarchical architectures. The study of the performance gap (if there is one) between airtight and non-airtight hierarchical models is left for future work.

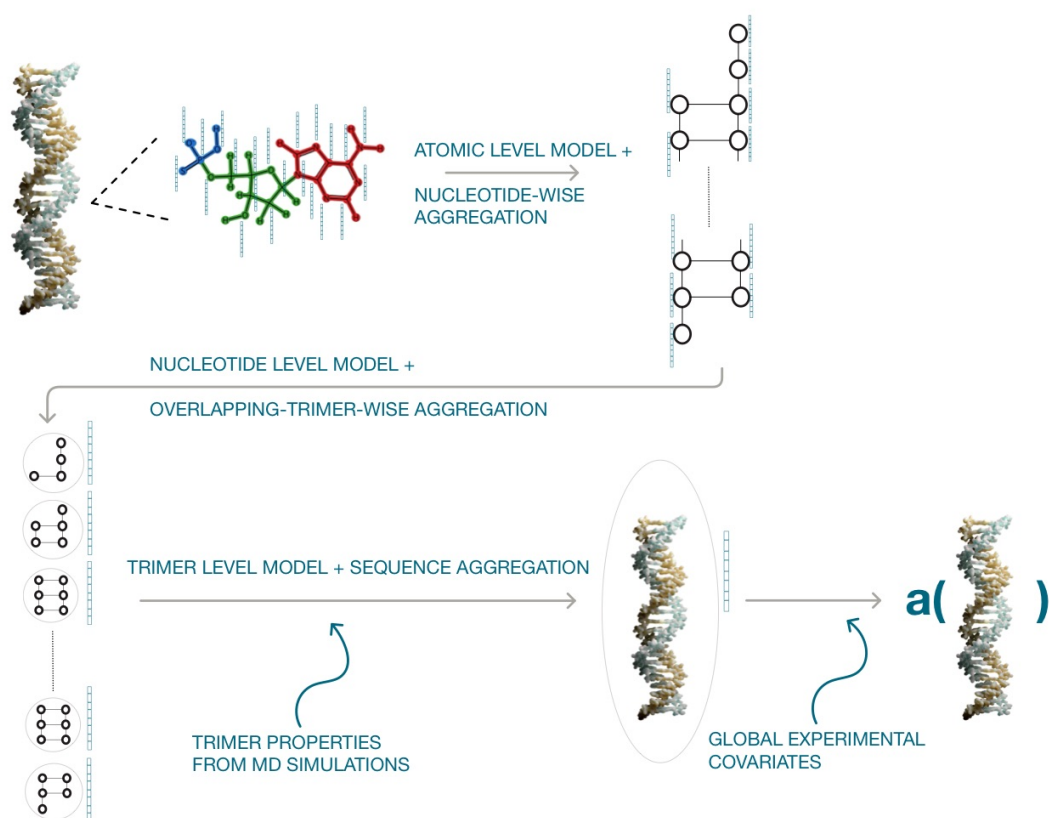


Figure 3.5: Overview of the proposed architecture

# Chapter 4

## Methods

This chapter dives into the research methods used in this thesis. Section 4.1 describes the research process. Section 4.2 focuses on the main dataset used in the project and the data pre-processing steps. Section 4.3 covers the details behind trimer MD simulation and feature extraction. Finally, Section 4.5 describes the ML methods used and introduces the baseline and main machine learning models used for siRNA activity prediction.

### 4.1 Research Process

The development of this thesis has followed five clear research stages following a literature review. The first stage consisted of studying the provided dataset and developing a framework to identify unique trimers, nucleotides and modifications which allowed cross-referencing with modXNA [4]. The next stage required evaluating the computational requirements for the simulation of the extracted trimers that are parametrized by modXNA, preparing the trimer files for simulation, establishing a procedure to manage high-throughput MD simulations and, lastly, executing them. The third stage covered the feature extraction from the trimer MD simulations and involved creating a workflow for high-throughput MD analysis, studying the statistical behaviour of the desired features to ensure their proper characterization and collecting them in a structured dataset. The fourth stage was the only stage which did not happen sequentially but rather in parallel to stages 2 and 5, and it involved the mathematical analysis of the siRNA molecular graph structure and its impact on standard message-passing architectures, and the mathematical description of the proposed hierarchical models. This stage is not addressed here as it is laid out in detail in Chapter 3. Finally, the fifth stage corresponds to

the implementation of a modular framework for the design of hierarchical architectures for modified **siRNA** activity prediction and the study of the performance of several instantiations of these models on a demanding data split.

## 4.2 Data Analysis and Pre-Processing

This section addresses the methodology behind the first stage of the research process. It begins by describing the studied dataset 4.2.1 and then goes on to describe the extraction of unique trimers, nucleotides and fragments and the cross-referencing with modXNA.

### 4.2.1 Dataset Analysis

The provided dataset has the following fields:

- **as\_smiles**: SMILES representation of the nucleotides in the *anti-sense* strand, corresponding to the guide strand in 5' to 3' orientation. When the same position in both the as\_smiles and the s\_smiles arrays is 0, it denotes padding - which is applied to ensure all sequences have the same length. A zero in only one of them indicates that the strand with the non-zero position has a termini, i.e., a strand end with no corresponding segment in the complementary strand.
- **s\_smiles**: SMILES representation of the nucleotides in the sense strand, corresponding to the passenger strand in 3' to 5' orientation. When the same position in both the as\_smiles and the s\_smiles arrays is 0, it denotes padding - which is applied to ensure all sequences have the same length. A zero in only one of them indicates that the strand with the non-zero position has a termini, i.e., a strand end with no corresponding segment in the complementary strand.
- **as\_seq**: Nucleotide sequence of the anti-sense (guide) strand, written in 5' to 3' order. When the same position in both the as\_seq and the s\_seq arrays is 0, it denotes padding - which is applied to ensure all sequences have the same length. A zero in only one of them indicates that the strand with the non-zero position has a termini, i.e., a strand end with no corresponding segment in the complementary strand.
- **s\_seq**: Nucleotide sequence of the sense (passenger) strand, written in 3' to 5' order. When the same position in both the as\_seq and the s\_seq

arrays is 0, it denotes padding - which is applied to ensure all sequences have the same length. A zero in only one of them indicates that the strand with the non-zero position has a termini, i.e., a strand end with no corresponding segment in the complementary strand.

- **siRNA\_concentration\_nM**: siRNA concentration used in the experiment, expressed in nanomolar.
- **celltype**: Cell type in which the siRNA activity was measured.
- **incubation\_time\_h**: Incubation time for the experiment, expressed in hours.
- **transfection\_method**: Method used to deliver or transfect the siRNA into cells.
- **activity\_detection**: Experimental readout or assay used to quantify siRNA activity.
- **activity**: Observed siRNA activity value from the experiment.
- **id**: Unique identifier for the dataset entry or experiment.
- **gene**: Target gene against which the siRNA is designed.
- **database**: Source database or origin from which the record was obtained.

We study the distribution of **siRNA** concentration, incubation time, cell types, genes, transfection methods, activity detection methods and database origins across the dataset through histograms. Furthermore, we extract the number of unique sequences - where two sequences are equal if both their `as_seq` and `s_seq` arrays are identical. Observe that, given that the 0 padding was done in a standardised manner, it is never the case that we have two identical sequences that differ only on the amount of padding (not including termini positions) added to each extreme of the strands.

#### 4.2.2 Extracting unique trimers, nucleotides and modifications

Extracting the unique trimers, nucleotides and modifications present in our dataset can provide further insights about our data, reveal the degree in which chemical modifications are present in it and enable us to simulate the identified

trimers to obtain their dynamic features. In order to achieve this, the first step is to set criteria that allow us to identify and label distinct elements in each of these three classes.

For individual nucleotides, we use **InChIKey** as an identifier. Given a nucleotide **SMILES**, `rdkit` makes it straightforward to obtain its **InChIKey**. Termini and padding positions (with a 0 instead of a valid **SMILES** string) are not considered. We add fields "as\_inchikeys" and "s\_inchikeys" to the original dataset which contain the lists of the **InChIKeys** of the nucleotides in each strand (in 5' to 3' order for the guide strand and 3' to 5' order for the passenger strand) and create a secondary dataset, named `sirna_unique_nucleotides`, with the extracted unique nucleotides. `sirna_unique_nucleotides` also includes an "inchikeys" field that enables quick search and retrieval from the original dataset and a "smiles" field with the **SMILES** of the nucleotide.

Choosing an identifier for the overlapping double-stranded trimers present in the dataset is slightly more elaborate. First, we obtain an identifier for the guide strand of the trimer by taking

$$\text{ID}(\text{trimer guide strand}) = \text{Hash}(\text{InChIKey}(n_1)|\text{InChIKey}(n_2)|\text{InChIKey}(n_3))$$

where  $n_1, n_2, n_3$  are the nucleotides in the guide strand of the trimer in order 5' to 3'. We repeat this operation to obtain an identifier for the passenger strand of the trimer:

$$\begin{aligned} \text{ID}(\text{trimer passenger strand}) = \\ \text{Hash}(\text{InChIKey}(\tilde{n}_1)|\text{InChIKey}(\tilde{n}_2)|\text{InChIKey}(\tilde{n}_3)) \end{aligned}$$

where  $\tilde{n}_1, \tilde{n}_2, \tilde{n}_3$  are the nucleotides in the passenger strand of the trimer in order 3' to 5'. Finally, we obtain the trimer's id as

$$\text{ID}(\text{trimer}) = \text{Hash}(\text{ID}(\text{trimer guide strand})||\text{ID}(\text{trimer passenger strand}))$$

Note that, exclusively this one time, || is not indicating concatenation - it is part of the string being hashed. Individual | in the trimer strand ID formula are also characters in the string that is being hashed. We use `sha256` as our hashing function.

We organize the extracted unique trimers in a trimer dataset named `sirna_unique_trimers` which contains fields `as_seq`, `s_seq`, `as_smiles` and `s_smiles` organised just as in the original dataset, but in this case with strings and lists of length 3. We also add a "trimer\_signatures" field in which

we write the obtained identifier for those trimers that contain no termini and a 0 for those trimers that contain termini in either strand. This is because we are not particularly interested in trimers containing termini: not because they do not play a role in the molecule’s activity but because they cannot be faithfully simulated with MD as a trimer of nucleotides. This is due to the fact that trimers with termini are subject to a different chemical environment than that of a trimer within a double-stranded chain, experiencing strong boundary effects. While traditional force-fields have been parametrised to account for such boundary effects in longer chains, they may not accurately capture their effect on a single trimer.

Finally, we address the extraction of the chemical modifications present in our dataset. This step is of utmost importance in order to cross-reference with modXNA and obtain parametrizations for the dataset’s trimers MD simulations. In order to identify these modifications, one must separate each nucleotide in three fragments: backbone, sugar and base. While this is a trivial task to do visually, it must be automated in order to be able to process the entirety of our dataset. To the best of our knowledge, there are no publicly available software tools that can perform this fragmentation in an automated manner. Thus, we develop a tool called `nucleotide_fragmentation` to efficiently (and robustly) identify the backbone, sugar and base in any nucleotide that can be constructed with the modifications covered by the union of modXNA and our dataset. Further, once backbone, sugar and base have been extracted one must “cap” the resulting molecular fragments as the resulting blocks would otherwise have invalid valences in the linking atoms.

In order to easily understand the procedure used to separate the nucleotide in backbone, sugar and base, it is convenient to introduce the concept of a “dead-end” in a molecular graph.

**Definition 46** (Dead-end). *Given a molecular graph  $G = (\mathcal{V}, \mathcal{E})$ , we say a node  $v$  is a dead-end if there only exists one neighbour of  $v$  that is not a Hydrogen.*

We are now ready to describe the steps followed by the fragmentation algorithm:

1. First, it identifies the unique Oxygen that links the backbone and sugar fragments as the unique Oxygen in the nucleotide graph which has exclusively two neighbours: a Phosphorus atom and a Carbon atom, neither of which are dead-ends. It holds that for every nucleotide that can be assembled with modXNA and every nucleotide in our database,

there exists only one such Oxygen atom and it always joins the backbone and the sugar fragments.

2. Second, it looks for a unique path "Oxygen-Carbon-Carbon-Oxygen/Sulfur-Carbon" starting from the identified Oxygen linker, where the last Carbon in the path cannot be a dead-end. The only exception to this rule is given by nucleotides with sugar fragments that are not technically sugars but rather ethers. These are handled separately, but ultimately also return a unique path ending in a Carbon which is not a dead-end. The last carbon in the uniquely identified path (irrespective of whether the sugar is an exception or not) is the base's anchor, this is, the carbon at which the base bounds to the sugar.
3. Third, it performs a BFS starting from the Phosphorus neighbour of the unique Oxygen linker identified in step 1 which stops if it reaches this unique Oxygen linker. This gives us the backbone fragment. Next, it performs a BFS starting from the unique Oxygen linker which stops if it reaches its Phosphorus neighbour or the Carbon anchor identified in step 2. Adding back the Carbon anchor to the nodes reached by this BFS, one obtains the entirety of the sugar fragment. Finally, the reminding unlabeled atoms form the base fragment.

Once the fragments have been identified, we cap the linking atoms (which have been left with invalid valences) following modXNA's conventions, which are specifically selected in order to mitigate boundary effects when parametrizing modified fragments. This allows us to obtain their **SMILES** and **InChIKeys**, which we can cross-reference with modXNA's catalogue.

We add a new field to the `sirna_unique_nucleotides` called "fragments\_inchikeys" which consists of a list with the **InChIKeys** of the backbone, sugar and base fragments, in this order.

After cross-referencing with modXNA, we create two new datasets: `sirna_modXNA_unique_nucleotides`, which only contains those nucleotides whose fragments are parametrized by modXNA, and `sirna_modXNA_unique_trimers`, which only contains trimers whose nucleotides are all in `sirna_modXNA_unique_nucleotides`. Given that each fragment in modXNA has a unique, 3-letter identifier, we include a new field in the `sirna_modXNA_unique_nucleotides` dataset called "modXNA\_ids", which consists of a list with the modXNA identifiers for the backbone, sugar and base in the nucleotide, in that order.

We also create datasets `bases_siRNAdatabase`, `sugarss_siRNAdatabase` and `backbones_siRNAdatabase` with the **SMILES**, **InChIKeys** and `modXNA` index (if it exists) for each modification.

Furthermore, we extend the `nucleotide_fragmentation` tool to also identify the Oxygen atoms which link the nucleotide to its adjacent nucleotides when it is part of a strand. As a result, we can easily link nucleotides together to get single-strand trimer molecular graphs. In turn, these can then be used to obtain trimer fingerprints that capture the structural information encoded in the bonds between the individual nucleotides which compose the full strand.

Lastly, we also implement a function to obtain the  $N$  **siRNA** molecules that contain the largest possible amount of trimers in a given trimer dataset. This is achieved through a greedy approach that, for  $N$  steps, chooses the **siRNA** which adds the most trimers that do not appear in the previously selected **siRNA** molecules. This will allow us to choose a reduced number of **siRNA** molecules to run **MD** simulations on that can validate the results of our trimer **MD** simulations.

## 4.3 Molecular dynamics preparation and simulation

The preparation of the double-stranded trimers in `siRNA_unique_trimers_modXNA_notermini` for **MD** simulations is implemented as a multi-stage pipeline that takes as input the `"as_smiles"` and `"s_smiles"` lists for a number of trimers and outputs simulation-ready coordinate and topology files and slurm scripts to run the **MD** simulations. This pipeline leverages pre-existing, unpublished scripts developed by Emilie Gerard and Marco Klähn, which at the same time build upon standard tools and practices for unmodified **DNA** and **RNA** chains.

### 4.3.1 Boundary effects and trimer caps

Running **MD** simulations of the selected trimers as-is would not yield faithful measurements of their dynamical properties because their boundaries, i.e. their extremes, would not be bonded to a larger chain but rather exposed to the solvent. As we are interested in trimer properties as units of a larger **siRNA** molecule, this naive simulation would not be representative of their features

within the chain.

Therefore, we need to modify the boundary conditions of the trimers to approximate their dynamics within a larger **siRNA** molecule while keeping the computational cost as low as possible. At the same time, there are specific trimer dynamics which we would never observe in a whole-**siRNA** simulation but are nonetheless known to be predictive of global features of the entire double-stranded molecule.

The two most obvious boundary choices are using A-U or C-G residue pairs as caps on both extremes of the trimer. We use the exact same caps in both extremes of the double-stranded trimers to avoid introducing biases towards one specific side of the whole **siRNA** molecule. We know the C-G pair is more stable because it shares one more hydrogen bond than the A-U pair [59, 60, 61]. Therefore, choosing C-G pairs as caps would mean imposing more rigid boundary conditions. This may restrain the dynamics of the trimer to a smaller set of its state-space.

We choose to favour more flexible boundary conditions instead, so all trimers are capped with two A-U residue pairs, one on each side of the chain. The A nucleotide is always in the guide strand and the U nucleotide is always in the passenger strand. We hypothesize that, if this choice really enables an increased exploration of the trimer's state-space, then it may also yield more distinct observations that allow our **ML** models to better differentiate between trimers as a result. However, testing this hypothesis is beneath the scope of this work.

### 4.3.2 Obtaining initial pentamer structures

The first step of the workflow consists on the creation of an input `csv` table from the "as\_smiles" and "s\_smiles" lists for each trimer, which are each assigned adjacent columns. The first column gets the guide **SMILES** - in order 5' to 3'- and the second column gets the passenger **SMILES** - in order 3' to 5'. Each row then corresponds to a residue pair in the chain and contains the **SMILES** for both of its nucleotides. Note that the first and fifth (last) rows are the same for every trimer and correspond to the selected A-U caps. Each trimer is no longer a trimer but a pentamer, with the underlying trimer encoded in rows 2-4. These are kept in a data folder.

`Oligo3Dgen` has been developed by Marco Klähn to enable the generation of three-dimensional duplex structures for modified **siRNA** molecules. It can read a `.csv` file in the introduced format and output a PDB file that serves as a starting structural model for subsequent parametrization.

Oligo3Dgen progressively builds the double-stranded chain by iteratively adding new nucleotides whose 3D structure is set through energy minimisation with RDKit's standard force field, and runs one final energy minimisation on the whole structure.

### 4.3.3 Building a modified nucleotide library with modXNA

In parallel, a reference `tsv` file was created for each trimer in `sirna_unique_trimers_modXNA_notermini`. This `tsv` file contains three columns with headers `index`, `triplet` and `strand`. The `index` field is numbered from 1 to 10, where indices 1 to 5 correspond to the nucleotides in the guide strand (remember we have added caps so they are technically pentamers) in the 5' to 3' sense and indices 6 to 10 correspond to the nucleotides in the passenger strand in the 3' to 5' sense. The `triplet` field contains three items which are the modXNA IDs of the backbone, sugar and base fragments of the nucleotide, in that order. Finally, the `strand` field is assigned 'A' if the nucleotide belongs to the guide strand and 'B' if the nucleotide belongs to the passenger strand.

These `tsv` files are kept in a separate parametrization folder - in particular, in a subfolder of the parametrization folder called `per_sirna`. Within the parametrization folder we also create a `nucleotide_library` subfolder that will store modXNA's parameters for all nucleotides present in our `sirna_unique_trimers_modXNA_notermini` dataset (which are the nucleotides in the `sirna_modXNA_unique_nucleotides` dataset).

The completeness of the nucleotide library is then evaluated using a script that compares the modified nucleotides present in the generated `tsv` files with those which are already available in the nucleotide library directory. This trivially means that the first time this step was executed all modified present in the `tsv` files were flagged as missing, but its appeal is that it enables a future incremental construction of the modified nucleotide library as new modified nucleotides are identified.

Any modified nucleotides not yet represented were collected into a `missing_combinations` directory as simple one-line `txt` files with the modXNA IDs of their backbone, sugar and base, in that order. Duplicate requests are removed. The modified nucleotides are then parametrized by modXNA, creating a subfolder within `nucleotide_library` which receives the name "`<Backbone fragment modXNA ID>_<Sugar fragment modXNA ID>_<Base fragment modXNA ID>`". Capping groups at the 3'

and 5' ends are identified and labelled as such with a prefix "3CAP\_" or "5CAP\_". Further, all 5' ends are normalized, with their modXNA ID changed to 5PO - which denotes a Hydrogen bonded to the Oxygen linking the sugar and backbone groups and thus substituting the phosphate group in the 5' cap. All parametrizations are also appended to a master file in `nucleotide_library` named "modXNA\_mod\_siRNA.lib", which also assigns a code to each modified nucleotide.

### 4.3.4 Format normalization

Before force-field assignment, the initial structural PDB files generated by `Oligo3Dgen` require substantial normalization. First, their capping residues are standardised by removing the 5' caps backbone group information from the file and substituting it for a single Hydrogen which is placed at a distance of 0.96Å in the direction of the old Phosphorus centre - linker Oxygen bond. These PDB files are given the suffix "\_sc", which stands for standardised caps, and are kept in the data folder.

In parallel, residue names from the nucleotide library are used to create annotated `tsv` files from the starting `tsv` files in the `per_siRNA` folder. The new annotated `tsv` files have an extra column with the `nucleotide_library` code for each modified nucleotide in the pentamer (trimer with caps). linking sequence positions to library residue names. Further, the order of the nucleotides in the passenger strand is inverted to match the indexing order in the PDB files (recall that the PDB files are generated by progressively adding nucleotides in the 5' to 3' sense, starting with the guide strand and then moving to the passenger strand). These files are given the suffix "\_annotated" and are stored in the same `per_siRNA` folder as the original `tsv` files.

The resulting residue numbering is used for the rest of this work. Here, we describe it more carefully for a general double-stranded **siRNA**  $N$ -mer, including terminal caps:

**Definition 47** (Residue Numbering in general  $N$ -mer). *Given any double-stranded **siRNA**  $N$ -mer with terminal caps, we number residues consecutively in the  $5' \rightarrow 3'$  direction, first along the guide strand and then along the passenger strand. Thus, the guide strand residues are indexed by*

$$1, 2, \dots, N,$$

and the passenger strand residues by

$$N + 1, N + 2, \dots, 2N.$$

By construction, residues 1 and  $N$  are the terminal caps of the guide strand, while residues  $N + 1$  and  $2N$  are the terminal caps of the passenger strand. Therefore, the nucleotides of interest are given by

$$2, 3, \dots, N - 1 \quad \text{and} \quad N + 2, N + 3, \dots, 2N - 1.$$

Each atom in our PDB files has a field which indicates what residue it belongs to. By default, this is set to the letter of the nucleotide's base - but this is not enough for the case of modified **siRNA** molecules. The next step therefore uses the annotated `tsv` files to update the PDB file with standardized cap's residue identities with the `nucleotide_library` code for each modified nucleotide. These PDB files are kept in a separate subfolder within the parametrization folder to facilitate monitoring the process.

Next, the `nucleotide_library`-annotated PDB files are processed again, this time changing their atom names to match AMBER naming conventions. This step relies on template PDB files for each nucleotide which are automatically generated when populating the `nucleotide_library`. Once again, the resulting files are kept in a separate subfolder within the parametrization folder to facilitate monitoring the process.

Finally, one last cleaning step resolves a small set of convention issues with the resulting PDB files that were not addressed by the previous normalization. This step is not as systematic as the others, but rather relies on a case-by-case approach to solve the small set of incompatibilities that could not be systematically resolved by the previous operations. The output are clean PDB files (which indeed receive the suffix "\_clean" and are, once again, kept in a separate folder to facilitate monitoring) suitable for `tleap`.

### 4.3.5 Molecular dynamics input files generation

System parametrization is then performed with `tleap` [62]. `tleap` takes as input the cleaned PDB files and the parametrization data in the `nucleotide_library` to produce AMBER topology and coordinate files. This step also creates the simulation box, which we set to be an orthorhombic box with a buffer of 12Å (i.e., the box dimensions span 12 extra Å from the  $x_{min}, x_{max}, y_{min}, y_{max}, z_{min}, z_{max}$  of the input system). The `tleap` setup must also perform ion addition, introducing  $\text{Na}^+$  and  $\text{Cl}^-$  to achieve

an approximate salt concentration of 150 mM, which matches the conditions inside a cell. Given that all our systems have very similar sizes, we determine a fixed number of ions based on the average resulting box volume. Note that water molecules are added automatically.

The AMBER output files (`.prmtop` and `.inpcrd`) are stored in a subsubfolder with name system's index in the `sirna_unique_trimers_modXNA_notermini` dataset which resides in a `running_tleap` subfolder within the parametrization folder.

Next, these AMBER output files are converted into GROMACS-compatible formats (`.top` and `.gro`) using ParmEd [20]. Afterwards, we generate the following system-specific files, which are stored within each system's subsubfolder in the `running_tleap` subfolder:

- index files using GROMACS [16] `make_ndx` (stored as `index.ndx`)
- position restraints using GROMACS `genrestr` (stored as `posre_duplex.itp`). The specified position restraints define an optional harmonic potential around the non-Hydrogen atoms of the pentamer.

Finally, the generated topology files are edited so that the position restraint files can be selectively activated during equilibration.

### 4.3.6 Pentamer simulation details

Each capped trimer, i.e. each resulting pentamer duplex, is simulated in four successive stages: energy minimization, NVT equilibration, NPT equilibration with simulated annealing, and a final production MD run. The procedure is summarised in Table 4.1, but we also include a short explanation below:

1. First, energy minimization relaxes the pentamer structure, removing any steric clashes and other high-energy configurations inherited from model building. The configuration file for this step is `minimization.mdp`. The exact parameter choices for this step are outlined in tables 4.1 & 4.2.
2. Next, velocities are generated from a Maxwell distribution at 100 K and a 100 ps NVT equilibration is run to warm-start the system and relax water and ions around the duplex, which is kept in place through the pre-defined position restraints. Further, bonds involving hydrogen atoms are

constrained with LINCS. For the purposes of this thesis, it is enough to note that these bond vibrations occur on very short timescales, and in explicit time-integration schemes the time-step must be small enough to resolve the fastest motions in the system. By constraining these fastest bond vibrations, LINCS provides a standard approximation that allows the time-step to be increased to 2 fs while maintaining stable integration. The same LINCS restraints and 2 fs time-step are also used for all the rest of the MD simulation stages. The exact parameter choices for this step are outlined in 4.1 & 4.2. The configuration file for this step is `nvt_equil.mdp`

3. The third stage progressively heats the system while keeping the pressure around 1 bar through pressure coupling (i.e., a barostat). This procedure is called NPT annealing, and it heats the system from 100 K to 300 K over 100 ps, thus reaching conditions that closely resemble real physiological conditions. During this stage, the duplex is also kept in place through the pre-defined position restraints. The configuration file for this step is `npt_anneal.mdp`. The exact parameter choices for this step are outlined in 4.1 & 4.2.
4. The final stage is an unrestrained production run performed under NPT conditions. Temperature is maintained at 300 K and pressure is maintained at 1 bar. Energies, log file and compressed coordinates are written out every 40 ps. We evaluate production run lengths of 15, 20 and 25 ns for 10 test systems by looking at the output whole-pentamer RMSD.

We will run 5 replicas (i.e., 5 distinct simulations) for each capped trimer in `sirna_unique_trimers_modXNA_notermini`.

### 4.3.7 Load optimization

The very last step in the MD simulations preparation and execution workflow is the optimization of computational load across the available SLURM partitions. We achieve this by formulating a load optimization problem, which is a particular type of **Mixed Linear Integer Programming (MILP)** problem, whose purpose is to distribute pending MD simulations across heterogeneous CPU and GPU queues so as to minimize the total wall-clock completion time, i.e. the make-span.

The optimization is formulated at the level of partitions, where each partition is represented by a `PartitionConfig` object. Each partition has as

Table 4.1: Simulation protocol used for capped trimer (pentamer) duplexes.

| Stage                            | Integrator       | Length             | Ensemble | Restraints |
|----------------------------------|------------------|--------------------|----------|------------|
| Energy minimization              | Steepest descent | Up to 50,000 steps | —        | No         |
| NVT equilibration                | Leap-frog MD     | 100 ps             | NVT      | Yes        |
| NPT equilibration with annealing | Leap-frog MD     | 100 ps             | NPT      | Yes        |
| Production MD                    | Leap-frog MD     | 25 ns              | NPT      | No         |

attributes an expected initial queueing time, an expected runtime per submitted job, an expected delay between successive job batches, the number of jobs that may run simultaneously in that partition (i.e., the number of jobs per batch), and the number of simulation runs bundled into a single job.

For each partition  $i$ , the configuration fields and their interpretation are summarised in Table 4.3. The `max_job_duration_to_use` attribute is also stored in the object, although it is not part of the optimization itself; rather, it is used later when generating the SLURM job scripts to allow for potential overruns due to crowding or particularly computationally intensive MD configurations.

Suppose that we have  $N$  systems in `sirna_unique_trimers_modXNA_notermini` and that each system requires  $m$  replicate simulations. In our case,  $m = 5$  by default. The total number of simulation tasks is therefore  $T = Nm$

Our load optimization problem may be formulated in terms of four sets of decision variables:  $n_i$ , the number of jobs assigned to partition  $i$ ;  $k_i$ , the number of batches required in partition  $i$ ;  $a_i$ , the number of complete systems assigned to partition  $i$ ; and  $x$ , the make-span, i.e. the maximum completion time across all partitions. However, these variables are not independent from each other:

- The auxiliary variable  $a_i$  enforces that systems are not split across partitions, this is, that all  $m$  replicas of a system are run in the same partition. Since each system contributes  $m$  simulation tasks and each job in partition  $i$  executes (at most)  $s_i$  tasks, the number of jobs assigned to partition  $i$  is determined by  $n_i = \lfloor \frac{a_i m}{s_i} \rfloor$ .

In practice we always have at least one partition with  $s_i = 1$ , which

Table 4.2: Main molecular dynamics parameters used across the pentamer simulation workflow.

| Parameter                              | Value / choice                                                                     |
|----------------------------------------|------------------------------------------------------------------------------------|
| Minimization algorithm                 | Steepest descent                                                                   |
| Minimization tolerance                 | 10.0 kJ mol <sup>-1</sup> nm <sup>-1</sup>                                         |
| Maximum minimization steps             | 50,000                                                                             |
| MD integrator                          | Leap-frog                                                                          |
| Time step                              | 2 fs                                                                               |
| Equilibration lengths                  | 100 ps NVT; 100 ps NPT annealing                                                   |
| Production length                      | 25 ns                                                                              |
| Bond constraints                       | Bonds involving H atoms with LINCS                                                 |
| Electrostatics                         | PME                                                                                |
| Coulomb cut-off                        | 1.0 nm                                                                             |
| van der Waals cut-off                  | 1.0 nm                                                                             |
| Cut-off scheme                         | Verlet                                                                             |
| Dispersion correction                  | EnerPres                                                                           |
| Periodic boundary conditions           | xyz                                                                                |
| Thermostat                             | V-rescale                                                                          |
| Temperature coupling groups            | RNA (pentamer) and Water_and_ions                                                  |
| Equilibration temperatures             | 100 K in NVT; annealed from 100 K to 300 K in restrained NPT                       |
| Production temperature                 | 300 K                                                                              |
| Pressure coupling during equilibration | Berendsen, isotropic, 1.0 bar                                                      |
| Pressure coupling during production    | Parrinello–Rahman, isotropic, 1.0 bar                                              |
| Position restraints                    | Active in NVT equilibration & NPT annealing; inactive in minimization & production |
| Compressed trajectory output           | Every 40 ps during production                                                      |
| Energy and log output                  | Every 40 ps during production                                                      |

Table 4.3: Attributes of the PartitionConfig objects used in load optimization.

| Attribute               | Symbol | Meaning                                                                                                       |
|-------------------------|--------|---------------------------------------------------------------------------------------------------------------|
| name                    | —      | Partition name used for bookkeeping and SLURM script generation.                                              |
| queue_time              | $q_i$  | Expected initial waiting time in the queue before the first batch of jobs starts.                             |
| job_duration            | $t_i$  | Expected runtime of one submitted job in that partition.                                                      |
| max_job_duration_to_use | —      | Wall-time to request in the generated SLURM script; stored for execution, not used in the optimization model. |
| requeue_time            | $r_i$  | Expected delay between consecutive batches of jobs in the same partition.                                     |
| parallel_capacity       | $l_i$  | Maximum number of jobs that can run simultaneously in that partition.                                         |
| tasks_per_job           | $s_i$  | Number of simulation tasks grouped into a single submitted job.                                               |

allows us to impose the stricter equality  $n_i = \frac{a_i m}{s_i}$  while preserving the feasibility of the problem. This ensures no job is submitted with empty task slots and keeps the interpretation simpler. Alternatively, one could use  $n_i s_i = a_i m + u_i$ , introducing an extra variable  $0 \leq u_i < s_i$  which would quantify the number of empty task slots in the last job for partition  $i$ .

- For partition  $i$  receiving  $n_i$  jobs, the number of required batches is determined by its parallel capacity  $l_i$  as  $k_i = \left\lceil \frac{n_i}{l_i} \right\rceil$ .
- The sum of the number of allocated systems to each partition must match the total number of systems to be simulated, this is,  $\sum_i a_i = N$ .

The completion time for a partition  $i$  is then modelled as

$$\tau_i = q_i + k_i t_i + (k_i - 1) r_i \quad (4.1)$$

where  $q_i$  is the initial queueing time,  $t_i$  is the runtime per job, and  $r_i$  is the expected delay between consecutive batches. Note that this completion estimate relies on a batch approximation which assumes all jobs within the same batch are run simultaneously. A slightly more realistic formulation

would explicitly model both the delay between consecutive jobs being launched and the delay between one job finishing and the next job in the same execution position, but in the following batch, being launched.

Assuming these latter conditions were a perfect description of our system, the potential error introduced by the batch approximation would stem from the fact that when the final set of jobs does not fill a complete batch, some of those jobs may already have started before the latest jobs of the preceding full batch have finished. In such cases, the batch approximation may overestimate the true completion time by counting an additional batch that does not actually extend the partition completion time. Thus, the discrepancy between the fixed-delay model and the batch approximation is upper bounded by at most one extra batch contribution, namely  $t_i + r_i$ . Therefore, assuming that the fixed-delay model is a reliable description of our system, if  $t_i + r_i$  is small relative to the total partition completion time then the batch approximation provides a reasonable and tractable model for scheduling.

While we know that the fixed-delay model is still far from the true behaviour of our system, it is difficult to improve substantially on these assumptions because doing so would require live, partition-specific scheduling information and a dynamic allocation procedure. In practice, this is remarkably inconvenient: for example, jobs that have already spent time in the queue typically gain scheduling priority, so the state of the queue evolves in a way that is difficult to model accurately.

Consequently, we settle for the batch approximation: we are dealing with  $t_i + r_i$  in the order of hours, while the simulation of all the systems in each partition will span multiple days, even weeks.

The resulting **MILP** problem is

$$\min x \quad (4.2)$$

$$\text{s.t. } \sum_i a_i = N \quad (4.3)$$

$$n_i s_i = a_i m \quad \forall i \quad (4.4)$$

$$n_i \leq l_i k_i \quad \forall i \quad (4.5)$$

$$q_i - r_i + k_i (t_i + r_i) \leq x \quad \forall i \quad (4.6)$$

$$n_i, k_i, a_i \in \mathbb{Z}_{\geq 0}, \quad x \in \mathbb{R}_{\geq 0} \quad (4.7)$$

Optional additional constraints can also be imposed in the implementation through user-provided dictionaries specifying minimum or maximum numbers

of jobs per partition,

$$n_i \leq n_i^{\max} \quad \text{and/or} \quad n_i \geq n_i^{\min} \quad (4.8)$$

for selected partitions.

The optimization is solved with `PuLP`, using the `CBC` solver by default. Once solved, the model returns the number of jobs, systems and simulation tasks assigned to each partition, together with the number of required batches and the optimal make-span. The solution is then checked to verify that the total number of allocated tasks matches  $Nm$  and that the total number of assigned systems matches  $N$ .

It must be remembered that in the real execution not all jobs will last the same amount of time, and delay times between consecutive jobs being launched will vary based on external factors such as total partition load. Consequently, the approximated delay between batches will also vary, which prompts us to put a series of tools in place that allow us to re-optimize the allocation when conditions change and also manually re-route systems for individual interventions.

We were given access to a series of partitions distributed across an AstraZeneca Swedish cluster and an AstraZeneca US cluster. From now on, we refer to these as the Swedish cluster and the US cluster. Once we obtain an allocation using the available partition's specifications, this result is used to create partition-specific execution directories and corresponding SLURM submission scripts. Separate script generation routines are used for CPU and GPU partitions to account for their distinct execution patterns and batching strategies. For each assigned subset of systems, the workflow copies the necessary input files, writes the appropriate job scripts and records both the allocation map and the partition configurations in respective `json` files.

## 4.4 Feature Extraction

Once the **MD** simulations for all systems in `sirna_unique_trimers_modXNA_notermini` have been run, we must extract the features that will then be used to inform our **ML** models. This section addresses not only the choice of what features to extract but also the process and techniques involved in their extraction.

### 4.4.1 Trajectory Cleaning and New Index File Creation

The raw **MD** trajectories obtained from the production step are not automatically suitable for feature extraction. First, we must remove artefacts introduced by periodic boundary conditions. Further, in order to extract observables linked to individual nucleotides, we need an index file containing the residue-level atom groups.

#### 4.4.1.1 Trajectory Cleaning

The production trajectories generated by GROMACS are stored in the file `md.xtc`. Because the simulations are run under periodic boundary conditions, the coordinates in this file may exhibit jumps when the molecule crosses the box boundaries.

The cleaning procedure is performed in two successive applications of `gmx trjconv`, both restricted to the atom group corresponding to the modified **siRNA** duplex.

In the first step, `gmx trjconv` is run with the `-pbc nojump` option. This reconstructs a continuous molecular path by removing artificial jumps caused by crossing the periodic boundaries; ensuring that the trajectory of any atom that leaves one side of the simulation box and re-enters through the opposite side is re-imaged so that the motion remains continuous from frame to frame. The output of this stage is an intermediate trajectory, `md_nojump.xtc`.

In a second pass, `gmx trjconv` is applied to the intermediate trajectory with the `-pbc mol` option. This re-images the system at the molecular level, ensuring that each molecule is kept whole rather than split across periodic images. The resulting output, `md_clean.xtc`, is the trajectory used in all subsequent analyses.

#### 4.4.1.2 New Index File Creation

Feature extraction in oligonucleotide chains usually looks at nucleotide-level properties and observables arising from the interaction between multiple nucleotides, on top of whole-chain features. Thus, we must identify the atom groups which compose each residue.

We create an additional index file in which each nucleotide residue is assigned its own index group, together with auxiliary groups for the whole RNA and for the solvent-plus-ion environment.

A call to `gmx make_ndx` (using the coordinate file `rna.gro` is used to generate a custom index file. The atom selections are the following:

- the full modified **siRNA** group, containing all **siRNA** atoms
- a `Water_and_ions` group, containing water molecules and the monovalent ions present in the simulation
- one group per nucleotide residue in the guide strand
- one group per nucleotide residue in the passenger strand

We create two of these custom files, one where the nucleotide residue groups include their corresponding hydrogen atoms and one in which they do not. This distinction is useful because some downstream analyses require explicit hydrogens (such as hydrogen bond analysis), whereas others are more naturally performed on heavy-atom representations (for example, **Root Mean Square Deviation (RMSD)** or **RMSF**).

#### 4.4.2 Observable time-series and equilibrium detection

Given a clean **MD** trajectory sampled every 40 ps, any observable we may extract from the trajectory will also be sampled every 40 ps. However, we are generally interested in using the **MD** simulations we have run as samplers of each trimer's state-space in the NPT ensemble rather than approximants of the true trimer's dynamics, which requires letting each simulation reach equilibrium before samples are considered (otherwise, one would not be sampling from the target equilibrium distribution).

In practice, this implies that all samples taken before equilibrium was reached must be discarded. There are two main approaches to select how many frames to discard from the beginning of the time-series to ensure that the remaining frames are samples from the equilibrium distribution.

The first option is to infer when is equilibrium reached for each observable in each run using automatic equilibration detection. This procedure, introduced in [63], consists on the following: given a time-series  $X_0, X_1, \dots, X_{T-1}$  for a certain observable, the statistical inefficiency  $g(t_0)$  for the truncated time-series  $X_{t_0}, X_{t_0+1}, \dots, X_{T-1}$  for a series of candidate truncation points  $t_0$  (which can be the whole time-series) is estimated. The effective number of uncorrelated samples remaining after truncation is then

computed as

$$N_{\text{eff}}(t_0) = \frac{T - t_0}{g(t_0)}.$$

The selected equilibration time is the value  $t^*$  that maximizes this quantity,

$$t^* = \arg \max_{t_0} \frac{T - t_0}{g(t_0)}.$$

It is important to note that this procedure is a heuristic rather than a proof that the system has reached global equilibrium.

The second option is also heuristic and consists on determining a safe truncation point that provides a solid guarantee that the system has reached equilibrium.

We will explore both approaches to determining equilibrium. The developed package `sirna_MDprep` allows the user to select an equilibrium determination technique.

#### 4.4.3 Processing equilibrium time-series as histograms

Once the time-series for the equilibrium dynamics has been extracted, it may be interpreted as a collection of (correlated) samples from a random variable. A common method to reconstruct the probability distribution of the captured observable is to employ a histogram.

A histogram represents a random variable  $X$  which has a piecewise-constant probability density: within each bin, the density is constant.

We will only work with 1-dimensional histograms. Let bin  $i$  have a centre  $m_i$  and width  $w_i$ . Then, its upper edge is  $b_i = m_i + \frac{w_i}{2}$  and its lower edge is  $a_i = m_i - \frac{w_i}{2}$ . Let  $f_i$  be the value of the probability density function of  $X$  at bin  $i$ .

Then the density can be written as

$$f(x) = \begin{cases} f_1, & x \in [a_1, b_1) \\ f_2, & x \in [a_2, b_2) \\ \vdots & \\ f_n, & x \in [a_n, b_n) \\ 0, & \text{otherwise} \end{cases},$$

where  $\sum_{i=1}^n f_i(b_i - a_i) = \sum_{i=1}^n f_i w_i = 1$ .

We always work with bin probability mass, as this value is restricted to

$[0, 1]$  and thus avoids magnitude issues when working with **ML** models. Note that  $p_i := p(x \in \text{bin}_i) = f_i w_i$  and then we recover  $\sum_{i=1}^n p_i = 1$ .

For each observable with a histogram, the histogram range is chosen by inspecting the pooled equilibrium samples across all systems simulated on AstraZeneca's Swedish cluster (systems were allocated to either the Swedish or the US cluster by the load optimization algorithm described in Section 4.3.7). More precisely, unless stated otherwise, we extract the 0.5 and 99.5 percentiles of the equilibrium observations and use these values to define a central interval covering the overwhelming majority of the data. Samples outside the selected interval are assigned to outlier bins.

To avoid overfitting, we do not inform our choice on any of the systems simulated on AstraZeneca's US cluster. Further, for those observables computed on repeated substructures of a pentamer, such as individual residues or residue pairs, the pooled sample used for bin selection includes all instances of the same type.

#### 4.4.4 Extracted features

There are two possible approaches to observable selection. If one disposes of previous knowledge identifying a small set of observables which determine the behaviour of the system, feature extraction can be limited to those properties.

Alternatively, if there is no such information available, one must resort to extracting as many features as possible - paying special attention to those which hold general biological relevance.

We settle on this second approach due to the absence of previous studies leveraging dynamical properties for **siRNA** activity production. This choice gives raise to a richer dataset of double-stranded, modified **siRNA** trimer properties which we hope will prove useful for the prediction of other modified **siRNA** properties.

The observables for which we have extracted features are **RMSD**, **RMSF**, base-pair and helical parameters, sugar pucker angle, interaction energies and several hydrogen bond-related properties.

##### 4.4.4.1 RMSD and RMSF

**RMSD** and **RMSF** both measure geometric variation, but they do so in different ways. On the one hand, **RMSD** quantifies how far a given configuration lies from a chosen reference structure, i.e., it measures structural deviation at any given time. On the other hand, **RMSF** quantifies the

magnitude of the fluctuations of an atom group around its average position over the trajectory, yielding one single scalar per trajectory.

Let the Cartesian reference coordinates of the  $N$  atoms selected for **RMSD** be denoted by  $\mathbf{r}_1^{\text{ref}}, \dots, \mathbf{r}_N^{\text{ref}}$  and let  $\mathbf{r}_1(t), \dots, \mathbf{r}_N(t)$  be their coordinates at a given time in the trajectory. Let  $O^*(t) \in SE(3)$  be the optimal rigid-body transformation from the given coordinates to the reference coordinates at time  $t$ , this is

$$O^*(t) = \operatorname{argmin}_{O \in SE(3)} \sum_{i=1}^N \|O\mathbf{r}_i(t) - \mathbf{r}_i^{\text{ref}}\|^2.$$

Then, the **RMSD** at time  $t$  is defined as

$$\text{RMSD}(t) = \sqrt{\frac{1}{N} \sum_{i=1}^N \|O^*(t)\mathbf{r}_i(t) - \mathbf{r}_i^{\text{ref}}\|^2}. \quad (4.9)$$

In our case, the reference structure is taken to be the first frame of the cleaned trajectory. Therefore, for each frame after, the reported **RMSD** measures the structural deviation of the selected group with respect to the initial sampled conformation. It is for this reason that we pay special attention to the **RMSD** of both the whole double-stranded pentamer and also each of the residues in the original trimer to determine equilibration times, as we expect these observables to stabilize after the initial transient state away from equilibrium. Nevertheless, it is important to note that the **RMSD** may shift between different modes (given by different conformational states) even after equilibrium has been reached.

**RMSF** is instead a time-averaged fluctuation measure. Using the previously defined  $O^*(t)$ , define

$$\bar{\mathbf{r}}_i = \langle O^*(t)\mathbf{r}_i(t) \rangle_t,$$

which denotes the average position of atom  $i$  over the analysed trajectory segment. Here,  $\langle \cdot \rangle_t$  denotes the time average over the frames in the trajectory. Then, the atom-level fluctuation is

$$\text{RMSF}_i = \sqrt{\left\langle \frac{1}{N} \sum_{i=1}^N \|O^*(t)\mathbf{r}_i(t) - \bar{\mathbf{r}}_i\|^2 \right\rangle_t}, \quad (4.10)$$

Since the **RMSF** is computed for the whole trajectory, it is not amenable to restrictions to a certain region of the system's state-space. As described in 4.4.5, we will only report equilibrium property distributions restricted to

the state-space region corresponding to the pentamer in the double-stranded configuration. The **RMSF** is one of the two only exceptions to this measure: for each nucleotide, we report the **RMSF** of the whole trajectory, including the transient state away from equilibrium and any potential frames outside of the double-stranded region of the state-space. While this measure is dictated by the difficulty of computing the **RMSF** across some specific frames only, there is some intrinsic value in quantifying the "total" fluctuation of the residues - even under strand dissociation.

We extract these observables from the cleaned trajectory `md_clean.xtc`. The considered groups for each of the observables as well as the specific commands and resulting file formats are presented in table 4.4.

| Property           | <b>RMSD</b>          | <b>RMSF</b>                |
|--------------------|----------------------|----------------------------|
| Whole pentamer     | Yes                  | No                         |
| Per nucleotide     | Yes                  | Yes                        |
| Hydrogens included | No                   | No                         |
| Output             | Time-series (.xvg)   | Scalar (.xvg)              |
| GROMACS command    | <code>gmx rms</code> | <code>gmx rmsf -res</code> |

Table 4.4: Summary of **RMSD** and **RMSF** extraction.

#### 4.4.4.2 Base-pair, base-pair step and helical parameters

The `nastruct` analysis implemented in `cpptraj` [64] extracts a series of structural parameters which provide a compact description of local duplex geometry at three different levels: individual base pairs, consecutive base-pair steps and the local helical frame.

The analysis is run on the cleaned trajectory `md_clean.xtc` using the topology file `rna.top`, after stripping water and ions from the system. The corresponding `cpptraj` routine produces three output files, namely `BP.nastruct.dat`, `BPstep.nastruct.dat` and `Helix.nastruct.dat`, containing the base-pair, base-pair step and helical parameters values as a time-series for each of the base-pair and base-pair pairs of the system (where base-pair step and helical parameters are both computed for consecutive base-pair pairs).

Since these descriptors are standard quantities (see 2.4 for a visual representation), we do not reproduce their mathematical definitions here. This choice is also motivated by the fact that, in our systems, these parameters proved substantially less robust than the remaining observables: for modified

trimers, `nastruct` often fails to compute part of the descriptors or returns values with an unclear physical interpretation.

The extracted parameters are listed in tables 4.5, 4.6 and 4.7, together with their natural type. Here, *linear* denotes a quantity that is not periodic and is therefore treated as an ordinary real-valued variable, while *angular* denotes a periodic variable defined on the interval  $[-180, 180]$ .

| Base-pair parameter | Type / range                     |
|---------------------|----------------------------------|
| Shear               | Linear ( $\mathbb{R}$ )          |
| Stretch             | Linear ( $\mathbb{R}$ )          |
| Stagger             | Linear ( $\mathbb{R}$ )          |
| Buckle              | Angular ( $[-180, 180]$ )        |
| Propeller           | Angular ( $[-180, 180]$ )        |
| Opening             | Angular ( $[-180, 180]$ )        |
| Major               | Linear ( $\mathbb{R}_{\geq 0}$ ) |
| Minor               | Linear ( $\mathbb{R}_{\geq 0}$ ) |

Table 4.5: Base-pair parameters extracted with `nastruct`

| Base-pair step parameter | Type / range              |
|--------------------------|---------------------------|
| Shift                    | Linear ( $\mathbb{R}$ )   |
| Slide                    | Linear ( $\mathbb{R}$ )   |
| Rise                     | Linear ( $\mathbb{R}$ )   |
| Tilt                     | Angular ( $[-180, 180]$ ) |
| Roll                     | Angular ( $[-180, 180]$ ) |
| Twist                    | Angular ( $[-180, 180]$ ) |
| Zp                       | Linear ( $\mathbb{R}$ )   |

Table 4.6: Base-pair step parameters extracted with `nastruct`.

#### 4.4.4.3 Pucker angle

The sugar pucker phase angle describes the conformation of the sugar in each nucleotide. We extract the pucker angle for each residue in the pentamer with `cpptraj` [64] as part of the same analysis stage used to compute the `nastruct` structural parameters. For each residue with a ringed sugar (some modifications introduce ring-less sugars, see 5.2), the pucker angle is computed from the ring atoms  $C1'$ ,  $C2'$ ,  $C3'$ ,  $C4'$  and  $O4'$ , and written to the file `pucker.agr` using the `range360` option. As a result, the pucker angle is reported as a periodic variable on the interval  $[0, 360]$ .

| Helical parameter | Type / range              |
|-------------------|---------------------------|
| X-displacement    | Linear ( $\mathbb{R}$ )   |
| Y-displacement    | Linear ( $\mathbb{R}$ )   |
| Rise              | Linear ( $\mathbb{R}$ )   |
| Inclination       | Angular ( $[-180, 180]$ ) |
| Tip               | Angular ( $[-180, 180]$ ) |
| Twist             | Angular ( $[-180, 180]$ ) |

Table 4.7: Helical parameters extracted with `nastruct`.

We classify the extracted sugar pucker time-series into 8 conformational regions. These are then used as the bins for the the sugar pucker histogram. The adopted partition of the interval  $[0, 360]$  is the following:

| ID | Conformation     | Angular interval          |
|----|------------------|---------------------------|
| 0  | C3'-endo (North) | $[0, 36) \cup [324, 360]$ |
| 1  | C4'-exo          | $[36, 72)$                |
| 2  | O4'-endo (East)  | $[72, 108)$               |
| 3  | C1'-exo          | $[108, 144)$              |
| 4  | C2'-endo (South) | $[144, 216)$              |
| 5  | C3'-exo          | $[216, 252)$              |
| 6  | C4'-endo         | $[252, 288)$              |
| 7  | O4'-exo (West)   | $[288, 324)$              |

Table 4.8: Classification of sugar pucker phase angles into standard conformational regions.

#### 4.4.4.4 Hydrogen bonds

Hydrogen bonds are one of the most informative observables for the structural characterization of double-stranded oligonucleotides, as they directly dictate inter-strand interactions. We analyse hydrogen bonds between base-pairs and also their closest intra- and inter-strand neighbours. For each such residue pair, we get the number of shared hydrogen bonds as a time series and identify the observed donor–hydrogen–acceptor triplets with GROMACS; and then characterize each detected hydrogen bond through its occupancy, lifetime, donor–acceptor distance and donor–hydrogen–acceptor angle with MDAnalysis [65, 66].

Since the passenger strand is annotated in the 5' to 3' direction instead of the 3' to 5' direction, the canonical base-pair partner of guide residue  $i$

is passenger residue  $2N + 1 - i$ . From this relation, we can enumerate the members of each of the three classes of residue pairs:

- **bp**: these are the canonical base-pair contacts between the two strands, excluding caps:

$$\mathcal{D}_{bp} = \{(i, 2N + 1 - i) \mid i = 2, \dots, N - 1\}. \quad (4.11)$$

- **c1**: these are first-neighbour pairs along the same strand, again excluding caps:

$$\mathcal{D}_{c1} = \{(i, i+1) \mid i = 2, \dots, N-2\} \cup \{(N+i, N+i+1) \mid i = 2, \dots, N-2\}. \quad (4.12)$$

- **d1**: these are diagonal inter-strand residue pairs that are not canonical base-pair partners but lie adjacent to them across the duplex; once again excluding caps:

$$\mathcal{D}_{d1} = \{(i, 2N-i) \mid i = 2, \dots, N-2\} \cup \{(i, 2N+2-i) \mid i = 3, \dots, N-1\}. \quad (4.13)$$

For example, in a pentamer we have guide residues 1,2,3,4,5 and passenger residues 6,7,8,9,10, with caps at 1, 5, 6 and 10. The relevant nucleotides are therefore 2,3,4 on the guide strand and 7,8,9 on the passenger strand. The canonical base pairs are (2,9), (3,8) and (4,7); the **c1** pairs are (2,3), (3,4), (7,8) and (8,9); and the **d1** pairs are (2,8), (3,9), (3,7) and (4,8). These relations are illustrated in Figure 4.1.

GROMACS and MDAnalysis have competing geometric definitions of a hydrogen bond. Let  $D$ ,  $H$  and  $A$  denote the donor, hydrogen and acceptor atoms, respectively. At a given frame  $t$ , GROMACS considers the triplet  $(D, H, A)$  to form a hydrogen bond if

$$d_{DA}(t) \leq r_G, \quad \theta_{ADH}(t) \leq \alpha_G, \quad (4.14)$$

where  $d_{DA}(t)$  is the donor–acceptor distance and  $\theta_{ADH}(t)$  is the acceptor–donor–hydrogen angle (i.e., with the donor as vertex)

Meanwhile, MDAnalysis considers the triplet  $(D, H, A)$  to form a hydrogen bond if

$$d_{DA}(t) \leq r_M, \quad \theta_{DHA}(t) \geq \alpha_M, \quad (4.15)$$

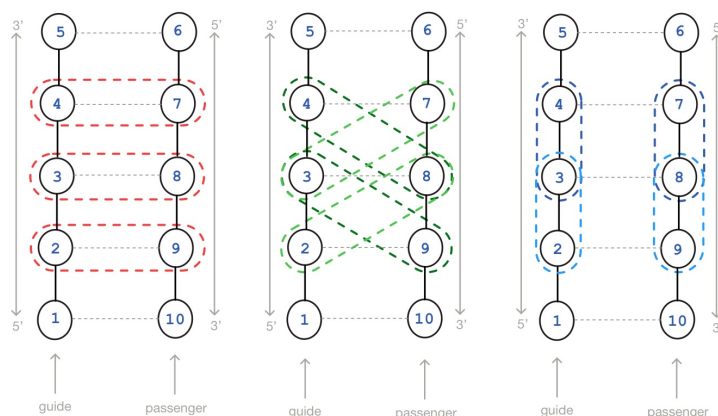


Figure 4.1: Illustration of the residue pair sets  $\mathcal{P}_{bp}$  (4.11) (left, in red),  $\mathcal{P}_{dl}$  (4.13) (middle, in green) and  $\mathcal{P}_{cl}$  (4.12) (right, in blue) on a generic pentamer with numbered residues

where  $\theta_{DHA}(t)$  is the donor–hydrogen–acceptor angle (i.e., with the hydrogen as vertex).

By default,  $r_G = 3.5\text{\AA}$ ,  $\alpha_G = 30^\circ$  and  $r_M = 3\text{\AA}$ ,  $\alpha_M = 150^\circ$ . Figure 4.2 illustrates a  $(D, H, A)$  triplet in its natural plane with its main geometric elements.

We favour GROMACS definition and parameters, but we must perform certain parts of the hydrogen bond analysis in MDAnalysis for two main reasons:

- GROMACS directly returns histograms for the hydrogen bond distances and angles, which would stop us from capturing only the distribution of these observables conditioned to the pentamer being in the double-stranded region of the state-space.
- MDAnalysis greatly facilitates the computation of the hydrogen bonds lifetimes.

Unfortunately, there is no natural equivalence relation between these two criteria. Crucially, note that it is not enough to set some lax MDAnalysis parameters to ensure all hydrogen bonds are detected, because that would mean some hydrogen bonds would be marked as present in frames where they should not be.

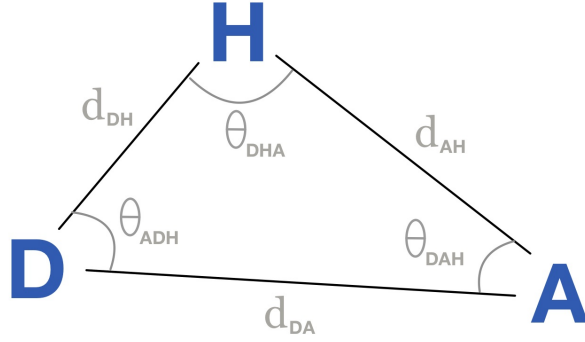


Figure 4.2: Illustration of a triplet  $(D, H, A)$  in its natural plane and the variables  $d_{DH}, d_{DA}, d_{AH}, \theta_{DHA}, \theta_{ADH}, \theta_{DAH}$

We derive a condition on MDAnalysis parameters which guarantees that every detected hydrogen bond by MDAnalysis is also a hydrogen bond according to GROMACS criterion.

**Lemma 48** (MDAnalysis to GROMACS H-bond criteria). *If  $r_M = r_G$ ,  $d_{DH} \in [b_m, b_M]$  and  $d_{DA} \geq c_m$  with  $b_m \leq b_M \leq c_m \in \mathbb{R}_{\geq 0}$  and  $\theta_{DHA} \geq 90^\circ$ , then solving*

$$\alpha_G = 180^\circ - x - \arcsin\left(\frac{b_m}{r_G} \sin x\right)$$

and choosing

$$\alpha_M = x, \quad r_M = r_G$$

guarantees that

$$(D, H, A) \text{ H-bond in MDAnalysis} \Rightarrow (D, H, A) \text{ H-bond in GROMACS}$$

if  $x \geq 90^\circ$

*Proof.* By the law of sines, we have that

$$\frac{\sin \theta_{ADH}}{d_{AH}} = \frac{\sin \theta_{DAH}}{d_{DH}} = \frac{\sin \theta_{DHA}}{d_{DA}}.$$

Taking the last two and using the fact that  $\theta_{DAH} = 180^\circ - \theta_{ADH} - \theta_{DHA}$  and

$\sin(180^\circ - \theta_{ADH} - \theta_{DHA}) = \sin(\theta_{ADH} + \theta_{DHA})$ , we get:

$$\theta_{DHA} + \theta_{ADH} = \begin{cases} \arcsin\left(\frac{d_{DH}}{d_{DA}} \sin \theta_{DHA}\right) & \in [-90^\circ, 90^\circ] \\ 180^\circ - \arcsin\left(\frac{d_{DH}}{d_{DA}} \sin \theta_{DHA}\right) & \in [90^\circ, 270^\circ] \end{cases} ,$$

which are the two branches of the inverse of the sinus function.

If we impose  $\alpha_M \geq 90^\circ$ , then we can restrict ourselves to the second branch because if  $(D, H, A)$  forms an MDAnalysis H-bond then  $\theta_{DHA} \geq \alpha_M \geq 90^\circ$  and automatically  $\theta_{DHA} + \theta_{ADH} \geq 90^\circ$ .

We want to guarantee that the supremum of  $\theta_{ADH}$  given the previous expression - and knowing that  $\theta_{DHA} \geq \alpha_M$  for the value of  $\alpha_M$  that we choose - is  $\alpha_G$ . Thus, we study the expression

$$\theta_{ADH} = 180^\circ - \theta_{DHA} - \arcsin\left(\frac{d_{DH}}{d_{DA}} \sin \theta_{DHA}\right).$$

From now on, assume  $(D, H, A)$  indeed forms a hydrogen bond in MDAnalysis, and thus  $\theta_{DHA} \geq \alpha_M \geq 90^\circ$ .

Let

$$f(\gamma) = 180^\circ - \gamma - \arcsin\left(\frac{d_{DH}}{d_{DA}} \sin \gamma\right)$$

with  $\gamma \in [\alpha_M, 180^\circ]$ . Then,

$$f'(\gamma) = -1 - \frac{\frac{d_{DH}}{d_{DA}} \cos \gamma}{\sqrt{1 - \left(\frac{d_{DH}}{d_{DA}} \sin \gamma\right)^2}}.$$

By assumption, we have that  $\frac{d_{DH}}{d_{DA}} \leq \frac{b_M}{c_m} =: U \leq 1$ . As  $\gamma \in [90^\circ, 180^\circ]$ , it holds that  $\cos \gamma \leq 0$  and therefore

$$-\frac{\frac{d_{DH}}{d_{DA}} \cos \gamma}{\sqrt{1 - \left(\frac{d_{DH}}{d_{DA}} \sin \gamma\right)^2}} = \frac{\frac{d_{DH}}{d_{DA}} (-\cos \gamma)}{\sqrt{1 - \left(\frac{d_{DH}}{d_{DA}} \sin \gamma\right)^2}} \geq 0.$$

Note also that  $\sqrt{1 - \left(\frac{d_{DH}}{d_{DA}} \sin \gamma\right)^2} \geq \sqrt{1 - (U \sin \gamma)^2}$ . As a result,

$$f'(\gamma) \leq -1 + \frac{U(-\cos \gamma)}{\sqrt{1 - (U \sin \gamma)^2}}.$$

Further, note that  $1 - U^2 \sin^2 \gamma = 1 - U^2 + U^2 \cos^2 \gamma \geq U^2 \cos^2 \gamma$  because  $r^2 \leq 1$ . Plugging this in the previous equation we get

$$f'(\gamma) \leq -1 - \frac{U \cos \gamma}{\sqrt{1 - (U \sin \gamma)^2}} \leq -1 + \frac{U(-\cos \gamma)}{U|\cos \gamma|} = -1 - \text{sign}(\cos \gamma).$$

As we have already established that  $\cos \gamma \leq 0$ , we arrive to

$$f'(\gamma) \leq 0.$$

It therefore follows that

$$\sup_{\gamma \in [\alpha_M, 180^\circ]} f(\gamma) = f(\alpha_M).$$

Observe that if  $(D, H, A)$  forms a hydrogen bond in MDAnalysis and we take  $r_M = r_G$  then  $\frac{d_{DH}}{d_{DA}} \geq \frac{b_m}{r_G} := L$ . We adopt this choice, which yields the following implication:

$$(D, H, A) \text{ H-bond in MDAnalysis} \Rightarrow 0 \leq L \leq \frac{d_{DH}}{d_{DA}} \leq U \leq 1.$$

Let now

$$g(s) = 180^\circ - \gamma - \arcsin(s \sin \gamma)$$

with  $s \in [L, U]$ .

Note that  $\frac{d}{ds} \arcsin(sC) > 0$  for  $sC > 0$  (where  $\arcsin$  here is interpreted as the first branch of the inverse function of the sinus), which means that  $g'(s) < 0$  and thus

$$\sup_{s \in [L, U]} g(s) = g(L).$$

We conclude that the function

$$\theta_{ADH}(\gamma, s) = 180^\circ - \gamma - \arcsin(s \sin \gamma)$$

satisfies

$$\sup_{(\gamma, s) \in [\alpha_M, 180^\circ] \times [L, U]} \theta_{ADH} = 180^\circ - \alpha_M - \arcsin(L \sin \alpha_M)$$

as long as  $\alpha_M \geq 90^\circ$ .

The condition  $(\gamma, s) \in [\alpha_M, 180^\circ] \times [L, U]$  is always satisfied if  $(D, H, A)$  is a hydrogen bond in MDAnalysis. For  $(D, H, A)$  to also be a hydrogen bond

in GROMACS, it must also hold that

$$\sup_{(\gamma,s) \in [\alpha_M, 180^\circ] \times [L, U]} \theta_{ADH} = \alpha_G.$$

We conclude that solving for  $\alpha_M$  such that

$$\alpha_G = 180^\circ - \alpha_M - \arcsin(L \sin \alpha_M)$$

and choosing, of course,  $r_M = r_G$ , guarantees that any hydrogen bond detected by MDAnalysis is also a GROMACS hydrogen bond as long as the obtained  $\alpha_M$  is larger than  $90^\circ$ .  $\square$

The derived condition ensuring every detected hydrogen bond in MDAnalysis is also a hydrogen bond in GROMACS relies on a series of assumptions that cannot be deterministically guaranteed for any possible system trajectory. Nevertheless, it is possible to give parameters  $b_m$ ,  $b_M$  and  $c_m$  such that, with a very high probability, every sample from equilibrium will fall within the defined range: as production MD runs are performed under the NPT ensemble, the conditions of the equipartition theorem hold. Force fields like AMBER model bonds as harmonic potentials around an equilibrium length  $r_0$ :

$$U(\Delta r) = k_r (\Delta r)^2,$$

where  $k_r$  is the bond force constant and  $\Delta r = r - r_0$ .

According to the equipartition theorem:

$$\left\langle \Delta r \frac{d}{d\Delta r} k_r (\Delta r)^2 \right\rangle = k_B T,$$

where  $k_B$  is the Boltzmann constant and  $T$  is the absolute temperature; and therefore we get

$$2k_r \langle (\Delta r)^2 \rangle = k_B T.$$

The standard deviation of the bond-length fluctuation is then

$$\sigma_r = \sqrt{\langle (\Delta r)^2 \rangle} = \sqrt{\frac{k_B T}{2k_r}}.$$

Therefore, the bond length interval at three standard deviations from equilibrium is  $\left[ r_0 - 3\sqrt{\frac{k_B T}{2k_r}}, r_0 + 3\sqrt{\frac{k_B T}{2k_r}} \right]$ .

If we take  $k_r \approx 500 \frac{\text{kcal}}{\text{mol} \cdot \text{\AA}^2}$  (taken from the .pmrtop files), given that we

set the production temperature to 300K and that  $k_B \approx 0.001987 \frac{\text{kcal}}{\text{mol}\cdot\text{K}}$ , we get a standard deviation  $\sigma \approx 0.02441 \text{ \AA}$ .

As seen in Figure 2.3, all standard hydrogen bonds between base pairs have a nitrogen as donor. The nitrogen-hydrogen bond equilibrium length is set to approximately  $1.01 \text{ \AA}$ , which means that the three standard deviations interval is approximately  $[0.937, 1.083]$ .

We take these values to be  $b_m$  and  $b_M$ . We cannot give an equally clean approximation for  $c_m$  because there is no bond potential between donor and acceptor but rather Lennard-Jones (and electrostatic) interactions. We need not worry about these because  $b_M$  sits significantly inside the Lennard-Jones potential repulsive region for heavy atoms, which guarantees that if one determines  $c_m$  with the same statistical accuracy employed for  $b_M$  it is guaranteed that  $b_M \leq c_m$  will hold.

Applying Lemma 48 and solving for  $\alpha_M$  with  $b_m = 0.937 \text{ \AA}$  and  $r_G = 3.5 \text{ \AA}$  we obtain  $\alpha_M \approx 140^\circ$ .

The extraction procedure takes place in two stages. First, GROMACS `gmx hbond` is used to obtain time-series of the number of hydrogen bonds between the two strands and also between each pair of residues in  $\mathcal{D}_{bc} \cup \mathcal{D}_{cl} \cup \mathcal{D}_{dl}$ . This also produces an index file listing the donor–hydrogen–acceptor triplets detected by GROMACS for each residue pair, which is then used to indicate to MDAnalysis what hydrogen bonds to look at.

In a second stage, each of the candidate triplets for every residue pair are re-processed with MDAnalysis to obtain donor–acceptor distance and donor–hydrogen–acceptor angle whenever the hydrogen bond is present, as well as to compute its lifetime curve.

The donor–acceptor distance time-series is given by

$$d_{DA}(t) = \|\mathbf{r}_D(t) - \mathbf{r}_A(t)\|, \quad (4.16)$$

and the donor–hydrogen–acceptor angle time-series is given by

$$\theta_{DHA}(t) = \angle(\mathbf{r}_D(t) - \mathbf{r}_H(t), \mathbf{r}_A(t) - \mathbf{r}_H(t)). \quad (4.17)$$

As the donor–hydrogen–acceptor angle is always constrained to  $[\alpha_M, 180^\circ]$ , there is no need to treat the hydrogen bond angles as periodic observables and can be safely processed as linear.

As for the hydrogen bond lifetime curve, if we define the indicator variable

$$I_{DHA}(t) = \mathbb{1}((D, H, A) \text{ forms a hydrogen bond at frame } t) \quad (4.18)$$

then its survival probability in  $\tau$  frames is defined as

$$S_{DHA}(\tau) = \mathbb{P}(I_{DHA}(t + \tau) = 1 | I_{DHA}(t) = 1), \quad \tau = 0, \dots, \tau_{\max} \quad (4.19)$$

and computed as

$$\frac{1}{|E|} \sum_{t \in E} I_{DHA}(t + \tau), \quad (4.20)$$

where  $E = \{t \mid I_{DHA}(t) = 1, t \leq T - \tau\}$  with  $T$  being the total number of frames.

The resulting survival curve (for which we take  $\tau_{\max} = 30$ ) provides a description of bond persistence and dynamical stability. Clearly, we will always have  $S_{DHA}(0) = 1$ .

Observe that the survival curves are computed only after the equilibration cut-off has been applied, but they are not conditioned to the pentamer being in the double-stranded region of the state-space.

Whenever the bond is not present at a given frame, these quantities are stored as missing values. Hydrogen bonds proposed by GROMACS but never recovered by MDAnalysis are explicitly marked as undetected.

Crucially, note that the number of hydrogen bonds between a given residue pair at each frame within the same run (and also across different runs) fluctuates. Further, the same residue pair in a different pentamer could have a different number of total registered hydrogen bonds. As a result, it would be very difficult to extract statistical summaries for both the geometric properties and the lifetimes of each detected hydrogen bond between a given residue pair and then feed those to a **ML** model.

What we do instead is the following: for each frame, we average out the geometric properties of all hydrogen bonds present in said frame for any given residue pair. Furthermore, we also average out all the lifetime curves of every hydrogen bond between any given residue pair. This ensures dimensions will remain consistent for all residue pairs, across all pentamers.

#### 4.4.4.5 Interaction Energies

Non-bonded interaction energies quantify the strength of the interactions between the two strands as well as between individual nucleotide residues. It is important to note that they do not necessarily match the true, physical interaction energies of the pentamer because force field parameters are adjusted to reproduce other experimental observables (for example, the interaction energy between two residues within the same pentamer would

understandably be difficult to measure experimentally). Nevertheless, non-bonded interaction energies are valuable in terms of the characterization of a system's interactions relative to other systems' interactions.

Given a fixed trajectory, GROMACS can recompute the non-bonded interaction energies associated with a prescribed set of energy groups without re-simulating it. What GROMACS does instead of re-simulating is computing the interaction energies between the indicated groups (which are provided through a modified `.mdp` file specifying the desired `energygrps`) from the trajectory's coordinates. We call this procedure an energy rerun.

We perform two such reruns for each simulation run. In both cases, the full production trajectory `md.xtc` is replayed with `gmx mdrun -rerun` and otherwise uses the same inputs as the original production run.

The first rerun targets strand-level interaction energies using an input file `md_guide_passenger.mdp`. This file differs from the standard production settings only in the definition

```
energygrps = guide passenger Na+ Cl-
```

This causes GROMACS to report the non-bonded interaction terms between all unordered pairs in the set `{guide passenger Na+ Cl-}`. A new run input file is first generated with `gmx grompp`, using the system topology `rna.top` and the standard index file `index.ndx`. The trajectory is then replayed with `gmx mdrun -rerun`, producing a new energy file `e_guide_passenger.edr`. Finally, `gmx energy` is used to extract all pairwise non-bonded contributions into the file `e_guide_passenger.xvg`.

The second rerun targets residue-level interaction energies. Here, the modified input file `md_residues.mdp` specifies as energy groups the residues of interest,

```
energygrps = res2 res3 res4 res7 res8 res9
```

for the pentamer case. In this rerun, the residue-level index file is supplied to `gmx grompp`, so that the resulting energy terms resolve the pairwise interactions between the selected nucleotides. The output of the rerun is written to `e_residues.edr`, and the corresponding energy time-series are extracted into `e_residues.xvg`.

For each pair of energy groups, GROMACS reports the four standard non-bonded contributions:

$$E_{\text{Coul-SR}} \quad E_{\text{Coul-14}} \quad E_{\text{LJ-SR}} \quad E_{\text{LJ-14}}$$

Therefore, the total interaction energy between two groups  $\mathcal{G}_a$  and  $\mathcal{G}_b$  is reconstructed as

$$E(\mathcal{G}_a, \mathcal{G}_b) = E_{\text{Coul-SR}}(\mathcal{G}_a, \mathcal{G}_b) + E_{\text{Coul-14}}(\mathcal{G}_a, \mathcal{G}_b) \quad (4.21)$$

$$+ E_{\text{LJ-SR}}(\mathcal{G}_a, \mathcal{G}_b) + E_{\text{LJ-14}}(\mathcal{G}_a, \mathcal{G}_b). \quad (4.22)$$

Some observables of interest are not reported directly by GROMACS as single group-group terms, but are instead reconstructed by summing the appropriate pairwise contributions. For example, the interaction energy between the RNA duplex and the ionic environment is obtained by combining the interactions of both strands with the sodium and chloride groups:

$$E(\text{RNA}, \text{ions}) = E(\text{guide}, \text{Na}^+) + E(\text{guide}, \text{Cl}^-) \quad (4.23)$$

$$+ E(\text{passenger}, \text{Na}^+) + E(\text{passenger}, \text{Cl}^-). \quad (4.24)$$

We report the total interaction energies between RNA and ions, between guide and passenger and between all the residue pairs in  $\mathcal{P}_{\text{bc}} \cup \mathcal{P}_{\text{cl}} \cup \mathcal{P}_{\text{dl}}$  (4.11), (4.12), (4.13).

This means that some interaction energies stored in the computed time-series are discarded: for example, the interaction energies between relevant residues that are two positions away in the same strand or twice-removed diagonally (visually, this corresponds to those residue pairs that are not included in 4.1) are not considered.

#### 4.4.5 Observable distributions under duplex dissociation

Given a double-stranded modified **siRNA**  $n$ -mer  $R$ , let  $\Gamma_R$  denote its state space, and let  $A$  be an observable. We partition  $\Gamma_R$  into three regions: a bonded region  $R_b$ , an open region  $R_o$ , and a dissociated region  $R_d$ .

Let  $L$  denote the length of the original  $n$ -mer, excluding the caps. A simple but effective definition that ensures the partition property is the following.

We define the dissociated region as

$$R_d := \{(p, q) \mid \text{RMSD}(q) \geq 1 \text{ and } \sum_{\text{bp}} \#(\text{H-bonds in bp}) = 0\}. \quad (4.25)$$

We define the open region as

$$R_o := \{(p, q) \mid \text{RMSD}(q) \geq 0.3 \text{ and } \sum_{\text{bp}} \#(\text{H-bonds in bp}) \leq 2(L - 1)\} \quad (4.26)$$

$$\cup \{(p, q) \mid \text{RMSD}(q) \geq 0.3 \text{ and } \sum_{\text{bp}} \mathbb{1}(\text{H-bonds in bp} = 0) \geq L/2\} \setminus R_d. \quad (4.27)$$

Finally, we define the bonded region by

$$R_b := \Gamma_R \setminus (R_o \cup R_d). \quad (4.28)$$

By construction,  $R_b \cup R_o \cup R_d = \Gamma_R$ , and all pairwise intersections are empty. Here,  $R_b$  represents the set of state-space configurations in which the two strands of  $R$  remain bonded,  $R_o$  represents configurations in which the duplex is open or displaced but still partially bound, and  $R_d$  represents configurations in which the two strands have dissociated.

The two conditions used in the definition of  $R_o$  for the case of (capped) trimers reflect either that there are only enough hydrogen bonds in the interior of the duplex to support at most two A–U base pairs, assuming the canonical two hydrogen bonds per pair, or that half or more of the interior base pairs are completely separated, with no hydrogen bonds between them.

The probability density function of  $A$  can therefore be decomposed as

$$p(a \mid R) = p(R_b) p(a \mid R_b) + p(R_o) p(a \mid R_o) + p(R_d) p(a \mid R_d).$$

The key observation is that we do not care about the specific values of an observable in the states  $R_d$  and  $R_o$ , as these are no longer informative of the properties of the trimer within a larger double-stranded **siRNA** molecule. While strand opening and dissociation are part of the **RNA** interference pathway, they take place only once the guide strand of the **siRNA** molecule binds to the **RISC**. Therefore, the properties of a single-stranded trimer in water are not relevant for our purposes.

As a result, for every observable we estimate the histogram only for the random variable  $A \mid R_b$ , while retaining a single feature that estimates

$$p(R_d) = \int_{\Gamma_R} \mathbb{1}((p, q) \in R_d) dp dq$$

and

$$p(R_o) = \int_{\Gamma_R} \mathbb{1}((p, q) \in R_o) dp dq.$$

#### 4.4.5.1 Run-weighted aggregation in the double-stranded region

For a given pentamer, a certain observable  $A$ , and simulation run  $r \in \{1, \dots, R\}$ , let

$$x_{r_1} := A(q(t_{r,1})), \dots, x_{r,n_r} := A(q(t_{r,n_r}))$$

denote the values of the observable after restricting to the trajectory frames belonging to the double-stranded region only. Here,  $n_r$  is therefore the number of retained frames in run  $r$  after the state-space filter. Since different runs may contribute different numbers of double-stranded frames, we must aggregate run-level summaries using weights  $w_r = n_r$ .

If  $\mu_r$  denotes the mean of the observable in run  $r$  after restricting to the trajectory frames belonging to the double-stranded region only, i.e.

$$\mu_r := \frac{1}{n_r} \sum_{i=1}^{n_r} x_{r_i},$$

then the weighted mean across runs is

$$\bar{\mu}_w = \frac{\sum_{r=1}^R w_r \mu_r}{\sum_{r=1}^R w_r}. \quad (4.29)$$

This gives more influence to runs that contain more data in the double-stranded region.

A corresponding weighted standard deviation of the run means can be defined from the weighted variance. Using the usual unbiased weighted form, one may write

$$s_w^2 = \frac{\sum_{r=1}^R w_r (\mu_r - \bar{\mu}_w)^2}{\sum_{r=1}^R w_r - \frac{\sum_{r=1}^R w_r^2}{\sum_{r=1}^R w_r}}, \quad s_w = \sqrt{s_w^2}. \quad (4.30)$$

This quantity is not a frame-wise standard deviation of the underlying trajectory values, but rather a weighted measure of cross-run dispersion of the run-level means. Note that variance estimation through multiple replicas becomes even more relevant in our scenario: not only can the studied

observable present multiple modes, but the system may also enter and leave the three defined regions multiple times; which make estimating a reliable within-run statistical inefficiency a significantly difficult endeavour.

**4.4.5.1.1 Circular mean and circular standard deviation.** Some observables are angular or periodic and must be treated on a bounded interval  $[l, h)$  rather than on the real line. Let the period be  $P = h - l$ , and let  $\mu_r^{(\text{circ})}$  denote the circular mean computed within run  $r$ . To combine such run-level circular means across runs, each value is first mapped to an angle  $\theta_r = 2\pi(\mu_r^{(\text{circ})} - l)/P$ . One then forms the weighted mean resultant components

$$C_w = \frac{\sum_{r=1}^R w_r \cos \theta_r}{\sum_{r=1}^R w_r}, \quad S_w = \frac{\sum_{r=1}^R w_r \sin \theta_r}{\sum_{r=1}^R w_r}. \quad (4.31)$$

The weighted circular mean is then

$$\bar{\mu}_w^{(\text{circ})} = l + \frac{P}{2\pi} \text{atan2}(S_w, C_w), \quad (4.32)$$

with the angle wrapped back to the interval  $[l, h)$  as needed.

The corresponding weighted circular standard deviation can be derived from the weighted mean resultant length  $R_w = \sqrt{C_w^2 + S_w^2}$ :

$$s_w^{(\text{circ})} = \frac{P}{2\pi} \sqrt{-2 \ln R_w}. \quad (4.33)$$

Again, this should be interpreted as a weighted dispersion of the run-level circular means, rather than as a direct measure of frame-wise uncertainty.

#### 4.4.5.2 Weighted aggregation of normalized histograms.

For histogram-based summaries, let  $h_{r,b}$  denote the normalized probability assigned by run  $r$  to bin  $b$ , so that  $\sum_b h_{r,b} = 1$ . The weighted average histogram is

$$\bar{h}_b = \frac{\sum_{r=1}^R w_r h_{r,b}}{\sum_{r=1}^R w_r}. \quad (4.34)$$

This provides a pooled estimate of the conditional distribution in the double-stranded region. A weighted per-bin dispersion can also be computed as

$$s_{h,b}^2 = \frac{\sum_{r=1}^R w_r (h_{r,b} - \bar{h}_b)^2}{\sum_{r=1}^R w_r - \frac{\sum_{r=1}^R w_r^2}{\sum_{r=1}^R w_r}}, \quad s_{h,b} = \sqrt{s_{h,b}^2}. \quad (4.35)$$

Although the per-bin standard deviations  $s_{h,b}$  do contain useful information, they distribute cross-run disagreement over all histogram bins. This may dilute the signal in a larger number of dimensions (one per bin). What is more, bin values are not independent from each other, so a precise statistical summary would require considering the covariance matrix of the per-bin estimates. However, doing this would introduce quadratically more dimensions (given by the square of the number of bins), most of which would carry an extremely low signal (if any at all). Therefore, we do not compute the covariance matrix of the per-bin estimates for our observables.

If cross-run disagreement is a property of the whole distribution (for example, this would be the case if cross-run differences are given by different runs occupying different metastable modes), it is convenient to use a compact whole-histogram disagreement metric. A natural choice is the **Jensen–Shannon (JS)** divergence. For two normalized histograms  $p$  and  $q$ , define their midpoint distribution  $m = \frac{1}{2}(p + q)$ . The Kullback–Leibler divergence is

$$D_{\text{KL}}(p\|q) = \sum_b p_b \log\left(\frac{p_b}{q_b}\right), \quad (4.36)$$

and the **JS** divergence is

$$D_{\text{JS}}(p, q) = \frac{1}{2}D_{\text{KL}}(p\|m) + \frac{1}{2}D_{\text{KL}}(q\|m). \quad (4.37)$$

Unlike the Kullback–Leibler divergence, the **JS** divergence is symmetric and always finite once a small numerical pseudo-count is added if needed.

Given  $R$  runs with normalized histograms  $h_1, \dots, h_R$ , one may define the overall disagreement as the average pairwise **JS** divergence:

$$\text{JS}_{\text{avg}} = \frac{\sum_{1 \leq i < j \leq R} \omega_{ij} D_{\text{JS}}(h_i, h_j)}{\sum_{1 \leq i < j \leq R} \omega_{ij}}, \quad (4.38)$$

where  $\omega_{ij} = 1$  gives equal-run weighting and  $\omega_{ij} = w_i w_j$  gives frame-count

weighting. This produces a single scalar that summarizes how differently the runs realize the conditional distribution in the double-stranded region.

**4.4.5.2.1 Toy example** Consider an observable whose conditional distribution in the double-stranded region has two well-separated modes. Suppose first that there are two runs and each run remains confined to a different mode:

$$h_1 = (1, 0), \quad h_2 = (0, 1). \quad (4.39)$$

The pooled histogram is then

$$\bar{h} = \frac{1}{2}h_1 + \frac{1}{2}h_2 = \left(\frac{1}{2}, \frac{1}{2}\right). \quad (4.40)$$

Now consider a second situation in which both runs sample both modes equally:

$$h_1 = \left(\frac{1}{2}, \frac{1}{2}\right), \quad h_2 = \left(\frac{1}{2}, \frac{1}{2}\right). \quad (4.41)$$

The pooled histogram is again

$$\bar{h} = \left(\frac{1}{2}, \frac{1}{2}\right). \quad (4.42)$$

Therefore, the averaged normalized bins alone cannot distinguish between two physically distinct situations: one in which different runs are trapped in different modes, and another in which every run samples both modes similarly. This is precisely why it is valuable to include, alongside the pooled histogram, an explicit cross-run disagreement summary such as weighted per-bin dispersion or, more compactly, a **JS** disagreement score. The pooled histogram captures the overall conditional distribution, whereas the disagreement metric captures whether that distribution is reproducible within each run or instead emerges only after averaging over heterogeneous runs.

#### 4.4.6 The modified **siRNA** trimers' dynamical properties dataset

In conclusion, for each observable we report its cross-run mean, dispersion (std), averaged histogram, dispersion histogram and **JS** disagreement; with the exception of hydrogen bond lifetime curves, for which we report the cross-run average and std lifetime curve for timesteps  $\tau$  ranging from 1 to 30 and also

the JS disagreement.

## 4.5 Machine Learning models

Next, we introduce the dataset splits that are used for ML model training and evaluation, the methodology employed to construct them, the feature construction and pre-processing procedure, and the ML models that are evaluated in this manuscript.

### 4.5.1 Dataset Splits

Our goal is to construct an 80/20 split (where 80% of the available data is destined to the training set and 20% of the data is destined to the validation and test sets and split randomly between them) that holds out a diverse set of data-points for evaluation while probing the generalization capacities of the proposed ML models.

A purely random split would inevitably incur in data leakage because many siRNA molecules in the dataset are linked through sequence similarity, shared constituent trimers, and shared implemented modifications. As a result, we propose two splits which we denote as "hard" and "easy". Perhaps differently to what one may expect, we initially test all proposed models only on the hard data split, and only use the "easy" data split to evaluate how does the performance of the best models plus the chosen baselines translate to a split in which there is higher degree of shared information between the train and test sets.

The reasoning behind this decision is that we want to identify those models with true generalization capabilities, which may prove to be a harder task if we evaluate them in the "easy" data split. This is because we would expect a larger class of models to be able to achieve a reasonable performance on the "easy" data split than on the "hard" data split.

Next, we lay out the strategies followed to construct the "easy" and "hard" data splits.

#### 4.5.1.1 The "hard" data split

The test & validation set of the "hard" data split is built by forcing into it several predefined subsets of siRNA molecules, each corresponding to a different generalization class. The training set is then formed from the

remaining molecules, with its final size chosen to remain close to the target 80% proportion.

The first forced test & validation subset contains all **siRNA** molecules that include at least one simulated trimer for which the `nastruct` base-pair parameter extraction failed for at least one relevant base-pair for three or more runs. This additional safeguard was introduced for three main reasons:

- to prevent leakage from problematic trimers into training
- to enable an explicit assessment of generalization to **siRNA** molecules containing trimers that are structurally challenging
- to avoid the need for a specific masking strategy which only targets the rare cases in which the base-pair parameters are not available

The second forced test & validation subset consists of molecules containing at least one sugar modification absent from the `modXNA` dataset. This ensures that the proposed models are tested for generalization to completely unseen modifications while retaining all the information extracted through the **MD** simulations and feature extraction procedure (recall that trimers containing sugar modifications missing from `modXNA` are not simulated).

The third forced test & validation subset is formed by the subset of modified **siRNA** molecules corresponding to  $M_3$  randomly selected sequences. This subset provides a sequence-level held-out sample which tests generalization to unseen sequences, which is relevant because different sequences are used to target different genes (synthesized by different **mRNA** molecules).

Finally, the fourth forced test & validation subset encompasses all modified **siRNA** molecules containing at least one trimer belonging to one of  $M_4$  randomly selected k-means trimer clusters. This cluster-based holdout is intended to probe generalization to unseen regions of trimer space.

#### 4.5.1.2 The "easy" data split

Previous to constructing the "easy" data split of **siRNA** molecules, we cluster every sequence present in our dataset. We use k-means and choose the Hamming distance to build our similarity metric. The imposed number of clusters is 1000.

The test & validation set of the "easy" data split was then built by forcing into it the subset of **siRNA** molecules whose sequences belong to one of  $M_5$  randomly selected k-medoids sequence clusters.

This choice provides a stronger evaluation of the models’ generalization to unseen sequences than the third forced subset in the ”strong” data split, which only held out a subset of **siRNA** molecules with sequences belonging to a group of randomly selected sequences. By withholding entire sequence clusters, we ensure that models are tested on chemically distinct sequences.

The training set was then formed from the remaining molecules, with its final size chosen to remain close to the target 80% proportion.

### 4.5.1.3 Trimer and sequence clustering

We close the data split construction section by outlining the procedures used to cluster the unique trimers in the

`sirna_unique_trimers_modXNA_notermini` dataset and the unique sequences extracted from the main dataset.

**4.5.1.3.1 Trimer Clustering** For each trimer in the `sirna_unique_trimers_modXNA_notermini` dataset, its fingerprint-based representation is obtained by concatenating the guide and passenger strand Morgan fingerprints. Each individual strand Morgan fingerprint is computed using the trimer molecular graph builder tool implemented in the `nucleotide_fragmentation`, with fingerprint radius 2 and 2048 bits. The resulting representation is a 4096 bit vector  $f(\text{trimer}) \in \{0, 1\}^{4096}$ .

Observe that, according to Lemma 41, the chosen fingerprint radius is not enough to guarantee nucleotide-wise permutation invariance. However, this is not relevant for our clustering purpose, as here we are not interested in trimer sequence similarity but rather shared, local chemical features.

The obtained fingerprint-based representations are used to build a distance matrix  $D_t \in [0, 1]^{N_t \times N_t}$  where  $N_t$  is the total number of trimers and  $[D_t]_{i,j} = 1 - T(f(\text{trimer}_i), f(\text{trimer}_j))$ . Recall that  $T(x, y)$  denotes the Tanimoto similarity as defined in definition 30. Finally, the distance matrix is fed to the k-medoids algorithm with the number of clusters set to 1000.

**4.5.1.3.2 Sequence Clustering** We interpret each unique base sequence extracted from the main dataset as a vector over an alphabet  $\{0, A, U, T, C, G, ., I\}$  with length the maximum sequence length in the dataset, where 0 represents padding positions, . represents abasic nucleotides and I represents inverted abasic nucleotides. Then, the sequence distance matrix  $D_s \in [0, 1]^{N_s \times N_s}$  (where  $N_s$  is the total number of sequences) is computed using the Hamming

distance, i.e., the proportion of mismatches between two sequences:

$$[D_s]_{i,j} = \frac{\sum_k^N \mathbb{1}([seq_i]_k \neq [seq_j]_k)}{N} \quad (4.43)$$

where we recall that  $N$  is the maximum sequence length.

The obtained distance matrix is fed to the k-medoids algorithm with the number of clusters set to 1000.

## 4.5.2 Feature construction and preprocessing

All machine learning models considered in this work rely on a common feature construction pipeline, even though they consume the resulting representations in different ways. The purpose of this pipeline is to transform each modified **siRNA** duplex into a collection of chemically and experimentally informative features while preserving, whenever required, its nucleotide-level, trimer-level, or full-sequence structure. In particular, we distinguish between three feature families: chemical features derived directly from sequence and **SMILES** representations, experimental covariates associated with the assay conditions, and optional trimer-level features derived from the **MD** simulations described in Section 4.3.

A key design principle of the preprocessing pipeline is that all transformations whose parameters may depend on the empirical distribution of the training data are fitted on the training set only and then applied unchanged to validation and test samples. This applies in particular to the standardisation of numerical assay variables, the extraction of categories for one-hot encoded assay descriptors, and the PCA-based dimensionality reduction used for chemical fingerprint and physicochemical descriptor features. In this way, the preprocessing pipeline avoids leakage of information from the held-out partitions into model fitting. The only exception concerns the auxiliary preprocessing of trimer-level **MD** descriptors, which we discuss separately below.

Crucially, the PCA on chemical fingerprint and physicochemical descriptor features is computed on unique nucleotides and trimers only, as we are interested on the underlying structure of the nucleotide and double-stranded trimer sub-graph sets of our input space  $\mathcal{G}_{25}$  (which is discrete, see Section 3.1). Considering repeated observations would bias the PCA computation towards those elements of the input space with a higher experimental frequency, which would be equivalent to computing a weighted PCA on the unique elements of the input space.

Observe that, given a feature vector  $x \in \mathbb{R}^d$ , a PCA fitted on the corresponding set feature matrix yields a mean vector  $\mu \in \mathbb{R}^d$  and a matrix  $W_k \in \mathbb{R}^{d \times k}$  whose columns are the first  $k$  principal directions. The corresponding PCA-reduced representation is  $W_k^\top (x - \mu) \in \mathbb{R}^k$ .

#### 4.5.2.1 Chemical features

Chemical features are constructed at two different structural resolutions: the nucleotide level and the duplex trimer level.

At the nucleotide level, each nucleotide may be represented through a concatenation of four feature blocks:

1. A one-hot encoding of the base identity,

$$\text{OneHot}(\text{nuc}) = \begin{cases} [1, 0, 0, 0, 0, 0, 0] & \text{if the nuc. base is A} \\ [0, 1, 0, 0, 0, 0, 0] & \text{if the nuc. base is C} \\ [0, 0, 1, 0, 0, 0, 0] & \text{if the nuc. base is G} \\ [0, 0, 0, 1, 0, 0, 0] & \text{if the nuc. base is U} \\ [0, 0, 0, 0, 1, 0, 0] & \text{if the nuc. base is T} \\ [0, 0, 0, 0, 0, 1, 0] & \text{if the nuc. is abasic} \\ [0, 0, 0, 0, 0, 0, 1] & \text{if the nuc. is inverted abasic} \\ [0, 0, 0, 0, 0, 0, 0] & \text{if the nuc. is a padding nuc.} \end{cases} \quad (4.44)$$

2. A PCA-reduced vector of `RDKit` descriptors,

$$W_{d,n_d}^\top (p(\text{nuc}) - \mu_d) \quad (4.45)$$

where  $p(\text{nuc}) \in \mathbb{R}^{d_d}$  is the vector of descriptors,  $W_{d,n_d}$  is the rank- $n_d$  PCA matrix fitted on the training dataset unique nucleotides' descriptors and  $\mu_d$  is their mean.

3. A PCA-reduced Morgan fingerprint,

$$W_{fp,n_f}^\top (f(\text{nuc}) - \mu_{fp}) \quad (4.46)$$

where  $f(\text{nuc})$  is the radius 2, 2048 bit Morgan fingerprint for the nucleotide,  $W_{fp,n_f}$  is the rank- $n_f$  PCA matrix fitted on the training dataset unique nucleotides' fingerprints and  $\mu_{fp}$  is their mean.

4. A PCA-reduced difference fingerprint defined relative to the corresponding “standard” nucleotide. Here, “standard” nucleotide refers to the corresponding unmodified nucleotide,

$$W_{\text{fpd},n_{fd}}^{\top} (f_d(\text{nuc}) - \mu_{\text{fpd}}) \quad \text{with} \quad f_d(\text{nuc}) = f(\text{nuc}) - f(\text{nuc}_{\text{w/o mods}}) \quad (4.47)$$

where  $f$  is the radius 2, 2048 bit Morgan fingerprint,  $W_{\text{fpd},n_{fd}}$  is the rank- $n_{fd}$  PCA matrix fitted on the training dataset unique nucleotides’ fingerprint differences and  $\mu_{\text{fpd}}$  is their mean.

Sequences are padded to a fixed maximum length, and padded positions are represented by zero vectors. From now on, let us use  $N$  to denote the maximum sequence length in the dataset.

Inclusion of each of the four nucleotide-level feature blocks in the final nucleotide input tokens is optional, which means that the total dimension of the nucleotide input tokens is given by

$$\begin{aligned} d_{\text{nuc},i} = & \mathbb{1}(\text{OneHot}(\text{nuc}) \text{ included}) \cdot 7 + \mathbb{1}(\text{PCAd descriptors included}) \cdot n_d \\ & + \mathbb{1}(\text{PCAd fingerprints included}) \cdot n_f \\ & + \mathbb{1}(\text{PCAd fingerprint diff included}) \cdot n_{fd}. \end{aligned}$$

All models in this work starting from the nucleotide level implement all four blocks to obtain the nucleotide input tokens, i.e.

$$\mathbf{n}_i(\text{nuc}) = [\text{OneHot}(\text{nuc}) \| W_{d,n_d}^{\top} (p(\text{nuc}) - \mu_d) \| \quad (4.48)$$

$$W_{\text{fp},n_f}^{\top} (f(\text{nuc}) - \mu_{\text{fp}}) \| W_{\text{fpd},n_{fd}}^{\top} (f_d(\text{nuc}) - \mu_{\text{fpd}})]. \quad (4.49)$$

This can be interpreted as a function  $\mathbf{n}_i : \mathcal{G}_1^* \rightarrow \mathbb{R}^{7+n_d+n_f+n_{fd}}$ .

Putting everything together, we can characterize the implemented models starting at the nucleotide level as having the identity function as their molecular-graph level propagator  $g_0^{I(0)}$ , and their molecular-graph to nucleotide level cross-level aggregation map being

$$p_{0,1} : \mathcal{G}_N \rightarrow (\mathbb{R}^{7+n_d+n_f+n_{fd}})^{2N} \quad (4.50)$$

which is defined as

$$p_{0,1}(G) = (\mathbf{n}_i([D(N)(G)]_1), \dots, \mathbf{n}_i([D(N)(G)]_{2N})), \quad (4.51)$$

where we recall that  $D(N) : \mathcal{G}_N \rightarrow (\mathcal{G}_1^*)^{2N}$  is the map that decomposes a

(modified) **siRNA** graph into its nucleotides.

Note that, for the case of nucleotide-level tabular baselines such as `XGBoost`, the obtained nucleotide input tokens are just concatenated to form a flat feature vector for the **siRNA** molecule.

At the trimer level, a fingerprint-based representation is computed for every overlapping duplex trimer following the next procedure:

1. Get a radius 2, 2048 bit Morgan fingerprint for each nucleotide in the trimer.
2. Get a radius 2, 2048 bit Morgan fingerprint for each strand (guide and passenger) using the trimer molecular graph builder tool implemented in the `nucleotide_fragmentation` package. Observe that, by Lemma 41, the strand fingerprint alone would not be enough to guarantee permutation invariance.
3. Get a guide strand fingerprint-based representation by concatenating the guide strand fingerprint with the fingerprints of each individual nucleotide in the guide strand in 5' to 3' order. Get a passenger strand fingerprint-based representation by concatenating the passenger strand fingerprint with the fingerprints of each individual nucleotide in the passenger strand in 3' to 5' order.
4. Get the final fingerprint-based representation for the trimer by concatenating the guide and passenger strands fingerprint-based representations. Let us denote the result  $\tilde{f}_{2,2}(\text{trimer}) \in \{0, 1\}^{16384}$ .

Thus, the whole procedure is encapsulated by the function

$$\tilde{f}_{2,2} : \mathcal{G}_3 \rightarrow \{0, 1\}^{16384}. \quad (4.52)$$

We keep these parameters set for all experiments. Future work would include evaluating a range of radius choices for the guide and passenger trimer strands fingerprints, obtaining the functions  $\tilde{f}_{r,2} : \mathcal{G}_3 \rightarrow \{0, 1\}^{16384}$ .

For convenience, from now on let the nucleotides in a trimer be numbered from 1 to 6, with nucleotides 1–3 being the nucleotides in the guide strand in the 5' to 3' sense and nucleotides 4–6 being the nucleotides in the passenger strand in the 3' to 5' sense, such that nucleotide  $i \in \{1, 2, 3\}$  base-pairs with nucleotide  $3 + i$ .

Observe that the chosen trimer representation breaks nucleotide permutation invariance, ensures local motifs are captured nucleotide-wise (if the same structure is present in different nucleotides, a strand Morgan fingerprint can only take it into account once by activating the corresponding bit and thus loses count resolution) and also accounts for inter-nucleotide connectivity. The price to pay is the large dimension of the resulting bit vector, which will contain large amounts of repeated information. This is nevertheless not a problem as a PCA is later performed on the obtained representations, thereby discarding those directions with the least variability. The resulting representation is

$$W_{\text{tfp},n_t}^\top (\tilde{f}_{2,2}(\text{trimer}) - \mu_{\text{tfp}}) \quad (4.53)$$

where  $W_{\text{tfp},n_t}$  is the rank- $n_t$  PCA matrix fitted on the training dataset unique trimers' fingerprint-based  $\tilde{f}_{2,2}$  vectors and  $\mu_{\text{tfp}}$  is their mean.

We further compute an analogous difference fingerprint, defined relative to the corresponding “standard” trimer and nucleotides:

$$\tilde{f}_{d,2,2}(\text{trimer}) = \tilde{f}_{2,2}(\text{trimer}) - \tilde{f}_{2,2}(\text{trimer w/o mods}). \quad (4.54)$$

The resulting trimer descriptors are again reduced by PCA, obtaining the representation

$$W_{\text{tfpd},n_{td}}^\top (\tilde{f}_{d,2,2}(\text{trimer}) - \mu_{\text{tfpd}})$$

where  $W_{\text{tfpd},n_{td}}$  is the rank- $n_{td}$  PCA matrix fitted on the training dataset unique trimers' fingerprint-difference-based  $\tilde{f}_{d,2,2}$  representations and  $\mu_{\text{tfpd}}$  is their mean.

All trimer positions are padded to a fixed maximum number of overlapping trimers, which is exactly  $N - 2$ . From now on, let  $T := N - 2$ .

Inclusion of the fingerprint and fingerprint-difference based trimer representations in the final trimer input tokens is optional, and one can also choose to include the concatenated representations of any of the nucleotide-level feature blocks for the 6 nucleotides which constitute the trimer. Thus, the dimension of the trimer input tokens is:

$$\begin{aligned} d_{\text{trimer},i} = & \mathbb{1}(\text{PCAd } \tilde{f}_{2,2} \text{ included}) \cdot n_t + \mathbb{1}(\text{PCAd } \tilde{f}_{d,2,2} \text{ included}) \cdot n_{td} \\ & + 6 \cdot (\mathbb{1}(\text{OneHot}(\text{nuc}) \text{ included}) \cdot 7 \\ & + \mathbb{1}(\text{PCAd descriptors included}) \cdot n_d \\ & + \mathbb{1}(\text{PCAd fingerprints included}) \cdot n_f \\ & + \mathbb{1}(\text{PCAd fingerprint diff included}) \cdot n_{fd}). \end{aligned}$$

All models in this work starting from the trimer level or incorporating PCA-based trimer features include both fingerprint and fingerprint-difference based trimer representations plus the concatenated PCAd nucleotide `RDKit` descriptors and nucleotide one-hot base embedding, yielding a final trimer input token dimension

$$d_{\text{trimer},i} = n_t + n_{td} + 6 \cdot (7 + n_d),$$

with the input token given by

$$\begin{aligned} \mathbf{t}_i(\text{trimer}) = & [W_{\text{tfp},n_t}^\top (\tilde{f}_{2,2}(\text{trimer}) - \mu_{\text{tfp}}) \| W_{\text{tfpd},n_{td}}^\top (\tilde{f}_{d,2,2}(\text{trimer}) - \mu_{\text{tfpd}}) \| \\ & \text{OneHot}(\text{nuc}_1) \| \dots \| \text{OneHot}(\text{nuc}_3) \| \end{aligned} \quad (4.55)$$

$$\begin{aligned} & W_{d,n_d}^\top (p(\text{nuc}_1) - \mu_d) \| \dots \| W_{d,n_d}^\top (p(\text{nuc}_3) - \mu_d) \| \\ & \text{OneHot}(\text{nuc}_4) \| \dots \| \text{OneHot}(\text{nuc}_6) \| \end{aligned} \quad (4.56)$$

$$W_{d,n_d}^\top (p(\text{nuc}_4) - \mu_d) \| \dots \| W_{d,n_d}^\top (p(\text{nuc}_6) - \mu_d)] \quad (4.57)$$

abusing the notation introduced for the nucleotide input tokens. This can be interpreted as a map

$$\mathbf{t}_i : \mathcal{G}_3 \rightarrow \mathbb{R}^{n_t + n_{td} + 6 \cdot (7 + n_d)}$$

which internally uses the map  $D(3) : \mathcal{G}_3 \rightarrow (\mathcal{G}_1^*)^6$  that decomposes a double-stranded (modified) **siRNA** trimer into its constituent nucleotides.

Putting everything together, we can characterize the implemented models starting at the trimer level as having the identity function as their molecular-graph level propagator  $g_0^{I(0)}$ , and their molecular-graph to trimer level cross-level aggregation map being

$$p_{0,2} : \mathcal{G}_N \rightarrow (\mathbb{R}^{n_t + n_{td} + 6 \cdot (7 + n_d)})^T \quad (4.58)$$

which is defined as

$$p_{0,2}(G) = (\mathbf{t}_i([D_t(N)(G)]_1), \dots, \mathbf{t}_i([D_t(N)(G)]_T)), \quad (4.59)$$

where we recall that  $D_t(N) : \mathcal{G}_N \rightarrow (\mathcal{G}_3)^{N-2}$  is the map that decomposes a (modified) **siRNA** graph into its overlapping trimers.

Note that, for the case of trimer-level tabular baselines such as `XGBoost`, the obtained trimer input tokens are just concatenated to form a flat feature vector for the **siRNA** molecule.

The preprocessing implementation follows a “fit wide, use narrow” strategy. PCA models for nucleotide fingerprints are fitted with 30 components and PCA models for trimer fingerprint-based representations are fitted with 50 components, but the number of active components used in downstream feature matrices can later be reduced without refitting the transformation.

In practice, unless stated otherwise, we take the number of PCA components that cover strictly more than 99% of the variance for the RDKit-derived properties, all of the variance for the nucleotide fingerprints (given that there is a reduced number of unique nucleotides in the dataset, we expect their fingerprints to be contained in a relatively lower-dimensional linear subspace) and 95% of the variance for the trimer fingerprints (more variability, richer underlying structure).

Our implementation caches those nucleotides and trimers whose chemical features have already been computed to speed up the preprocessing. For example, this means that nucleotide chemical features need only be computed once for each of the unique observed nucleotides instead of running approximately  $2NN_D$  computations, where  $N_D$  is the size of the dataset.

#### 4.5.2.2 Experimental covariates

In addition to the chemical descriptors, each siRNA sample is associated with a set of sequence-level experimental covariates describing the assay conditions under which its activity was measured. These include two numerical variables, siRNA concentration and incubation time, and three categorical variables: cell type, transfection method, and activity detection method.

The numerical variables are standardised using a standard scaler fitted on the training set (computes training set mean and standard deviation and then transforms each value subtracting the mean and dividing by the standard deviation). The categorical variables are one-hot encoded using the set of categories observed in the training partition.

Experimental covariates are stored separately from the chemical descriptors and are incorporated as global sequence-level features. In the neural architectures considered later, they are concatenated to the learned sequence representation after pooling; in tabular baselines such as XGBoost, they are concatenated directly to the flat feature vector.

#### 4.5.2.3 MD-derived trimer features

Our overarching goal is to inform the developed ML models with the extracted trimer-level features from the MD simulations. We distinguish

7 **MD** feature types: base pair parameters, sugar pucker angle, interaction energies, hydrogen bonds, **RMSD**, **RMSF** and open and dissociated duplex probabilities.

Within each of these feature types there are different groups with markedly different statistical behaviour: cross-run mean, cross-run standard deviation, cross-run normalized histograms, cross-run per-bin histogram standard deviation, cross-run histogram **JS** divergence, cross-run mean lifetime curve, cross-run lifetime curve standard deviation and cross-run lifetime curve **JS** divergence.

Out of all these groups, only cross-run normalized histograms and cross-run mean lifetime curves need not be standardised as their values are already restricted to the interval  $[0, 1]$ . For the case of histogram bin and lifetime curve standard deviations, standardisation is carried out separately for each element of the range. In every case, we use a standard scaler fit on the training set only.

After this group-wise normalization step, the full **MD** feature vector may either be used directly, restricted to selected feature groups, or projected into a lower-dimensional representation through a global PCA. Specific feature types can also be removed from the **MD** features model input, but this work only applies this option during the ablation stage.

A practical complication is that **MD**-derived descriptors are not available for every trimer appearing in the dataset. Consequently, different model classes require different strategies to handle missing trimer-level **MD** information. In the hierarchical neural models introduced later, missingness is handled explicitly through trimer-wise masks that allow the architecture to condition on whether **MD** data are available for a given trimer.

In contrast, for the auxiliary **XGBoost** experiments designed to assess the predictive utility of the **MD** descriptors, missing trimer-level **MD** features are imputed with the median value of the corresponding feature over the available trimer set. These **XGBoost** experiments are not primarily intended to define the strongest possible non-neural benchmark, but rather to evaluate whether the addition of **MD**-derived descriptors carries predictive signal beyond the chemical features alone.

Accordingly, throughout this chapter we distinguish between *chemical-only* models and *MD-augmented* models, which additionally incorporate trimer-level descriptors extracted from the simulations.

#### 4.5.2.4 Tensorization and model-specific input formats

Once the features have been computed, they are arranged into the input structures required by the different model families.

For the hierarchical models, the preprocessing and dataset-building pipeline is specifically designed to preserve the multilevel structure of the duplex. Chemical features are extracted at both the nucleotide and trimer levels, while experimental covariates are stored separately as global sequence-level features. From now on, let  $B$  be the batch size. The output of this preprocessing stage is a set of tensors summarized in Table 4.9.

Table 4.9: Tensors produced by the preprocessing pipeline and passed to the modular hierarchical model. Note that the default trimer input tokens  $[B, T, d_{\text{trimer},i}]$  as described in (4.55) are built by concatenating the trimer-level fingerprint-based representations and the nucleotide-level features for trimer tokens across their last dimension, i.e.,  $d_{\text{trimer},i} = d_{\text{trimer,fp}} + 6 \cdot d_{\text{nuc,trimer}}$ .

| Tensor                                         | Shape                                   | Description                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Nucleotide input tokens                        | $[B, 2N, d_{\text{nuc},i}]$             | Nucleotide input tensor. For each sample, it stores the features of all the nucleotides in the <b>siRNA</b> molecule. By default it is built using four different nucleotide-level feature blocks as described in (4.48).                                                                                                                                               |
| Trimer-level fingerprint-based representations | $[B, T, d_{\text{trimer,fp}}]$          | Per-trimer fingerprint-based representation tensor. For each sample, it stores the PCAd $\tilde{f}_{2,2}$ (4.52) and $\tilde{f}_{d2,2}$ (4.54) values of all the overlapping double-stranded trimers in the <b>siRNA</b> molecule. By default, $d_{\text{trimer,fp}} = n_t + n_{td}$ as both fingerprint and fingerprint-difference based representations are included. |
| Nucleotide-level features for trimer tokens    | $[B, T, 6 \cdot d_{\text{nuc,trimer}}]$ | Tensor including nucleotide features from each trimer constituent's nucleotides to be used in creating the trimer-level input tokens. By default the considered nucleotide features are one-hot base encodings and PCAd RDKit descriptors, yielding $d_{\text{nuc,trimer}} = 7 + n_d$ .                                                                                 |
| Optional <b>MD</b> features                    | $[B, T, d_{\text{MD}}]$                 | Per-trimer tensor containing preprocessed <b>MD</b> -derived descriptors when available.                                                                                                                                                                                                                                                                                |
| Trimer padding mask                            | $[B, T]$                                | Boolean mask identifying padded trimer positions so that they can be ignored by the sequence encoder and pooling operations.                                                                                                                                                                                                                                            |
| Global features                                | $[B, d_{\text{global}}]$                | Sequence-level experimental and assay covariates appended after sequence pooling.                                                                                                                                                                                                                                                                                       |

The XGBoost [28] baselines do not retain any explicit sequential or

hierarchical structure. Instead, the relevant nucleotide-level or trimer-level features (including MD features in the case of our MD feature importance XGBoost experiments) are flattened into fixed-length vectors and concatenated with the global experimental covariates.

### 4.5.3 A modular design for hierarchical models

The hierarchical framework whose mathematical description we laid out in Section 3.3.3 is instantiated through a modular architecture in which the full prediction pipeline is decomposed into nucleotide, trimer and sequence layers, with nucleotide-to-trimer, nucleotide-to-sequence and trimer-to-sequence cross-level interfaces. The last element of the pipeline is a prediction head. This design was chosen because it enables the quick iteration of different within-level propagation and cross-level aggregation strategies at each level and facilitates bringing together different feature sources, including nucleotide-level chemical descriptors, trimer-level chemical descriptors, and MD-derived trimer representations.

Note that any particular instantiation of this framework need not use all stages. For example, some models bypass the nucleotide-level stage and start directly from pre-computed trimer descriptors, while others go straight from nucleotide-level inputs to the full sequence embedding.

From now on, let the siRNA molecular graph be denoted as level 0, the nucleotide level as level 1, the trimer level as level 2 and the sequence level as level 3. This work does not consider any level-0 propagator other than the identity on the initial level-0 representation, and it also does not consider any cross-level maps leaving level 0 other than  $p_{0,1}$  as defined in (4.51) and  $p_{0,2}$  as defined in (4.59), which are induced by the fixed decomposition maps and the corresponding feature-construction pipeline.

In our padded representation, each trimer position contains six nucleotide slots, corresponding to the three guide-strand nucleotides and the three passenger-strand nucleotides that form the duplex trimer. The modular model then follows the sequence of maps

$$\begin{aligned} [B, 2N, d_{\text{nuc},i}] &\longrightarrow [B, 2N, d_{\text{nuc}}] \longrightarrow [B, T, d_{\text{trimer,learned}}] \\ &\longrightarrow [B, T, d_{\text{trimer}}] \longrightarrow [B, d_{\text{sequence}}] \longrightarrow [B, 1] \end{aligned}$$

where the first three transformations may be replaced by

$$[B, T, d_{\text{trimer},i}] \longrightarrow [B, T, d_{\text{trimer}}],$$

and one may also go directly from the nucleotide level to the sequence level through

$$[B, 2N, d_{\text{nuc}}] \longrightarrow [B, d_{\text{sequence}}].$$

The final output is the predicted activity.

#### 4.5.3.1 Nucleotide encoder

The first optional stage maps raw nucleotide features to learned nucleotide embeddings and corresponds to a within-level propagator at level 1. In the notation of Section 3.3.3, this is a map

$$g_1^{I^{(1)}} : (\mathbb{R}^{d_{\text{nuc},i}})^{2N} \rightarrow (\mathbb{R}^{d_{\text{nuc}}})^{2N}.$$

In the current implementation this stage supports two variants: a pass-through encoder and a shared **MLP** applied independently to each nucleotide position. Thus, if the input tensor is  $X \in \mathbb{R}^{B \times 2N \times d_{\text{nuc},i}}$ , the nucleotide encoder produces  $H^{(\text{nuc})} \in \mathbb{R}^{B \times 2N \times d_{\text{nuc}}}$ . This within-level propagator does not depend on the chosen structure  $I^{(1)}$ .

More explicitly, let  $x_{b,j} \in \mathbb{R}^{d_{\text{nuc},i}}$  denote the raw feature vector of nucleotide position  $j \in \{1, \dots, 2N\}$  in sample  $b \in \{1, \dots, B\}$ .

Then, in the pass-through variant, the implemented encoder is simply an optional pointwise projection from the raw nucleotide feature dimension to the working nucleotide embedding dimension

$$[g_1^{I^{(1)}}(X)]_{b,j} = \Pi_{\text{nuc}}(x_{b,j})$$

where  $\Pi_{\text{nuc}} : \mathbb{R}^{d_{\text{nuc},i}} \rightarrow \mathbb{R}^{d_{\text{nuc}}}$  is defined as

$$\Pi_{\text{nuc}} = \begin{cases} \text{Id}, & \text{if } d_{\text{nuc},i} = d_{\text{nuc}}, \\ \text{GELU} \circ \text{LN} \circ A, & \text{otherwise,} \end{cases}$$

where

$$A : \mathbb{R}^{d_{\text{nuc},i}} \rightarrow \mathbb{R}^{d_{\text{nuc}}}$$

is an affine map. Hence, when the input and output dimensions coincide, the raw nucleotide features are passed through unchanged; otherwise, they are transformed by a single affine map followed by layer normalisation and GELU activation.

In the **MLP** variant, the implemented encoder is a **MLP** composed of a sequence of affine maps, layer normalisation, GELU activations and dropout,

followed by a final affine map and output layer normalisation

$$[g_1^{I(1)}(X)]_{b,j} = \Phi_{\text{nuc}}(x_{b,j})$$

where, if the hidden widths are  $h_1, \dots, h_L$ , then  $\Phi_{\text{nuc}}$  has the form

$$\phi_{\text{nuc}} = \text{LN}_{L+1} \circ A_{L+1} \circ (\text{Drop}_L \circ \text{GELU} \circ \text{LN}_L \circ A_L) \circ \dots \quad (4.60)$$

$$\dots \circ (\text{Drop}_1 \circ \text{GELU} \circ \text{LN}_1 \circ A_1) \quad (4.61)$$

where

$$A_1 : \mathbb{R}^{d_{\text{nuc},i}} \rightarrow \mathbb{R}^{h_1}, \quad A_\ell : \mathbb{R}^{h_{\ell-1}} \rightarrow \mathbb{R}^{h_\ell} \text{ for } \ell = 2, \dots, L, \quad A_{L+1} : \mathbb{R}^{h_L} \rightarrow \mathbb{R}^{d_{\text{nuc}}}$$

are affine maps.

Because the same map  $\Pi_{\text{nuc}}$  or  $\Phi_{\text{nuc}}$  is applied independently to every nucleotide, this stage does not model interactions between different nucleotide positions. In particular, both the pass-through and **MLP** variants correspond to airtight within-level propagators.

#### 4.5.3.2 Nucleotide to sequence encoder

This optional stage maps learned nucleotide embeddings directly to a fixed-size sequence representation, bypassing the trimer level entirely. In the notation of Section 3.3.3, it is a cross-level aggregation module from level 1 to level 3,

$$p_{1,3}^{I(1)} : (\mathbb{R}^{d_{\text{nuc}}})^{2N} \rightarrow \mathbb{R}^{d_{\text{sequence}}}$$

applied independently to each sample in the batch. If the input tensor is  $X \in \mathbb{R}^{B \times 2N \times d_{\text{nuc}}}$  the nucleotide-to-sequence encoder produces  $H^{(\text{seq})} \in \mathbb{R}^{B \times d_{\text{sequence}}}$ .

Two implementations are considered in this work: a duplex **CNN** encoder and a transformer encoder. Both implementations define valid cross-level aggregation maps from the nucleotide level to the sequence level,

$$p_{1,3}^{\text{CNN}}, p_{1,3}^{\text{Tr}} : (\mathbb{R}^{d_{\text{nuc}}})^{2N} \rightarrow \mathbb{R}^{d_{\text{sequence}}}.$$

**4.5.3.2.1 CNN nucleotide-to-sequence encoder** The convolutional implementation treats the nucleotide-level duplex as a  $2 \times N$  grid (i.e.,  $I^{(1)}$  is chosen to be a  $2 \times N$  grid) whose two rows correspond to the guide and passenger strands and whose columns correspond to aligned nucleotide

positions.

Given  $X \in \mathbb{R}^{B \times 2N \times d_{\text{nuc}}}$  the tensor is reshaped as  $X \mapsto X^{\text{cnn}} \in \mathbb{R}^{B \times d_{\text{nuc}} \times 2 \times N}$  so that the nucleotide embedding dimension becomes the channel dimension of the convolutional network.

The encoder then applies a stack of two-dimensional convolutional blocks. At layer  $\ell$ , the block consists of a convolution  $\text{Conv2D}_\ell : \mathbb{R}^{c_{\ell-1} \times h_{\ell-1} \times w_{\ell-1}} \rightarrow \mathbb{R}^{c_\ell \times h_\ell \times w_\ell}$  followed by batch normalisation, a GELU activation and dropout.

The convolutional kernel has shape  $(k_\ell^{(h)}, k_\ell^{(w)})$  where  $k_\ell^{(h)} \in \{1, 2\}$  and  $k_\ell^{(w)} \in \{1, \dots, N\}$  is fixed across layers. Height padding is never applied, whereas width padding may optionally be used in order to preserve the sequence length along the strand direction (i.e., preserve strand length  $N$ ). Choosing kernel height of 2 merges the two strands, after which only kernels of height 1 are permitted.

After the convolutional stack, a final  $1 \times 1$  convolution projects the channel dimension to  $d_{\text{sequence}}$ . If the resulting tensor has spatial shape  $1 \times W$ , it is reshaped to  $\mathbb{R}^{B \times W \times d_{\text{sequence}}}$  whereas if the height remains equal to 2, it is reshaped to  $\mathbb{R}^{B \times 2W \times d_{\text{sequence}}}$ .

A final pooling operator is then applied across the remaining positions, yielding a sequence embedding in  $\mathbb{R}^{B \times d_{\text{sequence}}}$ . The implementation supports mean pooling, max pooling and learned attention pooling.

Thus, the CNN encoder has the form

$$p_{1,3}^{\text{CNN}} = \rho \circ R \circ \Gamma$$

where  $\Gamma$  denotes the convolutional stack together with the final  $1 \times 1$  projection,  $R$  is the reshaping operation that converts the final feature map into a sequence of latent tokens, and  $\rho$  is the chosen pooling map.

**4.5.3.2.2 Transformer nucleotide-to-sequence encoder** The transformer implementation applies a standard transformer encoder to the full sequence of nucleotide embeddings and then pools the resulting contextualised representations. The structure  $I^{(1)}$  imposed by the nucleotide transformer architecture is effectively a 1D  $2N$ -long sequence.

Given  $X \in \mathbb{R}^{B \times 2N \times d_{\text{nuc}}}$  the input is first projected to the model dimension  $d_{\text{sequence}}$  whenever  $d_{\text{nuc}} \neq d_{\text{sequence}}$ . Let the projected input be  $Z \in \mathbb{R}^{B \times 2N \times d_{\text{sequence}}}$ . We then optionally prepend a learnable CLS token, depending on the selected pooling strategy. Positional information is incorporated either through sinusoidal positional encodings or through learned positional embeddings.

The resulting sequence is passed through a transformer encoder consisting of  $L_{\text{tr}}$  layers of multi-head self-attention and position-wise feed-forward blocks, producing an output tensor  $\tilde{Z} \in \mathbb{R}^{B \times S \times d_{\text{sequence}}}$  where  $S = 2N$  in the absence of a CLS token and  $S = 2N + 1$  otherwise.

A final pooling operator then maps  $\tilde{Z}$  to a single sequence embedding in  $\mathbb{R}^{B \times d_{\text{sequence}}}$ . The implementation supports mean pooling, max pooling, CLS pooling and learned attention pooling. We may therefore write

$$p_{1,3}^{\text{Tr}} = \rho \circ \text{TrEnc} \circ \text{PE} \circ \Pi,$$

where  $\Pi$  denotes the optional input projection and optional CLS augmentation, PE denotes positional encoding, TrEnc denotes the transformer encoder, and  $\rho$  is the final pooling map.

Finally, observe that this stage is airtight in the sense introduced previously. Indeed, all nucleotide positions belong to the same level-3 aggregation context, namely the full duplex, so the corresponding aggregation-neighbourhood at level 1 is the whole set  $\{1, \dots, 2N\}$ .

#### 4.5.3.3 Nucleotide-to-trimer encoder

The second optional stage maps nucleotide-level embeddings to learned trimer representations. In the notation of Section 3.3.3, this stage is a cross-level aggregation module from level 1 to level 2,

$$p_{1,2}^{I(1)} : (\mathbb{R}^{d_{\text{nuc}}})^{2N} \rightarrow (\mathbb{R}^{d_{\text{trimer,learned}}})^T,$$

applied independently to each sample in the batch, where  $T = N - 2$  is the number of overlapping duplex trimers in the padded representation.

The first step of this module is fixed and consists of gathering, for each trimer position  $t \in \{1, \dots, T\}$ , the six nucleotide embeddings that define the corresponding duplex trimer. More precisely, if  $X \in \mathbb{R}^{B \times 2N \times d_{\text{nuc}}}$  then a deterministic gather map constructs  $G(X) \in \mathbb{R}^{B \times T \times 6 \times d_{\text{nuc}}}$  where for each sample  $b$  and trimer position  $t$ , the six gathered embeddings correspond to the three guide-strand nucleotides (given by positions  $t, t + 1, t + 2$ ) and the three passenger-strand nucleotides (positions  $N + t, N + t + 1, N + t + 2$ ) in the overlapping trimer at position  $t$ .

Two implementations of the nucleotide-to-trimer encoder are considered in this work, an MLP-based and a CNN-based cross-level aggregation functions.

The cross-level aggregation function at this stage may be written abstractly as

$$p_{1,2}^{I(1)} = \chi \circ G$$

where  $G$  is the fixed gather map determined by the duplex trimer incidence structure, and  $\chi$  is either the shared MLP-based or the CNN-based cross-level aggregator. Observe that the imposed underlying structure  $I^{(1)}$  is in both cases a 1D  $2N$ -long sequence because it is dictated by the input to the fixed gather map  $G$ .

**4.5.3.3.1 MLP nucleotide-to-trimer encoder** In the first implementation, the six gathered nucleotide embeddings are concatenated, obtaining  $\text{Concat}(G(X)) \in \mathbb{R}^{B \times T \times 6d_{\text{nuc}}}$ ; and then passed through a shared **MLP** which produces  $H^{(\text{tri}, \text{learned})} \in \mathbb{R}^{B \times T \times d_{\text{trimer}, \text{learned}}}$ .

More explicitly, if  $g_{b,t} \in \mathbb{R}^{6d_{\text{nuc}}}$  denotes the concatenated six-nucleotide representation of trimer  $t$  in sample  $b$ , then

$$[p_{1,2}^{\text{MLP}}(X)]_{b,t} = \Phi_{\text{nuc2tri}}(g_{b,t}),$$

where  $\Phi_{\text{nuc2tri}} : \mathbb{R}^{6d_{\text{nuc}}} \rightarrow \mathbb{R}^{d_{\text{trimer}, \text{learned}}}$  is a **MLP** composed of affine maps, layer normalisation, GELU activations and dropout, followed by a final affine map and output layer normalisation. For a more precise mathematical formulation of a **MLP**, see (4.60).

**4.5.3.3.2 CNN nucleotide-to-trimer encoder** In the second implementation, the six gathered nucleotide embeddings associated with each trimer are arranged as a  $2 \times 3$  array, where the first row contains the three guide-strand nucleotides and the second row contains the three passenger-strand nucleotides. Thus, after gathering we reshape into  $X^{\text{cnn}} \in \mathbb{R}^{BT \times d_{\text{nuc}} \times 2 \times 3}$  so that each trimer is treated as a small two-dimensional input with channel dimension  $d_{\text{nuc}}$ .

A stack of two-dimensional convolutional blocks is then applied. This stack is identical to the one introduced for the nucleotide-to-sequence encoder 4.5.3.2.1 with the exception that it accepts different kernel widths at each layer. This yields

$$p_{1,2}^{\text{CNN}}(X) \in \mathbb{R}^{B \times T \times d_{\text{trimer}, \text{learned}}}$$

Both proposed nucleotide to trimer encoders are airtight because each

learned trimer embedding depends only on the six nucleotide embeddings that belong to that same trimer.

#### 4.5.3.4 Trimer encoder

The trimer encoder is the central feature-integration stage of the architecture. Its role is to produce the final trimer token representation that will subsequently be contextualized along the **siRNA** sequence by the trimer-to-sequence encoder. In the notation of Section 3.3.3, this stage may be viewed as a within-level map at level 2 acting on the collection of per-trimer inputs available after preprocessing and, when applicable, after the nucleotide-to-trimer aggregation stage. Concretely, the implemented trimer encoders take as input some combination of:

- a learned trimer embedding  $H^{(\text{tri}, \text{learned})} \in \mathbb{R}^{B \times T \times d_{\text{trimer}, \text{learned}}}$  produced by the nucleotide encoder together with the nucleotide-to-trimer encoder
- a pre-computed trimer input token  $X^{(\text{tri}, i)} \in \mathbb{R}^{B \times T \times d_{\text{trimer}, i}}$  constructed from the fingerprint-based trimer representations and the nucleotide-level features associated with each trimer token as described in (4.55) and Table 4.9
- optional **MD**-derived descriptors  $X^{(\text{MD})} \in \mathbb{R}^{B \times T \times d_{\text{MD}}}$  together with an availability mask  $M^{(\text{MD})} \in \{0, 1\}^{B \times T}$

The output of the trimer encoder is a tensor  $H^{(\text{tri})} \in \mathbb{R}^{B \times T \times d_{\text{trimer}}}$  which is then passed to the trimer-to-sequence encoder.

Different model instantiations differ primarily in how this stage combines the available learned trimer embeddings, pre-computed chemical trimer descriptors, nucleotide-level trimer features, and optional **MD**-derived descriptors. Their common feature is that they do not depend on the underlying structure  $I^{(2)}$ .

**4.5.3.4.1 Pass-through chemical trimer encoder** The simplest trimer encoder is a pass-through variant, which simply concatenates the pre-computed trimer fingerprint features with the flattened nucleotide-level features associated with the same trimer to obtain

$$H^{(\text{tri})} = X^{(\text{tri}, i)} \in \mathbb{R}^{B \times T \times (d_{\text{trimer}, \text{fp}} + 6d_{\text{nuc}, \text{trimer}})}$$

as described in (4.55) and Table 4.9.

**4.5.3.4.2 Learned-only trimer encoder** A second family of variants uses only the learned trimer embeddings produced by the nucleotide encoder and the nucleotide-to-trimer encoder. If necessary, these learned trimer embeddings are projected to the token dimension expected by the downstream sequence encoder:

$$H^{(\text{tri})} = g_2^{I(2)}(H^{(\text{tri},\text{learned})})$$

where if  $h_{b,t} := [H^{(\text{tri},\text{learned})}]_{b,t} \in \mathbb{R}^{d_{\text{trimer},\text{learned}}}$  is the learned trimer representation for trimer  $t$  in sample  $b$  we can write

$$[g_2^{I(2)}(H^{(\text{tri},\text{learned})})]_{b,t} = \Pi_{\text{tri}}(h_{b,t})$$

where  $\Pi_{\text{tri}} : \mathbb{R}^{d_{\text{trimer},\text{learned}}} \rightarrow \mathbb{R}^{d_{\text{trimer}}}$  is either the identity map or a learned projection consisting of an affine map, layer normalisation, GELU activation and dropout.

**4.5.3.4.3 Hybrid chemical/learned trimer encoder** A third family of variants is hybrid: the learned trimer embedding is concatenated with the pre-computed trimer fingerprint representation and the result is projected into the trimer token space. More precisely,

$$H^{(\text{tri})} = \Phi_{\text{hyb}}([H^{(\text{tri},\text{learned})} \parallel X^{(\text{tri},\text{fp})}])$$

where  $\Phi_{\text{hyb}} : \mathbb{R}^{d_{\text{trimer},\text{learned}} + d_{\text{trimer},\text{fp}}} \rightarrow \mathbb{R}^{d_{\text{trimer}}}$  is a **MLP**.

This hybrid encoder tests whether the nucleotide-derived latent representation and the manually engineered trimer descriptors provide complementary information.

**4.5.3.4.4 Including trimer MD features** A key advantage of the modular architecture is that it enables the incorporation of trimer-level dynamical information extracted from the **MD** simulations described in Section 4.3. These features are not available for all trimers, either because a given trimer has not been simulated or because some derived descriptors could not be robustly extracted. Consequently, the model must handle missing **MD** information explicitly rather than assuming a fully observed trimer feature table.

Including all derived **MD** features directly would lead to a substantial increase in dimensionality and, therefore, in the number of learnable parameters in the downstream trimer encoder. We therefore evaluate three alternative strategies:

- using only the cross-run mean and cross-run standard deviation summaries of each extracted MD observable;
- using only the cross-run mean and cross-run mean normalised histogram of each extracted MD observable, plus cross-run mean hydrogen bond lifetime curves;
- applying PCA to the full concatenation of all extracted MD features and using the resulting lower-dimensional representation.

Two main integration modules are implemented for MD features. The chemical representation these modules take as input is either given by  $X^{(\text{tri},i)} \in \mathbb{R}^{B \times T \times (d_{\text{trimer,fp}} + 6d_{\text{nuc,trimer}})}$  as described in Paragraph 4.5.3.4.1, or by  $H^{(\text{tri,learned})} \in \mathbb{R}^{B \times T \times d_{\text{trimer,learned}}}$ . For the sake of generality, let us write their input as

$$X^{(\text{chem})} \in \mathbb{R}^{B \times T \times d_{\text{chem}}}$$

**4.5.3.4.5 Additive chemical + MD embedding** In the first integration module implemented, chemical and MD descriptors are passed through separate MLPs into the same token space and then combined additively.

The final trimer token is then computed as

$$H^{(\text{tri})} = \Phi_{\text{chem}}(X^{(\text{chem})}) + M^{(\text{MD})} \odot \Phi_{\text{MD}}(X^{(\text{MD})})$$

where

$$\Phi_{\text{chem}} : \mathbb{R}^{d_{\text{chem}}} \rightarrow \mathbb{R}^{d_{\text{trimer}}}, \quad \Phi_{\text{MD}} : \mathbb{R}^{d_{\text{MD}}} \rightarrow \mathbb{R}^{d_{\text{trimer}}}$$

are learnable MLPs and the mask  $M^{(\text{MD})}$  is broadcast along the feature dimension. Hence, only the chemical branch contributes when no MD representation exists.

**4.5.3.4.6 Masked concatenation + MLP** In the second strategy, the chemical representation, the masked MD representation, and an explicit scalar availability indicator are concatenated and passed jointly through a single MLP. This formulation allows the model not only to use the MD descriptor values when present, but also to condition on the very fact that MD information is available.

Optionally, the MD features are first passed through a separate MLP

$$\Psi_{\text{MD}} : \mathbb{R}^{d_{\text{MD}}} \rightarrow \mathbb{R}^{d_{\text{MD}}^*}$$

before concatenation. Let

$$\widehat{X}^{(\text{MD})} = \Psi_{\text{MD}}(X^{(\text{MD})}), \quad \widetilde{X}^{(\text{MD})} = M^{(\text{MD})} \odot \widehat{X}^{(\text{MD})} \quad (4.62)$$

Then, the trimer token is then computed as

$$H^{(\text{tri})} = \Phi_{\text{concat}}([X^{(\text{chem})} \parallel \widetilde{X}^{(\text{MD})} \parallel M^{(\text{MD})}])$$

where the scalar mask is appended as an additional one-dimensional feature for each trimer position and  $\Phi_{\text{concat}}$  is a **MLP** mapping the concatenated representation to  $\mathbb{R}^{d_{\text{trimer}}}$ .

During training, an additional **MD** dropout mechanism may be activated: for a random fraction of trimers with available **MD** features, the **MD** mask is artificially set to zero before concatenation, forcing the model to remain robust when dynamical information is absent.

#### 4.5.3.5 Trimer-to-sequence encoder

Once a token representation has been constructed for each overlapping duplex trimer, the full **siRNA** is represented as a sequence of trimer tokens; i.e. we set  $I^{(2)}$  to be a sequence (or 1D array) of length  $T$ . This sequence is then mapped to a single representation by the trimer-to-sequence encoder. In the notation of Section 3.3.3, this module may be viewed as

$$p_{2,3} : (\mathbb{R}^{d_{\text{trimer}}})^T \rightarrow \mathbb{R}^{d_{\text{seq}}}$$

Concretely, if the trimer token tensor is  $Z \in \mathbb{R}^{B \times T \times d_{\text{trimer}}}$  and the trimer padding mask is  $M \in \{0,1\}^{B \times T}$  where  $M_{b,t} = 1$  indicates that trimer position  $t$  in sample  $b$  is padded, then the trimer-to-sequence encoder produces  $H^{(\text{seq})} \in \mathbb{R}^{B \times d_{\text{sequence}}}$ .

Two implementations are considered in this work: a transformer-based encoder and a one-dimensional convolutional encoder. Both implementations internally combine contextual encoding with a final pooling stage.

**4.5.3.5.1 Transformer trimer-to-sequence encoder.** In the transformer implementation, the trimer token sequence is first projected, if necessary, from  $d_{\text{trimer}}$  to a model dimension  $d_{\text{sequence}}$ . Let  $\widehat{Z} \in \mathbb{R}^{B \times T \times d_{\text{sequence}}}$  be the projected input.

The encoder then optionally prepends a learnable **CLS** token, depending on the chosen pooling strategy, and augments the sequence with positional

information. We evaluate both standard sinusoidal positional encodings and learned positional embeddings.

The resulting sequence is passed through a stack of transformer encoder layers composed of multi-head self-attention and position-wise feed-forward blocks, yielding a contextualised representation  $\tilde{Z} \in \mathbb{R}^{B \times S \times d_{\text{sequence}}}$  where  $S = T$  if no CLS token is used and  $S = T + 1$  otherwise. The padding mask is propagated accordingly so that padded trimer positions do not contribute to attention. Finally, the contextualised trimer sequence is pooled into a single fixed-size sequence representation. The pooling operator is treated as a configurable component, and we consider mean pooling, max pooling, CLS-token pooling, and learned attention pooling.

Thus, the transformer trimer-to-sequence encoder has the form

$$p_{2,3}^{\text{Tr}} = \rho \circ \text{TrEnc} \circ \text{PE} \circ \Pi$$

where  $\Pi$  denotes the optional input projection together with optional CLS augmentation, PE denotes positional encoding, TrEnc denotes the transformer encoder, and  $\rho$  is the final pooling map.

**4.5.3.5.2 CNN trimer-to-sequence encoder** In the convolutional implementation, the trimer token sequence is instead processed by a one-dimensional convolutional network along the trimer axis. Given  $Z \in \mathbb{R}^{B \times T \times d_{\text{trimer}}}$  padded positions are first zeroed out when a mask is provided, and the tensor is transposed to  $Z^{\text{cnn}} \in \mathbb{R}^{B \times d_{\text{token}} \times T}$  so that the trimer token dimension becomes the channel dimension of the convolutional network.

A stack of one-dimensional convolutional blocks is then applied. Each block consists of a one-dimensional convolution, batch normalisation, a GELU activation and dropout. We consider both a length-preserving mode, in which asymmetric padding is used so that the trimer sequence length remains constant across layers, and a non-length-preserving mode, in which the sequence shrinks naturally after each convolution. In the latter case, the trimer padding mask is reduced after each layer so that an output position is marked as padded if and only if all input positions within the corresponding convolution window were padded.

After the convolutional stack, a final  $1 \times 1$  convolution projects the channel dimension to  $d_{\text{sequence}}$ , yielding a tensor in  $\mathbb{R}^{B \times d_{\text{sequence}} \times T'}$  for some resulting trimer length  $T'$  which is equal to  $T$  if length-preserving mode is used.

This tensor is then transposed back to  $\mathbb{R}^{B \times T' \times d_{\text{sequence}}}$ , the reduced mask is applied if present, and a final pooling operator maps the contextualised

trimer sequence to a fixed-size sequence embedding in  $\mathbb{R}^{B \times d_{\text{sequence}}}$ . The implementation supports mean pooling, max pooling and learned attention pooling.

Hence, the convolutional trimer-to-sequence encoder may be written as

$$p_{2,3}^{\text{CNN}} = \rho \circ \Gamma,$$

where  $\Gamma$  denotes the one-dimensional convolutional stack together with the final  $1 \times 1$  projection and the associated mask updates, and  $\rho$  is the final pooling map.

#### 4.5.3.6 Prediction head

The pooled sequence representation is finally concatenated with the global experimental feature vector and passed through a prediction head that outputs a single scalar corresponding to the predicted activity. The input to the prediction head is

$$X^{(\text{head})} = [H^{(\text{seq})} \parallel X^{(\text{global})}] \in \mathbb{R}^{B \times (d_{\text{sequence}} + d_{\text{global}})}$$

In the current implementation, the prediction head is a **MLP**

$$\Phi_{\text{seq}} : \mathbb{R}^{d_{\text{sequence}} + d_{\text{global}}} \rightarrow \mathbb{R}$$

applied independently to each sample in the batch. Thus, the final prediction is

$$\hat{y} = \Phi_{\text{seq}}(X^{(\text{head})}) \in \mathbb{R}^{B \times 1}.$$

Each hidden layer therefore consists of an affine map followed by layer normalisation, GELU activation and dropout, while the final layer is linear and produces the scalar regression output. If the hidden widths are  $h_1, \dots, h_L$ , then the implemented prediction head has the form

$$\Phi_{\text{seq}} = A_{L+1} \circ (\text{Drop}_L \circ \text{GELU} \circ \text{LN}_L \circ A_L) \circ \dots \circ (\text{Drop}_1 \circ \text{GELU} \circ \text{LN}_1 \circ A_1)$$

where

$$\begin{aligned} A_1 : \mathbb{R}^{d_{\text{sequence}} + d_{\text{global}}} &\rightarrow \mathbb{R}^{h_1}, & A_\ell : \mathbb{R}^{h_{\ell-1}} &\rightarrow \mathbb{R}^{h_\ell} \text{ for } \ell = 2, \dots, L, \\ A_{L+1} : \mathbb{R}^{h_L} &\rightarrow \mathbb{R} \end{aligned}$$

which only differs from the other **MLPs** implemented throughout the

hierarchical framework (whose structure is summarized in (4.60)) in the absence of the final layer normalization.

Observe that we did not implement a global covariates dropout, contrary to the MD features case. This is simply because global covariates are always available and we therefore need not worry about the model's robustness to their absence.

#### 4.5.4 Baselines and evaluation methodology

To contextualize the performance of the proposed hierarchical architectures, we compare them against two baseline model families: XGBoost [28] and Cm-siRPred [5].

These baselines serve complementary purposes. XGBoost provides a strong non-neural tabular reference model built on top of the same chemically informed descriptors used throughout this work. Cm-siRPred represents the strongest previously published deep learning architecture specifically designed for modified siRNA activity prediction. We must acknowledge that the original Cm-siRPred model uses a different fingerprint generator and also employs different nucleotide descriptors than the ones used in this work, but here we stick to Morgan fingerprints and RDKit descriptors for all models to ensure a fair comparison.

All models are trained on the hard data split training partition and selected on the basis of validation performance on the validation partition of the same data split.

Only the best performing hierarchical model, the best performing XGBoost baseline and the best performing Cm-siRPred baseline are later:

- evaluated on the test partition of the hard data split
- trained a second time, independently, on the training partition of the easy data split (i.e., technically, it is their architectures that are trained again) and evaluated on the test split of the same partition, in order to evaluate how their behaviour translates to a setting with a larger degree of shared information between training and held-out data.

Early stopping on the validation set is in place during the training process. The patience hyper-parameter (allowed number of epochs without improvement on the validation set MAE before training halts) is set to 200 for all of our hierarchical models, and to 500 for the Cm-siRPred baseline, to compensate for its significantly larger amount of learnable parameters.

We further set a series of training-related hyper-parameters that are shared across all the neural models to ensure a fair comparison between the evaluated models, as detailed in Table 4.10. A hyperparameter optimization using

Table 4.10: Hyperparameter values kept fixed across the evaluation of the machine learning models

| Hyperparameter              | Value               |
|-----------------------------|---------------------|
| Learning rate               | $3.0 \cdot 10^{-4}$ |
| Weight decay                | $2.0 \cdot 10^{-2}$ |
| Dropout rate                | 0.1                 |
| Gradient clipping threshold | 1.0                 |
| Maximum number of epochs    | 1500                |
| Batch size                  | 128                 |
| Random state                | 42                  |

Optuna [67] is run on the best performing architecture found. The final best model must beat the second best performing model across three random seeds which we set to 42, 73 and 1729.

#### 4.5.4.1 Ablation and Feature importance analyses

The contribution of the MD-derived descriptors is assessed through two complementary analyses. First, the auxiliary XGBoost experiments with median-imputed MD features provide a simple non-neural estimate of the additional predictive value carried by those descriptors. Second, feature ablation is performed on the best-performing hierarchical model by removing or restricting the MD input branch. Together, these analyses help distinguish improvements due to the hierarchical architecture itself from improvements attributable to the inclusion of trimer-level dynamical information.

The distinction between the performance of chemical-only and MD-augmented representations is also used to assess the relevance of the MD-derived descriptors themselves.

#### 4.5.4.2 XGBoost baselines

The XGBoost baselines operate on fixed-length tabular feature vectors obtained from the preprocessing pipeline introduced in the previous section. Two different versions are considered:

- One operates at nucleotide level, concatenating the flattened nucleotide input tokens and the global features of each siRNA molecule and

yielding a total dimension of

$$d_{\text{global}} + 2N \cdot (7 + n_d + n_f + n_{fd})$$

per **siRNA** molecule.

- One operates at the trimer level, concatenating the flattened trimer fingerprint-based representations, the flattened nucleotide one-hot base encoding and RDKit descriptors, the global features and optionally the flattened trimer **MD** features and yielding a total dimension of

$$d_{\text{global}} + T \cdot (n_t + n_{td} + d_{\text{MD}} \cdot \mathbb{1}(\text{MD included})) + 2N \cdot (7 + n_d)$$

per **siRNA** molecule. If **MD** features are used, those trimers without them get assigned the median of the trimer features present in the training set.

We evaluate two nucleotide-level `XGBoost` configurations and four trimer-level configurations. The tested hyper-parameter settings are summarized in Tables 4.11 and 4.12.

Table 4.11: Tested nucleotide-level `XGBoost` hyper-parameter configurations.

| Hyper-parameter         | N1         | N2        |
|-------------------------|------------|-----------|
| $n_{\text{estimators}}$ | 15000      | 1000      |
| Maximum depth           | 12         | 10        |
| Learning rate           | 0.05       | 0.1       |
| Subsample               | 0.8        | 0.8       |
| Column subsample        | 0.4        | 0.5       |
| Minimum child weight    | 5          | 10        |
| $\lambda$               | 1.0        | 1.0       |
| Early stopping          | 100 rounds | 50 rounds |

In all reported `XGBoost` baseline configurations, the assigned importance to each of the input features is retained in order to understand which of them are the most informative for the model.

#### 4.5.4.3 Cm-siRPred

We also compare against `Cm-siRPred` [5], which, to the best of our knowledge, is the strongest previously published model specifically tailored

Table 4.12: Tested trimer-level XGBoost hyper-parameter configurations.

| Hyper-parameter         | T1            | T2            | T3&4                       |
|-------------------------|---------------|---------------|----------------------------|
| $n_{\text{estimators}}$ | 15000         | 1000          | 5000                       |
| Maximum depth           | 12            | 10            | 11                         |
| Learning rate           | 0.05          | 0.1           | 0.075                      |
| Subsample               | 0.8           | 0.8           | 0.8                        |
| Column subsample        | 0.4           | 0.5           | 0.45                       |
| Minimum child weight    | 5             | 10            | 7                          |
| $\lambda$               | 1.0           | 1.0           | 1.0                        |
| Patience                | 100 rounds    | 50 rounds     | 75 rounds                  |
| Notes                   | Chemical only | Chemical only | MD study, with & w/o means |

to modified **siRNA** activity prediction. A mathematical description of its architecture was given in Section 2.4.10, so here we restrict ourselves to the implementation details relevant to its use as a baseline.

Cm-siRPred operates on three separate nucleotide-level input matrices: one-hot base encodings, PCA-reduced physicochemical property vectors, and PCA-reduced nucleotide fingerprint vectors. The guide and passenger strands are concatenated into a single padded sequence of length 50, corresponding to 25 nucleotide positions per strand. Thus, each sample is represented by three matrices of shapes

$$[2N, d_{\text{base}}], \quad [2N, d_{\text{prop}}], \quad [2N, d_{\text{fp}}]$$

together with the global experimental covariates. Padding positions are identified from the all-zero base encoding, and the corresponding rows in the property and fingerprint matrices are explicitly set to zero.

The chemical inputs for Cm-siRPred are produced with the same preprocessing pipeline described in the previous section, but arranged into the specific three-view format expected by the model. We keep the architectural hyper-parameters reported in the original publication: a shared embedding dimension of 512, a four-head cross-attention module, a two-layer convolutional network with channel progression  $3 \rightarrow 16 \rightarrow 32$ ,  $3 \times 3$  convolutions with stride 1 and padding 1,  $2 \times 2$  max pooling after each convolution, and a fully connected layer of width 128 followed by dropout with rate 0.5 and a final sigmoid output.

#### 4.5.4.4 Hierarchical Models

Our hierarchical model selection and evaluation procedure is organised in two different stages. The first stage considers hierarchical models which include a

transformer in either the nucleotide to sequence or trimer to sequence modules, while the second stage considers those hierarchical models which include a **CNN** in either the nucleotide to sequence or trimer to sequence modules.

Across both stages, we set  $d_{\text{trimer}} = d_{\text{seq}} = 128$  and use a head **MLP** with two hidden layers of sizes 128 and 64, respectively. This ensures a fair comparison between the explored models.

**4.5.4.4.1 Transformer-based Hierarchical Models** The exploration of transformer-based hierarchical models is carried out in 3 steps:

- First, we test whether nucleotide to sequence or trimer-to-sequence level transformers perform better. Initially, we use mean pooling.

For the nucleotide case, we test token dimensions  $d_{\text{nuc}} = 64, 128$  and always use a one-layer **MLP** with hidden dimension 64 as nucleotide encoder. This is because  $d_{\text{nuc},i}$  is considerably smaller than the tested  $d_{\text{nuc}}$ .

For the trimer case, we respect the convention  $d_{\text{trimer}} = 128$ . In both cases, we try feed-forward dimensions 512 and 256, 8 and 4 heads and 4 and 2 layers.

The best performing architecture is also tested using CLS, Attention and Max pooling. The best performing pooling strategy and transformer architecture is inherited by the subsequent models.

- Second, we test whether a learned trimer representation can outperform the preprocessed trimer input representations in the best performing trimer-to-sequence level transformer case. Here, we explore the learned only 4.5.3.4.2 and hybrid chemical/learned 4.5.3.4.3 trimer encoder architectures.

We use a one-layer **MLP** with hidden dimension 64 as nucleotide encoder, but set  $d_{\text{nuc}} = 32$ .

For the nucleotide to trimer encoder module, we only employ the **MLP** encoder 4.5.3.3.1. We set  $d_{\text{trimer,learned}} = d_{\text{trimer}} = 128$  and try using both one layer with hidden dimension 256 and two layers with hidden dimensions 256 and 128, respectively. The best performing choice coupled with the learned only trimer encoder is also applied to the hybrid chemical/learned trimer encoder.

- Third, we test whether the inclusion of MD features improves the performance of the best trimer-based transformer architecture found in the two previous steps.

Testing for the best MD-aware trimer encoder module is done using only cross-run means and standard deviations for all extracted MD observables. Initially, MD dropout is set to 0.1.

In the additive MD encoder 4.5.3.4.5 case, both the chemical features MLP and the MD features MLP use one single layer with hidden dimension 256, while respecting the convention  $d_{\text{trimer}} = 128$ . In the concatenation MD encoder 4.5.3.4.6 case, three different options are tested:

1. A one-layer MLP on the concatenated input with hidden dimension 256, and no previous MLP on the MD features.
2. A two-layer MLP on the concatenated input with hidden dimensions 256 and 128 respectively, and no previous MLP on the MD features.
3. A one-layer MLP on the MD features with hidden dimension 128 and output dimension also 128 (i.e., the MD feature vector that gets appended to the trimer input token has size 128), followed by a one-layer MLP on the concatenated input with hidden dimension 256.

The best performing option is tested across MD dropout values of 0.3 and 0.5 as well.

The resulting model is also tested using, as MD features:

- the cross-run mean and cross-run mean normalised histogram of each extracted MD observable, plus cross-run mean hydrogen bond lifetime curves
- the PCAd full extracted MD features

**4.5.4.4.2 CNN-based Hierarchical Models** In the CNN-based models, we impose all nucleotide to sequence and trimer to sequence CNNs to have 3 convolutional layers in order to limit the scope of our analysis. Nevertheless, different numbers of convolutional layers will later be explored during hyperparameter optimization if the most successful model turns out to be CNN-based.

The exploration of **CNN**-based hierarchical models is also carried out in 3 steps:

- First, we test whether nucleotide to sequence or trimer-to-sequence level **CNNs** perform better. Initially, we use mean pooling.

For the nucleotide case, we use  $d_{\text{nuc}} = 32$  and always employ a one-layer **MLP** with hidden dimension 64 as nucleotide encoder. We first test models with and without sequence padding, (constant) kernel width 3 and the following structural hyperparameters:

- channel dimensions 64, 128, 128 and 128, 128, 128, with kernel lengths 2, 1, 1 (strands are fused in the first convolutional layer)
- channel dimensions 64, 128, 128 with kernel lengths 1, 2, 1 (strands are not fused until the second convolutional layer)

The best performing architecture is subjected to further testing, and (constant) kernel widths  $k^{(w)} \in \{2, \dots, 6\}$  are considered. This is particularly relevant because the kernel width in the first **CNN** layer (irrespective to whether sequence padding is active or not) may be interpreted as the selected  $M$ -mer unit on which information is aggregated.

For the trimer case, we respect the convention  $d_{\text{trimer}} = 128$ . In this case we set the **CNN** channel dimensions to 128, 128, 128 and the kernel width to  $k^{(w)} = 5$ . We do not test a first channel dimension 64 because the trimer input dimension is considerably larger than the nucleotide input dimension, so the first test round only evaluates the use of sequence padding.

Afterwards, the best performing trimer model is tested with (constant) kernel widths  $k^{(w)} \in \{2, \dots, 10\}$ . This is an important evaluation because it assesses how far do relevant inter-trimer interactions for **siRNA** activity prediction span.

Finally, the best performing architecture from this step is also tested using Attention and Max pooling. The best performing pooling strategy and **CNN** architecture is inherited by the subsequent models.

- Second, we test
  - whether a learned trimer representation can outperform the preprocessed trimer input representations in the best performing

trimer-to-sequence level **CNN** case. Here, we explore the learned only 4.5.3.4.2 and hybrid chemical/learned 4.5.3.4.3 trimer encoder architectures.

- whether using a **CNN** on a  $2 \times 3$  array trimer representation (where each array unit is a nucleotide, see 4.5.3.3.2) can improve upon the **MLP** encoder 4.5.3.3.1

We use a one-layer **MLP** with hidden dimension 64 as nucleotide encoder, but set  $d_{\text{nuc}} = 32$ .

We set  $d_{\text{trimer,learned}} = d_{\text{trimer}} = 128$ . The **CNN** versus **MLP** nucleotide to trimer encoder performance is evaluated with the learned only trimer encoder.

The **MLP** nucleotide to trimer encoder inherits the best architecture for each trimer encoder option observed in the transformer models study. The **CNN** nucleotide to trimer encoder inherits the architecture from the best nucleotide-to-sequence **CNN** transformer, but we test reducing the number of layer to two convolutional layers, smaller kernel widths and impose that sequence length is preserved to accommodate the smaller size of the studied structures.

- Third, we test whether the inclusion of **MD** features improves the performance of the best trimer-based **CNN** architecture found in the two previous steps.

The best **MD**-aware trimer encoder module is inherited from the transformer-based models exploration. Initially, **MD** dropout is set to 0.1.

The input **MD** feature types tested are:

- the cross-run mean and cross-run standard deviation of each extracted **MD** observable
- the cross-run mean and cross-run mean normalised histogram of each extracted **MD** observable, plus cross-run mean hydrogen bond lifetime curves
- the PCAd full extracted **MD** features

The best performing option is tested with an **MD** dropout value of 0.3 as well as across the set of best-performing kernel widths from the **MD** feature-less trimer-to-sequence **CNN** case.



## Chapter 5

# Results and discussion

In this chapter, we present the obtained results for this thesis. Section 5.1 covers the data-preprocessing results, Section 5.2 discusses the results of the MD preparation and simulation pipeline, Section 5.3 addresses the results of the trimer MD features extraction, and finally Section 5.4 presents the ML model training and evaluation results.

### 5.1 Data analysis and pre-processing

We address here the results of the dataset analysis and trimer, nucleotide and modifications extraction that we carried out.

#### 5.1.1 Dataset analysis

The dataset contains 88833 siRNA molecules, among which we observed a total of 27625 unique sequences, with the most common sequence appearing 376 times. The most common sequences and their counts are presented in table 5.1. All the siRNA molecules in the dataset have been extracted from patents.

There are over 12000 with an activity between 0.0 and 0.033, but the remaining activity range is evenly distributed. The whole distribution can be found in figure 5.1a.

We observed strong differences in the concentration used (in nM), which ranged from 9.92e-07 nM to 1356895.35 nM. Thus, we will use z-scaling when normalizing the concentration for model training. The concentration distribution is plotted in figure 5.1b

Table 5.1: Top 10 repeated siRNA sequence pairs in the dataset, ranked by occurrence count. The anti-sense strand (`as_seq`) is the guide strand written in 5'–3' orientation, and the sense strand (`s_seq`) is the passenger strand written in 3'–5' orientation.

| <code>as_seq</code>       | <code>s_seq</code>        | Count |
|---------------------------|---------------------------|-------|
| ACAAAAGCAAAACAGGUCUAGAA00 | UGUUUUCGUUUUGUCCAGAU0000  | 376   |
| AAAAAGAUAUAUGUCUGCUUGCU00 | UUUUUCUAUUUACAGACGAAC0000 | 299   |
| 00ACUUCAUCUUUCUCCACUU00   | UUUGAAGUAGAAAGAAGGGUG0000 | 190   |
| 00GAGAACCAGUAGCAGCUGCUU00 | UUCUCUUGGUCAUCGUCGACG0000 | 122   |
| 00UUGCUC AUGACCUCGUGCTT00 | TTAACGAGUACUGGAGCGACG0000 | 108   |
| 00AAGUCACAGAUCCUCUATT00   | TTUUCAGUGUCUAGAGGAGAU0000 | 102   |
| ACAAAAGCAAAACAGGUCUAG0000 | UGUUUUCGUUUUGUCCAGA000000 | 96    |
| UUGUACCCUAGGAAAUACCAGAG00 | AACAUGGGAUCCUUUAUGGUC0000 | 96    |
| AGUGAAAAAGGUGUUCUAAAAUU00 | UCACUUUUUCCACAAGAUUUU0000 | 88    |
| AUAAAACCAACUCAGCUCAGAGG00 | UAUUUUGGUUGAGUCGAGUCU0000 | 86    |

The incubation time also showed some variability, but in this case it was constrained between 24 and 120 hours. Its distribution can be found in figure 5.1c

The siRNA molecules in the dataset target 102 different genes. VEFGA is the most targeted gene, with over 7000 siRNA molecules in the dataset targeting it. The number of siRNA molecules in the dataset decreases rapidly as we go down in the ranking, with the mean number of siRNA molecules targeting the 50 least targeted genes in the dataset being 193.58. The 3 least targeted genes (PTPRC, C9 and KISS1) have only 48, 46 and 46 siRNAs targeting them respectively, so it is interesting to ask whether this lower count deteriorates model performance on siRNA molecules targeting those genes. The full histogram of targeted gene counts appears in figure A.1 of the appendix due to its large size.

A similar trend is observed in the different cell types targeted. The dataset covers 32 distinct types of cells. There are over 25000 siRNA molecules in the dataset which target Hep3B cells (the most targeted cell type in the dataset), whereas the 16 least targeted cell types in the dataset are targeted by an average of 216.88 siRNA entries. The histogram of cell type counts is plotted in figure 5.1d.

The preferred transfection method is lipofectamine (for the purpose of this thesis, we only need to know that this involves using lipids to deliver the siRNA molecule into the cell), with 79577 siRNA molecules in the dataset delivered using it. 8882 siRNA molecules were delivered by simple free uptake, and the remaining 374 siRNA molecules in the dataset were delivered

using INTERFERin (for the purposes of this thesis, we only need to know that INTERFERin is a specialised reagent for **siRNA** delivery). The distribution of the employed transfection methods can be found in figure 5.1e

The preferred method for activity detection is qPCR, with the activity of almost 70000 **siRNA** molecules in the dataset having been measured using this technique. The dataset encompasses 7 different activity detection techniques, with all other techniques used for less than 10000 instances and the two least used detection techniques (ddPCR and Chromogenic) used in only 386 and 204 cases, respectively. The distribution of the employed detection methods can be found in figure 5.1f

### 5.1.2 Trimers, nucleotides and modifications

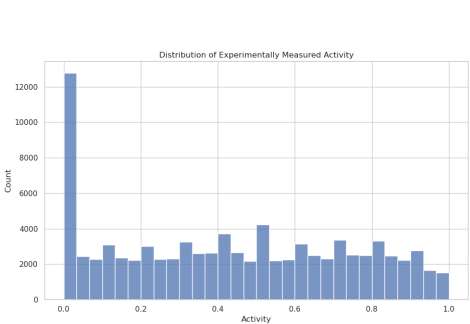
Our extraction methodology revealed that the dataset contains 41 different nucleotides, 8787 unique trimers and the following modifications:

- *Backbone* Only two distinct backbone fragments were observed. One corresponds to the standard phosphate backbone (with modXNA id RPO) and the other one corresponds to a modification in which the phosphorus shares a double bound with a Sulfur instead of an Oxygen. This makes the Phosphorus a chiral centre, and thus there exist two possible stereoisomers which modXNA encodes as PS1 and PS2. As we have no means to differentiate between both stereoisomers, we label all as PS1.
- *Sugar* There are 7 sugar variants in the dataset including the standard RNA ribose (with modXNA id RC3) and the standard DNA deoxyribose (with modXNA id DC2).

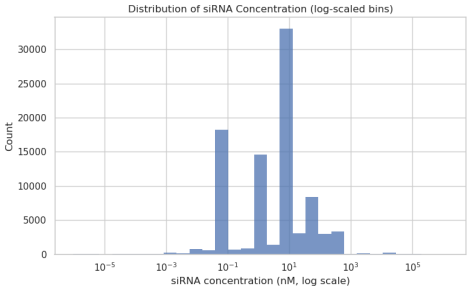
2 of the observed sugar modifications are not parametrized by modXNA. One of them is characterised by a long carbon chain attached to the 2' carbon in the sugar, and the other corresponds to an unlocked nucleic acid that is however not an ether, unlike modXNA's unlocked nucleic acid DUD.

The full `sugars_siRNA` database is shown in Table 5.2

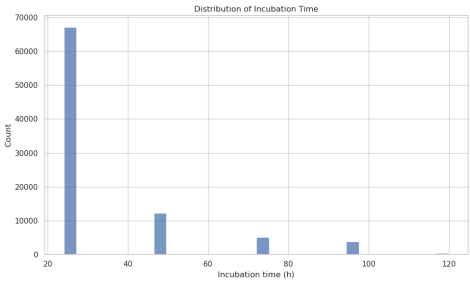
- *Base* The dataset only includes 5 different bases, which are just Adenine (A), Guanine (G), Cytosine (C), Uracil (U) and Thymine (T). This is, the only considered modification is **DNA's** Thymine instead of **RNA's** Uracil. Their modXNA codes are, respectively, RAA, RGG, RCC, RUU and DTT. There is one single nucleotide without a base.



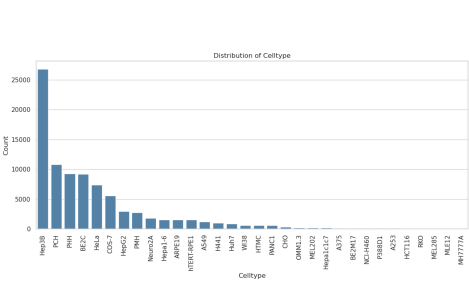
(a) Distribution of experimentally measured activity values across the dataset



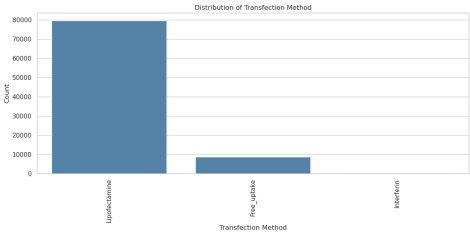
(b) Distribution of **siRNA** concentration (nM) used in the log-scale across the dataset



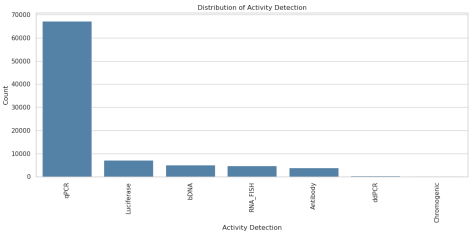
(c) Distribution of **siRNA** incubation times across the dataset



(d) Distribution of targeted cell types across the dataset



(e) Distribution of employed transfection methods across the dataset



(f) Distribution of employed activity detection methods across the dataset

Figure 5.1: Distribution of activity values, concentrations, incubation times, target cell types, transfection methods and activity detection methods across the dataset.

Table 5.2: Representation of `sugars_siRNA` database, including sugar **SMILES** and modXNA IDs but excluding **InChIKey**

| # | sugar_smiles                                   | modXNA_id |
|---|------------------------------------------------|-----------|
| 0 | <chem>CCCCCCCCCCCCCCCCCOC1C(C)OC(CO)C1O</chem> | —         |
| 1 | <chem>CC1OC(CO)C(O)C1O</chem>                  | RC3       |
| 2 | <chem>CC1OC(CO)C(O)C1F</chem>                  | AF2       |
| 3 | <chem>CC1CC(O)C(CO)O1</chem>                   | DC2       |
| 4 | <chem>COC1C(C)OC(CO)C1O</chem>                 | OME       |
| 5 | <chem>CCC(O)CO</chem>                          | —         |
| 6 | <chem>COCCOC1C(C)OC(CO)C1O</chem>              | MOE       |

Having identified the chemical modifications present in our dataset and cross-referenced those with modXNA, we are able to filter the extracted nucleotides and trimers to identify those which are fully parametrized by modXNA. The resulting `sirna_modXNA_unique_nucleotides` dataset contains 32 nucleotides, which reveals that 9 of the unique nucleotides found in the dataset are not parametrized by modXNA. The resulting `sirna_modXNA_unique_trimers` contains 7596 entries, which means that there are 1191 trimers which are not fully parametrized by modXNA. If we also eliminate those trimers which contain termini and thus cannot be faithfully simulated, we are left with 5925 unique trimers. We create a separate dataset called `sirna_unique_trimers_modXNA_notermini` which contains only those trimers which are fully parametrized and span no termini. These are the trimers we run **MD** simulations for. These results are summarized in table 5.3.

Table 5.3: Comparison of total components in the dataset versus those who additionally fulfill the criteria of being parametrized by modXNA, and those who satisfy both the previous criteria and the condition of spanning no termini

| Component   | In the dataset | & parametrized by modXNA | & containing no termini |
|-------------|----------------|--------------------------|-------------------------|
| Trimers     | 8787           | 7596                     | 5925                    |
| Nucleotides | 41             | 32                       | —                       |
| Backbones   | 2              | 2                        | —                       |
| Bases       | 5              | 5                        | —                       |
| Sugars      | 7              | 5                        | —                       |

Having datasets at 4 different levels (**siRNA**, trimers, nucleotides and modifications) allows one to implement a series of functions for the quick retrieval of information about the composition of our given dataset. These functions are all included in the `sirna_MDprep` package, but we include

here a series of examples of the information that can be retrieved through them.

For example, we observe that there are 70751 siRNAs in our dataset containing at least one trimer in

`siRNA_unique_trimers_modXNA_notermini` without a G base. Those 70751 contained an average of 3.397 trimers (out of a maximum 23 total trimers that a **siRNA** in our database may contain) in `siRNA_unique_trimers_modXNA_notermini` and with no G bases in any of their strands.

We also observe that the number of siRNAs that exclusively have trimers parametrized by modXNA is 64154, which means one could theoretically run whole-chain simulations for 72.22% of our dataset given the computational resources to do so.

We close this section with some illustrations of the `nucleotide_fragmentation` tool functionalities. In particular, we include the 2D visualization of different nucleotides that cover all the chemical modifications present in our dataset (Figure 5.2) and the 2D visualization of a single-stranded trimer obtained by linking its nucleotides together (Figure 5.3).

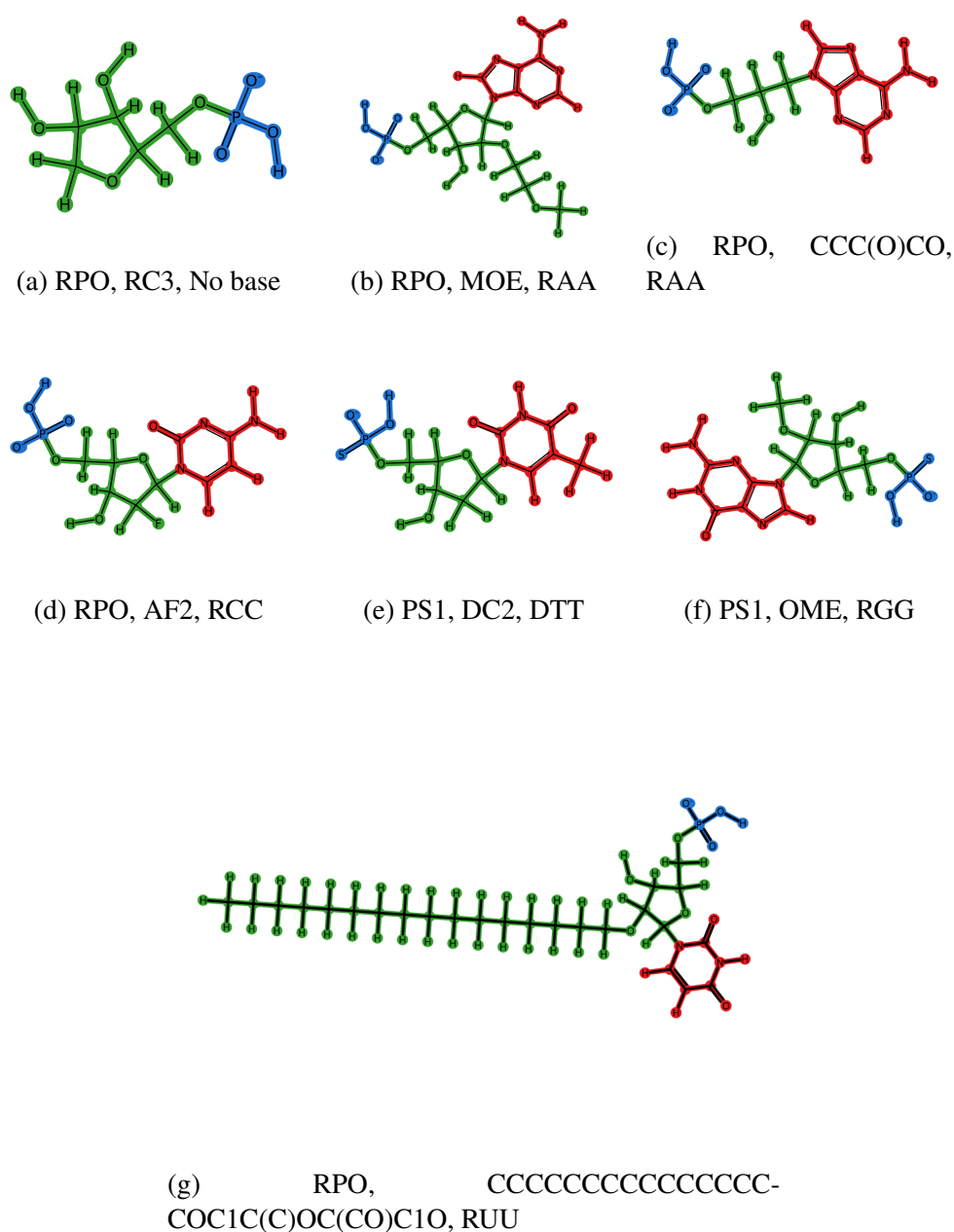


Figure 5.2: Nucleotide structures covering all chemical modifications present in the dataset with base fragments highlighted in red, sugar fragments highlighted in green and backbone fragments highlighted in blue. Each subfigure caption lists the backbone, sugar, and base modXNA IDs. A **SMILES** string in the place of the sugar fragment modXNA ID is used for those sugar modifications not available in modXNA.

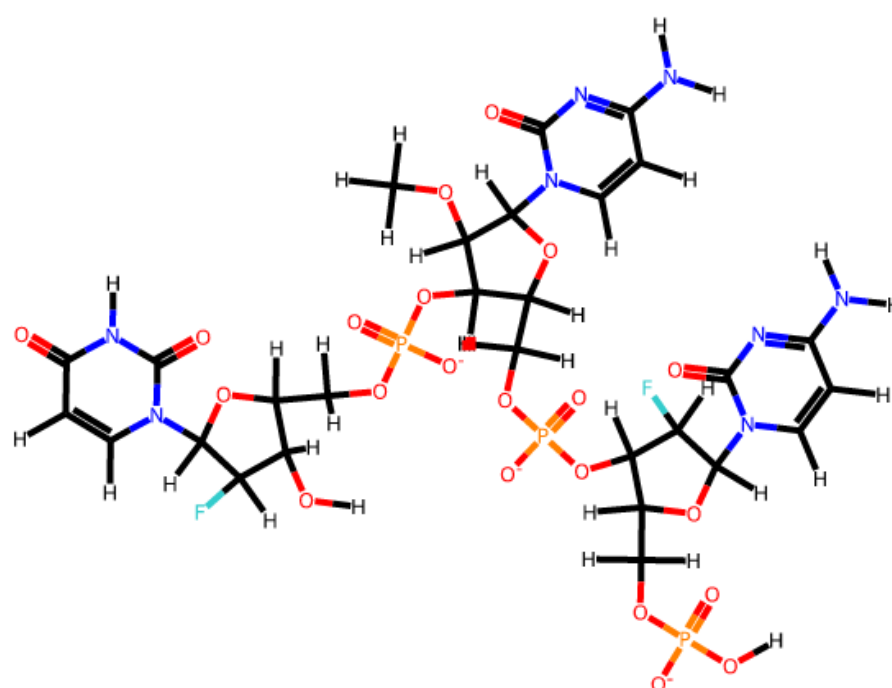


Figure 5.3: 2D representation of trimer #75 in the sirna\_unique\_trimers\_modXNA\_notermini dataset guide strand obtained by linking its conforming nucleotides

## 5.2 Molecular dynamics preparation and simulation

We present here the results of the molecular structure generation, system preparation, ion selection, production run calibration and computational load distribution carried out for the `sirna_unique_trimers_modXNA_notermini` dataset.

### 5.2.1 Structure generation and system preparation

Out of the 5925 candidate trimers whose constituent fragments are parametrized in the modXNA dataset, Oligo3D successfully generated structures for 5802 systems. Structure generation therefore failed for 123 trimers. Among the 5802 trimers for which a structure was obtained, `tLeap` failed for 11 systems. Consequently, the final set of successfully prepared systems contains 5791 trimers.

These results are summarized in Table 5.4.

Table 5.4: Summary of success and failure counts during molecular dynamics system construction and preparation for the 5925 unique non-cap trimers parameterized in modXNA. Percentages are reported relative to the initial set of 5925 trimers.

| Stage                                              | Count | Percentage |
|----------------------------------------------------|-------|------------|
| Initial trimers                                    | 5925  | 100.00%    |
| Oligo3D success                                    | 5802  | 97.92%     |
| Oligo3D failure                                    | 123   | 2.08%      |
| <code>tLeap</code> failure (after Oligo3D success) | 11    | 0.19%      |
| Final prepared systems                             | 5791  | 97.74%     |

We initially hypothesized that failures in structure generation or system preparation might be associated with trimers containing mismatched bases, that is, incorrect base pairings. However, only 65 of the 5925 trimers contain an incorrect base pairing but there are 123 Oligo3D failures, which means that there must be a deeper explanation for these errors.

A more detailed inspection shows that mismatched base pairs are strongly associated with `tLeap` failure but not with Oligo3D failure. In particular, all 11 `tLeap` failures correspond to trimers with mismatched bases. In contrast, only 1 of the 123 Oligo3D failures involves a trimer with an incorrect base pairing. Thus, incorrect base pairing appears to be a highly informative

predictor of failure at the `tLeap` preparation stage, but not of failure during initial structure generation.

## 5.2.2 Determination of the number of ions

We estimated the number of NaCl pairs required to reproduce a concentration of 150 mM in the simulation box. Across the 10 test systems, the average resulting box volume was approximately  $152433 \text{ \AA}^3$ .

Using the relation between concentration, box volume and number of molecules,

$$c = \frac{N_{\text{NaCl}}/N_A}{V_{\text{box}}},$$

the required number of NaCl pairs is

$$\begin{aligned} N_{\text{NaCl}} &= 150 \cdot 10^{-3} \frac{\text{mol}}{\text{L}} \cdot 152433 \text{ \AA}^3 \cdot \frac{10^{-27} \text{ L}}{\text{\AA}^3} \cdot 6.02214076 \cdot 10^{23} \frac{\text{molecules}}{\text{mol}} \\ &\approx 13.770 \end{aligned}$$

Since only an integer number of ions can be added, we truncate this value and add 13  $\text{Na}^+$  and 13  $\text{Cl}^-$  ions during the `tLeap` setup of each pentamer test system. This choice yields an ion count consistent with the target salt concentration while keeping the setup procedure simple and uniform across systems.

## 5.2.3 Selection of the production run length

We next evaluated the simulation length required to obtain stable production trajectories. The 10 test systems all show equilibration of the whole-pentamer **RMSD** more than 20 ns before the end of the tested production runs. This indicates that the considered timescales are sufficient to relax the initial pentamer structures.

However, some of the 25 ns trajectories also exhibit an additional change in the total **RMSD** near the end of the simulation, seemingly accessing a different mode. For example, in the test run for pentamer 3, this late transition corresponds upon visual inspection to a partial opening of the pentamer structure (see Figure 5.4).

This observation suggests that 25 ns simulations begin to access regions of conformational space that are not reached within shorter production runs.

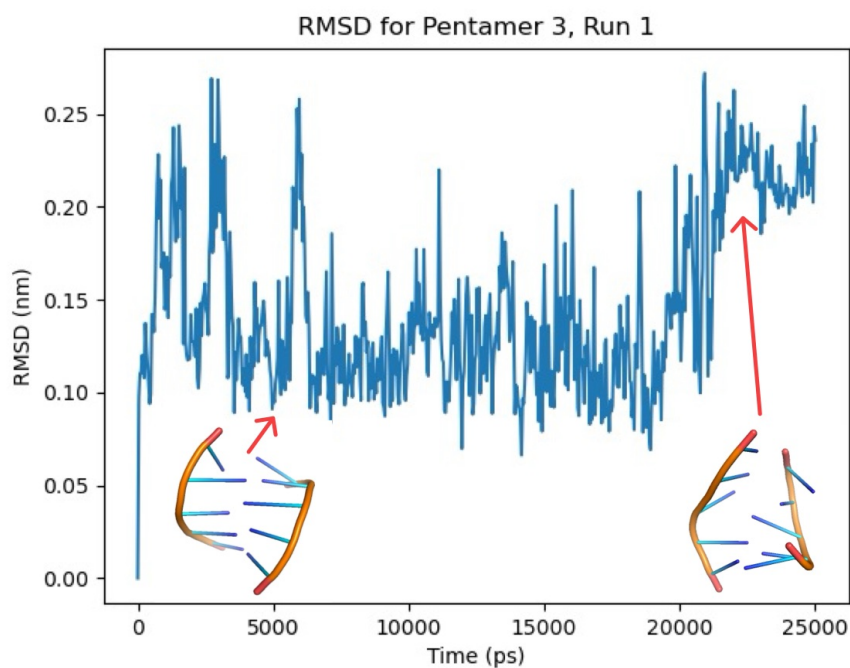


Figure 5.4: Whole-pentamer **RMSD** plot for the test run of trimer 3 in `siRNA_unique_trimers_modXNA_notermini` showcasing two snapshots of the pentamer's configuration: one right after the established equilibrium point (around 5 ns, left); and another one after the **RMSD** transitions to a different mode which corresponds to the pentamer opening (around 22 ns, right).

For this reason, and after confirming that this simulation length remained computationally affordable, we selected 25 ns as the production run length for the final simulations.

## 5.2.4 Load optimization and cluster usage

The final production workload was distributed across two computing environments. As a result of the load optimization procedure and fluctuating compute availability, 2710 trimers were run on the Swedish cluster and 3247 trimers were run on the US cluster.

These numbers imply that 166 trimers were executed twice. This duplication occurred because changing internal compute conditions required a re-optimization of the workload distribution. An early version of the re-optimization procedure contained a bug that reassigned some systems after they had already been scheduled, leading to duplicate execution. This issue has since been corrected. A detailed explanation of the bug is provided in the Appendix Section [A.2.1](#).

## 5.3 Feature Extraction

In the following, we present the results of the feature extraction procedure carried out on the simulated modified **siRNA** pentamers. We begin by discussing the final choice of equilibrium detection procedure and then address the outcomes of the extraction process for each family of observables. Unless stated otherwise, the summary statistics and distributional analyses reported in this section are computed from the simulations run on the Swedish cluster only. This choice was made to avoid overfitting in methodological decisions, such as histogram discretization.

### 5.3.1 Feature time-series and equilibrium detection

Each production run has a length of 25 ns and is sampled every 40 ps. Therefore, each trajectory contributes 626 frames, where the first frame corresponds to the initial configuration of the production run.

We initially implemented the automated equilibrium detection procedure of [63] on the whole-duplex **RMSD** time-series. In practice, however, this approach often yielded unreasonable equilibration estimates. For some systems, the estimated truncation point lay beyond frame 600, which would imply discarding more than 95.85% of the collected samples. Such outcomes

were clearly inconsistent with the qualitative behaviour of the trajectories and would leave us with an insufficient amount of samples to perform a significant statistical analysis.

Looking at the affected trajectories, we identified that these late truncation estimates were associated with the **RMSD** time-series displaying jumps between different states (or modes) or simply increased variability. For example, for the test run of pentamer 3, the predicted equilibration time from the whole-pentamer **RMSD** time-series plotted in Figure 5.4 is 20.76 ns, which closely matches the transition time to the open pentamer configuration. If we look instead at the predicted equilibration times for the relevant residues 2,3,4,7,8 and 9 **RMSD** time-series, they all fall between 2 and 3 ns.

The observed behaviour is not entirely unexpected. The automated equilibrium detection method introduced in [63] has also been reported in the literature to produce unrealistically late equilibration estimates in certain settings [68, 69]. Although more robust alternatives have been proposed [70], we ultimately opted for a simpler and more conservative rule: discarding the first 5 ns of every production run, corresponding to the first 125 frames.

This choice was motivated by two considerations. First, if one applies the same automated equilibrium detection method to the per-residue **RMSD** time-series instead of the whole-duplex **RMSD**, and then takes for each system the maximum predicted equilibration time across all runs and relevant residues, one standard deviation of that distribution lies below the 5 ns mark. Second, according to expert judgement, 5 ns is a safe estimate for the equilibration time of **RNA** pentamers under the present simulation conditions.

As a result, throughout the remainder of this chapter, equilibrium analyses are performed after discarding the first 125 frames of each run. Whenever a feature is intrinsically defined over the full trajectory, as is the case for **RMSF**, this exception is stated explicitly.

### 5.3.2 Choice of histogram bins

Recall that for those observables computed on repeated substructures of a pentamer, such as individual residues or residue pairs, the pooled sample used for bin selection includes all instances of the same type. Thus, for example, all residue-level **RMSD** values are pooled together, all base-pair hydrogen-bond distances are pooled together, and all interaction energies between residue pairs in  $\mathcal{D}_{c1}$  are also pooled together.

The default number of bins used to cover each of the ranges derived from the pooled distribution of a certain feature under a certain class is 10. In a few

cases, however, the empirical pooled distribution was sufficiently multimodal or otherwise irregular that a larger number of bins was used.

For discrete observables such as hydrogen-bond counts, we use bins adapted to their support on the natural numbers. Further, recall that the sugar pucker angle histogram bins are predefined by the conformational steps defined in 4.8.

### 5.3.3 Extracted trimer features

The final feature set combines structural, energetic and hydrogen-bond descriptors extracted from the trajectories after preprocessing and equilibrium truncation.

Overall, the extraction pipeline was remarkably successful. However, the base-pair, base-pair step and helical parameters returned by `nastruct` were considerably problematic. The time series produced by `nastruc` for these parameters is often missing some of the pentamer's base-pair steps (i.e., pairs of consecutive base pairs), which are the groups for which the BP step and the Helix parameters are computed. For some systems, some base pairs are also missing in the BP parameter time-series - but this is less common.

Dealing with frequently missing parameters whose absence does not necessarily carry a signal but is rather dictated by internal code inconsistencies would prove rather messy and, as a result, both BP-step and Helix parameters are discarded.

### 5.3.4 Feature distributions and discretization

We next summarize the pooled equilibrium distributions of the extracted observables and the corresponding histogram choices adopted for the final dataset. In each case, the reported number of samples refers to the total number of equilibrium observations obtained after pooling the relevant feature across all runs, systems and repeated substructures of the corresponding class in the Swedish cluster.

#### 5.3.4.1 Base-pair parameters

The pooled sample size for all base-pair parameters is 19,926,611 observations. The selected histogram ranges are summarized in Table 5.5.

The empirical pooled distribution for each of the base pair parameters is plotted in Figures 5.5(shear), 5.6 (stretch), 5.7 (stagger), 5.8(buckle), 5.9 (propeller), 5.10 (opening), 5.11(major), 5.12 (minor).

Table 5.5: Distribution summaries and selected histogram ranges for base-pair parameters.

| Parameter | P0.5     | P99.5   | Chosen binning                                                           |
|-----------|----------|---------|--------------------------------------------------------------------------|
| shear     | -1.1540  | 1.1639  | 10 in $[-1.2, 1.2]$ + 2 outlier bins                                     |
| stretch   | -0.4405  | 0.4780  | 10 in $[-0.5, 0.5]$ + 2 outlier bins                                     |
| stagger   | -1.6844  | 1.1196  | 10 in $[-1.7, 1.2]$ + 2 outlier bins                                     |
| buckle    | -36.0595 | 34.0300 | 10 in $[-40, 40]$ + 1 circ. outlier bin                                  |
| propeller | -35.6400 | 21.5206 | 10 in $[-40, 30]$ + 1 circ. outlier bin                                  |
| opening   | -15.3241 | 19.2567 | 10 in $[-25, 25]$ + 1 circ. outlier bin                                  |
| major     | 0.0000   | 20.5876 | 10 in $[16, 21]$ + state-0 bin + outlier bins $(0, 16)$ , $(21, \infty)$ |
| minor     | 0.0000   | 14.0493 | 10 in $[12, 15]$ + state-0 bin + outlier bins $(0, 12)$ , $(15, \infty)$ |

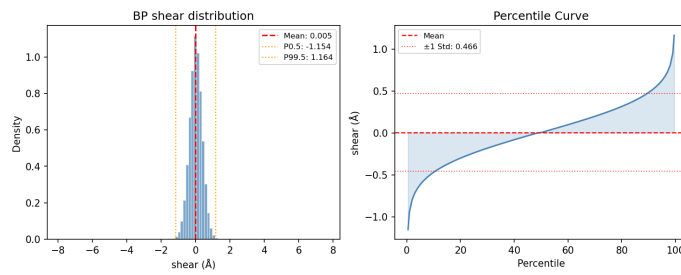


Figure 5.5: Empirical pooled distribution of the shear BP parameter

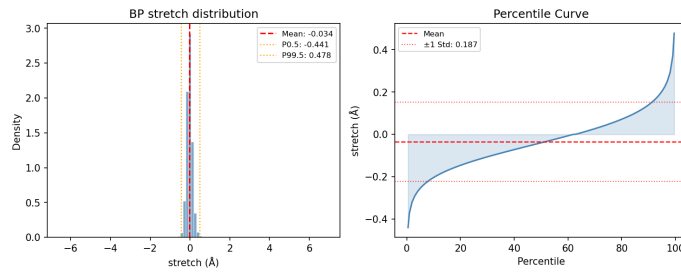


Figure 5.6: Empirical pooled distribution of the stretch BP parameter

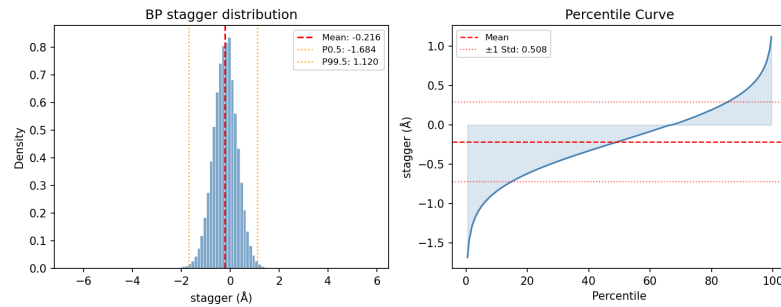


Figure 5.7: Empirical pooled distribution of the stagger BP parameter

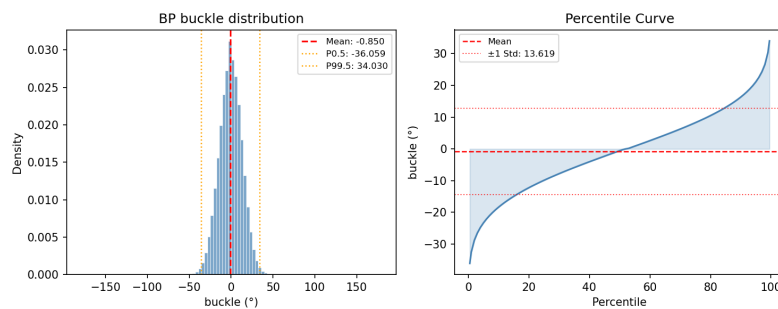


Figure 5.8: Empirical pooled distribution of the buckle BP parameter

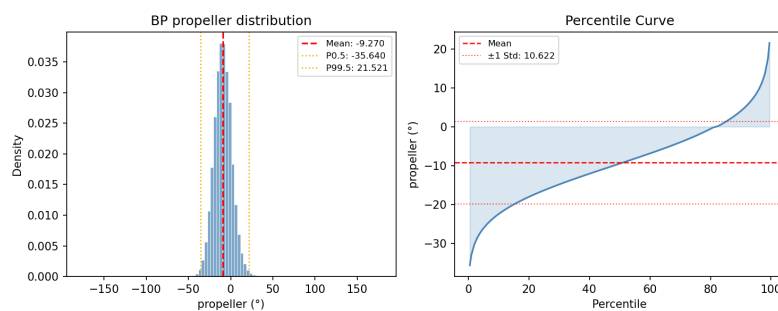


Figure 5.9: Empirical pooled distribution of the propeller BP parameter

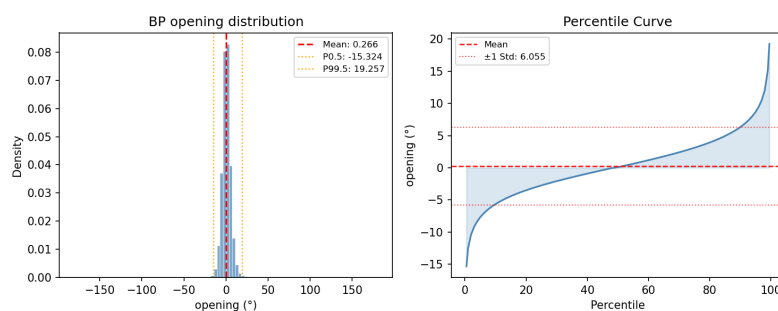


Figure 5.10: Empirical pooled distribution of the opening BP parameter

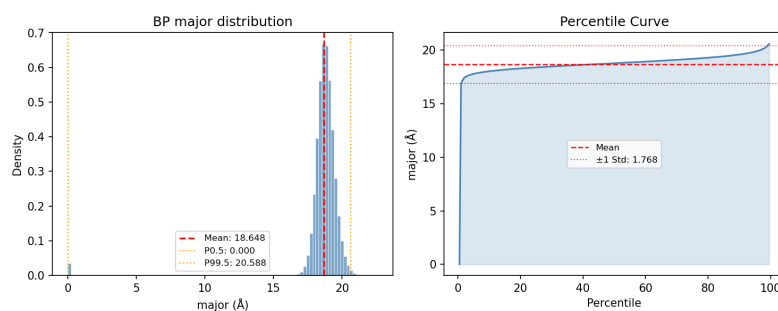


Figure 5.11: Empirical pooled distribution of the major BP parameter

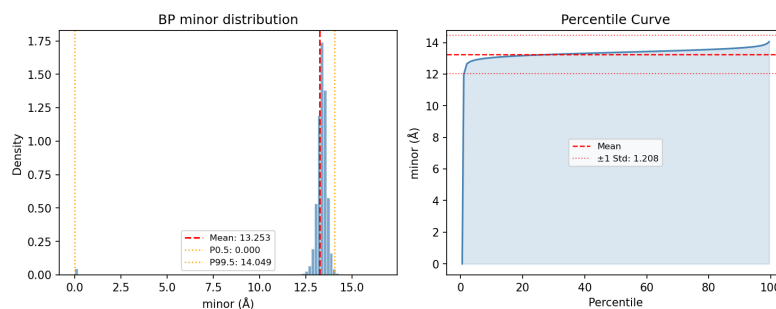


Figure 5.12: Empirical pooled distribution of the minor BP parameter

We observe that all base pair parameters distributions resemble a normal distribution, no matter whether they are linear or circular. The major and minor parameters do of course have a 0 bin that breaks the bell shape, but it is important to remember that this bin only captures frames for which the major/minor parameter could not be computed.

Although it may be tempting to conclude that cross-run mean and dispersion should be enough to capture these distributions, recall that what is presented here is the joint distribution for all systems and thus it is not necessarily the case that the individual distribution of these parameters for each of our pentamers is also normally distributed.

#### 5.3.4.2 Pucker angle

The pooled sample size for sugar pucker angles is 39,907,500 observations. The sample size is larger than for the BP parameter case because there are more individual relevant residues (6 per nucleotide) than base-pairs (3 per nucleotide). The observant reader may note that the pooled sugar pucker angles sample size is still slightly larger than double the sample size for BP parameters (which yields 39,853,222). The reason for this discrepancy is that, as covered in more detail in 5.3.3, there are systems for which a base pair is missing from the extracted parameter time-series.

The pooled percentiles are

$$P_{0.5} = 3.236 \times 10^{-1}, P_1 = 6.342 \times 10^{-1}, P_{99} = 3.593 \times 10^2, P_{99.5} = 3.597 \times 10^2$$

with minimum 0, maximum 360, mean 5.5692 and standard deviation 18.6093.

These values show that the distribution is highly concentrated on the North pucker conformation across all systems. The empirical sugar pucker

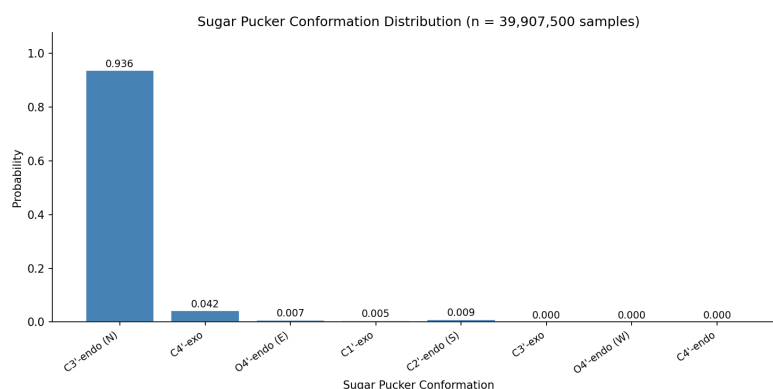


Figure 5.13: Empirical pooled distribution of the sugar pucker angle.

angle pooled distribution across the eight conformational states defined in 4.8 is plotted in Figure 5.13. The observed bias towards the North conformation is probably introduced by our MD simulations and thus does not necessarily reflect the physical pucker angle distribution. North (C3'-endo) and South (C2'-endo) minima are known to be separated by relatively modest interconversion barriers (typically on the order of a few kcal/mol) [71, 72]. It is therefore likely that our simulations, initialized in a North-puckered geometry, remain trapped in this basin on the accessible timescale, so that the sampled distribution reflects the initial conditions and force-field bias rather than the true equilibrium population between North and South conformers.

### 5.3.4.3 RMSD

For the whole-pentamer RMSD, the pooled sample size is 6,651,250 observations (which does exactly match the pooled sample size of the pucker angle divided by 6). The empirical distribution has

$$P_{0.5} = 0.7373, \quad P_1 = 0.7754, \quad P_{99} = 3.262, \quad P_{99.5} = 3.521,$$

with minimum 0.4245, maximum 7.003, mean 1.442 and standard deviation 0.514. Based on these values, we use 10 bins between 0.6 and 3.6 together with two outlier bins. The empirical whole-pentamer RMSD pooled distribution is plotted in Figure 5.14.

For the residue-level RMSD, the pooled sample size is 39,907,500 observations. The empirical distribution has

$$P_{0.5} = 0.1541, \quad P_1 = 0.1655, \quad P_{99} = 1.047, \quad P_{99.5} = 1.183,$$

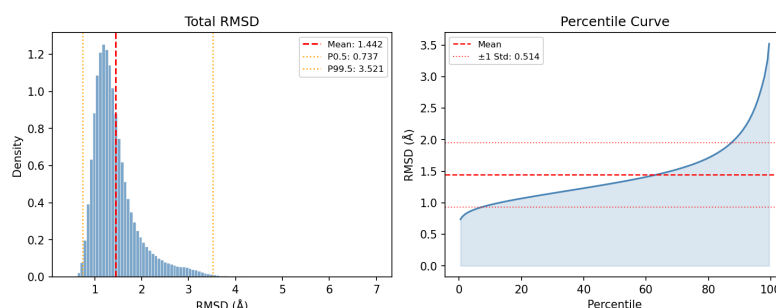


Figure 5.14: Empirical pooled distribution of the whole-pentamer **RMSD**

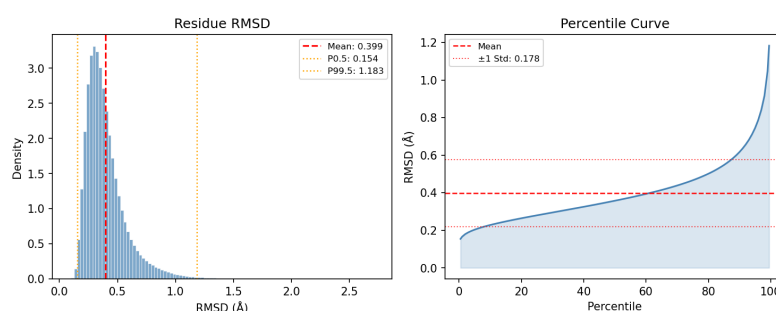


Figure 5.15: Empirical pooled distribution of the per-residue **RMSD**

with minimum 0.0721, maximum 2.7320, mean 0.3986 and standard deviation 0.1784. Here we use 10 bins between 0.1 and 1.2 together with two outlier bins. The empirical per-residue **RMSD** pooled distribution is plotted in Figure 5.15.

Interestingly, both the whole-pentamer and the per-residue **RMSD** empirical pooled distributions show a very similar skewed normal shape. The right-skewness may be explained by the pentamers drifting towards the boundary of the bonded region of the state-space (without necessarily crossing it). The shared shape could be a consequence of the whole-pentamer **RMSD** being directly related to the individual residues **RMSD**, but it is important to know that the optimal SE(3) transformation used to compute the **RMSD** is, in general, different from residue to residue and also different for the whole pentamer. Therefore, it may also be the case that the shared shape is caused by a deeper structural similarity in the **RMSD** observables' behaviour.

#### 5.3.4.4 Bonded, open and dissociated probabilities

After classifying frames into the bonded, open and dissociated regions of state space, we estimated the corresponding occupancy probabilities for each

system and run. Across all sampled instances, the open-state probability has mean 0.005820, standard deviation 0.04884, minimum 0 and maximum 1. The dissociated-state probability has mean  $1.037 \times 10^{-4}$ , standard deviation 0.04884, minimum 0 and maximum 0.7864.

These values indicate that, for the overwhelming majority of systems, the duplex remains in the bonded region for essentially all of the simulation. Open configurations are rare but clearly present in a subset of trajectories, whereas full dissociation is even less frequent.

Our state-space filtering thus removes relatively few frames in most systems, but remains important to avoid contaminating structural histograms with clearly non-duplex configurations and to quantify pentamer instability.

### 5.3.4.5 Interaction energies

The interaction energy between the different considered groups showcases rich, multimodal pooled distributions which are in stark contrast with those of the geometric observables. For this reason, several of them are discretized using 20 central bins rather than 10. The pooled summaries and resulting histogram choices are reported in Table 5.6.

Table 5.6: Distribution summaries and selected histogram ranges for interaction-energy features.

| Interaction groups | P0.5      | P99.5     | Chosen binning                        |
|--------------------|-----------|-----------|---------------------------------------|
| rna-ions           | -315.1603 | 21.8050   | 20 in $[-330, 30]$ + 2 outlier bins   |
| c1                 | -344.8635 | -179.2120 | 20 in $[-350, -170]$ + 2 outlier bins |
| c2                 | -2.6348   | -0.0554   | 10 in $[-3.0, 0.0]$ + 2 outlier bins  |
| d1                 | -43.2692  | 3.7458    | 20 in $[-45.0, 5.0]$ + 2 outlier bins |
| d2                 | -3.5210   | 0.2982    | 10 in $[-3.0, 0.0]$ + 2 outlier bins  |
| bp                 | -98.3706  | -3.5276   | 20 in $[-100, 0]$ + 2 outlier bins    |

The empirical pooled distributions for the interaction energies are plotted in Figures 5.16 (rna-ions), 5.17 (c1), 5.18 (c2), 5.19 (d1), 5.20 (d2), 5.21 (bp).

The two distinct modes in 5.21 were expected, as they correspond to the two types of standard base pairings,  $A-U$  or  $A-T$  and  $C-G$ . The number of trimers with mismatched base pairings is very low (only  $65 - 11 = 54$  trimers with incorrect base pairings were simulated), so we do not expect them to influence this pooled distribution.

The interaction energy distribution between inter-strand consecutive residues is slightly more interesting. The empirical pooled distribution for this observable shows four different modes. One possible explanation would be

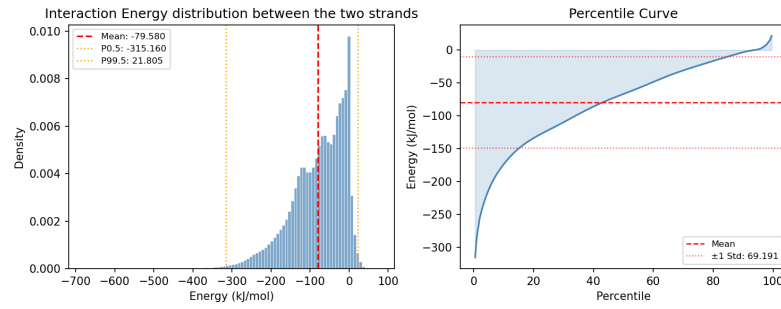


Figure 5.16: Empirical pooled distribution of the interaction energy between the pentamer and the ions in the system

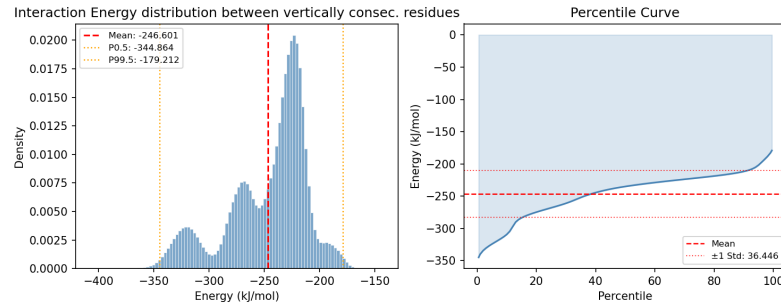


Figure 5.17: Empirical pooled distribution of the interaction energy between same-strand consecutive residue pairs, i.e., residue pairs belonging to  $\mathcal{D}_{c1}$

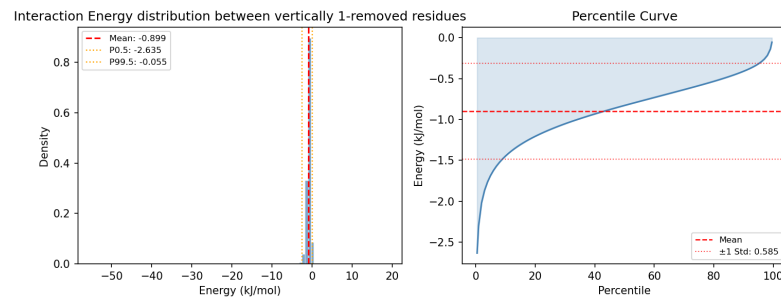


Figure 5.18: Empirical pooled distribution of the interaction energy between same-strand residue pairs that lie two positions away from each other, i.e., residue pairs belonging to  $\mathcal{D}_{c2}$

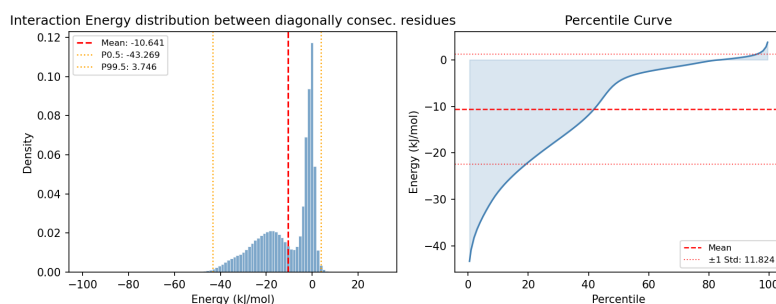


Figure 5.19: Empirical pooled distribution of the interaction energy between inter-strand diagonally consecutive residue pairs, i.e., residue pairs belonging to  $\mathcal{P}_{d1}$

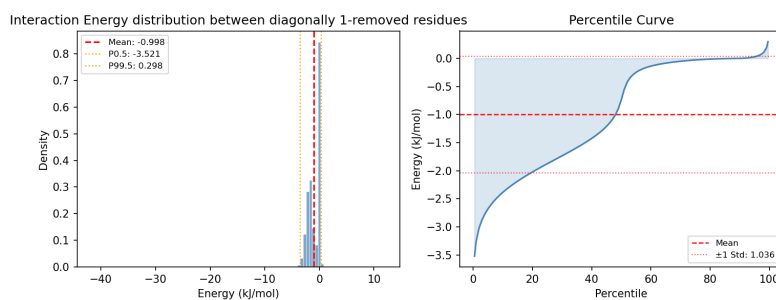


Figure 5.20: Empirical pooled distribution of the interaction energy between inter-strand once diagonally removed residue pairs, i.e., residue pairs belonging to  $\mathcal{P}_{d2}$

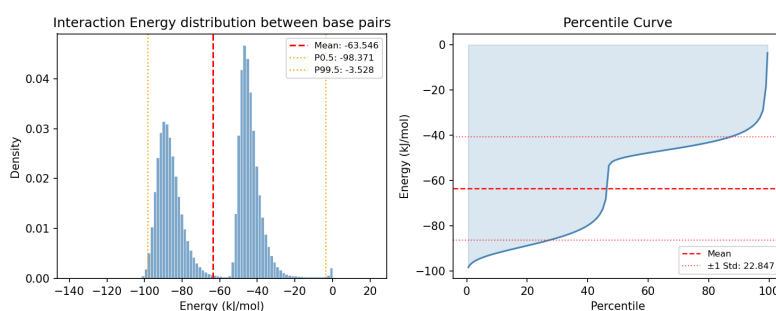


Figure 5.21: Empirical pooled distribution of the interaction energy between base pairs, i.e., residue pairs belonging to  $\mathcal{P}_{bp}$

that those modes are given by the possible consecutive base combinations, but there are actually 10 possible such combinations. It could of course be the case that some of them collapse to the same energy value, but it is also interesting to consider other potential explanations. For example, we know that there are 5 different sugar modifications in the simulated pentamers, whose different consecutive combinations may also play a role in giving rise to the observed modes.

#### 5.3.4.6 Hydrogen bonds

Even though we initially planned to extract different properties of the hydrogen bonds between every type of residue pair, the first stage of the hydrogen bond extraction procedure revealed that the number of hydrogen bonds between residue pairs in the  $\mathcal{D}_{c1}$  and  $\mathcal{D}_{d1}$  groups is frequently 0.

This does not mean that the hydrogen bonds between those residue pairs are irrelevant. However, including the hydrogen bond distance and angle for those residue pairs in every pentamer statistical summary would imply introducing large amounts of missing data into the final machine-learning representation - requiring yet another masking strategy. As a result, we do not include angle or distance summaries for residue pairs in  $\mathcal{D}_{c1}$  and  $\mathcal{D}_{d1}$ : but we do include the summary of the number of hydrogen bonds between them.

The agreement between GROMACS and MDAnalysis in the hydrogen-bond re-evaluation step is excellent. Only 8 hydrogen bonds identified by GROMACS were not later recovered by MDAnalysis. Since the total number of unique hydrogen bonds (i.e., a hydrogen bond present in multiple frames is only counted once) detected by MDAnalysis across all runs in the Swedish cluster is 108,633, this corresponds to an error rate of

$$\frac{8}{108633 + 8} = 7.3637 \times 10^{-5}.$$

This very small discrepancy is consistent with rare edge cases.

We next summarize the pooled hydrogen-bond count distributions for the different residue-pair classes. These results are collected in Table 5.7.

We have already mentioned that the number of hydrogen bonds between **c1** and **d1** residue pairs is generally very small and often 0. This is evidenced by the 0.5 percentile of the hydrogen-bond count pooled distribution being 0 for both classes (see Table 5.7). In fact, the 1 percentile is 0 for both classes too.

Nevertheless, just as established in Table 5.7, we still need upper outlier

Table 5.7: Summary statistics and discretization choices for hydrogen-bond count between different types of residue pairs

| Class        | Samples    | P0.5 | P99.5 | Binning                             |
|--------------|------------|------|-------|-------------------------------------|
| Both strands | 6,651,250  | 6    | 17    | $\{6, \dots, 17\} + 2$ outlier bins |
| <b>c1</b>    | 26,605,000 | 0    | 1     | $\{0, 1\} + 1$ outlier bin          |
| <b>d1</b>    | 26,605,000 | 0    | 1     | $\{0, 1\} + 1$ outlier bin          |
| <b>bp</b>    | 19,953,750 | 0    | 5     | $\{0, \dots, 5\} + 1$ outlier bin   |

bins in the hydrogen-bond count histogram for every pentamer **c1** and **d1** residue pairs because the maximum number of hydrogen bonds observed in a single frame between a **c1** residue pair is 4 and the maximum number of hydrogen bonds observed in a single frame between a **d1** residue pair is 6. A diagonally consecutive residue pair sharing 6 hydrogen bonds in a single sample is likely due to the base pairs having "slid" so that the residue pair that is theoretically classified as diagonally consecutive has begun functioning as a base pair.

The empirical pooled distributions of hydrogen-bond counts are plotted in Figures 5.22a (between both strands), 5.22b (between **c1** residue pairs), 5.22c (between **d1** residue pairs), 5.22d (between base-pairs).

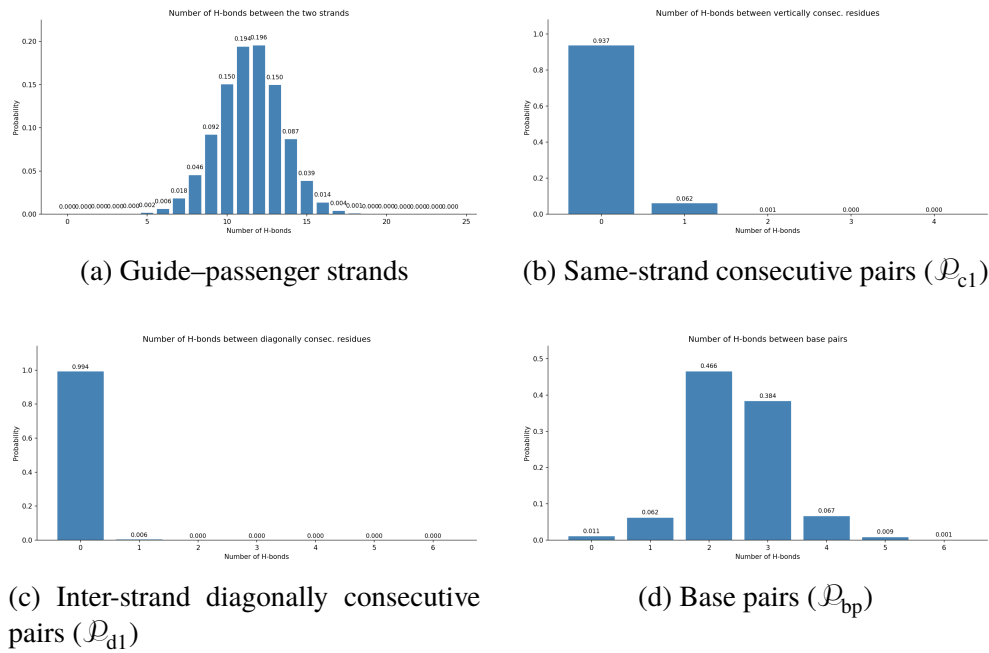


Figure 5.22: Empirical pooled distributions of hydrogen-bond counts for the four residue-pair classes considered in the dataset

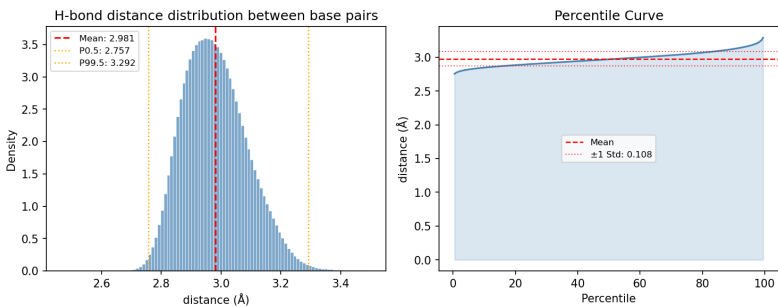


Figure 5.23: Empirical pooled distribution of the donor–acceptor distance in hydrogen bonds between base-pairs

For the geometric observables associated with base-pair hydrogen bonds, the summaries of their empirical pooled distributions are shown in Table 5.8.

Table 5.8: Summary statistics and discretization choices for geometric hydrogen-bond observables in hydrogen bonds between base pairs

| Observable | Samples    | P0.5     | P99.5    | Binning                               |
|------------|------------|----------|----------|---------------------------------------|
| Distance   | 19,759,738 | 2.7572   | 3.2921   | 10 bins in [2.7,3.3] + 2 outlier bins |
| Angle      | 19,759,738 | 139.8630 | 175.7324 | 20 bins in [130,180]                  |

Interestingly, we record hydrogen bonds for which the donor–hydrogen–acceptor angle exactly reaches the imposed lower bound of 130°, and we also record hydrogen bonds for which the donor–acceptor distance also exactly reaches the imposed upper bound of 3.5 Å.

The empirical pooled distribution for donor–acceptor distance in hydrogen bonds between base-pairs is plotted in Figure 5.23 and the empirical pooled distribution for donor–hydrogen–acceptor angle in hydrogen bonds between base-pairs is plotted in Figure 5.24.

Finally, the lifetime-curve results are summarized in Table 5.9.

Table 5.9: Summary of the mean hydrogen-bond lifetime curves across all detected hydrogen bonds within each residue-pair class

| Class      | Curves | First $\tau$ with mean $< 10^{-2}$ | Included in dataset |
|------------|--------|------------------------------------|---------------------|
| c1 pairs   | 40,507 | 3                                  | No                  |
| d1 pairs   | 28,062 | 2                                  | No                  |
| Base pairs | 40,064 | –                                  | Yes                 |

As evidenced by Table 5.9, hydrogen bonds between c1 and d1 residue pairs typically appear only transiently, often disappearing within adjacent

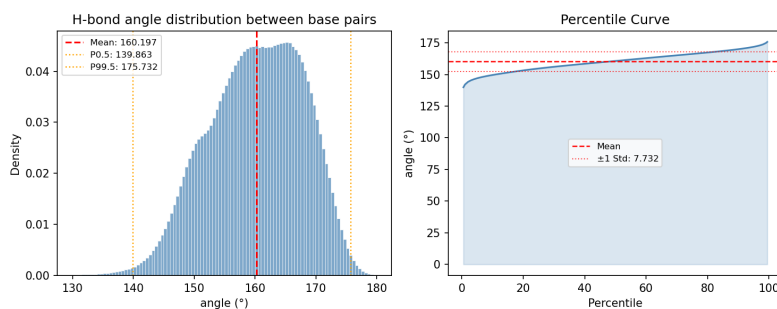


Figure 5.24: Empirical pooled distribution of the donor–hydrogen–acceptor angle in hydrogen bonds between base-pairs

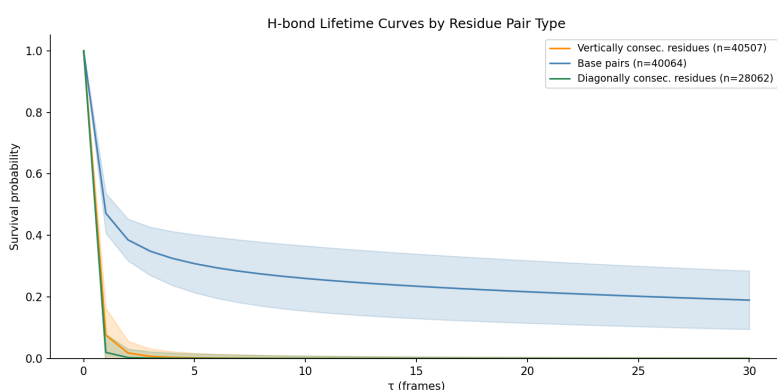


Figure 5.25: Mean and one standard deviation of the hydrogen bond lifetime curves across all hydrogen bonds between residue pairs in  $\mathcal{D}_{bp}$  (blue),  $\mathcal{D}_{c1}$  (orange) and  $\mathcal{D}_{d1}$  (green)

frames. Because this behaviour is essentially universal across systems, we do not include these lifetime curves in the final dataset.

The mean hydrogen bond lifetime curve across all hydrogen bonds between **bp**, **c1** and **d1** residue pairs is plotted in Figure 5.25. Each curve is surrounded by a shaded area which corresponds to one standard deviation from the mean.

### 5.3.5 Errors and excluded systems

The main source of system exclusion at the feature-extraction stage is failure in the computation of base-pair structural descriptors. In particular, systems for which base-pair calculations fail in three or more runs are currently not included in the final dataset. However, there is a small number of trimers with unidentified errors during the feature extraction process.

The excluded systems are:

{61, 801, 3772, 4127, 4337, 4339, 4837, 5027, 5028, 5673, 5715, 5720}

from the US cluster, and

{2980, 2981, 2982, 2987, 3150, 4718, 4895,  
5395, 5453, 5456, 5458, 5459, 5461}

from the Swedish cluster.

## 5.4 ML Results

This section addresses the results of the dataset split construction, feature pre-processing for the implemented ML models on the resulting splits and, finally, model performance evaluation. All Tables in this section comparing several models' performances highlight the best result in **bold**.

### 5.4.1 Dataset split and pre-processing

The final machine-learning dataset contains 88,833 siRNA molecules. A nominal 80/20 partition would therefore correspond to 71,066 molecules in the training set.

The maximum sequence length in the dataset is  $N = 25$ , and therefore we have  $T = N - 2 = 23$ .

We begin by describing the hard and easy dataset splits and the result of the data pre-processing stage on each.

#### 5.4.1.1 Hard data split

The results of the hard data split construction in terms of the predefined validation&test-set categories are summarized in Table 5.10. We emphasize that these categories are not necessarily disjointed, since a given siRNA may simultaneously contain, for example, both an unsupported sugar modification and a trimer from a held-out cluster.

The resulting training split contains 71638 siRNA molecules, the validation split contains 8597 siRNA molecules and the test split contains 8598 siRNA molecules. A total of 5 random trimer clusters and 100 random sequences were used. The IDs of the excluded trimers due to feature-extraction failure are outlined in subsection 5.3.5.

Table 5.10: Predefined categories of **siRNA** molecules forced into the test split. Percentages are reported relative to the full dataset of 88,833 molecules.

| Forced test&validation set category                                   | Count  | Percentage |
|-----------------------------------------------------------------------|--------|------------|
| Contains at least one trimer with failed base-pair feature extraction | 2,311  | 2.60%      |
| Contains at least one held-out sugar modification (MOE or CCC (O) CO) | 12,808 | 14.42%     |
| Belongs to the 100 randomly selected held-out sequences               | 355    | 0.40%      |
| Contains at least one trimer from a selected k-medoids cluster        | 2,020  | 2.27%      |

The held-out trimer clusters are labelled 840, 668, 120, 750 and 606. To assess the degree of separation between the selected clusters and the remaining ones, we examined similarities between their medoids. The results of this procedure are summarized in Figure 5.26. The distribution of similarities between selected and non-selected medoids is shifted toward smaller values relative to the full pairwise medoid-similarity distribution, with mean similarity 0.645. However, complete separation is not achieved: if one computes, for each non-selected medoid, its maximum similarity to any selected medoid, the mean of this quantity is 0.765, and 41 out of 995 non-selected medoids have maximum similarity greater than or equal to 0.9. Thus, the selected clusters define a meaningful but not isolated holdout region. This is consistent with the fact that the global medoid-pair similarity distribution retains non-negligible density above 0.9. Increasing the number of held-out clusters would not fundamentally eliminate this overlap, while it would substantially increase the number of **siRNA** molecules forced into the test split and could reduce the training set below a useful size.

Finally, Figure 5.27 and Table 5.11 lay out the results of the data pre-processing on the computed Hard dataset split.

At the nucleotide level, only 12 **Principal Component Analysis (PCA)** components are required to explain all variance in the nucleotide fingerprints  $f(\text{nuc})$  and only 7 **PCA** components are required to explain all variance in the nucleotide fingerprint differences  $f_d(\text{nuc})$ . Likewise, 11 **PCA** components are required to explain 99% of the variance in the nucleotide RDKit descriptors  $p(\text{nuc})$ .

At the trimer level, 43 components are required to explain 95% of the variance in the fingerprint-based trimer representations  $\tilde{f}_{2,2}(\text{trimer})$  and 27 components are required to explain 95% of the variance in the fingerprint-based difference trimer representations  $\tilde{f}_{d,2,2}(\text{trimer})$ .

Regarding the input **MD** features, 478 **PCA** components are required to explain 95% of the variance.

There are 34 unique nucleotides in the training set of the Hard data split

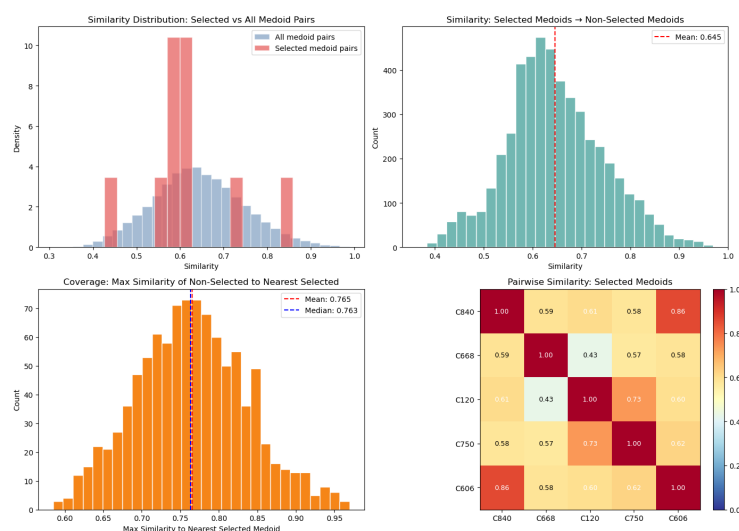


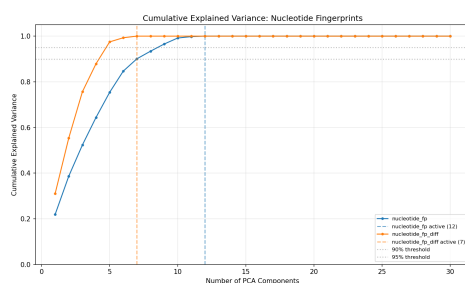
Figure 5.26: Exploration of the similarity between the medoids of the selected trimer clusters (labelled 840, 668, 120, 750 and 606) to be held out from the Hard split training set. Top-left plot illustrates the distribution of pairwise similarities between all medoid pairs and selected-clusters-only medoid pairs. Top-right plot illustrates the distribution of pairwise similarities between medoid pairs where one is the medoid of a selected cluster and the other is the medoid of an unselected cluster. Bottom-left plot illustrates the distribution of the coverage, defined as the pairwise similarity between the medoid of an unselected cluster and its closest (i.e., most similar) medoid of a selected cluster. Bottom-right plot illustrates the selected clusters medoids pair-wise similarity matrix

Table 5.11: Summary of the PCA dimensionality reduction results on the Hard training split.

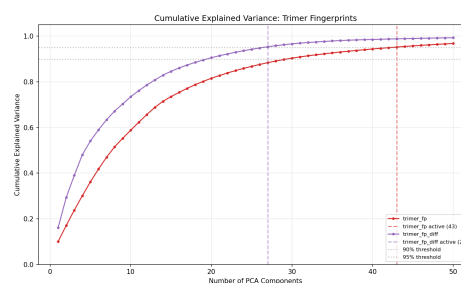
| Representation                                                                   | Target variance | Components required |
|----------------------------------------------------------------------------------|-----------------|---------------------|
| Nucleotide fingerprints $f(\text{nuc})$                                          | 100%            | 12                  |
| Nucleotide fingerprint differences $f_d(\text{nuc})$                             | 100%            | 7                   |
| Nucleotide RDKit descriptors $p(\text{nuc})$                                     | 99%             | 11                  |
| Trimer fingerprint-based representations $\tilde{f}_{2,2}(\text{trimer})$        | 95%             | 43                  |
| Trimer fingerprint-difference representations $\tilde{f}_{d,2,2}(\text{trimer})$ | 95%             | 27                  |
| MD input features                                                                | 95%             | 478                 |

out of 41 total unique nucleotides, which together with the shared structural similarity among nucleotide graphs explains the relatively low number of **PCA** components required to explain all variance in their fingerprints.

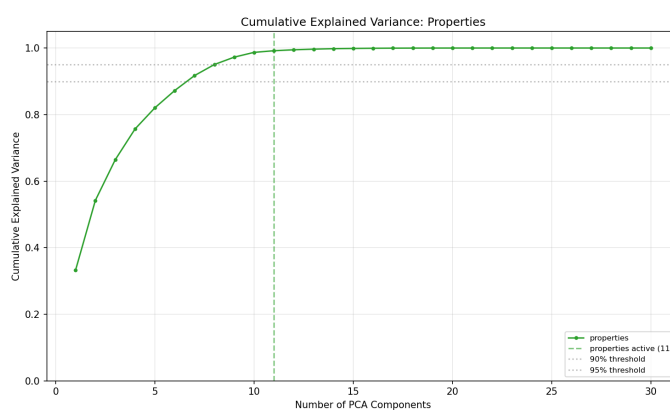
There are 7914 unique double trimers (including termini) in the training dataset out of 8787 total unique trimers. The larger number of unique trimers present in the training dataset together with the larger complexity of the fingerprint-based trimer representation comfortably explain why a larger number of **PCA** components would be needed to capture 95% of their variance.



(a) Explained variance as a function of the number of retained PCA components for nucleotide fingerprints and nucleotide fingerprint differences.



(b) Explained variance as a function of the number of retained PCA components for trimer fingerprint-based representations and trimer fingerprint-difference representations.



(c) Explained variance as a function of the number of retained PCA components for nucleotide RDKit descriptors.

Figure 5.27: Explained variance curves for the PCA-based preprocessing steps used in the chemical feature construction pipeline for the Hard data split. Top left: nucleotide fingerprints and nucleotide fingerprint differences. Top right: trimer fingerprint-based representations and trimer fingerprint-difference representations. Bottom: nucleotide RDKit descriptors.

### 5.4.1.2 Easy data split

For the easy data split, a new feature table is recomputed, this time excluding the base pair parameters in order to maximize the amount of trimers included in it.

However, there are a number of trimers for which feature table generation failed, even with no base pair features. Their IDs are 2982, 2987, 3150, 4718, 61, 801, 4127. This represents a much smaller number of failed systems than that of the original feature table 5.3.5. Unfortunately, the failure cause for these trimers is unclear and seemingly heterogeneous.

It is important to note that these trimers for which feature table generation failed are not specifically excluded from the training split, contrary to the hard data split case.

We choose 200 random sequence clusters, excluding all siRNA molecules with sequences belonging to one of the selected clusters from the training set. This exclusion yields a training set size of 71032 siRNA molecules, with 8900 siRNA molecules assigned to the validation set and 8901 siRNA molecules assigned to the test set.

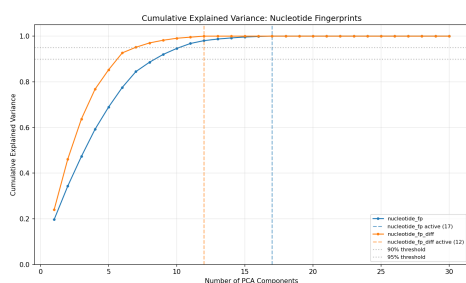
Finally, Figure 5.28 and Table 5.12 lay out the results of the data pre-processing on the computed Hard dataset split.

Table 5.12: Summary of the PCA dimensionality reduction results on the Hard training split.

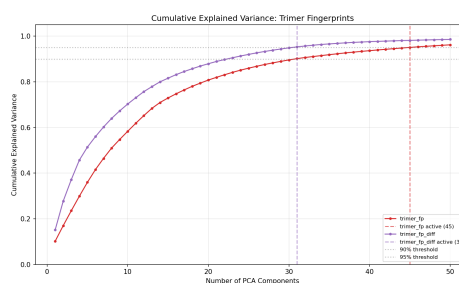
| Representation                                                                   | Target variance | Components required |
|----------------------------------------------------------------------------------|-----------------|---------------------|
| Nucleotide fingerprints $f(\text{nuc})$                                          | 100%            | 17                  |
| Nucleotide fingerprint differences $f_d(\text{nuc})$                             | 100%            | 12                  |
| Nucleotide RDKit descriptors $p(\text{nuc})$                                     | 99%             | 13                  |
| Trimer fingerprint-based representations $\tilde{f}_{2,2}(\text{trimer})$        | 95%             | 45                  |
| Trimer fingerprint-difference representations $\tilde{f}_{d,2,2}(\text{trimer})$ | 95%             | 31                  |
| MD input features                                                                | 95%             | 340                 |

The fact that a larger number of PCA components is required to achieve the same target variance than in the Hard data split for chemical features is hardly surprising if we note that all 41 unique nucleotides and 8112 out of 8787 unique double-stranded trimers are now present in the training set of the Easy data split.

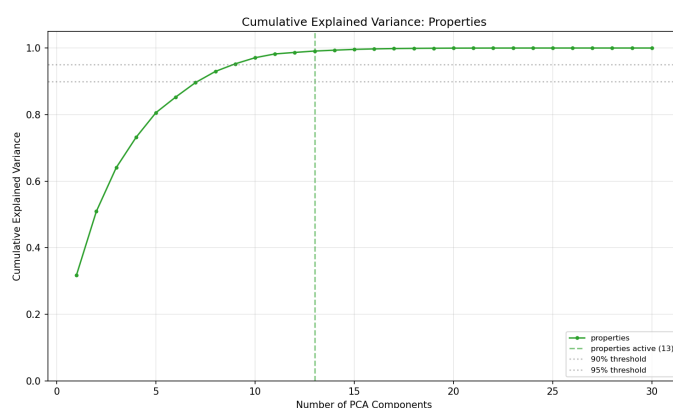
On the opposite, a significantly smaller number of **PCA** components is required to explain 95% of the trimer **MD** features variance because we have excluded all base pair features, yielding a considerably smaller total **MD** dimension.



(a) Explained variance as a function of the number of retained PCA components for nucleotide fingerprints and nucleotide fingerprint differences.



(b) Explained variance as a function of the number of retained PCA components for trimer fingerprint-based representations and trimer fingerprint-difference representations.



(c) Explained variance as a function of the number of retained PCA components for nucleotide RDKit descriptors.

Figure 5.28: Explained variance curves for the PCA-based preprocessing steps used in the chemical feature construction pipeline for the Hard data split. Top left: nucleotide fingerprints and nucleotide fingerprint differences. Top right: trimer fingerprint-based representations and trimer fingerprint-difference representations. Bottom: nucleotide RDKit descriptors.

### 5.4.2 Cm-siRPred Baseline Evaluation on the Hard Data Split

Table 5.13 reports the validation-set MAE obtained by the three tested Cm-siRPred configurations. Due to the limited performance the architecture reported in the original publication, we also evaluate a longer-training variant and a reduced-capacity variant.

Table 5.13: Validation-set MAE for the tested Cm-siRPred configurations.

| Config. | Architecture                                                                                                                                                         | Training hyper-parameters                                                                                                                              | # Params   | Val. MAE      |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------|---------------|
| C1      | $d_{\text{model}} = 512$ ; cross-attention with 4 heads and dropout 0.0; CNN channels $3 \rightarrow 16 \rightarrow 32$ ; classifier hidden dim. 128 and dropout 0.5 | batch size 128; max epochs 1500; learning rate $10^{-3}$ ; weight decay $10^{-4}$ ; warmup epochs 5; patience 200; gradient clip 1.0; random state 42  | 11,050,209 | <b>0.2285</b> |
| C2      | $d_{\text{model}} = 512$ ; cross-attention with 4 heads and dropout 0.0; CNN channels $3 \rightarrow 16 \rightarrow 32$ ; classifier hidden dim. 128 and dropout 0.5 | batch size 128; max epochs 5000; learning rate $10^{-3}$ ; weight decay $10^{-4}$ ; warmup epochs 5; patience 750; gradient clip 1.0; random state 42  | 11,050,209 | 0.2300        |
| C3      | $d_{\text{model}} = 256$ ; cross-attention with 4 heads and dropout 0.1; CNN channels $3 \rightarrow 8 \rightarrow 16$ ; classifier hidden dim. 128 and dropout 0.1  | batch size 128; max epochs 1500; learning rate $10^{-3}$ ; weight decay $10^{-4}$ ; warm-up epochs 5; patience 200; gradient clip 1.0; random state 42 | 4,350,945  | 0.2318        |

The Cm-siRPred architecture struggles to generalize across the validation split. This observation may easily be explained by its large size: the original model has 11,050,209 trainable parameters, which correspond to a ratio of more than 100 learnable parameters per dataset entry. A larger patience value does not seem to help.

### 5.4.3 XGBoost Baseline Evaluation on the Hard Data Split

The XGBoost baselines were evaluated in order to provide non-neural reference models based on the fixed tabular representations described in the methods section. We report the validation-set MAE for the nucleotide-level and trimer-level configurations, as well as for the auxiliary trimer-level models used to assess the predictive contribution of the MD-derived descriptors.

Table 5.14 reports the validation-set MAE obtained by the two nucleotide-level XGBoost configurations. The proposed nucleotide-level XGBoost

Table 5.14: Validation-set MAE for the nucleotide-level XGBoost baselines.

| Configuration | Val. MAE      | Notes                  |
|---------------|---------------|------------------------|
| N1            | 0.1923        | Chemical features only |
| N2            | <b>0.1917</b> | Chemical features only |

models are able to fit the training set successfully, but struggle to generalize across the validation split. Nevertheless, their performance comfortably improves upon the Cm-siRPred, highlighting the capabilities of seemingly simpler, non-neural models.

The normalised feature importance for each of the elements fed to the XGBoost models is plotted in Figure 5.29a. Both nucleotide-level models assign the most importance to nucleotide fingerprints and RDKit properties/descriptors. This is hardly a surprising observation, as these features encode the chemistry of the siRNA molecule’s nucleotides, which are its building blocks.

Perhaps more interestingly, we observe that all global features receive the least importance. However, it is important to stress that this does not mean that global features do not carry an informative signal. It must be remembered that these features constitute a significantly smaller part of the input vector for each siRNA molecule. If we were to consider the normalised per nucleotide passenger fingerprint importance (obtained by dividing the normalised passenger strand nucleotides fingerprint importance by 25) we would obtain  $0.1840/25 = 0.00736$ , which is significantly lower than the importance given to global properties such as the cell-type (0.021 normalised importance) or the employed detection method (0.0032 normalised importance). Meanwhile, the whole set of passenger fingerprints receive the largest importance out of all features, with a normalised value of 0.1840.

It is therefore interesting to look at the per-nucleotide importance of the nucleotide-level features. For example, Figure 5.29b illustrates the per-nucleotide importance of the fingerprint-difference representation in the guide strand. We can observe that all fingerprint-differences receive similar importances, except for the fingerprint-difference of the 25th nucleotide in the guide strand, whose importance is 0. This must mean that the 25th position of the guide strand is always occupied by an unmodified nucleotide (when it exists), thus always yielding a 0-valued vector as its (PCAd) fingerprint difference.

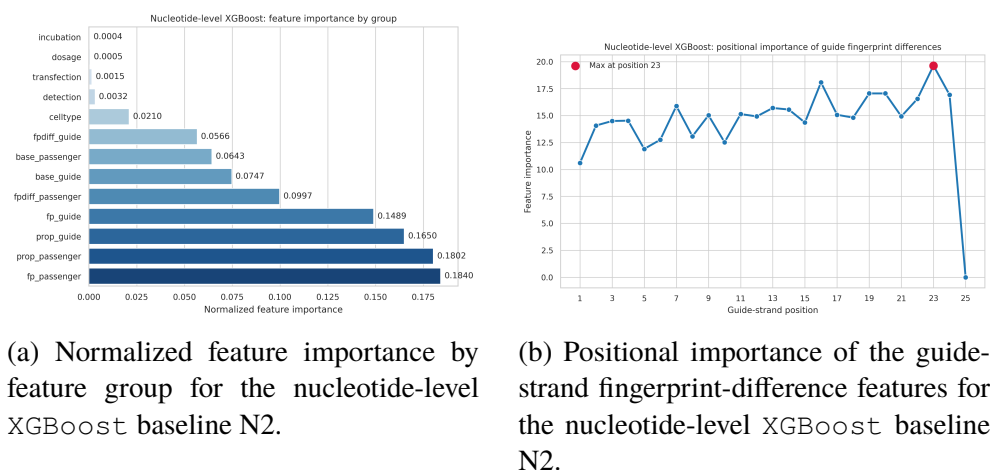


Figure 5.29: Feature-importance analysis for the nucleotide-level XGBoost baseline N2. Left: normalized importance aggregated by feature group. Right: positional importance of the guide-strand fingerprint-difference features.

Table 5.15 reports the validation-set MAE for the trimer-level XGBoost baselines using only the chemical feature representation. The trimer-level

Table 5.15: Validation-set MAE for the trimer-level chemical-only XGBoost baselines.

| Configuration | Val. MAE      | Notes                  |
|---------------|---------------|------------------------|
| T1            | 0.1917        | Chemical features only |
| T2            | <b>0.1914</b> | Chemical features only |

baseline XGBoost model immediately matches the performance of the largest and best performing nucleotide-level XGBoost model, N2. The largest trimer-level XGBoost model, T2, is able to bring some improvement, although it is very limited. T2 is able to achieve a training loss of 0.0334 but fails to bring significant improvement over N2 and T1, which seems to hint at a fundamental limitation of XGBoost models given the available data.

Figure 5.30a illustrates the normalised importance given to the input features in the trimer level XGBoost model T2. The most striking observation is that there is now a single feature, the trimer fingerprint based representation  $\tilde{f}_{2,2}$ , which dominates the importance ranking.

Nevertheless, one must note that the fingerprint-based features are no longer separated in guide strand and passenger strand contributions. In fact, if one adds the normalised importance of the guide and passenger fingerprints in the N2 model (see Figure 5.29a), the resulting normalised, combined

nucleotide fingerprint importance is  $0.1840 + 0.1489 = 0.3329$  - which sits significantly close to the 0.3952 feature importance attained by the trimer fingerprint-based representations in the T2 model.

Further, if we once again divide the trimer fingerprint-based representation normalised importance by 23 to obtain the normalised per trimer fingerprint-based representation importance, the resulting value is approximately 0.01718. This is only slightly larger than the normalised importance of the highest-rated global feature, the cell-type, which receives a normalised importance of 0.0123.

The most interesting observation regarding feature importance in the T2 model is illustrated in Figure 5.30b. This plot represents the feature importance assigned to the trimer fingerprint-based representation as a function of the trimer position. This importance follows an increasing trend which rapidly slows down from positions 1 to 19, and then sees one final explosion - assigning the maximum importance to the very last trimer in the chain, which is approximately 2.5 times more important than the first trimer in the chain. This is a significant difference in assigned importance which may harm our MD-informed approach. If the neural models agree on assigning the most importance to the last trimer in the chain, the MD features will not be available to improve the representation of the most important element of the trimer-level chain - as the last trimer contains termini and thus has not been simulated.

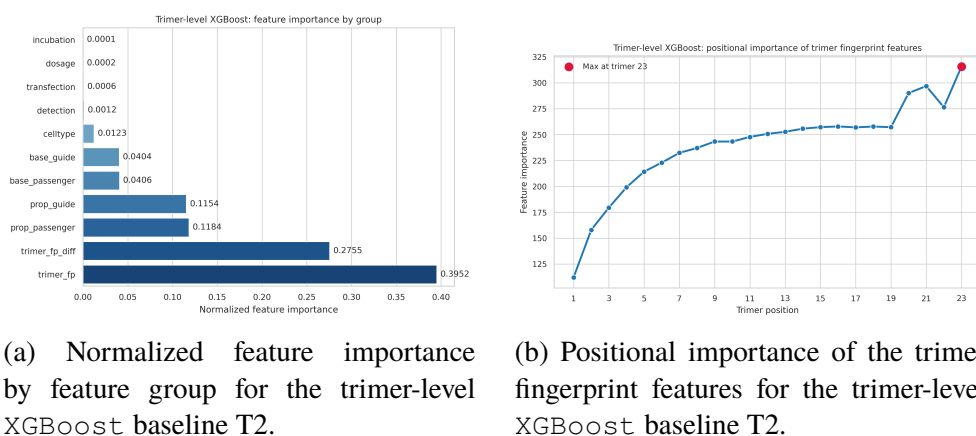


Figure 5.30: Feature-importance analysis for the trimer-level XGBoost baseline T2. Left: normalized importance aggregated by feature group. Right: positional importance of the trimer fingerprint features.

Table 5.16 summarizes the auxiliary trimer-level XGBoost experiments

used to assess whether the addition of MD-derived descriptors improves over the corresponding chemical-only representation. Although both T3 and T4

Table 5.16: Validation-set MAE for the trimer-level XGBoost models used in the MD-feature importance study.

| Configuration | Val. MAE      | Features                                                  |
|---------------|---------------|-----------------------------------------------------------|
| T3            | 0.1925        | Chemical features only                                    |
| T4            | <b>0.1918</b> | Chemical features + cross-run means of all MD observables |

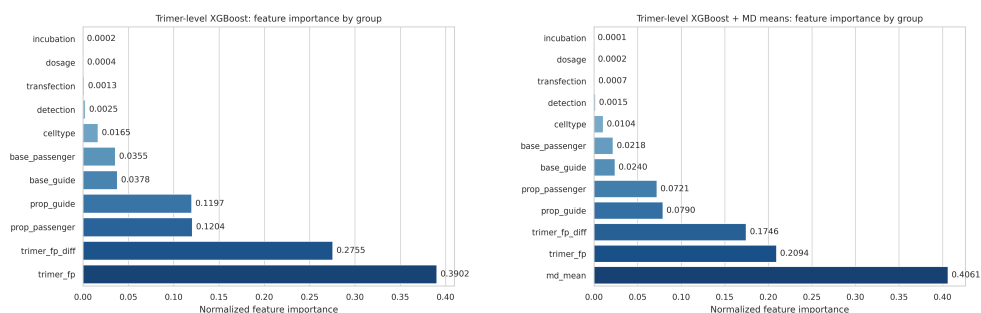
are worse performers than T1 and T2, what these models aim to evaluate is the importance given to MD-derived features by an unsaturated XGBoost model.

Therefore, it is positive to see that the performance improves from T3 to T4 upon including the cross-run means extracted from the MD simulations. Further, it is interesting to see the model transition from assigning the largest importance to the fingerprint based trimer representations in T3 (see Figure 5.31a) - exactly as it did for model T2 - to assigning the largest importance to the supplied cross-run means of the extracted MD observables (see Figure 5.31b).

It is important to bear in mind that the input MD means have a total dimension of 79, and that if we divide by 23 to obtain the per-trimer MD means importance we obtain a normalised importance of 0.01766. This value is once again comparable to that of the most important global feature, the cell-type, which is assigned an importance of 0.0104.

Nevertheless, the fact that the model automatically assigns the largest normalised importance to the provided MD means (even when those trimers with missing MD features simply are assigned the median value) is a strong indicator of the predictive power of the computed MD features.

We also looked at the average importance of each provided cross-run mean across the 23 trimers in a chain and found that these values are all contained within the interval [0.012, 0.0145]. Therefore, there are no clearly uninformative features which may be discarded right away.



(a) Normalized feature importance by feature group for the trimer-level XGBoost baseline T3, using only chemical descriptors.

(b) Normalized feature importance by feature group for the trimer-level XGBoost baseline T4, augmented with cross-run means of the extracted MD observables.

Figure 5.31: Comparison of normalized feature-group importances for the trimer-level XGBoost baselines. Left: chemical-only model (T3). Right: model augmented with cross-run means of the extracted MD observables (T4).

#### 5.4.4 Transformer-based Hierarchical Models Evaluation on the Hard Data Split

The exploration of transformer-based hierarchical models was carried out in three stages, following the procedure described in Paragraph 4.5.4.4.1. In all cases, model selection was based on the validation-set mean absolute error (MAE). To facilitate comparison between predictive performance and model complexity, we also report the number of trainable parameters of each model.

Table 5.17 reports the validation-set MAE obtained by the initial transformer backbone exploration, comparing nucleotide-to-sequence and trimer-to-sequence architectures across the tested token dimensions and transformer hyper-parameters. As evidenced by the results in Table 5.17, transformers can beat the XGBoost baseline, but they struggle to do so. In particular, no nucleotide to sequence transformer model was able to outperform the XGBoost baseline, and all trimer to sequence transformer models were noticeably superior to the nucleotide to sequence variants.

There are several possible explanations to this observation. First, trimer-to-sequence models operate with double-stranded trimers as their input unit; while nucleotide-to-sequence models operate with individual nucleotides as their input unit. In the latter, the nucleotides in both strands are just concatenated and represented as a single sequence. While the transformer model should theoretically be able to capture relevant nucleotide interactions,

Table 5.17: Validation-set MAE for the initial exploration of transformer backbones. Nucleotide-to-sequence models with  $d_{nuc} = 64$  use a one-layer nucleotide encoder with hidden dimension 64, while nucleotide-to-sequence models with  $d_{nuc} = 32$  simply implement a projection layer from the input  $d_{nuc,i} = 37$ . Trimer-to-sequence models use the default trimer token representation with  $d_{trimer} = 128$

| Model type      | Token dim. | FF dim. | Heads | Layers | # Params | Val. MAE      |
|-----------------|------------|---------|-------|--------|----------|---------------|
| Nuc.→Sequence   | 32         | 512     | 8     | 4      | 170,561  | 0.2021        |
| Nuc.→Sequence   | 32         | 512     | 4     | 2      | 95,233   | 0.2117        |
| Nuc.→Sequence   | 32         | 256     | 8     | 4      | 104,001  | 0.2081        |
| Nuc.→Sequence   | 32         | 256     | 4     | 2      | 61,953   | 0.2169        |
| Nuc.→Sequence   | 32         | 256     | 8     | 2      | 61,953   | 0.2093        |
| Nuc.→Sequence   | 64         | 512     | 8     | 4      | 361,665  | 0.2097        |
| Nuc.→Sequence   | 64         | 512     | 4     | 2      | 195,649  | 0.2052        |
| Nuc.→Sequence   | 64         | 256     | 8     | 4      | 229,569  | 0.2086        |
| Nuc.→Sequence   | 64         | 256     | 4     | 2      | 129,601  | 0.2029        |
| Nuc.→Sequence   | 64         | 256     | 8     | 2      | 129,601  | 0.2112        |
| Trimer→Sequence | 128        | 256     | 4     | 2      | 319,233  | 0.1894        |
| Trimer→Sequence | 128        | 256     | 8     | 2      | 319,233  | <b>0.1889</b> |
| Trimer→Sequence | 128        | 256     | 8     | 4      | 584,193  | 0.1900        |
| Trimer→Sequence | 128        | 512     | 4     | 2      | 450,817  | 0.1900        |
| Trimer→Sequence | 128        | 512     | 8     | 4      | 847,361  | 0.1926        |

it may struggle to do so due to the lack of structural inductive bias and the smaller size of the nucleotide-level transformers.

The lower learnable parameter count of the nucleotide-level transformers is precisely the second possible explanation to their underperformance. In fact, two of the evaluated nucleotide-to-sequence transformers have less learnable parameters than our dataset has entries. The smaller size is not only reflected on their performance on the validation split but also on their capacity to fit the training dataset, with the best performing nucleotide-to-sequence model only achieving a MAE of 0.1216 on the training split before coming to a halt. By contrast, the best performing trimer-to-sequence model had achieved a MAE of 0.0798 on the training split before stopping. However, when we extended the nucleotide-to-sequence model training by assigning a larger patience parameter of 500 it was not able to achieve further improvement.

To test whether these hypothesis are able to explain the nucleotide-to-sequence transformer models underperformance, we design a new evaluation round in which we test:

1. A larger nucleotide token size  $d_{nuc} = 128$ , to see if increasing model size can yield improved results.

2. A new nucleotide-to-sequence transformer model design which operates on nucleotide base-pairs, obtaining each nucleotide base-pair input token by concatenating the representations of both nucleotides in the base-pair. This model’s objective is to see whether a stronger structural inductive bias with nucleotide base-pairs as the input unit can match the performance of models using double-stranded trimers as input unit.

Table 5.18 illustrates the results of this new evaluation round. The transformer architectures tested correspond to those of the best performing nucleotide-to-sequence transformer with  $d_{\text{nuc}} = 64$  (4 heads, 2 layers, feed-forward dimension 256) and the best performing trimer-to-sequence transformer (8 heads, 2 layers, feed-forward dimension 256). The results of this extended

Table 5.18: Validation-set **MAE** for the further exploration of nucleotide-to-sequence and nucleotide base-pair-to-sequence transformer architectures. Nucleotide-to-sequence models all have  $d_{\text{nuc}} = 128$  and use a one-layer nucleotide encoder with hidden dimension 64. Nucleotide base-pair-to-sequence models with  $d_{\text{nuc,bp}} = 64$  implement a projection layer from the input  $d_{\text{nuc,i}} = 37$  to  $d_{\text{nuc}}$ ; while nucleotide base-pair-to-sequence models with  $d_{\text{nuc,bp}} = 128$  use a one-layer nucleotide encoder with hidden dimension 64 and  $d_{\text{nuc}} = 64$ .

| Input Token     | Token dim. | FF dim. | Heads | Layers | # Params | Val. MAE      |
|-----------------|------------|---------|-------|--------|----------|---------------|
| Individual nuc. | 128        | 256     | 8     | 2      | 307,201  | 0.2086        |
| Individual nuc. | 128        | 256     | 4     | 2      | 307,201  | 0.2134        |
| Nuc. base-pair  | 64         | 256     | 8     | 2      | 124,033  | 0.2055        |
| Nuc. base-pair  | 64         | 256     | 4     | 2      | 124,033  | 0.2037        |
| Nuc. base-pair  | 128        | 256     | 8     | 2      | 302,913  | 0.2037        |
| Nuc. base-pair  | 128        | 256     | 4     | 2      | 302,913  | <b>0.2020</b> |

evaluation round indicate that, while model size and the choice of nucleotide-level inductive bias may play a role in overall performance, they are insufficient to explain the performance gap between nucleotide-level and trimer-level models.

This naturally suggests a third possible explanation to nucleotide-level models underperformance when compared to trimer-level models: double-stranded trimers may be a better information/input unit for our models, i.e., trimers constitute a better inductive bias to extract sequence-level representations. This claim will be strengthened if we are to observe the same results for **CNN**-based models. Nevertheless, it must also be stressed out that there are other possible nucleotide-level architectures that have not been considered due to time limitations, such as the Edge Set Transformer [73].

From now on, we focus on trimer-to-sequence transformer models. The best performing trimer-to-sequence models correlate with the lowest amounts of learnable parameters, which is reasonable given that our dataset has less than 100k samples and there were no signs of grokking on the Cm-siRPred runs.

This seems to suggest that, for our dataset, larger models are at high-risk of over-fitting on the training partition; which highlights the importance of capturing the correct inductive bias in order to achieve strong performance.

After selecting the best-performing transformer backbone under mean pooling, Table 5.19 compares the alternative pooling strategies considered for that architecture. Both CLS and Max pooling beat the mean pooling

Table 5.19: Validation-set MAE for different pooling strategies applied to the best-performing transformer backbone from Table 5.17.

| Pooling strategy | # Params | Val. MAE      |
|------------------|----------|---------------|
| Mean             | 319,233  | 0.1889        |
| CLS              | 319,489  | 0.1880        |
| Attention        | 319,362  | 0.1906        |
| Max              | 319,233  | <b>0.1857</b> |

baseline. As CLS pooling adds an extra token which represents the sequence as a whole with a very small parameter overhead, it is not surprising to see it edge past mean pooling. However, Max pooling performance achieves a significantly better result with no added learnable parameters, which makes it a very convincing choice for the subsequent models.

Thus, for the next steps, we set a trimer to sequence transformer encoder with 8 heads, 2 layers, feed-forward dimension 256 and Max pooling. Note that this model’s performance clearly surpasses that of the best baselines.

Having fixed the trimer-to-sequence transformer backbone and pooling strategy, Table 5.20 evaluates whether a learned trimer representation can outperform, or complement, the pre-computed trimer chemical descriptors. The results are negative: the best performing trimer to sequence transformer with pre-processed trimer input tokens (validation MAE 0.1857) performs significantly better than the best trimer to sequence transformer with learned trimer tokens (validation MAE 0.1885). In fact, the latter performs worse than, for example, the trimer to sequence transformer with pre-processed trimer input tokens and the sub-optimal CLS attention pooling (validation MAE 0.1880).

There are two possible explanations to this observation:

Table 5.20: Validation-set MAE for transformer-based trimer models using learned and hybrid trimer encoders. All models inherit the best-performing trimer-to-sequence transformer backbone and pooling strategy selected in Stage 1; corresponding to 8 heads, 2 layers, feed-forward dimension 256 and Max pooling. The nucleotide encoder is a one-layer **MLP** with hidden dimension 64 and output dimension  $d_{\text{nuc}} = 32$ .

| Trimer encoder          | Nucleotide→trimer encoder                  | $d_{\text{trimer,learned}}$ | # Params | Val. MAE      |
|-------------------------|--------------------------------------------|-----------------------------|----------|---------------|
| Learned only            | 1-layer <b>MLP</b> , hidden dim. 256       | 128                         | 383,841  | 0.1895        |
| Learned only            | 2-layer <b>MLP</b> , hidden dims. 256, 128 | 128                         | 451,905  | <b>0.1885</b> |
| Hybrid chemical/learned | 1-layer <b>MLP</b> , hidden dim. 256       | 128                         | 409,569  | 0.1887        |
| Hybrid chemical/learned | 2-layer <b>MLP</b> , hidden dims. 256, 128 | 128                         | 426,337  | 0.1890        |

1. It could be the case that our chosen nucleotide to trimer encoder  $p_{1,2}^{\text{MLP}}$  is not able to effectively merge the input nucleotide tokens into a single learned trimer token. The chosen **MLP**-based encoder does not impose an underlying structure, and thus lacks inductive bias.

Having already established that - in our setting - a larger learnable parameter count harms generalization, it would not be unreasonable to think that the nucleotide to trimer **MLP** is simply unable to learn a transformation from the concatenated features of the trimer nucleotides to an appropriate trimer representation.

This explanation does, however, seem rather unlikely. 6 nucleotides with  $d_{\text{nuc},i} = 37$  each yield a manageable input dimension, so the lack of inductive bias should not have this large of an impact.

An interesting test for this hypothesis is to look at the behaviour of the proposed **CNN**-based nucleotide-to-trimer encoder, whose results are outlined in the **CNN**-based models subsection 5.4.5. As this encoder imposes an underlying  $2 \times 3$  array structure on the trimer, we should be able to observe whether a stronger inductive bias improves the performance of models with a learned nucleotide to trimer encoder.

2. The second possible explanation is that the obtained trimer input tokens through the  $t_i$  function (4.55) is already a significantly good description of the input trimer. This is a more defensible explanation because the pre-processed trimer input tokens already include the PCAd

trimer and composing nucleotides fingerprints as well as the composing nucleotides one-hot base encodings and PCAd RDKit descriptors.

Therefore, if we are to discard the first possible explanation to the underperformance of models with learned trimer representations (through supervised learning, that is), the most viable route for further improvement seems to be using an atom-level graph based **ML** method to obtain nucleotide and trimer representations which complement those computed in our pre-processing stage.

Next, Table 5.21 compares the **MD**-aware trimer encoder variants when cross-run means and standard deviations are used as the trimer-level **MD** representation. **MD** features dropout is initially set to 0.1 and the best performing trimer to sequence transformer architecture is used (8 heads, 2 layers, feed-forward dimension 256 and Max pooling). The failure of the

Table 5.21: Validation-set MAE for **MD**-aware transformer-based trimer encoders using cross-run means and standard deviations of all extracted **MD** observables. All models inherit the best-performing transformer backbone selected in the previous stages; corresponding to 8 heads, 2 layers, feed-forward dimension 256 and Max pooling. The initial **MD** dropout is fixed to 0.1.

| <b>MD encoder</b>          |           | <b>Architecture details</b>                                                                                         | <b># Params</b> | <b>Val. MAE</b> |
|----------------------------|-----------|---------------------------------------------------------------------------------------------------------------------|-----------------|-----------------|
| Additive <b>MD</b> encoder |           | Chemical <b>MLP</b> : 1 layer, hidden dim. 256; <b>MD MLP</b> : 1 layer, hidden dim. 256                            | 449,921         | 0.1915          |
| Concatenation encoder      | <b>MD</b> | Concatenated-input <b>MLP</b> : 1 layer, hidden dim. 256; no prior <b>MD MLP</b>                                    | 416,257         | <b>0.1846</b>   |
| Concatenation encoder      | <b>MD</b> | Concatenated-input <b>MLP</b> : 2 layers, hidden dims. 256, 128; no prior <b>MD MLP</b>                             | 433,025         | 0.1893          |
| Concatenation encoder      | <b>MD</b> | <b>MD MLP</b> : 1 layer, hidden dim. 128, output dim. 128; concatenated-input <b>MLP</b> : 1 layer, hidden dim. 256 | 445,953         | 0.1914          |

additive **MD** encoder is relatively unsurprising. Not only does it have the largest amount of parameters, but it also must add together two vectors that live in completely different spaces, one being the space of trimer chemical representations and the other being the space of trimer **MD**-feature based representations.

Regarding the concatenation MD encoder, the most positive news are that it achieves a noticeable performance increase with respect to the best previous MD-featureless transformer architecture (0.1846 vs 0.1857 validation set MAE). However, the improvement is limited and highly sensitive to the concatenation MD encoder architecture: both an increase in the number of layers of the concatenated input MLP and the use of a previous MLP on the MD features before concatenation significantly harm performance.

After selecting the best-performing MD-aware trimer encoder, Table 5.22 studies the effect of the MD dropout rate on validation performance. The

Table 5.22: Validation-set MAE for different MD dropout values applied to the best-performing MD-aware trimer encoder from Table 5.21; consisting on the concatenation MD encoder with a one-layer MLP of hidden dimension 256 on the concatenated chemical & MD inputs and a trimer-to-sequence transformer with 8 heads, 2 layers, feed-forward dimension 256 and Max pooling.

| MD dropout | # Params | Val. MAE      |
|------------|----------|---------------|
| 0.1        | 416,257  | <b>0.1846</b> |
| 0.3        | 416,257  | 0.1858        |
| 0.5        | 416,257  | 0.1878        |

results displayed in Table 5.22 indicate that a larger MD dropout yields performance losses. This trend is expected for a larger dropout rate like 0.5, as it would imply losing a considerable amount of MD information to the dropout on top of the non-negligible amount of already missing MD features - only 75.9% of the trimers in the training dataset and 66.2% of the trimers in the validation dataset have MD features available.

However, it was not entirely clear whether a MD dropout rate of 0.3 would improve or harm the model’s performance. Given the high MD features absence rate, it could be the case that a MD dropout rate of this magnitude helps generalization by ensuring the concatenation MD encoder appropriately learns how to deal with MD features missing-ness and does not over-rely on the MD feature information as was the case for the XGBoost baseline with MD features. In fact, the model’s performance with a 0.3 dropout rate is practically indistinguishable from that of the best MD feature-less transformer architecture. For these reasons, we will still test a 0.3 dropout rate in our CNN-based models.

Finally, Table 5.23 compares the alternative MD feature representations using the best-performing MD-aware transformer architecture; which consists on the concatenation MD encoder with a & 0.1 MD dropout and a trimer-

to-sequence transformer with 8 heads, 2 layers, feed-forward dimension 256 and Max pooling. The underwhelming performance of the PCA-reduced **MD**

Table 5.23: Validation-set MAE for different trimer-level **MD** feature representations using the best-performing **MD**-aware trimer encoder and **MD** dropout value.

| MD feat. type & encoder architecture                                                                                                      | MD input dim. | # Params  | Val. MAE      |
|-------------------------------------------------------------------------------------------------------------------------------------------|---------------|-----------|---------------|
| Cross-run means and std;<br>Concatenated-input <b>MLP</b> : 1 layer, hidden dim. 256                                                      | 158           | 416,257   | <b>0.1846</b> |
| Cross-run means, normalised histograms and mean h-bond lifetime curves;<br>Concatenated-input <b>MLP</b> : 2 layers, hidden dim. 512, 256 | 1028          | 1,167,361 | 0.1879        |
| PCA-reduced full <b>MD</b> feature vector;<br>Concatenated-input <b>MLP</b> : 2 layers, hidden dim. 512, 256                              | 478           | 798,977   | 0.1956        |

features informed model seems to imply that **PCA** is not able to capture the information carried by the **MD** features. This may be because the directions with the lowest variance actually carry the most information.

This argument is reinforced by the solid performance of the model informed by the cross-run means and normalised histograms together with the cross-run mean hydrogen bond lifetime curves despite being, by far, the largest model so far. Even though it falls behind the best **MD** feature-less transformer model, its performance with 1,167,361 learnable parameters is much better than that of the next larger models that have been evaluated. This suggests that the cross run means, histograms and mean hydrogen bond lifetime curves carry a large amount of information, but are too high-dimensional for the size of our dataset.

To conclude, Table 5.24 summarizes the best-performing transformer-based hierarchical model with and without **MD** features.

Table 5.24: Summary of the best-performing transformer-based hierarchical models. Both models employ the observed optimal trimer-to-sequence transformer with 8 heads, 2 layers, feed-forward dimension 256 and Max pooling

| <b>MD features</b>   | <b>Selected model</b>                                                                                                             | <b># Params</b> | <b>Val. MAE</b> |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------|-----------------|
| No                   | Observed optimal trimer-to-sequence transformer                                                                                   | 319,233         | 0.1857          |
| Cross-run mean & std | Concatenated-input <b>MLP</b> : 1 layer, hidden dim. 256, <b>MD</b> dropout 0.1 + Observed optimal trimer-to-sequence transformer | 416,257         | <b>0.1846</b>   |

### 5.4.5 CNN-based Hierarchical Models Evaluation on the Hard Data Split

The exploration of **CNN**-based hierarchical models followed the three-stage procedure described in Paragraph 4.5.4.4.2. In all cases, model selection was based on the validation-set mean absolute error (MAE). As in the transformer-based exploration, we also report the number of trainable parameters in order to compare predictive performance against model complexity.

Table 5.25 reports the validation-set MAE obtained in the initial **CNN** backbone exploration, comparing nucleotide-to-sequence and trimer-to-sequence architectures under the default mean-pooling setting. Trimer-to-

Table 5.25: Validation-set MAE for the initial exploration of **CNN** backbones. Nucleotide-to-sequence models use  $d_{\text{nuc}} = 32$  and a one-layer **MLP** nucleotide encoder with hidden dimension 64. Trimer-to-sequence models use  $d_{\text{trimer}} = 128$ . All models use mean pooling

| <b>Model type</b> | <b>Channels / <math>k^{(h)}</math></b> | <b>Pad</b> | <b><math>k^{(w)}</math></b> | <b># Params</b> | <b>Val. MAE</b> |
|-------------------|----------------------------------------|------------|-----------------------------|-----------------|-----------------|
| Nuc.→Sequence     | 64, 128, 128 / 2, 1, 1                 | No         | 3                           | 139,553         | 0.1798          |
| Nuc.→Sequence     | 64, 128, 128 / 2, 1, 1                 | Yes        | 3                           | 139,553         | 0.1809          |
| Nuc.→Sequence     | 128, 128, 128 / 2, 1, 1                | No         | 3                           | 176,609         | 0.1813          |
| Nuc.→Sequence     | 128, 128, 128 / 2, 1, 1                | Yes        | 3                           | 176,609         | 0.1825          |
| Nuc.→Sequence     | 64, 128, 128 / 1, 2, 1                 | No         | 3                           | 157,985         | 0.1832          |
| Nuc.→Sequence     | 64, 128, 128 / 1, 2, 1                 | Yes        | 3                           | 157,985         | 0.1832          |
| Trimer→Sequence   | 128, 128, 128                          | No         | 5                           | 326,785         | 0.1828          |
| Trimer→Sequence   | 128, 128, 128                          | Yes        | 5                           | 326,785         | <b>0.1751</b>   |

sequence **CNN** models with padding emerge as the best performing model evaluated so far, beating the best performing transformer models with and without **MD** features. In fact, observe that all **CNN**-based models considered

in Table 5.25 outperform the best performing transformer models with and without MD features.

Further, the best performing trimer-based CNN model (validation split MAE 0.1751) is significantly better than the best performing nucleotide-based CNN model (validation split MAE 0.1798). To further confirm this performance gap, we also train a nucleotide-level CNN model with the same architecture as the best performing nucleotide-level model in 5.25 (channel sizes 64, 128, 128; kernel heights 2, 1, 1; no padding;  $k^{(w)} = 3$ ) but changing the nucleotide encoder MLP dimension to  $d_{\text{nuc}} = 64$  which still achieves an inferior MAE of 0.1824 in the validation split.

Therefore, the results in Table 5.25 also strengthen the hypothesis formulated in the previous section: double-stranded trimers may be a better information/input unit for our models, i.e., trimers constitute a better inductive bias to extract sequence-level representations.

Next, Table 5.26 evaluates the effect of the convolutional kernel width on the best nucleotide-to-sequence CNN model, which determines the local nucleotide context aggregated by the first convolutional layer. These results

Table 5.26: Validation-set MAE for the kernel-width sweep of the best-performing nucleotide-to-sequence CNN backbone from Table 5.25.

| $k^{(w)}$ | # Params | Val. MAE      |
|-----------|----------|---------------|
| 2         | 110,881  | 0.1880        |
| 3         | 139,553  | <b>0.1798</b> |
| 4         | 168,225  | 0.1802        |
| 5         | 196,897  | 0.1809        |
| 6         | 225,569  | 0.1840        |

are more easily visualised in a plot, see Figure 5.32a. The observed tendency further reinforces the argument that the trimer representation is naturally favoured as model input: the best performing nucleotide-to-sequence CNN models are those which first aggregate the local context corresponding to either trimers, 4-mers and pentamers. A smaller or larger context significantly harms performance. Out of the three best  $M$ -mers, trimers emerge as the best choice because their simulation is less costly and we expect to observe less unique trimers than 4-mers or pentamers for any sufficiently large chain.

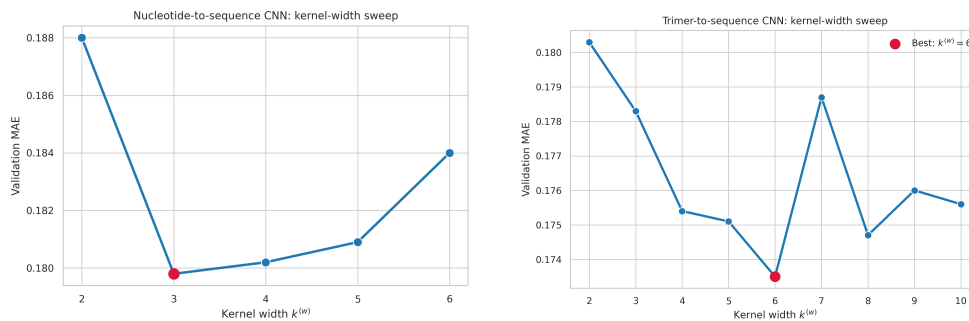
Table 5.27 then studies how the validation performance of the best trimer-to-sequence CNN varies with the trimer-level convolutional kernel width, thereby probing the span of relevant inter-trimer interactions. These results are more easily visualised in a plot, see Figure 5.32b. A kernel width  $k^{(w)} =$

Table 5.27: Validation-set MAE for the kernel-width sweep of the best-performing trimer-to-sequence **CNN** backbone from Table 5.25.

| $k^{(w)}$ | # Params | Val. MAE      |
|-----------|----------|---------------|
| 2         | 160,129  | 0.1803        |
| 3         | 215,681  | 0.1783        |
| 4         | 271,233  | 0.1754        |
| 5         | 326,785  | 0.1751        |
| 6         | 382,337  | <b>0.1735</b> |
| 7         | 437,889  | 0.1787        |
| 8         | 493,441  | 0.1747        |
| 9         | 548,993  | 0.1760        |
| 10        | 604,545  | 0.1756        |

6 undisputedly yields the best performing trimer-to-sequence **CNN** model, which is also the best model evaluated so far (validation set **MAE** of 0.1735).

The fact that the optimal local overlapping-trimer context to be aggregated by the model has width  $k^{(w)} = 6$  supports our decision to move away from molecular graph-based **ML** models. 6 overlapping trimers span 8 nucleotides, so even in the case of a **siRNA** molecular graph with a highway (see Definition 43), by Lemma 44 we would get an over-squashing factor of  $(\frac{3}{5})^{7-2} = 7.776 \cdot 10^{-2}$  on the right hand side of (3.15).



(a) Validation-set MAE for the kernel-width sweep of the best-performing nucleotide-to-sequence **CNN** backbone.

(b) Validation-set MAE for the kernel-width sweep of the best-performing trimer-to-sequence **CNN** backbone.

Figure 5.32: Validation-set MAE as a function of the convolutional kernel width  $k^{(w)}$  for the best-performing nucleotide-to-sequence and trimer-to-sequence **CNN** backbones.

Having selected the best-performing **CNN** backbone under mean pooling, Table 5.28 compares the alternative pooling strategies considered for that

architecture. Contrary to the transformer models case, mean pooling is, by

Table 5.28: Validation-set MAE for different pooling strategies applied to the best-performing **CNN** backbone identified in Stage 1; corresponding to a trimer-to-sequence CNN model with 3 convolutional layers, channel dimension 128, kernel width  $k^{(w)} = 6$  and sequence padding.

| Pooling strategy | # Params | Val. MAE      |
|------------------|----------|---------------|
| Mean             | 382,337  | <b>0.1735</b> |
| Attention        | 382,466  | 0.1783        |
| Max              | 382,337  | 0.1761        |

a safe margin, the best pooling strategy for trimer-to-sequence **CNN** models. Thus, all the subsequent models inherit the 3 convolutional layers, channel dimension 128, kernel width  $k^{(w)} = 6$ , sequence padding and mean pooling trimer-to-sequence **CNN** architecture.

Table 5.29 evaluates whether learned trimer representations improve upon the pre-computed trimer input tokens in the **CNN**-based setting, and whether a **CNN**-based nucleotide-to-trimer encoder can outperform the corresponding **MLP** alternative. The learned-only trimer encoder paired with

Table 5.29: Validation-set MAE for **CNN**-based hierarchical models using learned and hybrid trimer encoders, together with alternative nucleotide-to-trimer encoders. All models inherit the best-performing trimer-to-sequence **CNN** backbone and pooling strategy selected in Stage 1; consisting of 3 convolutional layers, channel dimension 128, kernel width  $k^{(w)} = 6$ , sequence padding and mean pooling.

| Trimer encoder          | Nucleotide→trimer encoder                                                                        | # Params | Val. MAE      |
|-------------------------|--------------------------------------------------------------------------------------------------|----------|---------------|
| Learned only            | 2-layer <b>MLP</b> , hidden dims. 256, 128                                                       | 744,161  | 0.1786        |
| Learned only            | <b>CNN</b> , 3 layers with channel sizes 64, 128, 128, $k^{(h)} = 2, 1, 1$ & $k^{(w)} = 1, 2, 2$ | 452,385  | 0.1789        |
| Learned only            | <b>CNN</b> , 2 layers with channel sizes 64, 128, $k^{(h)} = 2, 1$ & $k^{(w)} = 1, 2$            | 419,233  | <b>0.1741</b> |
| Hybrid chemical/learned | 1-layer <b>MLP</b> , hidden dim. 256                                                             | 408,289  | 0.1756        |

a smaller **CNN** nucleotide-to-trimer encoder achieves the best result among the evaluated nucleotide-to-trimer-to-sequence models and gets remarkably close (validation split **MAE** of 0.1741) to the best performing trimer-to-sequence (validation split **MAE** of 0.1735).

We conclude that imposing a stronger inductive bias in the nucleotide-to-trimer encoder can improve upon a basic **MLP** nucleotide-to-trimer encoder, opening the door to possible future improvement through learned trimer representation. Nevertheless, we focus on the best performing trimer-to-sequence model with pre-processed trimer input tokens for now.

For the final step, we slightly alter the procedure order by first determining the best **MD** dropout rate in  $\{0.1, 0.3\}$  for the best performing trimer-to-sequence **CNN** model informed with the cross-run means and standard deviations of every extracted **MD** observable. The **MD** aware trimer encoder is inherited from that of the best performing trimer-to-sequence **MD**-informed transformer model, which corresponds to a concatenated-input **MLP** with 1 layer and hidden dim. 256. Table 5.30 displays the results of the **MD** dropout optimization. The convolutional kernel is kept fixed as  $k^{(w)} = 6$ , as this has been observed to be the undisputed optimal kernel width for trimer-to-sequence **CNN** models.

Table 5.30: Validation-set MAE for a **MD**-aware trimer-to-sequence **CNN**, varying the **MD** dropout rate and setting the convolutional kernel to  $k^{(w)} = 6$

| <b>MD dropout</b> | <b># Params</b> | <b>Val. MAE</b> |
|-------------------|-----------------|-----------------|
| 0.1               | 464,129         | 0.1735          |
| 0.3               | 464,129         | <b>0.1718</b>   |

Finally, Table 5.31 compares the alternative trimer-level **MD** feature representations when incorporated into the best-performing **CNN**-based hierarchical model and using a **MD** dropout rate of 0.3. The underperformance

Table 5.31: Validation-set MAE for different **MD** feature representations applied to the best-performing **CNN**-based trimer architecture. The **MD**-aware trimer encoder is inherited from the transformer-based exploration and the initial **MD** dropout is fixed to 0.3.

| <b>MD feat. type &amp; encoder architecture</b>                                                                                           | <b>MD input dim.</b> | <b># Params</b> | <b>Val. MAE</b> |
|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-----------------|-----------------|
| Cross-run means and std;<br>Concatenated-input <b>MLP</b> : 1 layer, hidden dim. 256                                                      | 158                  | 464,129         | <b>0.1718</b>   |
| Cross-run means, normalised histograms and mean h-bond lifetime curves;<br>Concatenated-input <b>MLP</b> : 2 layers, hidden dim. 512, 256 | 1028                 | 1,215,233       | 0.1767          |
| PCA-reduced full <b>MD</b> feature vector;<br>Concatenated-input <b>MLP</b> : 2 layers, hidden dim. 512, 256                              | 478                  | 846,849         | 0.1772          |

of both the model informed by cross-run means, normalised histograms and mean h-bond lifetime curves and the model informed by the PCAd concatenation of all extracted **MD** features match the results obtained for the case of transformer based models. The largest difference between the observations in Table 5.23 and Table 5.31 is the absence of a large performance drop for the PCAd **MD** features case. This may however be explained by the strong overall performance of the **CNN** architecture.

To conclude, Table 5.32 summarizes the best-performing **CNN**-based hierarchical model with and without **MD** features.

Table 5.32: Summary of the best-performing transformer-based hierarchical models. Both models employ the observed optimal trimer-to-sequence **CNN** with  $d_{\text{trimer}} = d_{\text{seq}} = 128$ , 3 convolutional layers with channel size 128, kernel width  $k^{(w)} = 6$ , mean pooling and sequence padding.

| <b>MD features</b>   | <b>Selected model</b>                                                                                                               | <b># Params</b> | <b>Val. MAE</b> |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------|-----------------|
| No                   | Observed optimal trimer-to-sequence <b>CNN</b>                                                                                      | 319,233         | 0.1735          |
| Cross-run mean & std | Concatenated-input <b>MLP</b> : 1 layer, hidden dim. 256, <b>MD</b> dropout 0.3 +<br>Observed optimal trimer-to-sequence <b>CNN</b> | 464,129         | <b>0.1718</b>   |

#### 5.4.6 Best Model Hyper-parameter Optimization & Ablation on the Hard Data Split

Once we have identified the best performing model, the next step is to run a hyper-parameter optimization on its architecture to maximize its performance. Recall that the best identified model is a **MD**-informed trimer-to-sequence **CNN** whose architecture is summarized in Table 5.33. Further, Figure 5.33 illustrates the best performing model generalized architecture, without specifying hyper-parameters nor the specific set of **MD** features used.

Hyper-parameter tuning was carried out using Optuna [67]. The search was performed for 50 trials due to time and resource limitations. In each trial, the kernel width, sequence padding strategy and pooling type were kept fixed at the values already selected in the architecture study, namely  $k^{(w)} = 6$ , sequence padding enabled and mean pooling. The search therefore focused on continuous and discrete choices affecting optimization, regularization and model capacity. Trials were trained for at most 1000 epochs. The objective

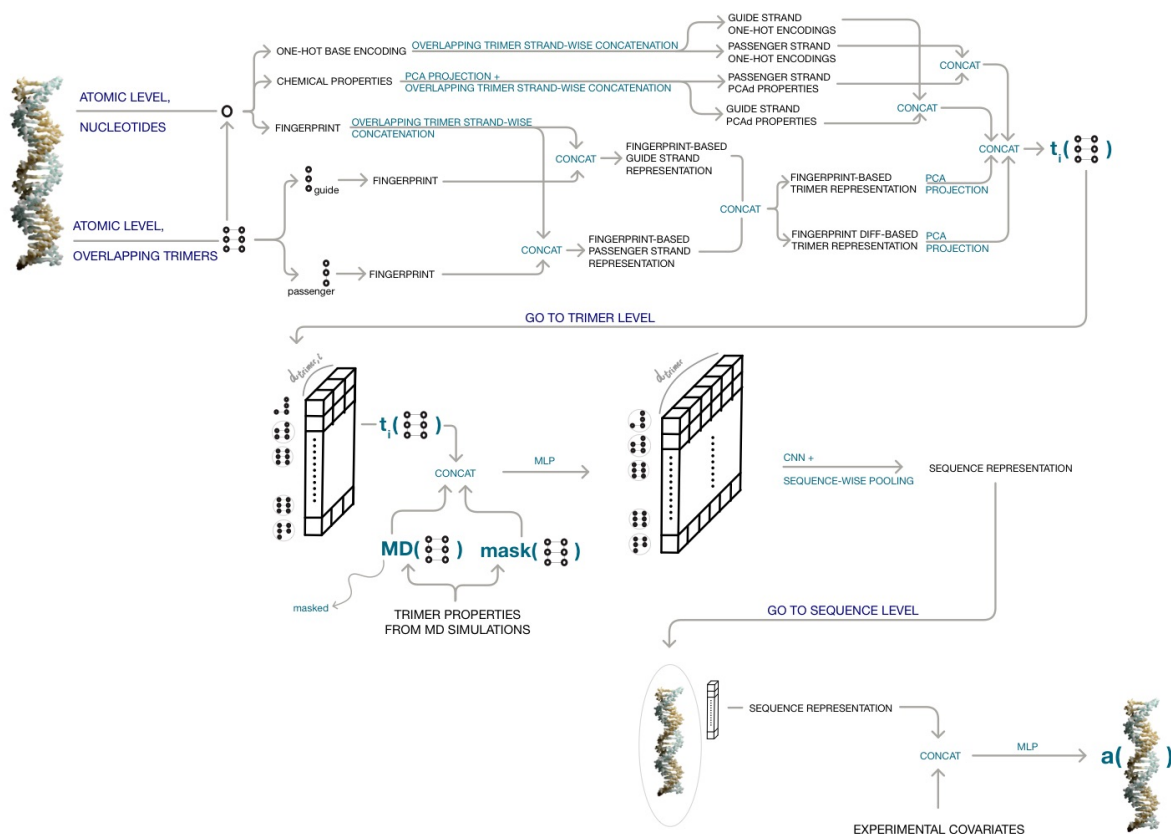


Figure 5.33: Diagram of the best performing model's generalized architecture, with unspecified hyper-parameters and **MD** features. The map  $t_i(\cdot)$  is defined rigorously in (4.55). The map  $MD(\cdot)$  simply refers to the concatenated **MD** features of the input trimer, while the map  $mask(\cdot)$  indicates whether **MD** features for the input trimer are available. Finally,  $a(\cdot)$  denotes the predicted activity for the input **siRNA** molecule.

Table 5.33: Architecture summary of the best-performing model.

| Component                      | Specification                                                                                                                         |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Model family                   | Trimer-to-sequence <b>CNN</b> with <b>MD</b> -aware trimer encoder                                                                    |
| Trimer encoder type            | Concatenation-based <b>MD</b> -aware trimer encoder                                                                                   |
| Chemical / MD integration      | Concatenate chemical representation, masked <b>MD</b> representation, and <b>MD</b> availability indicator, followed by an <b>MLP</b> |
| Concatenated-input <b>MLP</b>  | 1 layer, hidden dimension 256                                                                                                         |
| MD dropout                     | 0.3                                                                                                                                   |
| Trimer token dimension         | $d_{\text{trimer}} = 128$                                                                                                             |
| Sequence embedding dimension   | $d_{\text{seq}} = 128$                                                                                                                |
| Sequence encoder type          | Trimer-to-sequence <b>CNN</b>                                                                                                         |
| Number of convolutional layers | 3                                                                                                                                     |
| Convolutional channel sizes    | 128, 128, 128                                                                                                                         |
| Kernel width                   | $k^{(w)} = 6$                                                                                                                         |
| Sequence padding               | Enabled                                                                                                                               |
| Pooling strategy               | Mean pooling                                                                                                                          |

value returned to Optuna was the best validation loss achieved during training.

The searched hyper-parameters are summarized in Table 5.34. The trimer encoder remained the concatenation-based **MD**-aware encoder, while the trimer-to-sequence stage remained a **CNN**; however, their internal widths and depths were allowed to vary. To avoid exploring architectures that were either unrealistically small or unnecessarily large, candidate models were pruned whenever their number of trainable parameters fell below  $10^5$  or exceeded  $5 \times 10^6$ .

Although Optuna did identify one model configuration which achieved a validation set loss of 0.1701, this result could not be replicated for any of the considered random seeds 42, 73 and 1729. In fact, all three seeds performed significantly worse than the best performing model identified through our evaluation procedure (whose architecture is described in Table 5.33).

Meanwhile, the best performing model described in Table 5.33 achieved a validation set **MAE** of 0.1718 for seed 73 and 0.1711 for seed 1729. Table 5.35 compares these results to those of the second best performing model identified by our search, which is the **MD** feature-less variant of the best performing model.

These results suggest that the best performing, **MD**-informed model not

Table 5.34: Optuna search space for the hyper-parameter optimization of the best-performing MD-aware trimer-to-sequence CNN model. Fixed architectural choices inherited from Table 5.33 were  $k^{(w)} = 6$ , sequence padding enabled and mean pooling.

| Hyper-parameter                 | Type        | Search space                            |
|---------------------------------|-------------|-----------------------------------------|
| Learning rate                   | log-uniform | $[10^{-5}, 10^{-3}]$                    |
| Weight decay                    | log-uniform | $[10^{-4}, 10^{-1}]$                    |
| Batch size                      | categorical | $\{64, 128, 256\}$                      |
| Loss function                   | categorical | $\{\text{mae}, \text{huber}\}$          |
| Huber $\delta$                  | uniform     | $[0.05, 0.5]$ if Huber loss is selected |
| Dropout                         | uniform     | $[0.05, 0.3]$                           |
| MD dropout                      | uniform     | $[0.1, 0.5]$                            |
| Trimer token dimension          | categorical | $\{64, 128\}$                           |
| $d_{\text{token}}$              |             |                                         |
| Trimer encoder hidden dimension | categorical | $\{128, 256\}$                          |
| Number of CNN layers            | integer     | $\{2, 3, 4, 5\}$                        |
| CNN hidden channel size         | categorical | $\{64, 128\}$                           |
| CNN output dimension            | categorical | $\{64, 128\}$                           |
| $d_{\text{model}}$              |             |                                         |
| Prediction head hidden dim. 1   | categorical | $\{64, 128, 256\}$                      |
| Prediction head hidden dim. 2   | categorical | $\{32, 64, 128\}$                       |

Table 5.35: Validation set MAE of the best-performing hierarchical model (MD-informed, see Table 5.33) and the second best performing model, which corresponds to its MD-free variant, evaluated across three independent seeds. Runs for seeds 73 and 1729 used an early-stopping patience of 500 to minimise cross-seed variance. The last two columns report the cross-seed mean and sample standard deviation ( $n = 3$ ).

| Model       | Seed 42 | Seed 73 | Seed 1729 | Mean          | Std    |
|-------------|---------|---------|-----------|---------------|--------|
| MD-informed | 0.1718  | 0.1718  | 0.1711    | <b>0.1716</b> | 0.0004 |
| MD-free     | 0.1735  | 0.1784  | 0.1739    | 0.1753        | 0.0027 |

only achieves a noticeable performance increase with respect to its **MD** feature-less variant but also features a greater cross-seed stability. This warrants further investigation on the potential of **MD**-informed models for reducing prediction uncertainty.

In lack of a more thorough hyper-parameter optimization, let us declare the model described in Table 5.33 as the best performing model found, and denote it as *B* from now on.

To assess the sensitivity of the best-performing architecture to its main training and capacity choices, we carried out a one-factor-at-a-time ablation study in which each selected hyper-parameter was varied once while all remaining settings were kept fixed. Its results are summarized in Table 5.36 and highlight a significant sensitivity of the best performing model *B* to any alterations to its configuration. While none of the studied ablations yield a catastrophic performance decrease, their validation set **MAEs** all lie below that of the second best performing model.

Table 5.36: One-factor-at-a-time ablation study of the best-performing model. Each row reports a single variation with respect to the reference configuration in Table 5.33, while all other hyper-parameters are kept fixed.

| Ablated factor                           | Reference setting | Tested variation | # Params | Val. MAE |
|------------------------------------------|-------------------|------------------|----------|----------|
| Batch size                               | 128               | 64               | 464,129  | 0.1744   |
| Number of <b>CNN</b> layers              | 3                 | 2                | 365,441  | 0.1778   |
| <b>CNN</b> channel size                  | 128 in each layer | 64 in each layer | 258,753  | 0.1790   |
| Dropout                                  | 0.10              | 464,129          | 464,129  | 0.1783   |
| MD-aware trimer encoder hidden dimension | 256               | 128              | 404,225  | 0.1759   |
| Prediction head hidden dimensions        | [128, 64]         | [64, 32]         | 446,657  | 0.1755   |
| Prediction head number of layers         | 2                 | 1                | 455,809  | 0.1769   |

To assess the relative contribution of the different **MD**-derived descriptor families, we carried out a leave-one-group-out feature ablation study in which the best-performing model was retrained after removing one **MD** feature group at a time. The results of this procedure are summarized in Table 5.37. These results match the observations from the toy XGBoost models with and without **MD** features: there is no single feature group carrying all the signal, but removing any single feature group harms performance.

**RMSD** observables and the introduced open and dissociated probabilities emerge as critical **MD** features, as their absence yields performances that are

Table 5.37: Feature-group ablation study for the MD descriptors in the best-performing model. Each row reports the result obtained after removing one MD feature group from the full MD input while keeping all other feature groups unchanged. The feature group with the largest negative effect is indicated with an asterisk \*.

| Removed MD feature group            | Resulting input MD dimension | Val. MAE      |
|-------------------------------------|------------------------------|---------------|
| None (best model)                   | 158                          | <b>0.1718</b> |
| Base-pair parameters                | 110                          | 0.1741        |
| Hydrogen bonds                      | 122                          | 0.1740        |
| RMSD                                | 144                          | 0.1768*       |
| RMSF                                | 146                          | 0.1749        |
| Open and dissociation probabilities | 154                          | 0.1762        |
| Pucker                              | 146                          | 0.1746        |
| Interaction energies                | 126                          | 0.1731        |

noticeably below that of the best MD feature-less model (validation MAE of 0.1735). The importance of the cross-run mean and standard deviation open and dissociated probabilities is particularly interesting, as these provide a compact description (they only constitute 4 of the dimensions of the input MD feature vector) of trimer stability.

Finally, Table 5.38 displays the achieved validation and test set MAE by the best performing baselines and the best performing model on the Hard data split.

Table 5.38: Final comparison of the best-performing baseline models and the best overall model on the validation and test sets of the Hard data split.

| Model                       | # Params   | Val. MAE      | Test MAE      |
|-----------------------------|------------|---------------|---------------|
| C1 (Cm-siRPreD baseline)    | 11,050,209 | 0.2285        | 0.2299        |
| T2 (XGBoost baseline)       | –          | 0.1914        | 0.1962        |
| B (best hierarchical model) | 464,129    | <b>0.1718</b> | <b>0.1754</b> |

As a final note, it is important to remember that we have only performed a limited hyper-parameter optimization in which some architectural details were left fixed, informed by the initial evaluation in Section 5.4.5. However, this evaluation was performed using a single seed and a moderate value of 200 for the patience hyper-parameter, and the difference in validation MAE between some of the studied architectural choices was relatively small. While we have reasons to be confident in both the identified best model’s performance advantage over the best lines and the positive impact of the extracted

**MD** features, future work would include a more extensive hyper-parameter optimization – including a more fine-grained search on the employed **MD** groups and features – which may yield even stronger results by means of a slightly different set of hyper-parameters for the best performing model.

#### 5.4.7 Best Model & Baseline Evaluation on the Easy Data Split

The very last step of our study consists on training the best baselines and the best performing model on the Easy data split (using validation set **MAE** as stopping criteria) and report the models' performance on the validation and test sets of the Easy data split. Table 5.39 summarizes these results. As expected,

Table 5.39: Final comparison of the best-performing baseline models and the best overall model on the validation and test sets of the Easy data split.

| Model                       | # Params   | Val. MAE      | Test MAE      |
|-----------------------------|------------|---------------|---------------|
| C1 (Cm-siRPred baseline)    | 11,053,793 | 0.2023        | 0.2035        |
| T2 (XGBoost baseline)       | –          | 0.1762        | 0.1750        |
| B (best hierarchical model) | 456,449    | <b>0.1427</b> | <b>0.1420</b> |

all models achieve better predictions, but the performance gap between the identified best performing model B and the baseline models is even wider in the Easy data split.



## Chapter 6

# Conclusions and Future work

Finally, this Chapter summarizes the main findings of this work 6.1, identifying its main limitations 6.2 and pointing at possible future research directions 6.3.

### 6.1 Conclusions

The starting point of this thesis was the question of whether mRNA knockdown — a global, experimentally measured property of a modified siRNA duplex — could be predicted from the structural and dynamical properties of the duplex's constituent double-stranded trimers, and whether such a prediction could be made to generalise to sequences and chemical modifications that were not seen during training. Answering this question required contributions at the dataset, simulation, mathematical and modelling levels, and the picture that emerges from putting these contributions together is, on balance, an encouraging one.

At the dataset level, we have curated and characterised a collection of more than 88,000 experimentally measured modified siRNA activities, spanning a wide range of cell types, transfection methods, incubation times and activity detection assays. The associated fragmentation and hashing procedures introduced in Chapter 4 have made it possible to identify the unique nucleotides and overlapping trimers underlying this dataset in a fully automated fashion, and to cross-reference the extracted chemical fragments with the modXNA parametrisation catalogue. To the best of our knowledge, this is the first time such an analysis has been carried out at this scale for chemically modified siRNA.

Around this curated dataset we have built an end-to-end pipeline that takes modified double-stranded trimers to MD-ready pentamer constructs, runs the

corresponding simulations using modXNA-derived parameters, and extracts a set of structural and dynamical descriptors per trimer. The pipeline is incremental by design: as new **siRNA** molecules are added to the dataset and new unique trimers are identified, only the previously unseen trimers need to be simulated and appended to the **MD** feature table. This modular structure means that the resulting trimer-level feature dataset is not tied to activity prediction alone, but can in principle be reused as an input for the prediction of other relevant **siRNA** properties, such as thermodynamic stability, off-target propensity or duplex asymmetry.

At the mathematical level, the analysis in Chapter 3 has clarified why standard message-passing architectures are structurally ill-suited to this problem. The permutation-invariance pathologies of Lemma 41, the exponential decay of the sensitivity bounds implied by Lemma 42 and the persistence of this decay even in the presence of a stacking-based highway (Lemma 44) together provide a rigorous justification for moving beyond **MPNNs** and for adopting a hierarchical formulation in which physical, trimer-level information can be injected at the level at which it is most informative. The hierarchical models introduced in Subsection 3.3.3 offers a principled way of doing so.

At the modelling level, the empirical results obtained on a demanding, generalisation-oriented split confirm that the proposed hierarchical models are viable in practice. Although the tested models are deliberately kept modest in size, they clearly surpass the best previously available baseline for modified **siRNA** activity prediction, and the ablations show that the **MD**-derived trimer features contribute measurably to this improvement. Taken together, these observations support a positive answer to our original research question: trimer-level structural and dynamical descriptors do carry information that is predictive of the duplex's global activity, and hierarchical, physics-assisted models are a natural vehicle to exploit it.

This positive answer should nevertheless be read with the appropriate caution. The absolute accuracy of the proposed models, while state of the art on this task, remains modest in absolute terms. The gap between our predictions and the experimental measurements is a reminder that the complexity of the **RNA** interference pathway, the intracellular environment in which it unfolds and its ultimate outcomes are only partially captured by our approach. The limitations that this entails are examined in more detail in the next section.

## 6.2 Limitations

The main limitations of this work concern the size and coverage of the available data on one hand, and the biological realism of the underlying simulations on the other.

The publicly available data on modified **siRNA** activity remains, despite recent progress, comparatively scarce. While the dataset used in this thesis contains close to  $10^5$  entries, we have consistently observed that larger **ML** models struggle to generalise between the training, validation and test splits, particularly when the splits are constructed to probe unseen sequences and unseen chemical modifications rather than a purely random partition. This imposed a ceiling on the depth and width of the architectures that can be trained without overfitting. It also implies that some of the more expressive instantiations of the hierarchical framework proposed in Chapter 3 could not be evaluated under statistically meaningful conditions.

A second, more fundamental limitation stems from the biology of the **RNA** interference pathway itself. Activity, as experimentally measured, is the outcome of a multi-step intracellular process involving delivery across the cellular membrane, avoidance of degradation, loading of the guide strand into the **RISC** complex and, ultimately, cleavage of the target **mRNA** by the Argonaute enzyme. Our **MD** simulations are however very limited in their scope, as we consider modified trimers embedded in pentamer constructs in explicit water only. The physical environment we probe is therefore only a slice of the true cellular environment which determines **siRNA** activity, and any features extracted from these simulations should be understood as informative correlates rather than direct proxies for the biological outcome.

The extent of the performed hyper-parameter optimization is a secondary limitation of this work. An extensive hyper-parameter optimization on the best performing architecture should yield an even better model whose hyper-parameter set may slightly differ from that of the current best performing model.

## 6.3 Future work

The workflow developed in this thesis suggests a number of natural extensions, some of which are relatively concrete refinements of the current pipeline and some of which are more ambitious research directions in their own right.

A first and arguably most desirable direction is the use of more biologically

realistic simulations. Introducing an explicit representation of the Argonaute protein, or at least of the local environment provided by the **RISC** complex, would bring the simulated setting substantially closer to the process that determines activity in vivo. This is, however, severely constrained by the computational cost of such simulations, and would likely require the combination of coarse-grained models, enhanced sampling techniques and a careful selection of the trimers for which the extra cost is justified.

On the feature extraction side, an immediate refinement consists in incorporating the full cross-run bin correlation matrix, rather than the per-bin, cross-run standard deviations. Further, it would be interesting to move beyond simple averaging when aggregating the information from all the hydrogen bonds between a given residue pair in a pentamer; alternatives ranging from attention-based aggregation to physically motivated weightings by bond occupancy or lifetime are all worth exploring.

Another relevant future direction concerns the validation of the extracted trimer **MD** features from pentamer simulations. We are currently working on contrasting those with the extracted trimer **MD** features from whole-siRNA simulations. While open and dissociation probabilities – which we have identified as a significantly informative feature – cannot be verified through this procedure, their robustness may also be improved by re-formulating the definition of the open and dissociated state-space regions to be independent from the whole-pentamer **RMSD**, which may be too system-specific and does not naturally translate to other M-mers.

At the theoretical level, the obvious next step is to formulate and prove a universality theorem for the proposed hierarchical architecture in Chapter 3. At the moment of submitting this thesis, this proof is underway. Further steps would involve developing a theoretical and practical understanding of the differences between *airtight* and *non-airtight* hierarchical models.

Two natural extensions concern the architecture of the hierarchical models themselves. The first is to use a graph-based **ML** method as a propagator at the molecular-graph level, paired with a pooling function towards the nucleotide and/or trimer levels, and to study whether such a component can complement the successful, pre-processed fingerprint-based trimer input tokens rather than replace them. At this representational level, it would also be interesting to extend the construction of trimer representations by using the more general family of functions  $\tilde{f}_{r,2}$  introduced in Subsection 4.5.2, rather than restricting ourselves to the (2,2) case adopted in the present experiments. This would allow a finer control over the trade-off between locality and expressivity in the pre-processed trimer input tokens.

The second is to use a graph-based **ML** method operating on nucleotides as nodes to encode the extracted **MD** trimer properties into a dedicated **MD** features embedding for the whole trimer, effectively learning the aggregation from local, physically meaningful descriptors to a trimer-level physical representation. Observe that these two extensions would naturally suit the use of an AI agent which proposes different **ML** models for each of the hierarchical model levels (atom, nucleotide, trimer and sequence), evaluates their performance and iterates in an automated manner.

From a broader perspective, it would be valuable to investigate whether ensembles of **MD**-informed models are less epistemically uncertain than ensembles of **MD** feature-less models trained on the same data. A positive answer would support the intuition that the physical features not only shift the mean prediction but also constrain the space of plausible models, which would in turn have consequences for downstream tasks such as active learning and uncertainty-aware experimental design.

## 6.4 Reproducibility

A Python package covering every stage of the pipeline presented in this Thesis has been developed and will be made public soon together with the data produced during the development of this Thesis.

This package includes the nucleotide fragmentation tool for extracting modifications from modified nucleotides; a series of specialized functions for creating and managing datasets at the fragment, nucleotide, trimer and **siRNA** molecule level; an automated pipeline for modified trimers **MD** preparation, simulation and feature extraction; and a modular **ML** architecture that facilitates the implementation of the proposed hierarchical models.

## 6.5 Impact Statement

The thesis contributes to the **United Nations (UN) Sustainable Development Goals (SDGs)** in ways that are, we believe, both concrete and modest, and it is worth being explicit about both sides of that statement.

The most direct contribution is to *SDG 3 (Good Health and Well-being)*. Modified **siRNA** therapeutics have the potential to address metabolic, hepatic and rare genetic disorders for which no satisfactory small-molecule or biologic treatment currently exists, and a substantial fraction of the cost and time associated with their development is spent on the experimental

exploration of chemical modifications. By providing a curated, incremental dataset, a reproducible MD pipeline for modified trimers and a family of hierarchical models that surpass the best previously available baseline for activity prediction, this work contributes a small but tangible step towards making that exploration more efficient. In the long run, more efficient design translates into a broader and cheaper access to therapies for patients affected by conditions that are currently under-served.

A secondary contribution is to *SDG 9 (Industry, Innovation and Infrastructure)*. The end-to-end nature of the proposed pipeline — from raw siRNA sequences to trimer-level MD features and hierarchical model training — and the fact that it has been designed to be incremental means that it can be adopted, extended and audited by other practitioners without having to reproduce the entire chain from scratch. The mathematical analysis of graph-based models on siRNA molecular graphs, and in particular the notion of airtight hierarchical models, is not specific to nucleic acids and may be of use to other biomolecular systems whose natural description involves multiple, well-separated structural scales.

Some indirect contributions to *SDG 12 (Responsible Consumption and Production)* are also worth mentioning. Molecular dynamics simulations are computationally expensive and, consequently, energetically expensive. The trimer-based decomposition adopted in this thesis is explicitly motivated by the intractability of exhaustively simulating full modified duplexes, and can be viewed as an attempt to extract as much reusable physical information as possible from a comparatively small amount of compute, by ensuring that each simulated trimer contributes to the prediction of many duplexes rather than a single one.

Finally, it is important to acknowledge what this work does *not* do. It does not, on its own, deliver a clinically usable design tool, and its predictions should be understood as an aid to, rather than a substitute for, experimental characterisation. The main ethical responsibility associated with this line of work therefore lies, in our view, in being transparent about its limitations and in ensuring that the resulting tools remain openly accessible to the research community that they are intended to serve.

## References

- [1] S. Harikrishna and P. I. Pradeepkumar, “Probing the binding interactions between chemically modified sirnas and human argonaute 2 using microsecond molecular dynamics simulations,” *Journal of Chemical Information and Modeling*, vol. 57, no. 4, pp. 883–896, 2017. doi: 10.1021/acs.jcim.6b00773
- [2] A. Sahoo, S. Gupta, G. Das, S. N. Sharma, A. Ghosh, S. S. Bagale, S. Sinha, and K. R. Gore, “Thermodynamic destabilization and structural adaptability of 2 -alkoxy/fluoro-n3-methyl pyrimidine nucleotides in sirna enhance rna interference activity,” *Journal of Medicinal Chemistry*, vol. 68, no. 15, pp. 16 371–16 394, 2025. doi: 10.1021/acs.jmedchem.5c01269
- [3] D. D. Martinelli, “From sequences to therapeutics: Using machine learning to predict chemically modified sirna activity,” *Genomics*, vol. 116, no. 2, p. 110815, 2024. doi: 10.1016/j.ygeno.2024.110815
- [4] O. Love, R. Galindo-Murillo, D. R. Roe, P. D. Dans, T. E. Cheatham III, and C. Bergonzo, “modxna: A modular approach to parametrization of modified nucleic acids for use with amber force fields,” *Journal of Chemical Theory and Computation*, vol. 20, no. 21, pp. 9354–9363, 2024. doi: 10.1021/acs.jctc.4c01164
- [5] T. Liu, J. Huang, D. Luo, L. Ren, L. Ning, J. Huang, H. Lin, and Y. Zhang, “Cm-sirpred: Predicting chemically modified sirna efficiency based on multi-view learning strategy,” *International Journal of Biological Macromolecules*, vol. 264, p. 130638, 2024. doi: 10.1016/j.ijbiomac.2024.130638
- [6] S. He, C. Chen, X. Pan, G. Xue, Y. Yang, J. Feng, H. Zulfiqar, Y. Zhang, and K. Deng, “Cmsirnadb: a database of chemically modified sirna

- silencing efficiency for nucleic acid drug design,” *BMC Bioinformatics*, 2026. doi: 10.1186/s12859-025-06359-y
- [7] D. Gallego Pérez, “Parmbsc2: Development of an accurate force field for nucleic acids simulations,” PhD thesis, Universitat de Barcelona, 2023. [Online]. Available: <https://diposit.ub.edu/dspace/handle/2445/190966>
- [8] G. J. Hannon, “Rna interference,” *Nature*, vol. 418, pp. 244–251, 2002. doi: 10.1038/418244a
- [9] D. R. Corey, “Chemical modification: the key to clinical application of rna interference?” *Journal of Clinical Investigation*, vol. 117, no. 12, pp. 3615–3622, 2007. doi: 10.1172/JCI33474
- [10] R. E. Dickerson, “Definitions and nomenclature of nucleic acid structure components,” *Nucleic Acids Research*, vol. 17, no. 5, pp. 1797–1803, 1989. doi: 10.1093/nar/17.5.1797
- [11] X.-J. Lu and W. K. Olson, “3dna: A software package for the analysis, rebuilding and visualization of three-dimensional nucleic acid structures,” *Nucleic Acids Research*, vol. 31, no. 17, pp. 5108–5121, 2003. doi: 10.1093/nar/gkg680
- [12] I. J. MacRae and J. A. Doudna, “Ribonuclease revisited: structural insights into ribonuclease iii family enzymes,” *Current Opinion in Structural Biology*, vol. 16, no. 1, pp. 100–108, 2006.
- [13] A. Birmingham, E. Anderson, K. Sullivan, A. Reynolds, Q. Boese, D. Leake, J. Karpilow, and A. Khvorova, “A protocol for designing siRNAs with high functionality and specificity,” *Nature Protocols*, vol. 2, no. 9, pp. 2068–2078, 2007. doi: 10.1038/nprot.2007.278
- [14] D. Frenkel and B. Smit, *Understanding Molecular Simulation: From Algorithms to Applications*, 2nd ed. San Diego, CA: Academic Press, 2001.
- [15] A. D. Sokal, “Monte carlo methods in statistical mechanics: Foundations and applications,” in *Cours de Troisième Cycle de la Physique en Suisse Romande*, 1989, lecture Notes.
- [16] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, and E. Lindahl, “Gromacs: High performance molecular simulations through

- multi-level parallelism from laptops to supercomputers,” *SoftwareX*, vol. 1–2, pp. 19–25, 2015.
- [17] S. Páll, A. Zhmurov, P. Bauer, M. Abraham, M. Lundborg, A. Gray, B. Hess, and E. Lindahl, “Heterogeneous parallelization and acceleration of molecular dynamics simulations in gromacs,” *The Journal of Chemical Physics*, vol. 153, no. 13, p. 134110, 2020.
- [18] D. A. Case, T. E. I. Cheatham, T. Darden, H. Gohlke, R. Luo, K. M. J. Merz, A. Onufriev, C. Simmerling, B. Wang, and R. J. Woods, “The amber biomolecular simulation programs,” *Journal of Computational Chemistry*, vol. 26, no. 16, pp. 1668–1688, 2005. doi: 10.1002/jcc.20290
- [19] A. Pérez, I. Marchán, D. Svozil, J. Šponer, T. E. I. Cheatham, C. A. Laughton, and M. Orozco, “Refinement of the amber force field for nucleic acids: Improving the description of  $\alpha/\gamma$  conformers,” *Biophysical Journal*, vol. 92, no. 11, pp. 3817–3829, 2007. doi: 10.1529/biophysj.106.097782
- [20] ParmEd Developers, “Parmed,” <https://github.com/ParmEd/ParmEd>, 2026, gitHub repository, accessed 2026-08-02.
- [21] S. Axelrod and R. Gomez-Bombarelli, “Geom: Energy-annotated molecular conformations for property prediction and molecular generation,” *arXiv preprint arXiv:2006.05531*, 2020. doi: 10.48550/arXiv.2006.05531. [Online]. Available: <https://arxiv.org/abs/2006.05531>
- [22] RDKit contributors, “Rdkit: Open-source cheminformatics,” <https://www.rdkit.org>, accessed: 2026-08-09.
- [23] A. H. Lipkus, “A proof of the triangle inequality for the tanimoto distance,” *Journal of Mathematical Chemistry*, vol. 26, no. 1–3, pp. 263–265, 1999. doi: 10.1023/A:1019186204558
- [24] H. Hotelling, “Analysis of a complex of statistical variables into principal components,” *Journal of Educational Psychology*, vol. 24, no. 6, pp. 417–441, 1933. doi: 10.1037/h0071325
- [25] K. Pearson, “On lines and planes of closest fit to systems of points in space,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, no. 11, pp. 559–572, 1901. doi: 10.1080/14786440109462720

- [26] L. Kaufman and P. J. Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*. New York: Wiley, 1990.
- [27] E. Schubert and P. J. Rousseeuw, “Faster k-medoids clustering: Improving the pam, clara, and clarans algorithms,” *arXiv preprint*, 2018. [Online]. Available: <https://arxiv.org/abs/1810.05691>
- [28] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016. doi: 10.1145/2939672.2939785 pp. 785–794.
- [29] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [30] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [31] H. Robbins and S. Monro, “A stochastic approximation method,” *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.
- [32] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *International Conference on Learning Representations (ICLR)*, 2015.
- [33] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein, “Muon: An optimizer for hidden layers in neural networks,” <https://kellerjordan.github.io/posts/muon/>, 2024.
- [34] K. Fukushima, “Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position,” *Biological Cybernetics*, vol. 36, no. 4, pp. 193–202, 1980.
- [35] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [36] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, 2012.

- [37] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 2017, pp. 1263–1272. [Online]. Available: <https://proceedings.mlr.press/v70/gilmer17a.html>
- [38] J. Klicpera, S. Weißenberger, and S. Günnemann, “Diffusion improves graph learning,” *CoRR*, vol. abs/1911.05485, 2019. [Online]. Available: <http://arxiv.org/abs/1911.05485>
- [39] B. P. Chamberlain, J. Rowbottom, M. I. Gorinova, S. Webb, E. Rossi, and M. M. Bronstein, “GRAND: graph neural diffusion,” *CoRR*, vol. abs/2106.10934, 2021. [Online]. Available: <https://arxiv.org/abs/2106.10934>
- [40] M. Eliasof, E. Haber, and E. Treister, “Graph neural reaction diffusion models,” *SIAM Journal on Scientific Computing*, vol. 46, no. 4, pp. C399–C420, 2024. doi: 10.1137/23M1576700. [Online]. Available: <https://doi.org/10.1137/23M1576700>
- [41] A. A. A. Elhag, G. Corso, H. Stärk, and M. M. Bronstein, “Graph anisotropic diffusion,” *arXiv preprint*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.00354>
- [42] T. K. Rusch, M. M. Bronstein, and S. Mishra, “A survey on oversmoothing in graph neural networks.” [Online]. Available: <http://arxiv.org/abs/2303.10993>
- [43] W. Zhao, C. Wang, X. Wang, C. Han, T. Guo, and T. Yu, “Understanding oversmoothing in diffusion-based gnns from the perspective of operator semigroup theory,” 2025. [Online]. Available: <https://arxiv.org/abs/2402.15326>
- [44] U. Alon and E. Yahav, “On the bottleneck of graph neural networks and its practical implications.” [Online]. Available: <http://arxiv.org/abs/2006.05205>
- [45] S. Akansha, “Over-squashing in graph neural networks: A comprehensive survey.” [Online]. Available: <http://arxiv.org/abs/2308.15568>

- [46] J. Topping, F. Di Giovanni, B. P. Chamberlain, X. Dong, and M. M. Bronstein, “Understanding over-squashing and bottlenecks on graphs via curvature,” 2021. [Online]. Available: <https://arxiv.org/abs/2111.14522>
- [47] E. Heid, K. P. Greenman, Y. Chung, S.-C. Li, D. E. Graff, F. H. Vermeire, H. Wu, W. H. Green, and C. J. McGill, “Chemprop: A machine learning package for chemical property prediction,” *Journal of Chemical Information and Modeling*, vol. 64, no. 1, pp. 9–17, 2024. doi: 10.1021/acs.jcim.3c01250
- [48] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [49] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” 2017.
- [50] D. Buterez, J. P. Janet, D. Oglic, and P. Liò, “An end-to-end attention-based approach for learning on graphs,” *Nature Communications*, vol. 16, no. 1, p. 5244, 2025. doi: 10.1038/s41467-025-60252-z
- [51] A. Brandt, “Multi-level adaptive solutions to boundary-value problems,” *Mathematics of Computation*, vol. 31, no. 138, pp. 333–390, 1977.
- [52] R. Ying, J. You, C. Morris, X. Ren, W. L. Hamilton, and J. Leskovec, “Hierarchical graph representation learning with differentiable pooling,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [53] D. Kim, S. Yim, S. Park, K. Yoo, K. Lee, D. Hwang, S. Lee, J. Jang, and K. Kim, “Hierarchical multi-scale modeling of absolute binding affinity in protein complexes,” in *Learning Meaningful Representations of Life (LMRL) Workshop at ICLR 2026*, 2026.
- [54] R. Bentivoglio, E. Isufi, S. N. Jonkman, and R. Taormina, “Multi-scale hydraulic graph neural networks for flood modelling,” *Natural Hazards and Earth System Sciences*, vol. 25, no. 1, pp. 335–351, 2025.
- [55] P. Garnier, J. Viquerat, and E. Hachem, “Multi-grid graph neural networks with self-attention for computational mechanics,” *arXiv preprint arXiv:2402.07672*, 2024.
- [56] B. Miquel-Oliver, M. Gil-Sorribes, V. Guallar, and A. Molina, “Effective resistance rewiring: A simple topological correction for over-squashing,” *arXiv preprint arXiv:2603.11944*, mar 2026.

- [57] K. Oono and T. Suzuki, “Graph neural networks exponentially lose expressive power for node classification,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [58] M. Scholkemper, X. Wu, A. Jadbabaie, and M. T. Schaub, “Residual connections and normalization can provably prevent oversmoothing in GNNs,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [59] L. D. Williams, B. Chawla, and B. R. Shaw, “The hydrogen bonding of cytosine with guanine: Calorimetric and  $^1\text{H}$ -nmr analysis of the molecular interactions of nucleic acid bases,” *Biopolymers*, vol. 26, pp. 591–603, 1987. doi: 10.1002/bip.360260411
- [60] Y. Kyogoku, R. C. Lord, and A. Rich, “The effect of substituents on the hydrogen bonding of adenine and uracil derivatives,” *Proceedings of the National Academy of Sciences of the United States of America*, vol. 57, no. 2, pp. 250–257, 1967. doi: 10.1073/pnas.57.2.250
- [61] M. Monajjemi and B. Chahkandi, “Theoretical investigation of hydrogen bonding in Watson–Crick, Hoogsteen and their reversed and other models: comparison and analysis for configurations of adenine–thymine base pairs in 9 models,” *Journal of Molecular Structure: THEOCHEM*, vol. 714, no. 1, pp. 43–60, 2005. doi: 10.1016/j.theochem.2004.10.023
- [62] D. A. Case, H. M. Aktulga, K. Belfon, D. S. Cerutti, G. A. Cisneros, V. W. D. Cruzeiro, N. Forouzesheh, T. J. Giese, A. W. Götz, H. Gohlke, S. Izadi, K. Kasavajhala, M. C. Kaymak, E. King, T. Kurtzman, T.-S. Lee, P. Li, J. Liu, T. Luchko, R. Luo, M. Manathunga, M. R. Machado, H. M. Nguyen, K. A. O’Hearn, A. V. Onufriev, F. Pan, S. Pantano, R. Qi, A. Rahnamoun, A. Risheh, S. Schott-Verdugo, A. Shajan, J. Swails, J. Wang, H. Wei, X. Wu, Y. Wu, S. Zhang, S. Zhao, Q. Zhu, T. E. Cheatham, III, D. R. Roe, A. Roitberg, C. Simmerling, D. M. York, M. C. Nagan, and K. M. Merz, Jr., “AmberTools,” *Journal of Chemical Information and Modeling*, vol. 63, no. 20, pp. 6183–6191, 2023. doi: 10.1021/acs.jcim.3c01153
- [63] J. D. Chodera, “A simple method for automated equilibration detection in molecular simulations,” *Journal of Chemical Theory and Computation*, vol. 12, no. 4, pp. 1799–1805, 2016. doi: 10.1021/acs.jctc.5b00784

- [64] D. R. Roe and T. E. Cheatham, III, "PTRAJ and CPPTRAJ: Software for processing and analysis of molecular dynamics trajectory data," *Journal of Chemical Theory and Computation*, vol. 9, no. 7, pp. 3084–3095, 2013. doi: 10.1021/ct400341p
- [65] R. J. Gowers, M. Linke, J. Barnoud, T. J. E. Reddy, M. N. Melo, S. L. Seyler, J. Domanski, D. L. Dotson, S. Buchoux, I. M. Kenney, and O. Beckstein, "MDAnalysis: A Python package for the rapid analysis of molecular dynamics simulations," in *Proceedings of the 15th Python in Science Conference*, S. Benthall and S. Rostrup, Eds. Austin, TX: SciPy, 2016. doi: 10.25080/majora-629e541a-00e pp. 98–105.
- [66] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, and O. Beckstein, "MDAnalysis: A toolkit for the analysis of molecular dynamics simulations," *Journal of Computational Chemistry*, vol. 32, no. 10, pp. 2319–2327, 2011. doi: 10.1002/jcc.21787
- [67] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [68] F. Clark, G. R. Robb, D. J. Cole, and J. Michel, "Automated adaptive absolute binding free energy calculations," *Journal of Chemical Theory and Computation*, vol. 20, pp. 7806–7828, 2024. doi: 10.1021/acs.jctc.4c00806
- [69] S. Fan, B. I. Iorga, and O. Beckstein, "Prediction of octanol-water partition coefficients for the sampl6-log p molecules using molecular dynamics simulations with opls-aa, amber and charmm force fields," *Journal of Computer-Aided Molecular Design*, vol. 34, no. 5, pp. 543–560, 2020.
- [70] F. Clark, D. J. Cole, and J. Michel, "Robust automated truncation point selection for molecular simulations," *Journal of Chemical Theory and Computation*, vol. 20, no. 18, pp. 7935–7949, 2024. doi: 10.1021/acs.jctc.4c01359
- [71] K. A. Brameld and W. A. Goddard, III, "Ab initio quantum mechanical study of the structures and energies for the pseudorotation of 5'-dehydroxy analogues of 2'-deoxyribose and ribose sugars," *Journal of*

*the American Chemical Society*, vol. 121, no. 5, pp. 985–993, 1999. doi: 10.1021/ja982995f

- [72] N. Foloppe and A. D. MacKerell, Jr., “Conformational properties of the deoxyribose and ribose moieties of nucleic acids: A quantum mechanical study,” *The Journal of Physical Chemistry B*, vol. 102, no. 34, pp. 6669–6678, 1998. doi: 10.1021/jp9818683
- [73] D. Buterez *et al.*, “An end-to-end attention-based approach for learning on graphs,” *Nature Communications*, vol. 16, no. 1, p. 5244, 2025. doi: 10.1038/s41467-025-60252-z



# **Appendix A**

## **Supporting materials**

## A.1 Short discussion on the ergodicity of MD simulations

Unlike Langevin dynamics, for which rigorous convergence-to-equilibrium results are available under suitable assumptions, commonly used thermostat and barostat couplings in biomolecular molecular dynamics are usually justified at the level of preserving or approximately sampling the target ensemble rather than through general proofs of ergodic convergence. In particular, some methods, such as stochastic velocity-rescaling thermostats and extended-system barostats, are constructed so that the canonical or isothermal-isobaric distribution is invariant in continuous time, whereas others, such as the Berendsen barostat, are primarily designed for efficient equilibration and do not rigorously generate the target ensemble.

This does not undermine the MD method, because MD simulations may be interpreted both as realizations of an approximate dynamical model and, when appropriate ensemble-preserving couplings are employed, as a means of sampling equilibrium configurations.

## A.2 Errors

### A.2.1 Initial load re-optimization bug

The result of a load optimization problem tells us how many systems to allocate to each available partition, and we use this to partition our lexicographically ordered trimer dataset - assigning a partition to each of the trimers. Additionally, we keep a resource allocation dictionary that, for each partition, lists the systems allocated to it. This dictionary facilitates load re-optimization as, given new counts for each of the partitions, we can proceed in the following way: for any partition that is now assigned less systems than it originally had, the last systems in the resource allocation dictionary entry for that partition are taken away and moved to partitions that are now allocated more systems. The problem was that the added systems could arrive in a non-lexicographic order, which was not accounted for originally.

For example: after a re-optimization, a partition could first receive system 400 and then system 300, which would be written down as [..., 400,300] in the resource allocation dictionary. However, when the job script for the new systems is generated, they are lexicographically ordered: [300,400]. Thus, if there is a second re-optimization that allocates one less system than the

previous iteration to the partition, system 300 would be taken away from the partition and given to another partition. Even though the script indicated how many jobs to cancel to avoid double runs, once the user cancels the last job (as indicated by the script), the job for system 400 is cancelled but the job for system 300 is still run, now on two different partitions. Thankfully, this does not also result in system 400 being lost thanks to other safety checks.

### **A.3 Complementary Figures**

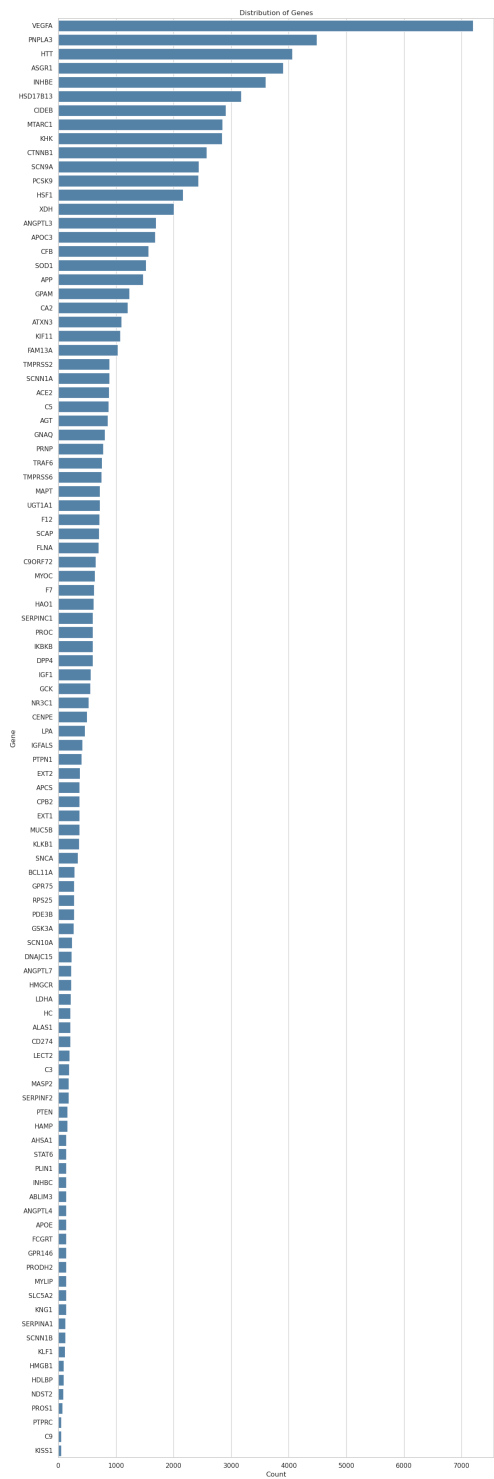


Figure A.1: Distribution of targeted genes across the dataset





# €€€€ For DIVA €€€€

```
{
  "Author1": { "Last name": "Gallardo Saenz",
    "First name": "Javier",
    "Local User Id": "u1st5bmy",
    "E-mail": "javgal@kth.se",
    "organisation": {"L1": " and ",
  },
},
"Cycle": "2",
"Course code": "SF259X",
"Credits": "30.0",
"Degree1": {"Educational program": "Master's Programme, Computer Simulations for Science and Engineering, 120 credits",
  "programcode": "TDTNM",
  "Degree": "Degree of Master of Science in Engineering",
  "subjectArea": "Computer Simulations for Science and Engineering (Applied Mathematics)"
},
},
"Title": {
  "Main title": "Molecular-dynamics-assisted hierarchical machine learning for modified siRNA activity prediction",
  "Language": "eng",
  "Alternative title": {
    "Main title": "Molekyldynamikassisterad hierarkisk maskininlärning för aktivitetsprediktion av modifierade siRNA",
    "Language": "swe"
  },
},
"Supervisor1": { "Last name": "Gerard",
  "First name": "Emilie",
  "E-mail": "emilie.gerard@astrazeneca.com",
  "Other organisation": "AstraZeneca AB"
},
"Supervisor2": { "Last name": "Klähn",
  "First name": "Marco",
  "E-mail": "marco.klahn@astrazeneca.com",
  "Other organisation": "AstraZeneca AB"
},
"Supervisor3": { "Last name": "Runborg",
  "First name": "Olof",
  "Local User Id": "u1XXXXXX",
  "E-mail": "olofr@kth.se",
  "organisation": {"L1": "",
    "L2": "Numerical Analysis"
  },
},
"Examiner1": { "Last name": "Runborg",
  "First name": "Olof",
  "Local User Id": "u1XXXXXX",
  "E-mail": "olofr@kth.se",
  "organisation": {"L1": "",
    "L2": "Numerical Analysis"
  },
},
"Cooperation": { "Partner_name": "AstraZeneca AB - MolecularAI department",
  "National Subject Categories": "10105, 10201, 10602",
  "SDGs": "3, 9",
  "Other information": {"Year": "2026", "Number of pages": "xxviii,241"},
  "Copyrightleft": "copyright",
  "Series": { "Title of series": "TRITA – XXX-EX", "No. in series": "2025:0000" },
  "Opponents": { "Name": "XXX",
  "Presentation": { "Date": "2022-03-15 13:00"
  },
  "Language": "XXX",
  "Room": "via Zoom https://kth-se.zoom.us/j/ddddddddddd",
  "Address": "Isafjordsgatan 22 (Kistagången 16)",
  "City": "Stockholm",
  "Number of lang instances": "2",
  "Abstract[eng ]": €€€€
```

Small interfering RNA (siRNA) molecules are short double-stranded oligonucleotides capable of silencing disease-associated genes through the RNA interference pathway. Their therapeutic promise is conditioned on the introduction of chemical modifications that simultaneously ensure stability, cellular delivery, target engagement and minimal off-target activity. The resulting design space is combinatorially vast and, at present, essentially inaccessible to brute-force experimental or fully atomistic exploration: a single 21-nucleotide modified duplex requires several hours of GPU time to simulate in explicit water, while the number of chemically distinct duplexes of that length easily exceeds  $\sim 10^{40}$  even under conservative assumptions on the available modifications.

This thesis investigates whether the messenger RNA knockdown activity of a modified siRNA duplex --- a global, experimentally measured property --- can be predicted from the structural and dynamical properties of its constituent double-stranded trimers, and whether a model built on this premise can generalise to unseen sequences and chemical modifications. To address this question, we curate a dataset of more than 88\,000 experimentally characterised modified siRNAs, develop an automated fragmentation procedure that separates each modified nucleotide into backbone, sugar and base

fragments, and cross-reference the extracted fragments with the \textit{modXNA} parametrisation catalogue in order to identify the unique nucleotides and overlapping trimers that can be simulated. The compatible trimers are embedded in standardised pentamer constructs and simulated by molecular dynamics in an incremental, high-throughput pipeline that also extracts a set of structural and dynamical descriptors per trimer. In parallel, we conduct a mathematical analysis of graph-based machine learning models on siRNA molecular graphs, formalising the problem as a hierarchical learning task over a graph of graphs and proving that standard message-passing neural networks suffer from exponential over-squashing on siRNA duplexes even when hydrogen bonds and base-stacking interactions are explicitly encoded as edges. Motivated by this analysis, we introduce a modular hierarchical architecture that integrates atom-, nucleotide-, trimer- and molecule-level representations, together with the notion of \textit{airtight} hierarchical models, in which within-level propagation is restricted to elements that share a common aggregation context.

Evaluated on a demanding, generalisation-oriented split, the proposed hierarchical models clearly surpass the best previously available baseline for modified siRNA activity prediction, and ablations show that the molecular dynamics-derived trimer features contribute measurably to this improvement. The resulting workflow provides an end-to-end, reproducible pipeline covering dataset curation, trimer identification, molecular dynamics simulation, feature extraction and hierarchical model training, and constitutes a step towards a physics-assisted, computationally tractable exploration of the modified siRNA chemical space.

"Abstract[swe]": €€€€

Små interfererande RNA-molekyler (siRNA) är korta dubbelsträngade oligonukleotider som kan tysta sjukdomsassocierade gener genom RNA-interferensvägen. Deras terapeutiska potential förutsätter att kemiska modifieringar införs, vilka samtidigt måste säkerställa stabilitet, cellulär leverans, bindning till målsekvensen och minimal oavsiktlig påverkan på andra gener. Det resulterande designutrymmet är kombinatoriskt enormt och för närvarande i praktiken otillgängligt för uttömmande experimentell eller helt atomistisk utforskning: en enda modifierad duplex bestående av 21 nukleotider kräver flera timmars beräkningstid på en grafikprocessor för att simuleras i explicit vatten, medan antalet kemiskt skilda duplex av samma längd med lätthet överstiger  $(10^{40})$  även under konservativa antaganden om vilka modifieringar som finns tillgängliga.

Denna avhandling undersöker om förmågan hos en modifierad siRNA-duplex att tysta budbärar-RNA -- en global, experimentellt mätbar egenskap -- kan förutsägas utifrån de strukturella och dynamiska egenskaperna hos duplexens ingående dubbelsträngade trimerer, och om en modell som bygger på denna premiss kan generalisera till sekvenser och kemiska modifieringar som inte förekommit under träningen. För att besvara denna fråga sammanställer vi ett dataset med fler än 88\,000 experimentellt karakteriserade modifierade siRNA, utvecklar en automatiserad fragmenteringsprocedur som delar upp varje modifierad nukleotid i ryggrads-, socker- och basfragment och korsrefererar de extraherade fragmenten mot parametriseringskatalogen \textit{modXNA} för att identifiera de unika nukleotider och överlappande trimerer som kan simuleras. De kompatibla trimererna bäddas in i standardiserade pentamerkonstrukt och simuleras med molekylodynamik i ett inkrementellt arbetsflöde med hög genomströmning som även extraherar en uppsättning strukturella och dynamiska deskriptorer för varje trimer. Parallellt genomför vi en matematisk analys av grafbaserade maskininlärningsmodeller tillämpade på molekylära grafer för siRNA, formulerar problemet som en hierarkisk inlärningsuppgift över en graf av grafer och visar att konventionella neurala nätverk baserade på meddelandeöverföring uppvisar exponentiell överkomprimering (\textit{over-squashing}) på siRNA-duplex, även när vätebindningar och basstaplingsinteraktioner uttryckligen representeras som kanter. Med utgångspunkt i denna analys introducerar vi en modulär hierarkisk arkitektur som integrerar representationer på atom-, nukleotid-, trimer- och molekylnivå, tillsammans med begreppet \textit{lufttäta} hierarkiska modeller, där informationsspridningen inom en nivå begränsas till element som delar ett gemensamt aggregeringssammanhang.

Vid utvärdering på en krävande datauppdelning, utformad för att pröva modellernas generaliseringsförmåga, överträffar de föreslagna hierarkiska modellerna tydligt den bästa tidigare tillgängliga referensmodellen för aktivitetsprediktion av modifierade siRNA, och ablationsstudier visar att de trimeregenskaper som härletts ur molekylodynamik bidrar mätbart till denna förbättring. Det resulterande arbetsflödet tillhandahåller en reproducerbar helhetslösning som omfattar kurering av data, identifiering av trimerer, molekylodynamiksimuleringar, extraktion av egenskaper och träning av hierarkiska modeller, och utgör ett steg mot en fysikassisterad och beräkningsmässigt hanterbar utforskning av det kemiska rummet för modifierade siRNA.

€€€€

"Keywords[eng]": €€€€

Small interfering RNA, Chemical modifications, Molecular dynamics, Hierarchical Machine Learning, Physics-assisted Machine Learning, Graph Neural Networks, Transformers, Convolutional Neural Network, Over-squashing, Activity prediction. €€€€

€€€€

"Keywords[swe]": €€€€

Små interfererande RNA, kemiska modifieringar, molekylodynamik, hierarkisk maskininlärning, fysikassisterad maskininlärning, grafneurala nätverk, transformers, konvolutionellt neuralt nätverk, överkomprimering (over-squashing), aktivitetsprediktion. €€€€, }



# acronyms.tex

```
%%% Local Variables:
%%% mode: latex
%%% TeX-master: t
%%% End:
% The following command is used with glossaries-extra
\setabbreviationstyle[acronym]{long-short}
% The form of the entries in this file is \newacronym{label}{acronym}{phrase}
% or \newacronym[options]{label}{acronym}{phrase}
% see "User Manual for glossaries.sty" for the details about the options, one example is shown below
% note the specification of the long form plural in the line below
%\newacronym[longplural={Debugging Information Entities}]{DIE}{DIE}{Debugging Information Entity}
%
% The following example also uses options
%\newacronym[shortplural={OSes}, firstplural={operating systems (OSes)}]{OS}{OS}{operating system}

% note the use of a non-breaking dash in long text for the following acronym
%\newacronym[IQL]{IQL}{Independent -QLearning}

% example of putting in a trademark on first expansion
%\newacronym[first={NVIDIA OpenSHMEM Library (NVSHMEM\texttrademark)}]{NVSHMEM}{NVSHMEM}{NVIDIA OpenSHMEM Library}

\newacronym{API}{API}{Application Programming Interface}
\newacronym{DiVA}{DiVA}{Digitala Vetenskapliga Arkivet}
\newacronym{IMRAD}{IMRAD}{Introduction, Methods, Results, and Discussion}
\newacronym{JSON}{JSON}{JavaScript Object Notation}
\newacronym{KI}{KI}{Karolinska Institutet}
\newacronym{KOPPS}{KOPPS}{Kurs- och programplaneringssystemet}
\newacronym{LADOK}{LADOK}{Lokalt -addbaserat dokumentationssystem}
\newacronym{TIMTM}{TIMTM}{Interactive Media Technology}
\newacronym{TMMTM}{TMMTM}{Media Managemen}
\newacronym{KTH}{KTH}{KTH Royal Institute of Technology}

\newacronym{LAN}{LAN}{Local Area Network}
\newacronym{VM}{VM}{virtual machine}
% note the use of a non-breaking dash in the following acronym
%\newacronym{WiFi}{-WiFi}{Wireless Fidelity}

\newacronym{WLAN}{WLAN}{Wireless Local Area Network}
\newacronym{UN}{UN}{United Nations}
\newacronym{SDG}{SDG}{Sustainable Development Goal}

#####
% MY ACRONYMS
#####
\newacronym{RNA}{RNA}{Ribonucleic Acid}
\newacronym{DNA}{DNA}{Deoxyribonucleic Acid}
\newacronym{siRNA}{siRNA}{Small interfering RNA}
\newacronym{RNAi}{RNAi}{RNA interference}
\newacronym{ML}{ML}{Machine Learning}
\newacronym{GNN}{GNN}{Graph Neural Network}
\newacronym{mRNA}{mRNA}{Messenger RNA}
\newacronym{RISC}{RISC}{RNA-induced silencing complex}
\newacronym{MD}{MD}{Molecular Dynamics}
\newacronym{PBC}{PBC}{Periodic Boundary Condition}
\newacronym{FFN}{FFN}{Feed Forward Network}
\newacronym{NN}{NN}{Neural Network}
\newacronym{MPNN}{MPNN}{Message Passing Neural Network}
\newacronym{D-MPNN}{D-MPNN}{Directed Message Passing Neural Network}
\newacronym{MLP}{MLP}{Multi Layer Perceptron}
\newacronym{BFS}{BFS}{Breadth First Search}
\newacronym{DFS}{DFS}{Depth First Search}
\newacronym{MAE}{MAE}{Mean Absolute Error}
\newacronym{CNN}{CNN}{Convolutional Neural Network}
\newacronym{SMILES}{SMILES}{Simplified Molecular Input Line Entry System}
\newacronym{InChI}{InChI}{IUPAC International Chemical Identifier}
\newacronym{InChIKey}{InChIKey}{IUPAC International Chemical Identifier Key}
\newacronym{PCA}{PCA}{Principal Component Analysis}
\newacronym{PDE}{PDE}{Partial Differential Equation}
\newacronym{MILP}{MILP}{Mixed Linear Integer Programming}
\newacronym{JS}{JS}{Jensen--Shannon}
\newacronym{RMSD}{RMSD}{Root Mean Square Deviation}
\newacronym{RMSF}{RMSF}{Root Mean Square Fluctuation}
```